AWS Certified Generative AI Developer - Professional

AIP-C01 Service Study Guides

Comprehensive service-by-service study material covering all 80+ in-scope AWS services from Appendix B of the exam guide, ordered by exam relevance across four tiers.

Tier	Exam Relevance	Key Services
Tier 1	Core GenAI (Most tested)	Amazon Bedrock, Knowledge Bases, Agents, Prompt Management, Prompt Flows
Tier 2	High-Frequency (5+ references)	SageMaker, Comprehend, Kendra, Q Business, Step Functions, Lambda, Titan
Tier 3	Important (3-5 references)	CloudWatch, IAM, API Gateway, OpenSearch, DynamoDB, S3, KMS
Tier 4	Supporting (1-2 references)	X-Ray, Amplify, ECS, EKS, DataSync, Amazon Connect

Table of Contents

Amazon Bedrock -- Comprehensive Study Guide (AIP-C01) . . . 10

1. Service Overview . . . 10
2. Foundation Models Available . . . 10
3. Key APIs . . . 11
4. Sub-services and Features . . . 13
5. Key CloudWatch Metrics . . . 18
6. TypeScript AWS SDK v3 Usage . . . 19
7. Integration Patterns . . . 23
8. Cost Optimization . . . 25
9. Security . . . 25
10. Structured Outputs . . . 26
11. Exam Gotchas and Traps . . . 26
12. Quick Reference Decision Framework . . . 28
13. Key Terminology Quick Reference . . . 29

Amazon Bedrock Knowledge Bases -- Comprehensive Study Guide (AIP-C01) . . . 30

1. Service Overview . . . 30
2. Data Sources . . . 31
3. Chunking Strategies . . . 32
4. Embedding Models . . . 33
5. Vector Stores . . . 34
6. Search and Retrieval . . . 36
7. Grounding and Hallucination Reduction . . . 37
8. Query Decomposition . . . 38
9. Parsing Options . . . 38
10. TypeScript AWS SDK v3 Usage . . . 39
11. Integration Patterns . . . 42
12. Monitoring and Logging . . . 43
13. Cost Considerations . . . 44
14. Exam Gotchas and Common Traps . . . 45
15. When to Use / When NOT to Use . . . 46
16. Quick Reference Card . . . 47

Bedrock Prompt Management, Prompt Flows, and Agentic AI Study Guide 49

AIP-C01 Exam Coverage 49

1. Amazon Bedrock Prompt Management 49

2. Amazon Bedrock Prompt Flows 51

3. Amazon Bedrock AgentCore 54

4. Strands Agents SDK 56

5. AWS Agent Squad (Multi-Agent Orchestrator) 59

6. Model Context Protocol (MCP) 60

7. Tool Use / Function Calling 62

8. Agentic Design Patterns 64

9. Exam Gotchas 66

10. When to Use Each: Decision Matrix 67

11. TypeScript SDK v3 Usage 68

12. Key Metrics and Monitoring 71

Quick Reference Card 72

Amazon SageMaker AI Ecosystem -- Comprehensive Study Guide (AIP-C01) 75

1. SageMaker AI Core -- Inference and Deployment 75

2. SageMaker JumpStart -- Foundation Models 76

3. SageMaker Model Registry 77

4. SageMaker Clarify -- Bias, Explainability, and FM Evaluation 79

5. SageMaker Ground Truth -- Data Labeling 80

6. SageMaker Data Wrangler -- Data Preparation 81

7. SageMaker Processing -- Data Processing Jobs 82

8. SageMaker Model Monitor -- Production Monitoring 83

9. SageMaker Neo -- Model Compilation and Edge Optimization 84

10. SageMaker Unified Studio 85

11. Model Customization for GenAI 86

12. Model Cards -- Governance and Compliance 88

13. Exam Gotchas -- SageMaker vs Bedrock Decision Points 89

14. When to Use / When NOT to Use -- Decision Matrix 91

15. Key CloudWatch Metrics 91

16. TypeScript SDK v3 Usage 93

17. Cost Considerations 96

Quick Reference Card . . . 97

AI/ML Individual Services -- Comprehensive Study Guide (AIP-C01) . . . 98

Table of Contents . . . 98

1. Amazon Comprehend . . . 98

2. Amazon Kendra . . . 101

3. Amazon Q Business and Q Apps . . . 104

4. Amazon Q Developer . . . 107

5. Amazon Titan Foundation Models . . . 109

6. Amazon Lex . . . 112

7. Amazon Transcribe . . . 114

8. Amazon Textract . . . 117

9. Amazon Rekognition . . . 120

10. Amazon Augmented AI (A2I) . . . 123

11. Service Comparison Decision Guide . . . 126

Application Integration Services -- Comprehensive Study Guide (AIP-C01) . . . 129

1. AWS Step Functions (Heavily Tested) . . . 129

2. Amazon EventBridge . . . 141

3. Amazon SQS . . . 143

4. Amazon SNS . . . 145

5. Amazon AppFlow . . . 146

6. AWS AppConfig . . . 148

7. Cross-Service Integration Patterns for GenAI . . . 151

8. Quick-Reference Comparison Table . . . 151

9. Exam Strategy Tips . . . 152

Compute Services for Generative AI -- Study Guide . . . 153

1. AWS Lambda (Primary Focus) . . . 153

2. Amazon EC2 for GenAI . . . 163

3. AWS App Runner . . . 165

4. AWS Lambda@Edge . . . 166

5. AWS Outposts . . . 167

6. AWS Wavelength . . . 167

Quick Reference: Service Selection Matrix . . . 169

Exam Tips Summary . . . 169

Management and Governance Services for the AIP-C01 Exam . . . 172

Table of Contents . . . 172

1. Amazon CloudWatch . . . 172

2. Amazon CloudWatch Logs . . . 180

3. Amazon CloudWatch Synthetics . . . 183

4. AWS CloudTrail . . . 184

5. AWS Auto Scaling . . . 185

6. Cost Tools . . . 186

7. AWS Well-Architected Tool . . . 188

8. Other Services . . . 189

Quick-Reference: When to Use What . . . 190

Common Exam Gotchas . . . 191

Security, Identity, and Compliance -- Comprehensive Study Guide (AIP-C01) . . . 192

1. IAM (Identity and Access Management) . . . 192

2. IAM Identity Center . . . 200

3. Amazon Cognito . . . 201

4. AWS KMS (Key Management Service) . . . 204

5. AWS Secrets Manager . . . 207

6. Amazon Macie . . . 210

7. AWS WAF (Web Application Firewall) . . . 212

8. AWS Encryption SDK . . . 214

9. Cross-Cutting Exam Themes . . . 217

Networking and Content Delivery -- Comprehensive Study Guide (AIP-C01) . . . 219

1. Amazon API Gateway (Heavily Tested) . . . 219

2. AWS AppSync . . . 229

3. Amazon CloudFront . . . 231

4. Amazon VPC . . . 232

5. AWS PrivateLink . . . 235

6. Elastic Load Balancing (ELB) . . . 236

7. AWS Global Accelerator . . . 237

8. Amazon Route 53 . . . 238

9. Exam Cheat Sheet -- Key Facts . . . 240

Analytics Services -- Comprehensive Study Guide (AIP-C01) . . . 243

1. Amazon OpenSearch Service . . . 243
2. AWS Glue . . . 253
3. Amazon Athena . . . 257
4. Amazon EMR . . . 259
5. Amazon Kinesis . . . 261
6. Amazon QuickSight . . . 264
7. Amazon Managed Streaming for Apache Kafka (MSK) . . . 265
8. Service Selection Quick Reference . . . 268
9. Key Metrics to Monitor . . . 269

Database Services -- Comprehensive Study Guide (AIP-C01) . . . 270

Exam Relevance Map . . . 270
1. Amazon Aurora (pgvector Focus) . . . 270
2. Amazon DynamoDB . . . 275
3. Amazon ElastiCache . . . 281
4. Amazon Neptune . . . 285
5. Amazon DocumentDB . . . 288
6. Amazon RDS (PostgreSQL with pgvector) . . . 290
7. Database Selection Decision Guide for GenAI . . . 291
8. Exam Gotchas and Traps . . . 292

Developer Tools for GenAI -- Study Guide . . . 295

1. AWS X-Ray . . . 295
2. AWS Amplify . . . 299
3. CI/CD for GenAI (CodePipeline, CodeBuild, CodeDeploy) . . . 301
4. AWS CDK and CloudFormation . . . 305
5. AWS CodeArtifact . . . 307
6. AWS CLI for Bedrock . . . 308
7. AWS Tools and SDKs . . . 309
Quick Reference: When to Use Each Service . . . 313
Exam Gotchas Summary . . . 314

Container Services for GenAI -- Study Guide . . . 316

1. Amazon ECS (Elastic Container Service) . . . 316
2. Amazon EKS (Elastic Kubernetes Service) . . . 320
3. AWS Fargate . . . 321

4. Amazon ECR (Elastic Container Registry) . . . 322
5. Container Deployment Patterns for GenAI . . . 324
6. Exam Gotchas . . . 326
7. When to Use Each -- Decision Matrix . . . 327
8. TypeScript SDK v3 Usage . . . 328
9. Key Metrics -- CloudWatch . . . 331
Summary -- Key Facts for Exam Day . . . 332
Storage and Migration Services for GenAI . . . 334
Amazon S3 . . . 334
Amazon EBS . . . 348
Amazon EFS . . . 350
AWS DataSync . . . 351
AWS Transfer Family . . . 352
Cross-Cutting Exam Gotchas . . . 354
Amazon Connect -- GenAI Study Guide (AIP-C01) . . . 355
1. Service Overview . . . 355
2. GenAI Features . . . 355
3. Integration with Bedrock and Lex . . . 356
4. Exam Relevance . . . 357
5. Exam Gotchas . . . 358
6. When to Use / When NOT to Use . . . 358
7. Key Concepts Quick Reference . . . 358
8. Architecture Patterns for Exam . . . 359
Summary for Exam Day . . . 359

Tier 1: Core GenAI Services

These services appear in virtually every exam domain and task statement. Amazon Bedrock alone accounts for the majority of exam questions. Expect deep integration knowledge, API-level details, and architectural patterns.

Guides in this tier:

- Amazon Bedrock
- Bedrock Knowledge Bases & RAG
- Agents, Prompt Management & Flows

Amazon Bedrock -- Comprehensive Study Guide (AIP-C01)

Amazon Bedrock is the **most heavily tested service** on the AWS Certified Generative AI Developer - Professional exam. It appears across virtually every domain and task statement. This guide covers the core Bedrock service itself; Knowledge Bases, Agents, and Flows are covered separately.

---

1. Service Overview

Amazon Bedrock is a **fully managed service** that provides secure API access to high-performing foundation models (FMs) from multiple AI providers. It lets you build and scale generative AI applications without managing infrastructure.

What Bedrock Does

- Provides a **single unified API** to access 100+ FMs from 17+ providers
- Supports text generation, image generation, embeddings, video generation, speech-to-speech, and multimodal understanding
- Enables model customization (fine-tuning, continued pre-training) with your own data
- Offers content safety controls through Guardrails
- Provides model evaluation capabilities (automatic, human, LLM-as-a-judge)
- Supports document and media processing through Data Automation
- Handles scaling, security, and compliance as a fully managed service

When to Use Bedrock

- You need access to **multiple FM providers** through a single API
- You want a **serverless, fully managed** inference experience (no infrastructure to manage)
- You need **enterprise security**: data never leaves your account, never used to train base models
- You need **content safety controls** (Guardrails) built into the inference pipeline
- You want to **switch between models** without rewriting application code (Converse API)
- You require **compliance**: SOC, ISO, HIPAA-eligible, FedRAMP High, GDPR

When NOT to Use Bedrock

- You need to **train a model from scratch** -- use SageMaker AI instead
- You need to host a **custom open-weight model** with full control over infrastructure -- use SageMaker AI endpoints
- You need models **not available on Bedrock** -- check Bedrock Marketplace first, then SageMaker
- You need **real-time fine-tuning** during inference -- Bedrock fine-tuning is an offline batch process

---

2. Foundation Models Available

Model Providers and Key Families

Provider	Key Models	Primary Use Cases
Amazon	Nova Micro, Nova Lite, Nova Pro, Nova Premier, Nova Canvas, Nova Reel, Nova Sonic, Titan Embeddings V2, Titan Image Generator	Text, multimodal, image gen, video gen, speech-to-speech, embeddings
Anthropic	Claude Opus 4.6, Claude Sonnet 4.6, Claude Haiku 4.5, Claude Opus 4.5	Complex reasoning, coding, analysis, agentic workflows
Meta	Llama 4 Maverick, Llama 4 Scout, Llama 3.3 70B, Llama 3.1 (8B/70B/405B)	Open-weight text generation, multilingual
Mistral AI	Mistral Large 3, Devstral 2, Ministral, Pixtral Large	Code, multilingual, vision
Cohere	Command R/R+, Embed v4, Rerank 3.5	RAG, search, embeddings, reranking
AI21 Labs	Jamba 1.5 Large/Mini	Long-context text generation
Stability AI	Stable Image (various)	Image generation and manipulation
DeepSeek	DeepSeek-R1, DeepSeek-V3.1, DeepSeek-V3.2	Reasoning, code
OpenAI	GPT-OSS 20B/120B, GPT-OSS Safeguard	Text generation, safety
Google	Gemma 3 (4B/12B/27B)	Lightweight inference
Qwen	Qwen3 235B, Qwen3 32B, Qwen3 Coder	Code, multilingual
Writer	Palmyra X4, Palmyra X5	Enterprise text
TwelveLabs	Marengo Embed	Video embeddings
Others	MiniMax, Moonshot (Kimi), NVIDIA Nemotron	Various

Model Selection Considerations (Exam-Critical)

- **Claude Opus 4.6**: Most capable model for complex tasks, highest cost
- **Claude Sonnet 4.6**: Best balance of capability and cost for most production workloads
- **Claude Haiku 4.5**: Fastest, cheapest Claude model for high-volume, latency-sensitive tasks
- **Nova Micro**: Text-only, lowest latency and cost for simple text tasks
- **Nova Lite**: Multimodal (text + image + video), low cost
- **Nova Pro**: Best balance for multimodal tasks within Amazon's family
- **Nova Premier**: Complex reasoning, agentic workflows, model distillation teacher
- **Titan Embeddings V2**: Cost-effective embeddings for RAG
- **Cohere Embed v4**: Multimodal embeddings with leading retrieval quality

---

3. Key APIs

Amazon Bedrock provides **four main API families** for inference. Knowing when to use each is heavily tested.

API Family Comparison

API	Endpoint	Best For	Key Feature
Responses API	`bedrock-mantle.{region}.amazonaws.com`	Stateful conversations, agentic apps	Built-in tool use, multimodal, stateful -- **recommended for new apps**
Chat Completions	`bedrock-mantle.{region}.amazonaws.com`	Stateless multi-turn chat	OpenAI-compatible, lightweight
Converse / ConverseStream	`bedrock-runtime.{region}.amazonaws.com`	Multi-turn chat, model portability	Unified interface across all Bedrock models
InvokeModel / InvokeModelWithResponseStream	`bedrock-runtime.{region}.amazonaws.com`	Single transactions, embeddings, image gen	Direct model access, full control over request/response format

Converse API (Exam Priority: HIGH)

The Converse API provides a **consistent, model-agnostic interface** for conversational inference. Write code once, switch models by changing the `modelId`.

Key characteristics:

- Unified request/response format across all models that support messages
- Supports `Converse` (synchronous) and `ConverseStream` (streaming)
- Permission: `bedrock:InvokeModel` for Converse, `bedrock:InvokeModelWithResponseStream` for ConverseStream
- Supports: text, images, documents, video, tool use, guardrails, prompt caching
- Automatically applies model-specific prompt templates (e.g., for Mistral, Meta)
- Returns standardized `usage` (inputTokens, outputTokens) and `metrics` (latencyMs)

Request structure:

```
modelId (required) -- model ID, inference profile ID, or provisioned model ARN
messages[] -- array of {role, content[]} objects
  role: "user" | "assistant"
  content[]: text | image | document | video | toolUse | toolResult | cachePoint
system[] -- system prompt content blocks
inferenceConfig -- temperature, maxTokens, topP, stopSequences
toolConfig -- tool definitions for function calling
guardrailConfig -- guardrail ID, version, and trace settings
additionalModelRequestFields -- model-specific parameters (e.g., top_k for Claude)
requestMetadata -- custom metadata for invocation logs
serviceTier -- "standard" | "priority" | "flex"
```

Response structure:

- `output.message` -- the generated message with role and content
- `stopReason` -- "end_turn", "tool_use", "max_tokens", "stop_sequence", "guardrail_intervened"
- `usage` -- { inputTokens, outputTokens, totalTokens }
- `metrics` -- { latencyMs }
- `trace` -- guardrail assessment details (if trace enabled)

InvokeModel API (Exam Priority: HIGH)

Direct, model-specific API for single-turn inference.

Key characteristics:

- Request body is **model-specific JSON** -- different format for Claude vs. Llama vs. Titan

- Returns raw model response in the format specified by `contentType`
- Best for: embeddings, image generation, single-turn text, or when you need model-specific features
- Permission: `bedrock:InvokeModel`
- Does NOT support documents or video directly (Converse API restriction applies)

When to use InvokeModel vs. Converse:

Scenario	Use
Multi-turn conversation	Converse
Need to switch models easily	Converse
Generating embeddings	InvokeModel
Image generation	InvokeModel
Need model-specific parameters not in Converse	InvokeModel
Consistent interface across models	Converse
Legacy integration with model-specific format	InvokeModel

Streaming APIs

- **ConverseStream**: Streaming version of Converse. Returns events: `messageStart`, `contentBlockStart`, `contentBlockDelta`, `contentBlockStop`, `messageStop`, `metadata`
- **InvokeModelWithResponseStream**: Streaming version of InvokeModel. Returns raw chunks.
- The **AWS CLI does NOT support streaming operations** -- must use SDK
- Check `responseStreamingSupported` field from `GetFoundationModel` to confirm model support

Async and Batch APIs

- **StartAsyncInvoke**: For long-running operations like video generation. Returns an `invocationArn` immediately; results stored in S3. Permission: `bedrock:InvokeModel`
- **Batch Inference**: Process thousands of prompts asynchronously. Input/output via S3. Supports both InvokeModel and Converse format. NOT supported for provisioned models.
- **InvokeModelWithBidirectionalStream**: Full-duplex streaming for real-time speech-to-speech (Nova Sonic)

---

4. Sub-services and Features

4.1 Bedrock Guardrails

Configurable safeguards that work with **any** FM (Bedrock-hosted, self-hosted, or third-party via `ApplyGuardrail` API).

Six safeguard policies:

Policy	What It Does	Action Options
Content Filters	Detects harmful text/image content across categories: Hate, Insults, Sexual, Violence, Misconduct, Prompt Attacks	Block, Detect (no action)

Policy	What It Does	Action Options
Denied Topics	Blocks specific topics you define	Block
Word Filters	Exact-match blocking of custom words/phrases and profanity	Block
Sensitive Information Filters	Detects PII (names, SSNs, emails, etc.)	Block or Mask (anonymize with tags)
Contextual Grounding Checks	Detects hallucinations by checking if responses are grounded in source material	Block, Detect
Automated Reasoning Checks	Validates responses against logical rules and policies using formal mathematical techniques	Block, Detect

Tiers:

- **Classic Tier**: Basic content filters and denied topics
- **Standard Tier**: Enhanced contextual understanding, typo detection, 60+ language support, code domain protection, distinguishes jailbreaks from prompt injection. Requires opt-in to cross-Region inference.

Key exam points:

- Guardrails can be applied in **synchronous** or **asynchronous** streaming mode
- In **synchronous** streaming mode: buffers chunks, applies policies before sending (more latency, better accuracy)
- In **asynchronous** streaming mode: sends chunks immediately, applies policies in background (no latency impact, but may initially pass inappropriate content). Does NOT support masking.
- `ApplyGuardrail` API: Apply guardrails **independently** without invoking a model -- use for self-hosted or third-party models
- Blocked content appears as plain text in model invocation logs (can be disabled)
- Guardrails support **cross-Region inference** for guardrail policy evaluation
- Image content filtering supported (max 8000x8000 px, 4 MB, 20 images per request)

4.2 Bedrock Data Automation (BDA)

Automates transformation of **unstructured multimodal data** (documents, images, video, audio) into structured formats using generative AI.

Key capabilities:

- Document processing: parse, classify, extract structured data from PDFs, forms, invoices
- Media analysis: video summaries, unsafe content detection, text extraction, brand classification
- Supports documents up to 3,000 pages
- Provides **confidence scores** and **bounding box** data for extracted fields
- **Blueprints**: Define custom extraction schemas (what fields to extract)
- **Projects**: Group standard and custom output configurations; support LIVE and DEVELOPMENT stages

APIs:

- `InvokeDataAutomationAsync` -- asynchronous processing, returns invocation ARN
- `InvokeDataAutomation` -- synchronous processing for documents and images
- `CreateBlueprint`, `CreateDataAutomationProject` -- configuration

Integration with RAG: BDA can be used as a parser for Knowledge Bases to convert unstructured docs into RAG-ready chunks.

4.3 Bedrock Model Evaluation

Evaluate and compare FMs for your specific use cases.

Three evaluation methods:

Method	Description	Use Case
Automatic (Programmatic)	Uses predefined metrics (accuracy, robustness, toxicity) with built-in or custom datasets	Quick, objective model comparison
Human Evaluation	Human raters assess model outputs using custom metrics (relevance, style, brand voice)	Subjective quality assessment
LLM-as-a-Judge	Uses a second LLM to evaluate model responses with metrics like correctness, completeness, harmfulness	Human-like quality at lower cost

Key exam points:

- Can evaluate **any model** on Bedrock: serverless, Marketplace, customized, distilled, imported, prompt routers
- "Bring your own inference responses": Evaluate models hosted anywhere (not just Bedrock)
- RAG evaluation: Evaluate retrieval quality and generation quality of Knowledge Bases
- **Compare feature**: View results of multiple evaluation jobs side-by-side
- Supported use cases: text generation, classification, Q&A, summarization

4.4 Model Invocation Logging

Collects invocation logs, model input data, and model output data for all invocations in an account/Region.

Configuration:

- **Disabled by default** -- must be explicitly enabled
- Destinations: **CloudWatch Logs** and/or **Amazon S3**
- S3 is required for large data (images, embeddings, video) and binary outputs
- Can selectively enable: text, image, embedding, video data delivery
- Logs stored until logging configuration is deleted

APIs:

- `PutModelInvocationLoggingConfiguration`
- `GetModelInvocationLoggingConfiguration`
- `DeleteModelInvocationLoggingConfiguration`

Exam tip: S3 bucket must be in the **same Region** as the logging configuration. If using SSE-KMS, add a KMS encryption policy to the bucket.

4.5 Cross-Region Inference (CRIS)

Automatically routes inference requests to other AWS Regions with available capacity for higher throughput.

Two types:

Type	Data Boundary	Throughput	Cost
Geographic CRIS	Stays within geographic boundary (e.g., US, EU)	Higher than single-region	Standard pricing
Global CRIS	Any supported commercial AWS Region worldwide	Highest available	~10% cost savings

Key exam points:

- **No additional cost** for using cross-Region inference (geographic); ~10% savings for global
- All data transmitted **stays on AWS network**, encrypted in transit between Regions
- CloudTrail logs **only in the source Region** (simplifies monitoring)
- Can route to Regions **not manually enabled** in your account
- Identified by inference profile IDs: geographic profiles use region prefix, global profiles use `global.` prefix
- Supported for: on-demand inference, batch inference, agents, model evaluation, prompt management, prompt flows
- **IAM requirements**: Three-part policy for global CRIS (regional inference profile, regional FM, global FM)
- **SCP considerations**: Must update `aws:RequestedRegion` conditions to allow `"unspecified"` for global CRIS
- Prioritizes source Region when capacity is available (minimizes latency)

Quota management:

- Quotas measured in Tokens Per Minute (TPM) and Requests Per Minute (RPM)
- **Burndown rate**: Output tokens consume **5x** more quota than input tokens
- Request quota increases through Service Quotas console in the **source** Region

4.6 Provisioned Throughput

Purchase dedicated throughput capacity for a model to guarantee consistent performance.

Two purchasing methods:

- **By Model Units (MUs)**: Request MUs through AWS Support, then purchase via console or `CreateProvisionedModelThroughput` API
- **By Tokens**: Purchase tokens-per-minute capacity directly

Key characteristics:

- Returns a `provisionedModelArn` to use as `modelId` in inference calls
- Can be used with InvokeModel, InvokeModelWithResponseStream, Converse, ConverseStream
- Can be associated with agent aliases
- Commitment terms available (1-month, 6-month) for discounts, or no-commitment
- Custom models **require** Provisioned Throughput for inference
- **Batch inference is NOT supported** for provisioned models
- Can change associated model only for custom model Provisioned Throughputs

When to use:

- Predictable, high-volume production workloads
- Need guaranteed throughput that won't be throttled
- Running custom (fine-tuned) models -- they require it

4.7 Service Tiers

On-demand inference offers four tiers to balance cost and performance:

Tier	Pricing	Latency	Best For
Reserved	Fixed monthly per 1K TPM	Guaranteed, 99.5% uptime SLA	Mission-critical, predictable workloads; 1-month or 3-month reservations
Priority	Standard + premium	Up to 25% better OTPS latency	Customer-facing apps, real-time interaction

Tier	Pricing	Latency	Best For
Standard	Base rate	Consistent	General-purpose AI tasks
Flex	Discounted	Higher latency tolerated	Non-urgent: evaluations, summarizations, agentic workflows

Set the tier **per API call** using the `serviceTier` parameter.

4.8 Model Customization

Customize FMs with your own data to improve domain-specific performance.

Two methods:

Method	Data Type	Purpose	Supported Models
Fine-tuning	Labeled input-output pairs (JSONL from S3)	Teach model specific task behavior	Claude 3 Haiku, Llama, Nova, Titan, Cohere, Mistral
Continued Pre-training	Unlabeled domain text	Expand model knowledge for a domain/industry	Amazon Titan, Nova

Additional methods via SageMaker AI recipes for Nova:

- Supervised Fine-Tuning (PEFT, FFT)
- Alignment (DPO, PPO)
- Knowledge Distillation
- Reinforcement Fine-Tuning (RFT)

Key exam points:

- Both create a **private copy** of the FM accessible only to your account
- Data transferred securely through **VPC**
- Custom models **require Provisioned Throughput** for inference
- Training and validation data stored in **S3**
- Encrypted with **AWS KMS** (customer-managed keys supported)
- **EventBridge** integration for job status change notifications
- Use **canaries** in training data + prompt logging to detect training data leakage

4.9 Bedrock Marketplace

Access and deploy additional models from third-party providers not available as serverless Bedrock models.

How it works:

- Subscribe to a model in the Bedrock Marketplace catalog
- Deploy to a **SageMaker AI endpoint** (specify instance type and count)
- Register the endpoint with Bedrock
- Invoke using standard Converse or InvokeModel APIs
- Deployment takes approximately 10-15 minutes

Key points:

- Endpoints hosted by SageMaker AI but accessed through Bedrock APIs
- Supports `Converse` and `InvokeModel` operations
- Do NOT modify endpoints directly in SageMaker (may cause failures) -- use Bedrock Marketplace console/API
- If endpoint fails due to SageMaker modifications, deregister and reregister

4.10 Intelligent Prompt Routing

Dynamically routes prompts to different models within a model family based on complexity.

- Routes simple prompts to smaller/cheaper models, complex prompts to larger models
- Can reduce costs by up to **30%** without compromising accuracy
- Uses a prompt router that you configure

4.11 Model Distillation

Creates smaller, faster, more cost-effective models from larger teacher models.

- Up to **500% faster** and **75% less expensive** than original models
- Less than **2%** accuracy loss
- Useful for RAG and other specific use cases where you can trade minimal accuracy for significant speed/cost gains
- Nova Premier serves as a teacher model for distillation

4.12 Prompt Caching

Caches frequently used prompt prefixes to reduce latency and cost.

Key characteristics:

- Reduces inference latency by up to **85%**
- Reduces input token costs by up to **90%**
- Supported in Converse, ConverseStream, InvokeModel, InvokeModelWithResponseStream
- Uses **cache checkpoints** to define what to cache
- Cache has a **TTL**: most models support 5-minute TTL; some support extended 1-hour TTL
- TTL resets with each cache hit; cache expires after TTL window with no hits
- Cached tokens charged at a reduced rate
- For Claude models: automatic cache management with a single breakpoint (checks ~20 content blocks back)

Best use cases:

- Chat with document (static document, varying questions)
- Coding assistants (static codebase context)
- Agentic workflows (repeated tool/system prompts)
- Few-shot learning (reused examples)

Supported models: Claude (Opus, Sonnet, Haiku 4.5+), Amazon Nova (Micro, Lite, Pro, Premier, Nova 2 Lite)

---

5. Key CloudWatch Metrics

Runtime Metrics (AWS/Bedrock namespace)

Metric	Description	Exam Relevance
`Invocations`	Count of successful invocations	Usage tracking
`InvocationLatency`	Time from request to complete response (ms)	Performance monitoring

Metric	Description	Exam Relevance
`TimeToFirstToken`	Time from request to first token in streaming (ms)	Streaming performance; set alarms for latency SLAs
`InputTokenCount`	Number of input tokens processed	Cost attribution
`OutputTokenCount`	Number of output tokens generated	Cost attribution
`OutputImageCount`	Number of images generated	Cost tracking for image models
`InvocationClientErrors`	4xx errors	Application debugging
`InvocationServerErrors`	5xx errors	Service health monitoring
`InvocationThrottles`	Throttled requests	Capacity planning; trigger for CRIS or Provisioned Throughput
`EstimatedTPMQuotaUsage`	Estimated TPM quota consumed (includes burndown)	Proactive capacity management; request increases before throttling

Logging Delivery Metrics

Metric	Description
`ModelInvocationLogsCloudWatchDeliverySuccess`	Successful log deliveries to CloudWatch
`ModelInvocationLogsCloudWatchDeliveryFailure`	Failed log deliveries to CloudWatch
`ModelInvocationLogsS3DeliverySuccess`	Successful log deliveries to S3
`ModelInvocationLogsS3DeliveryFailure`	Failed log deliveries to S3

Pre-built CloudWatch Dashboards

CloudWatch provides pre-configured Model Invocations dashboards:

- Invocation count, latency, token counts by model
- Daily token counts by model ID
- Requests grouped by input tokens
- Invocation throttles and error counts

---

6. TypeScript AWS SDK v3 Usage

InvokeModel (Synchronous)

```
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

// InvokeModel -- request body is model-specific
```

```
const response = await client.send(
  new InvokeModelCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    contentType: "application/json",
    accept: "application/json",
    body: JSON.stringify({
      anthropic_version: "bedrock-2023-05-31",
      max_tokens: 1024,
      messages: [
        {
          role: "user",
          content: "Explain quantum computing in one paragraph.",
        },
      ],
    }),
  })
);

const responseBody = JSON.parse(new TextDecoder().decode(response.body));
console.log(responseBody.content[0].text);
```

Converse API (Synchronous)

```
import {
  BedrockRuntimeClient,
  ConverseCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

// Converse API -- unified format across all models
const response = await client.send(
  new ConverseCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    messages: [
      {
        role: "user",
        content: [{ text: "Explain quantum computing in one paragraph." }],
      },
    ],
    system: [{ text: "You are a physics professor. Be concise." }],
    inferenceConfig: {
      maxTokens: 1024,
      temperature: 0.7,
      topP: 0.9,
    },
  })
);

console.log(response.output?.message?.content?.[0]?.text);
console.log("Input tokens:", response.usage?.inputTokens);
console.log("Output tokens:", response.usage?.outputTokens);
console.log("Latency (ms):", response.metrics?.latencyMs);
console.log("Stop reason:", response.stopReason);
```

ConverseStream (Streaming)

```
import {
  BedrockRuntimeClient,
  ConverseStreamCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new ConverseStreamCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    messages: [
      {
        role: "user",
        content: [{ text: "Write a short story about a robot." }],
      },
    ],
    inferenceConfig: {
      maxTokens: 2048,
      temperature: 0.8,
    },
  })
);

// Process the stream of events
if (response.stream) {
  for await (const event of response.stream) {
    if (event.contentBlockDelta?.delta?.text) {
      process.stdout.write(event.contentBlockDelta.delta.text);
    }
    if (event.messageStop) {
      console.log("\nStop reason:", event.messageStop.stopReason);
    }
    if (event.metadata) {
      console.log("Usage:", event.metadata.usage);
      console.log("Latency:", event.metadata.metrics?.latencyMs, "ms");
    }
  }
}
```

InvokeModelWithResponseStream (Streaming)

```
import {
  BedrockRuntimeClient,
  InvokeModelWithResponseStreamCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new InvokeModelWithResponseStreamCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    contentType: "application/json",
    accept: "application/json",
    body: JSON.stringify({
      anthropic_version: "bedrock-2023-05-31",
      max_tokens: 2048,
      messages: [
        { role: "user", content: "Write a haiku about cloud computing." },
      ],
```

```
    }),
  })
);

if (response.body) {
  for await (const event of response.body) {
    if (event.chunk?.bytes) {
      const chunk = JSON.parse(new TextDecoder().decode(event.chunk.bytes));
      if (chunk.type === "content_block_delta") {
        process.stdout.write(chunk.delta?.text ?? "");
      }
    }
  }
}
```

Converse with Tool Use

```
import {
  BedrockRuntimeClient,
  ConverseCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

const toolConfig = {
  tools: [
    {
      toolSpec: {
        name: "get_weather",
        description: "Get current weather for a city",
        inputSchema: {
          json: {
            type: "object",
            properties: {
              city: { type: "string", description: "City name" },
            },
            required: ["city"],
          },
        },
      },
    },
  ],
};

const response = await client.send(
  new ConverseCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    messages: [
      {
        role: "user",
        content: [{ text: "What's the weather in Seattle?" }],
      },
    ],
    toolConfig,
  })
);

// If stopReason is "tool_use", the model wants to call a tool
if (response.stopReason === "tool_use") {
```

```
  const toolUseBlock = response.output?.message?.content?.find(
    (block) =&gt; "toolUse" in block
  );
  console.log("Tool call:", toolUseBlock?.toolUse?.name);
  console.log("Tool input:", toolUseBlock?.toolUse?.input);
}
```

Converse with Guardrails

```
const response = await client.send(
  new ConverseCommand({
    modelId: "anthropic.claude-sonnet-4-20250514",
    messages: [
      {
        role: "user",
        content: [{ text: "Tell me something harmful." }],
      },
    ],
    guardrailConfig: {
      guardrailIdentifier: "my-guardrail-id",
      guardrailVersion: "1",
      trace: "enabled", // returns guardrail assessment in response trace
    },
  })
);

if (response.stopReason === "guardrail_intervened") {
  console.log("Guardrail blocked the response");
  console.log("Trace:", JSON.stringify(response.trace, null, 2));
}
```

Using Cross-Region Inference

```
// Use the inference profile ID instead of a regional model ID
const response = await client.send(
  new ConverseCommand({
    // Geographic CRIS profile (US)
    modelId: "us.anthropic.claude-sonnet-4-20250514-v1:0",
    // Or Global CRIS profile:
    // modelId: "global.anthropic.claude-sonnet-4-20250514-v1:0",
    messages: [
      {
        role: "user",
        content: [{ text: "Hello, world!" }],
      },
    ],
  })
);
```

---

7. Integration Patterns

Lambda + Bedrock

The most common pattern. Lambda function invokes Bedrock models.

```
API Gateway --&gt; Lambda --&gt; Bedrock Runtime (Converse/InvokeModel)
```

- Lambda IAM role needs `bedrock:InvokeModel` permission
- For streaming: use Lambda response streaming with function URL or API Gateway WebSocket
- Consider Lambda timeout (max 15 minutes) for long model responses

API Gateway + Lambda + Bedrock

- REST API: Standard request/response, good for Converse API
- WebSocket API: Required for real-time streaming to clients
- HTTP API: Lower latency, lower cost alternative to REST API

Step Functions + Bedrock

- **Direct Bedrock integration**: Step Functions can invoke Bedrock models directly without Lambda
- Use for orchestrating multi-step AI workflows
- Combine with other AWS services (S3, DynamoDB, SQS)
- Supports the Bedrock `InvokeModel` and `Converse` API actions natively

EventBridge + Bedrock

- Receive notifications for model customization job status changes
- Trigger workflows when invocation logging detects anomalies
- React to Bedrock Data Automation job completions

S3 + Bedrock

- Store training data for model customization
- Store batch inference input/output
- Store model invocation logs (large data)
- Store async invocation results (video generation, embeddings)
- Store Data Automation results

CloudWatch + Bedrock

- Monitor runtime metrics (invocations, latency, tokens, throttles)
- Set alarms on `InvocationThrottles` to trigger scaling actions
- Use CloudWatch Logs Insights to query invocation logs
- Create dashboards for operational visibility

CloudTrail + Bedrock

- Logs all Bedrock API calls for auditing
- Cross-Region inference requests logged in the **source Region** only
- Track who invoked which models, when, and from where

---

8. Cost Optimization

Pricing Models

Model	Description	Best For
On-Demand	Pay per input/output token, no commitment	Variable or unpredictable workloads
Provisioned Throughput	Purchase dedicated capacity (Model Units or Tokens)	Predictable high-volume workloads, custom models
Batch Inference	~50% cost reduction vs. on-demand	Non-time-sensitive bulk processing
Service Tiers	Flex (~discount), Standard, Priority (~premium), Reserved	Match tier to urgency

Token Cost Optimization Strategies

1. **Optimize prompt length**: Keep prompts concise; remove unnecessary context
2. **Control response length**: Set `maxTokens` appropriately; use stop sequences
3. **Prompt caching**: Cache static prompt prefixes for up to 90% cost reduction on cached tokens
4. **Model selection**: Use the smallest model that meets quality requirements
   - Route simple tasks to Nova Micro or Haiku
   - Route complex tasks to Sonnet or Opus
5. **Intelligent Prompt Routing**: Auto-route to optimal model based on complexity (up to 30% savings)
6. **Model Distillation**: Create smaller specialized models (up to 75% cheaper)
7. **Batch inference**: Use for non-urgent workloads (50% savings)
8. **Cross-Region Inference (Global)**: ~10% savings on token pricing
9. **Flex tier**: Discount pricing for latency-tolerant workloads
10. **Request-response caching**: Cache full responses externally (ElastiCache, DynamoDB) for identical queries

Quota Management

- Monitor `EstimatedTPMQuotaUsage` in CloudWatch
- Set alarms at **70-80%** quota utilization
- Output token burndown rate is **5:1** (output tokens consume 5x more quota than input)
- Request quota increases **proactively** through Service Quotas console

---

9. Security

Data Protection

- **Data isolation**: Bedrock never shares your data with model providers or uses it to train base models
- **Encryption in transit**: TLS 1.2 for all inter-network data, SSL for API/console requests
- **Encryption at rest**: All resources encrypted using AWS KMS
- **Customer-managed keys**: Supported for model customization jobs, agents, knowledge bases, evaluations
- **No data storage**: Bedrock does NOT store text, images, or documents provided as inference content

Network Security

- **VPC endpoints**: Use AWS PrivateLink for private connections between VPC and Bedrock
- **VPC-protected features**: Model customization jobs, batch inference, Knowledge Bases
- **S3 gateway endpoints**: Use for private access to S3 during model customization
- **VPC Flow Logs**: Monitor network traffic for security analysis

Identity and Access Management

- **IAM permissions**:
  - `bedrock:InvokeModel` -- for Converse, InvokeModel, StartAsyncInvoke
  - `bedrock:InvokeModelWithResponseStream` -- for ConverseStream, InvokeModelWithResponseStream
  - Deny both actions to fully block inference access to a resource
- **Service roles**: Required for model customization, evaluation, and Knowledge Bases
- **Least privilege**: Use SCPs to restrict access to specific models/Regions
- **Cross-account**: S3 cross-account access supported for custom model import

Compliance

- SOC 1/2/3, ISO 27001, CSA STAR Level 2
- HIPAA-eligible
- FedRAMP High authorized
- GDPR compliant
- Comprehensive CloudTrail audit logging

Abuse Detection

Built-in automated abuse detection mechanisms identify and respond to potential misuse.

Prompt Injection Security

- Content filters detect prompt attacks (jailbreaks, prompt injection)
- Standard Tier Guardrails distinguish between jailbreaks and prompt injection
- **Prompt leakage** detection available in Standard Tier
- Defense-in-depth approach recommended: combine model controls + guardrails + application-level validation

---

10. Structured Outputs

A capability for guaranteeing **schema-compliant JSON responses** through constrained decoding.

- Ensures valid, type-safe JSON matching a specified JSON Schema
- Two approaches: **JSON Schema output format** (controls response format) and **strict tool use** (validates tool parameters)
- Eliminates parsing failures, missing fields, type mismatches
- Works with: Cross-Region inference, batch inference, streaming
- Supported providers: Anthropic, DeepSeek, Google, MiniMax, Mistral AI, Moonshot AI, NVIDIA, OpenAI, Qwen

---

11. Exam Gotchas and Traps

API Selection Traps

Trap	Correct Answer
"Which API provides a consistent interface across all Bedrock models?"	**Converse API** (not InvokeModel)
"Which API should you use for generating embeddings?"	**InvokeModel** (Converse does not support embeddings)
"Which API should you use for image generation?"	**InvokeModel** (or StartAsyncInvoke for video)
"Which API is recommended for new applications?"	**Responses API** (for modern apps) or **Converse API** (for native Bedrock apps needing model portability)
"How do you stream responses from Bedrock?"	**ConverseStream** or **InvokeModelWithResponseStream** -- NOT the AWS CLI (CLI does not support streaming)
"What permission does ConverseStream require?"	`bedrock:InvokeModelWithResponseStream` (NOT `bedrock:InvokeModel`)

Cross-Region Inference Traps

Trap	Correct Answer
"Is there extra cost for geographic CRIS?"	**No** -- same pricing as on-demand
"Is there extra cost for global CRIS?"	**No additional surcharge** -- actually ~10% cheaper
"Where are CloudTrail logs recorded for CRIS?"	**Source Region only**
"Can CRIS route to disabled Regions?"	**Yes** -- even Regions not manually enabled
"What must you update in SCPs for global CRIS?"	Allow `aws:RequestedRegion: "unspecified"`

Provisioned Throughput Traps

Trap	Correct Answer
"Can you use batch inference with provisioned models?"	**No**
"Do custom (fine-tuned) models require Provisioned Throughput?"	**Yes** -- custom models require Provisioned Throughput for inference
"What do you pass as modelId for provisioned throughput?"	The **provisionedModelArn** (not the base model ID)

Guardrails Traps

Trap	Correct Answer
"Can Guardrails work with non-Bedrock models?"	**Yes** -- use `ApplyGuardrail` API

Trap	Correct Answer
"Does async streaming guardrail mode support masking?"	**No** -- only synchronous mode supports masking
"What happens in Detect mode?"	Guardrail detects but takes **no action** -- content passes through; useful for testing

General Traps

Trap	Correct Answer
"Does Bedrock store your inference data?"	**No** -- data is used only to generate the response
"Is model invocation logging enabled by default?"	**No** -- disabled by default, must be explicitly enabled
"What is the output token burndown rate for quotas?"	**5:1** -- output tokens consume 5x more quota
"Where must the S3 bucket for invocation logging be?"	**Same Region** as the logging configuration
"Can you change the model for a base model Provisioned Throughput?"	**No** -- only for custom model Provisioned Throughputs

---

12. Quick Reference Decision Framework

```
Need to invoke a model?
  |
  +-- New app / OpenAI migration? --&gt; Responses API or Chat Completions
  |
  +-- Need model-agnostic interface? --&gt; Converse API
  |
  +-- Need embeddings or image gen? --&gt; InvokeModel
  |
  +-- Need real-time streaming? --&gt; ConverseStream or InvokeModelWithResponseStream
  |
  +-- Need async (video gen)? --&gt; StartAsyncInvoke
  |
  +-- Need bulk processing? --&gt; Batch Inference
  |
  +-- Need speech-to-speech? --&gt; InvokeModelWithBidirectionalStream (Nova Sonic)

Need higher throughput?
  |
  +-- Data residency requirements? --&gt; Geographic Cross-Region Inference
  |
  +-- Maximum throughput + cost savings? --&gt; Global Cross-Region Inference
  |
  +-- Guaranteed capacity for production? --&gt; Provisioned Throughput or Reserved Tier
  |
  +-- Running a custom model? --&gt; Provisioned Throughput (required)

Need content safety?
  |
  +-- Using Bedrock models? --&gt; Guardrails via guardrailConfig parameter
```

```
  |
  +-- Using external models? --&gt; ApplyGuardrail API
  |
  +-- Need hallucination detection? --&gt; Contextual Grounding Checks
  |
  +-- Need factual accuracy? --&gt; Automated Reasoning Checks
  |
  +-- Need PII protection? --&gt; Sensitive Information Filters (Block or Mask)

Need to reduce costs?
  |
  +-- Repeated prompt prefixes? --&gt; Prompt Caching (up to 90% savings)
  |
  +-- Variable complexity? --&gt; Intelligent Prompt Routing (up to 30% savings)
  |
  +-- Non-urgent workload? --&gt; Batch Inference (50% savings) or Flex tier
  |
  +-- Can use a smaller model? --&gt; Model Distillation (up to 75% savings)
  |
  +-- Global workload? --&gt; Global CRIS (~10% savings)
```

---

13. Key Terminology Quick Reference

Term	Definition
Foundation Model (FM)	Large AI model trained on vast amounts of diverse data
Base Model	Pre-packaged FM from a provider, available in Bedrock
Inference	Process of a model generating output from input
Token	Sequence of characters that a model treats as a single unit
Inference Profile	Defines an FM and the Regions to which requests can be routed
Provisioned Throughput	Dedicated capacity purchased for consistent model performance
Model Customization	Adjusting model parameters using training data (fine-tuning or continued pre-training)
Hyperparameters	Settings that control the training process during customization
Inference Parameters	Settings that influence response generation (temperature, maxTokens, topP)
RAG	Retrieval Augmented Generation -- querying data sources to augment prompts
Blueprint	Custom extraction schema for Bedrock Data Automation
Cache Checkpoint	Point in a prompt that defines what gets cached for prompt caching
Burndown Rate	Ratio at which output tokens consume quota (5:1 for Bedrock)

Amazon Bedrock Knowledge Bases -- Comprehensive Study Guide (AIP-C01)

Amazon Bedrock Knowledge Bases is the **fully managed RAG (Retrieval Augmented Generation) service** on the exam. It appears heavily in Domain 1 (31% of the exam), particularly around vector stores, chunking strategies, retrieval mechanisms, and grounding to reduce hallucinations.

Exam relevance: Tasks 1.4 (vector stores), 1.5 (retrieval mechanisms), 3.1.3 (grounding to reduce hallucinations)

---

1. Service Overview

What It Is

Amazon Bedrock Knowledge Bases is a fully managed capability that implements the **entire RAG workflow** -- ingestion, vectorization, storage, retrieval, and response generation -- without requiring custom integrations or data flow management.

How RAG Works with Knowledge Bases

```
■■■■■■■■■■■■■■■■■■■■■■ INGESTION TIME ■■■■■■■■■■■■■■■■■■■■■■
■                                                          ■
■  Data Source (S3, Confluence, etc.)                      ■
■       ■                                                  ■
■       ▼                                                  ■
■  Parse Documents (default / FM / BDA parser)             ■
■       ■                                                  ■
■       ▼                                                  ■
■  Chunk Documents (fixed / hierarchical / semantic)       ■
■       ■                                                  ■
■       ▼                                                  ■
■  Generate Embeddings (Titan V2, Cohere, etc.)            ■
■       ■                                                  ■
■       ▼                                                  ■
■  Store in Vector Database (OpenSearch, Aurora, etc.)      ■
■                                                          ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■■■■■■■■■■■■■■■■■■■■■■ QUERY TIME ■■■■■■■■■■■■■■■■■■■■■■■■■■
■                                                          ■
■  User Query                                              ■
■       ■                                                  ■
■       ▼                                                  ■
■  Convert Query to Embedding (same embedding model)       ■
■       ■                                                  ■
■       ▼                                                  ■
■  Search Vector Store (semantic / hybrid)                 ■
■       ■                                                  ■
■       ▼                                                  ■
■  (Optional) Rerank Results                               ■
■       ■                                                  ■
■       ▼                                                  ■
■  Augment Prompt with Retrieved Chunks                    ■
■       ■                                                  ■
```

```
■      ▼                                                ■
■  Generate Response with FM (Claude, Nova, etc.)       ■
■      ■                                                ■
■      ▼                                                ■
■  Return Response with Citations                       ■
■                                                       ■
■■■
```

- **S3 metadata files**: Must be named `<filename>.metadata.json` and stored alongside the source document. This is how you add filterable metadata attributes for S3 sources.
- **Custom data sources**: Use the `KnowledgeBaseDocuments` API to directly ingest, update, or delete individual documents without a full sync. Supports streaming data ingestion.
- **You cannot change the parsing strategy or chunking strategy after connecting a data source.** You must create a new data source to use different settings.
- Each data source connector stores authentication credentials in **AWS Secrets Manager**.

Sync Process

1. Bedrock fetches documents from the data source
2. Documents are parsed using the configured parser
3. Content is chunked according to the chunking strategy
4. Chunks are converted to embeddings using the configured embedding model
5. Embeddings are stored in the vector database with associated text and metadata

---

3. Chunking Strategies

Chunking determines how source documents are split into smaller pieces for embedding and retrieval. **You cannot change the chunking strategy after connecting a data source.**

Strategy Comparison

Strategy	How It Works	When to Use	Key Parameters
Default	Splits into ~300-token chunks, preserving sentence boundaries	Quick start; general-purpose	None (automatic)
Fixed-size	Splits into chunks of a configured token count with configurable overlap	Predictable chunk sizes; uniform documents	`maxTokens`, `overlapPercentage`
Hierarchical	Two-level structure: large parent chunks containing smaller child chunks	Need precise retrieval AND broad context; retrieves child chunks but returns parent chunks	`parentMaxTokens`, `childMaxTokens`, `overlapTokens`
Semantic	Uses NLP to group semantically similar sentences together	Documents with diverse topics; want chunks to be coherent units of meaning	`maxTokens`, `bufferSize`, `breakpointPercentileThreshold`
None	Treats each file as a single chunk	Pre-processed documents; small files	None

Detailed Parameter Reference

Fixed-Size Chunking

- **maxTokens**: Maximum number of tokens per chunk
- **overlapPercentage**: Percentage of overlap between consecutive chunks (ensures context continuity at chunk boundaries)

Hierarchical Chunking

- **Parent chunk size**: Maximum tokens for the larger (parent) chunks
- **Child chunk size**: Maximum tokens for the smaller (child) chunks derived from parents
- **overlapTokens**: Absolute number of overlapping tokens between consecutive chunks at each level
- At retrieval time, child chunks are found first, then **replaced with their parent chunks** to give the model broader context
- The returned number of results may be fewer than requested because multiple child chunks can share the same parent

Semantic Chunking

- **maxTokens**: Maximum tokens per chunk (sentence boundaries honored)
- **bufferSize**: Number of surrounding sentences to consider when computing embeddings for boundary detection. Buffer size of 1 = current + previous + next sentence (3 total)
- **breakpointPercentileThreshold**: Percentile of sentence dissimilarity at which to split. Higher threshold = fewer, larger chunks; lower threshold = more, smaller chunks
- **Additional cost**: Semantic chunking uses a foundation model to compute sentence embeddings for boundary detection

No Chunking

- Each document becomes a single chunk
- Documents must be pre-processed into individual files
- **Cannot view page numbers in citations** and cannot filter by `x-amz-bedrock-kb-document-page-number`

Multimodal Content Chunking

For audio, video, and images, chunking works differently:

- **Nova Multimodal Embeddings**: Chunking happens at the embedding model level. Audio/video chunk duration is configurable (1-30 seconds, default 5 seconds). Text chunking strategies do NOT apply to multimodal files.
- **Bedrock Data Automation (BDA) parser**: Converts multimedia to text first (transcripts, scene summaries), then applies standard text chunking.

Custom Transformation (Lambda)

You can use a **Lambda function** during ingestion to post-process chunks. This allows:

- Custom chunking logic
- Custom chunk-level metadata assignment
- Can be applied after one of Bedrock's built-in chunking options

Configuration requires a `CustomTransformationConfiguration` with the Lambda ARN and an S3 location for intermediate storage.

---

4. Embedding Models

Supported Embedding Models for Knowledge Bases

Model	Model ID	Dimensions	Vector Types	Max Tokens	Key Feature
Amazon Titan Embeddings G1 - Text	`amazon.titan-embed-text-v1`	1536 (fixed)	Floating-point	8192	Legacy; widest availability
Amazon Titan Text Embeddings V2	`amazon.titan-embed-text-v2:0`	256, 512, **1024** (configurable)	Floating-point, Binary	8192	Flexible dimensions; binary support; normalization; most regions
Cohere Embed English	`cohere.embed-english-v3`	1024	Floating-point, Binary	512	English-optimized
Cohere Embed Multilingual	`cohere.embed-multilingual-v3`	1024	Floating-point, Binary	512	100+ languages
Amazon Titan Multimodal Embeddings G1	`amazon.titan-embed-image-v1`	1024	Floating-point	N/A	Text + image embeddings
Amazon Nova Multimodal Embeddings	N/A	1024	Floating-point	N/A	Text, image, audio, video in unified vector space
Cohere Embed v3 (Multimodal)	N/A	1024	Floating-point, Binary	N/A	Multimodal embeddings

Key Exam Points for Embedding Models

- **Titan V2 is the default/recommended choice** for most use cases: configurable dimensions, binary vector support, widest region availability
- **Binary vectors** use 1 bit per dimension instead of 32 bits (floating-point). They reduce storage cost significantly but are less precise. Only supported with **Amazon OpenSearch Serverless** and **Amazon OpenSearch Managed Clusters**.
- **Lower dimensions** (e.g., 256) = faster search, less storage, slightly less accuracy. Higher dimensions = more accurate, more storage, slower.
- **Cohere Embed Multilingual** is the choice when you need **multilingual support** (100+ languages)
- **You cannot change the embedding model after creating the knowledge base.** Changing models requires creating a new knowledge base.
- The same embedding model used during ingestion MUST be used during query time.

---

5. Vector Stores

Supported Vector Databases

Vector Store	Type	Quick Create	Hybrid Search	Binary Vectors	Key Strengths
Amazon OpenSearch Serverless	AWS-managed serverless	Yes (default)	Yes	Yes	Default choice; full-text search + vector search; auto-scaling; richest filter support (`startsWith`, `stringContains`, `listContains`)
Amazon OpenSearch Service Managed Clusters	AWS-managed (provisioned)	No	Yes	Yes (v2.16+)	Fine-grained configuration; performance tuning; existing OpenSearch investments
Amazon Aurora PostgreSQL (pgvector)	AWS-managed relational	Yes (quick create)	Yes	No	SQL + vector capabilities; familiar PostgreSQL; good for existing Aurora users
Amazon S3 Vectors	AWS-managed object storage	Yes	No	No	Lowest cost; subsecond query performance; ideal for cost-sensitive large KBs
Amazon Neptune Analytics	AWS-managed graph	Yes	No	No	GraphRAG; entity relationships; multi-hop reasoning; cross-document correlation
Pinecone	Third-party SaaS	No	No	No	Pure vector search performance; managed SaaS
Redis Enterprise Cloud	Third-party SaaS	No	No	No	Ultra-low latency; real-time use cases
MongoDB Atlas	Third-party SaaS	No	Yes	No	Document database + vector search; existing MongoDB investments

Vector Store Selection Decision Framework

```
Need lowest cost for large KBs?
  → Amazon S3 Vectors

Need hybrid search (keyword + semantic)?
  → OpenSearch Serverless (default) or Aurora PostgreSQL or MongoDB Atlas

Need binary vector support?
  → OpenSearch Serverless or OpenSearch Managed Clusters

Need graph-based relationships / cross-document reasoning?
```

```
    → Neptune Analytics (GraphRAG)

  Need ultra-low latency?
    → Redis Enterprise Cloud

  Already using PostgreSQL?
    → Aurora PostgreSQL with pgvector

  Already using MongoDB?
    → MongoDB Atlas

  Default / general purpose?
    → OpenSearch Serverless (auto-created by Bedrock)
```

Vector Store Configuration

When setting up a vector store (not quick-create), you must create:

1. A **vector field** for storing embeddings (dimensions must match your embedding model)
2. A **text field** for storing raw text chunks (e.g., `AMAZON_BEDROCK_TEXT_CHUNK`)
3. A **metadata field** for Bedrock-managed metadata (e.g., `AMAZON_BEDROCK_METADATA`)

For OpenSearch Serverless:

- Use the **faiss** engine (not nmslib) -- nmslib is NOT supported
- Use **Euclidean** distance metric for floating-point embeddings
- Use **Hamming** distance metric for binary embeddings

---

6. Search and Retrieval

Search Types

Search Type	How It Works	When to Use
Semantic	Converts query to embedding, searches vector index for nearest neighbors	Default behavior; works with all vector stores
Hybrid	Combines vector embedding search AND raw text keyword search	Better recall; handles exact-match terms (names, IDs, codes) that semantic search may miss
Default	Bedrock chooses the best strategy for your vector store configuration	Let Bedrock decide

Hybrid search is only supported with: Amazon OpenSearch Serverless, Amazon Aurora PostgreSQL (RDS), and MongoDB Atlas (requires a filterable text field).

Metadata Filtering

Metadata filtering narrows search results by applying conditions on document metadata attributes BEFORE vector similarity search.

Filter Operators

Operator	API Name	Supported Types
Equals	`equals`	string, number, boolean

Operator	API Name	Supported Types
Not Equals	`notEquals`	string, number, boolean
Greater Than	`greaterThan`	number
Greater Than or Equals	`greaterThanOrEquals`	number
Less Than	`lessThan`	number
Less Than or Equals	`lessThanOrEquals`	number
In	`in`	string list
Not In	`notIn`	string list
Starts With	`startsWith`	string (OpenSearch Serverless only)
String Contains	`stringContains`	string (OpenSearch Serverless only)
List Contains	`listContains`	string (OpenSearch Serverless only)

Logical Operators

- `andAll`: All filters must match (up to 5 per group)
- `orAll`: At least one filter must match (up to 5 per group)
- You can nest up to 5 filter groups within a top-level logical operator (one level of nesting)

Implicit Metadata Filtering

A feature that **automatically generates and applies filters** based on the user query and a metadata schema. The system analyzes the query, identifies relevant metadata constraints, and applies them without the developer writing explicit filter logic. Currently only supported with **Anthropic Claude 3.5 Sonnet**.

Reranking

Reranking uses a foundation model to **re-score and reorder** initial vector search results for better relevance.

- Uses models like **Cohere Rerank 3.5** or Amazon Bedrock foundation models
- Applied AFTER the initial vector search
- Can specify **numberOfRerankedResults** (1-100) to limit how many results the reranker processes
- Can configure **metadata fields** to include or exclude during reranking (`selectionMode`: ALL or SELECTIVE)
- Reranking results **override** the default ranking from the vector store
- Only applicable to **textual data**
- Reduces cost and latency by returning fewer but more relevant results

Query Decomposition

A technique that breaks **complex, multi-part queries** into smaller, more manageable sub-queries. Each sub-query is processed independently against the knowledge base, and the results are combined to generate a comprehensive response.

Example: "What is the difference between Lambda and Fargate for running containers, and which is cheaper?" gets broken into separate sub-queries about Lambda capabilities, Fargate capabilities, and their pricing.

---

7. Grounding and Hallucination Reduction

How Knowledge Bases Provide Grounding

RAG is fundamentally a **grounding mechanism**. Instead of relying solely on the model's training data (which may be outdated or incorrect), Knowledge Bases ground the model's response in your actual data:

1. **Contextual grounding**: Retrieved chunks provide factual context that the model uses to generate responses
2. **Citation tracking**: Every response includes citations linking back to the specific source chunks used
3. **Source attribution**: Users can verify claims by tracing back to original documents
4. **Reduced hallucination**: The model is constrained to generate responses based on retrieved content rather than fabricating information

Citations in Responses

The `RetrieveAndGenerate` response includes a `citations` array. Each citation contains:

- **generatedResponsePart**: The specific segment of the generated text that is based on a source
- **retrievedReferences**: The actual source chunks with their content, location (S3 URI, page number), and metadata

Combining with Guardrails

You can apply **Bedrock Guardrails** to knowledge base queries for additional safety:

- **Contextual grounding checks**: Detect and filter hallucinations by checking if the response is grounded in the retrieved source material AND relevant to the user query
- **Grounding threshold**: Configurable threshold for how closely the response must match source material
- **Relevance threshold**: Configurable threshold for how relevant the response must be to the query
- Content filters, denied topics, PII detection all apply

---

8. Query Decomposition

How It Works

When a user asks a complex question that spans multiple topics or requires information from different parts of the knowledge base:

1. The orchestration model analyzes the query
2. It breaks the query into focused sub-queries
3. Each sub-query is independently searched against the knowledge base
4. Retrieved results from all sub-queries are combined
5. The generation model synthesizes a comprehensive answer from all retrieved context

Configuration

- Enabled through the **orchestrationConfiguration** in `RetrieveAndGenerate` requests
- You can customize the **orchestration prompt** that controls how queries are decomposed
- Prompt placeholders and XML tags are used to structure the decomposition instructions

---

9. Parsing Options

Available Parsers

Parser	What It Does	When to Use
Default parser	Extracts text from supported file formats (PDF, DOCX, TXT, HTML, CSV, etc.)	Standard text documents
Foundation model parser	Uses Claude or Nova vision models to parse complex documents including tables, charts, images	Documents with rich visual content (PDFs with tables, diagrams)
Bedrock Data Automation (BDA) parser	Converts multimodal content (audio, video) to text representations	Audio/video content; preview feature

- Foundation model parsing incurs **additional model inference charges**
- You can provide a **custom parsing prompt** when using a foundation model parser
- BDA parser is currently available in **US West (Oregon)** only (preview)

---

10. TypeScript AWS SDK v3 Usage

Creating a Knowledge Base

```
import {
  BedrockAgentClient,
  CreateKnowledgeBaseCommand,
  CreateDataSourceCommand,
  StartIngestionJobCommand,
} from "@aws-sdk/client-bedrock-agent";

const client = new BedrockAgentClient({ region: "us-east-1" });

// Step 1: Create the Knowledge Base
const createKbResponse = await client.send(
  new CreateKnowledgeBaseCommand({
    name: "my-knowledge-base",
    roleArn: "arn:aws:iam::123456789012:role/BedrockKBRole",
    knowledgeBaseConfiguration: {
      type: "VECTOR",
      vectorKnowledgeBaseConfiguration: {
        embeddingModelArn:
          "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-embed-text-v2:0",
        embeddingModelConfiguration: {
          bedrockEmbeddingModelConfiguration: {
            dimensions: 1024, // 256, 512, or 1024 for Titan V2
          },
        },
      },
    },
    storageConfiguration: {
      type: "OPENSEARCH_SERVERLESS",
      opensearchServerlessConfiguration: {
        collectionArn: "arn:aws:aoss:us-east-1:123456789012:collection/abc123",
        vectorIndexName: "bedrock-kb-index",
        fieldMapping: {
          vectorField: "bedrock-knowledge-base-default-vector",
          textField: "AMAZON_BEDROCK_TEXT_CHUNK",
          metadataField: "AMAZON_BEDROCK_METADATA",
```

```
        },
      },
    },
  })
);

const knowledgeBaseId = createKbResponse.knowledgeBase!.knowledgeBaseId!;

// Step 2: Create a Data Source (S3)
const createDsResponse = await client.send(
  new CreateDataSourceCommand({
    knowledgeBaseId,
    name: "my-s3-data-source",
    dataSourceConfiguration: {
      type: "S3",
      s3Configuration: {
        bucketArn: "arn:aws:s3:::my-kb-documents-bucket",
      },
    },
    vectorIngestionConfiguration: {
      chunkingConfiguration: {
        chunkingStrategy: "HIERARCHICAL",
        hierarchicalChunkingConfiguration: {
          levelConfigurations: [
            { maxTokens: 1500 }, // parent chunk size
            { maxTokens: 300 },  // child chunk size
          ],
          overlapTokens: 60,
        },
      },
    },
  })
);

const dataSourceId = createDsResponse.dataSource!.dataSourceId!;

// Step 3: Start Ingestion (Sync)
await client.send(
  new StartIngestionJobCommand({
    knowledgeBaseId,
    dataSourceId,
  })
);
```

Querying with RetrieveAndGenerate

```
import {
  BedrockAgentRuntimeClient,
  RetrieveAndGenerateCommand,
} from "@aws-sdk/client-bedrock-agent-runtime";

const runtimeClient = new BedrockAgentRuntimeClient({ region: "us-east-1" });

const response = await runtimeClient.send(
  new RetrieveAndGenerateCommand({
    input: { text: "What are the key features of our product?" },
    retrieveAndGenerateConfiguration: {
      type: "KNOWLEDGE_BASE",
      knowledgeBaseConfiguration: {
```

```
        knowledgeBaseId: "ABCDEF1234",
        modelArn:
          "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-5-sonnet-20241022-v2:0",
        retrievalConfiguration: {
          vectorSearchConfiguration: {
            numberOfResults: 10,
            overrideSearchType: "HYBRID",
          },
        },
        generationConfiguration: {
          inferenceConfig: {
            textInferenceConfig: {
              temperature: 0.0,
              maxTokens: 2048,
            },
          },
        },
      },
    },
  })
);

// The generated response
console.log(response.output?.text);

// Citations linking response segments to source documents
for (const citation of response.citations ?? []) {
  console.log("Generated text segment:", citation.generatedResponsePart?.textResponsePart?.text);
  for (const ref of citation.retrievedReferences ?? []) {
    console.log("  Source:", ref.location?.s3Location?.uri);
    console.log("  Content:", ref.content?.text);
  }
}
```

Querying with Retrieve (Chunks Only)

```
import {
  BedrockAgentRuntimeClient,
  RetrieveCommand,
} from "@aws-sdk/client-bedrock-agent-runtime";

const runtimeClient = new BedrockAgentRuntimeClient({ region: "us-east-1" });

const response = await runtimeClient.send(
  new RetrieveCommand({
    knowledgeBaseId: "ABCDEF1234",
    retrievalQuery: { text: "deployment best practices" },
    retrievalConfiguration: {
      vectorSearchConfiguration: {
        numberOfResults: 20,
        overrideSearchType: "HYBRID",
      },
    },
  })
);

for (const result of response.retrievalResults ?? []) {
  console.log("Score:", result.score);
  console.log("Content:", result.content?.text);
```

```
  console.log("Source:", result.location?.s3Location?.uri);
  console.log("Metadata:", result.metadata);
}
```

Metadata Filtering

```
const response = await runtimeClient.send(
  new RetrieveCommand({
    knowledgeBaseId: "ABCDEF1234",
    retrievalQuery: { text: "security policies" },
    retrievalConfiguration: {
      vectorSearchConfiguration: {
        numberOfResults: 10,
        filter: {
          andAll: [
            {
              equals: { key: "department", value: "engineering" },
            },
            {
              greaterThan: { key: "year", value: 2023 },
            },
          ],
        },
      },
    },
  })
);
```

Reranking Configuration

```
const response = await runtimeClient.send(
  new RetrieveCommand({
    knowledgeBaseId: "ABCDEF1234",
    retrievalQuery: { text: "How do I configure auto-scaling?" },
    retrievalConfiguration: {
      vectorSearchConfiguration: {
        numberOfResults: 50, // retrieve more initial results
        rerankingConfiguration: {
          type: "BEDROCK_RERANKING_MODEL",
          bedrockRerankingConfiguration: {
            numberOfRerankedResults: 10, // return top 10 after reranking
            modelConfiguration: {
              modelArn:
                "arn:aws:bedrock:us-east-1::foundation-model/cohere.rerank-v3-5:0",
            },
          },
        },
      },
    },
  })
);
```

---

11. Integration Patterns

With Bedrock Agents

The most common integration pattern. Associate a knowledge base with an agent so the agent can automatically query it when relevant:

1. Create the knowledge base independently
2. Associate it with the agent via `AssociateAgentKnowledgeBase`
3. Provide the agent with **instructions** on when/how to use the knowledge base
4. The agent autonomously decides when to invoke the knowledge base based on user queries
5. Modify query configurations at runtime using `sessionState` in `InvokeAgent`

With Custom Applications

Use the `Retrieve` API for maximum control:

1. Call `Retrieve` to get relevant chunks
2. Build your own prompt template incorporating the chunks
3. Call `InvokeModel` or `Converse` with the augmented prompt
4. Handle citations and post-processing in your application

This pattern gives you control over prompt engineering, chunk selection, and response formatting.

With Lambda Functions

Two integration points:

Ingestion-time Lambda (Custom Transformation):

- Post-process chunks during data ingestion
- Add custom metadata to chunks
- Implement custom chunking logic
- Invoked by Bedrock during the sync process

Query-time Lambda (Application Layer):

- Lambda function calls `Retrieve` or `RetrieveAndGenerate` API
- Typically behind API Gateway or invoked by an agent
- Handles business logic, authentication, session management

With Amazon Kendra GenAI Index

You can use an **Amazon Kendra GenAI index** as the retrieval backend for a Bedrock Knowledge Base. This lets you:

- Reuse existing Kendra-indexed content across multiple applications
- Leverage Kendra's high-accuracy retrieval with Bedrock's generation capabilities
- Avoid re-ingesting and re-indexing data

---

12. Monitoring and Logging

CloudWatch Logs

Knowledge bases support **APPLICATION_LOGS** for data ingestion monitoring:

- Track file processing status during ingestion jobs
- Log destinations: CloudWatch Logs, Amazon S3, or Amazon Data Firehose
- Useful queries include: files ignored during ingestion, vector embedding exceptions, deletion exceptions

Enable via console or CloudWatch API (`PutDeliverySource` / `PutDeliveryDestination` / `CreateDelivery`).

Key Debugging Queries (CloudWatch Logs Insights)

```
# All documents ignored during ingestion
fields @timestamp, @message
| filter @message like /IGNORED/

# All embedding/indexing exceptions
fields @timestamp, @message
| filter @message like /EXCEPTION/

# All exceptions during a specific ingestion job
fields @timestamp, @message
| filter ingestionJobId = "your-job-id" and @message like /EXCEPTION/
```

CloudWatch Metrics

Bedrock publishes runtime metrics under the `AWS/Bedrock` namespace:

- **Invocations**: Count of API calls
- **InvocationLatency**: End-to-end latency
- **InvocationClientErrors** / **InvocationServerErrors**: Error rates
- **InputTokenCount** / **OutputTokenCount**: Token usage
- **InvocationThrottles**: Throttle events

Agent-specific metrics (when KB is used via agents):

- Agent invocation counts, latency, TTFT, token counts, error rates

---

13. Cost Considerations

Cost Components

Component	Pricing Model	Notes
Embedding generation	Per-token (on-demand) or per-hour (provisioned throughput)	Charged during ingestion AND re-ingestion on sync
Vector store storage	Varies by store (OpenSearch Serverless OCUs, Aurora ACUs, S3 storage, Pinecone pods, etc.)	OpenSearch Serverless has a minimum of 2 OCUs; S3 Vectors is cheapest
Retrieval (query-time)	Embedding model cost (to embed query) + vector store query cost	Each query embeds the query text
Response generation	Per-token for the generation FM (input + output tokens)	Includes retrieved chunks in the input context
Reranking	Per-query ($2.00 per 1,000 queries for Cohere Rerank 3.5)	Query = up to 100 document chunks

Component	Pricing Model	Notes
Semantic chunking	Per-token (uses an FM for boundary detection)	Additional cost compared to fixed-size chunking
FM-based parsing	Per-token for the parser model	Only if you use a foundation model parser
Data source connectors	No additional charge for the connector itself	But you pay for the compute/storage of the source

Cost Optimization Strategies

- **Use Titan V2 with lower dimensions** (256 or 512) to reduce vector storage costs with minimal accuracy loss
- **Use binary vectors** (with OpenSearch Serverless) for significant storage reduction
- **Use Amazon S3 Vectors** as the vector store for the lowest storage cost
- **Use fixed-size chunking** instead of semantic chunking to avoid the FM cost during ingestion
- **Manage sync frequency** carefully -- every sync re-embeds changed documents
- **Use** `Retrieve` **instead of** `RetrieveAndGenerate` when you want to control the generation model and prompt (potentially use a cheaper model)
- **Use reranking** to retrieve fewer but more relevant results, reducing the tokens sent to the generation model

---

14. Exam Gotchas and Common Traps

Chunking Strategy Traps

1. **"Cannot change chunking strategy after connecting a data source"** -- if you need to change, create a new data source
2. **Semantic chunking costs extra** -- it uses a foundation model for boundary detection during ingestion
3. **No chunking = no page number citations** -- you lose `x-amz-bedrock-kb-document-page-number` metadata
4. **Hierarchical chunking returns fewer results than requested** -- because child chunks are replaced by parents, and multiple children may share a parent
5. **Hierarchical chunking is not recommended with S3 vector bucket** -- metadata size limitations with high token counts

Vector Store Traps

1. **OpenSearch Serverless must use** `faiss` **engine** -- `nmslib` is NOT supported for filtering
2. **Binary vectors only work with OpenSearch** (Serverless or Managed Clusters) -- not Aurora, not Pinecone, not others
3. **Hybrid search requires specific vector stores** -- only OpenSearch Serverless, Aurora PostgreSQL, and MongoDB Atlas (must have filterable text field)
4. `startsWith` **and** `stringContains` **filters are OpenSearch Serverless only** -- other vector stores do not support these

Embedding Model Traps

1. **Cannot change the embedding model after creating a knowledge base** -- must create a new KB
2. **Same embedding model at ingestion and query time** -- mismatched models produce garbage results
3. **Titan V1 is fixed at 1536 dimensions** -- cannot reduce; Titan V2 is configurable (256, 512, 1024)
4. **Cohere Embed has 512 max tokens** -- shorter context window than Titan (8192)

Retrieval and Configuration Traps

1. **Implicit metadata filtering only works with Claude 3.5 Sonnet** -- not other models
2. **Reranking only applies to textual data** -- not images, audio, or video
3. **Maximum 100 results per query** (numberOfResults min 1, max 100)
4. **RetrieveAndGenerate has a session concept** -- sessionId is auto-generated and required for multi-turn context
5. **Filter nesting is limited to one level** -- you can combine up to 5 filter groups, and each group can have up to 5 filters

Data Source Traps

1. **Cannot change parsing strategy after connecting a data source**
2. **Cannot add S3 multimodal storage location after creating a knowledge base** -- must create a new KB
3. **Confluence, SharePoint, Salesforce connectors do not support multimodal** -- only S3 and custom sources do
4. **Web crawler is/was in preview** -- exam may ask about limitations

---

15. When to Use / When NOT to Use

Use Knowledge Bases When

- You need to **ground FM responses in your private data** (the core RAG use case)
- You want a **fully managed** RAG pipeline without building custom ingestion/retrieval
- You need **citations and source attribution** in generated responses
- You have **unstructured documents** (PDFs, docs, web pages) that need to be searchable
- You need **incremental sync** to keep the knowledge base current
- You're building with **Bedrock Agents** and need to give them access to domain knowledge
- You need **structured data querying** via Natural Language to SQL

Do NOT Use Knowledge Bases When

- Your data is **rapidly changing in real-time** and you need sub-second freshness -- use custom pipeline with streaming
- You need **full control over the embedding pipeline** (custom models, custom preprocessing) -- build your own RAG with SageMaker + OpenSearch
- Your data is **already in a well-indexed search engine** (e.g., Kendra) -- use Kendra GenAI Index integration or Kendra directly
- You need **cross-account or cross-region vector store access** -- limited to same-account configurations
- You need to use an **embedding model not supported by Knowledge Bases** -- build custom RAG
- Your documents are **extremely large** and you need fine-grained control over how they're split -- consider custom chunking with Lambda or pre-processing

Knowledge Bases vs. Other RAG Approaches

Approach	When to Choose
Bedrock Knowledge Bases	Fully managed RAG; fastest to deploy; built-in sync, chunking, retrieval, generation, citations
Amazon Kendra	Enterprise search with fine-grained access control; 40+ native connectors; when search accuracy is the primary goal
Kendra GenAI Index + KB	Combine Kendra's retrieval accuracy with Bedrock's generation capabilities
Custom RAG (SageMaker + OpenSearch)	Full control over every pipeline stage; custom embedding models; custom retrieval logic
Long-context window models	Small datasets that fit in context; no need for persistent vector storage
Fine-tuning	When the knowledge needs to be baked into model weights rather than retrieved at query time

---

16. Quick Reference Card

API Clients (SDK v3)

Client	Package	Purpose
`BedrockAgentClient`	`@aws-sdk/client-bedrock-agent`	Build-time: create/update/delete KB, data sources, start sync
`BedrockAgentRuntimeClient`	`@aws-sdk/client-bedrock-agent-runtime`	Runtime: Retrieve, RetrieveAndGenerate, RetrieveAndGenerateStream

Critical Numbers

Limit	Value
Max results per query	100
Default results per query	5
Max filter groups	5
Max filters per group	5
Max filter nesting depth	1 level
Default chunk size (default chunking)	~300 tokens
Titan V2 dimension options	256, 512, 1024
Titan V1 fixed dimensions	1536
Cohere Embed dimensions	1024
Cohere Embed max input tokens	512
Titan Embed max input tokens	8192
Max reranking results	100

IAM Permissions for Knowledge Base Service Role

The Bedrock KB service role needs:

- `bedrock:InvokeModel` on the embedding model ARN
- `s3:GetObject` and `s3:ListBucket` on the data source bucket
- Vector store access (e.g., `aoss:APIAccessAll` for OpenSearch Serverless)
- `secretsmanager:GetSecretValue` for third-party data source credentials

Bedrock Prompt Management, Prompt Flows, and Agentic AI Study Guide

AIP-C01 Exam Coverage

Topic	Domain	Task	Weight
Prompt Management	Domain 1	Task 1.6	Part of 30%
Prompt Flows	Domain 1 / Domain 2	Task 1.6 / Task 2.5	Part of 30% / 26%
Agentic AI (AgentCore, Strands, Agent Squad)	Domain 2	Task 2.1	Part of 26%
Tool Use / Function Calling	Domain 2	Task 2.1	Part of 26%

---

1. Amazon Bedrock Prompt Management

What It Is

A managed service for creating, saving, versioning, testing, and reusing prompt templates across different workflows. It eliminates ad-hoc prompt engineering by centralizing prompt governance.

Core Concepts

Term	Definition
Prompt	An input provided to a model to guide response generation
Variable	A placeholder (e.g., `{{genre}}`) filled at runtime or during testing
Prompt Variant	An alternative configuration of the same prompt -- different model, different inference params, or different wording
Prompt Builder	Visual console tool for creating, editing, and testing prompts
Draft Version	The mutable working copy, created when you first save a prompt
Version	An immutable snapshot of a prompt at a point in time, used for deployment

Parameterized Prompt Templates

Variables use double-brace syntax in prompt text. At invocation time, variable values are supplied either:

- Directly in the `Converse` API via the `promptVariables` field
- Through a Prompt node's inputs in a Bedrock Flow
- During testing in the Prompt Builder

```
Create a {{genre}} playlist with {{count}} songs.
```

Each prompt can have **up to 3 variants** for side-by-side comparison. Variants can differ in:

- The prompt text itself

- The model selected (e.g., Claude Sonnet vs. Amazon Titan)
- Inference parameters (temperature, topP, maxTokens, stopSequences)
- System prompt content
- Tool configuration (for function-calling models)

Versioning and Lifecycle

```
Draft (mutable) --&gt; Version 1 (immutable snapshot) --&gt; Version 2 --&gt; ...
```

Workflow:

1. Create a prompt (creates a DRAFT)
2. Iterate on the draft -- modify text, model, parameters
3. When satisfied, **create a version** (immutable snapshot)
4. Deploy the version by referencing its ARN in your application
5. To update, modify the DRAFT and create a new version

Version Comparison: The console lets you diff two versions side-by-side, showing JSON differences with `+` (added) and `-` (removed) markers. You can also run both versions against test inputs to compare outputs.

Integration with Other Services

Integration Point	How It Works
Converse API	Pass the prompt version ARN as `modelId` plus `promptVariables`
InvokeModel API	Reference prompt version ARN directly
Bedrock Flows	Add a **Prompt node** referencing the prompt ARN
Bedrock Agents	Use prompts in agent orchestration templates

API Example (CreatePrompt):

```
import boto3

client = boto3.client("bedrock-agent")

response = client.create_prompt(
    name="playlist-generator",
    description="Generates playlists by genre",
    defaultVariant="variant-one",
    variants=[
        {
            "name": "variant-one",
            "templateType": "TEXT",
            "templateConfiguration": {
                "text": {
                    "text": "Create a {{genre}} playlist with {{count}} songs.",
                    "inputVariables": [
                        {"name": "genre"},
                        {"name": "count"}
                    ]
                }
            },
            "modelId": "anthropic.claude-sonnet-4-20250514-v1:0",
            "inferenceConfiguration": {
                "text": {
```

```
                    "temperature": 0.7,
                    "maxTokens": 500
                }
            }
        }
    ]
)

prompt_id = response["id"]

# Create immutable version
version_response = client.create_prompt_version(promptIdentifier=prompt_id)
```

Governance and Auditing

- All Prompt Management API calls are logged in **AWS CloudTrail** (CreatePrompt, CreatePromptVersion, DeletePrompt, etc.)
- Prompts can be shared across teams via ARNs and IAM policies
- Version immutability provides an audit trail of what was deployed when
- **Prompt Optimization** feature can automatically rewrite prompts for better accuracy

Exam Tips for Prompt Management

- A **Draft** is the only mutable version; all numbered versions are **immutable**
- You invoke a prompt version via Converse API by passing its **ARN** as the `modelId`
- Up to **3 variants** can be compared side-by-side
- Variables use `{{variableName}}` syntax
- Structured prompts can include system instructions, tools configuration, and conversational history (for models supporting Converse API)

---

2. Amazon Bedrock Prompt Flows

What It Is

A visual workflow builder for creating end-to-end generative AI pipelines by connecting nodes (prompts, models, Lambda functions, knowledge bases, agents, S3, conditions, loops). Flows link foundation models, AWS services, and custom logic without writing orchestration code.

Core Architecture

```
Create Flow --&gt; Design nodes + connections --&gt; Prepare (working draft)
  --&gt; Test --&gt; Create Version (immutable) --&gt; Create Alias --&gt; Deploy
```

Key Concepts:

- **Flow**: A series of interconnected nodes; initiated via `InvokeFlow`
- **Node**: A step in the flow with inputs, outputs, and configuration
- **Connection**: Links output of one node to input of another (data connection or conditional connection)
- **Expression**: Extracts relevant parts from a node's whole input (e.g., `$.data.name`)
- **Working Draft**: Mutable version for iterative testing

- **Version**: Immutable snapshot of a flow
- **Alias**: A pointer to a version, used in production `InvokeFlow` calls

Node Types

Logic Nodes (Control Flow)

Node Type	Purpose	Key Details
Flow Input	Entry point; one per flow	Takes `content` from `InvokeFlow` request
Flow Output	Returns results; multiple allowed	One per branch/path
Condition	Routes data based on conditions	Up to 5 conditions; evaluated in order; includes default path
Iterator	Processes array items one at a time	Splits array into individual items
Collector	Gathers iterated results back into array	Pairs with Iterator
DoWhile Loop	Repeats nodes until condition met	`maxIterations` default=10; no nested loops; up to 5 exit conditions

Data Processing Nodes

Node Type	Purpose	Key Details
Prompt	Invokes an FM with a prompt	Can reference Prompt Management or define inline; supports guardrails
Agent	Calls a Bedrock Agent	Supports multi-turn within a flow; inputs: prompt, session/prompt attributes
Knowledge Base	Queries a knowledge base	With `modelId`: returns generated text; without: returns raw retrieval results
Lambda Function	Runs custom code	Input event has `messageVersion`, `flow`, `node`, `input` fields
Inline Code	Python code within the flow	No external dependencies; last expression is the return value; 128KB memory
S3 Storage	Stores data to S3	Input: content + objectKey; Output: s3Uri
S3 Retrieval	Reads data from S3	Input: objectKey; Output: file content (must be UTF-8)
Lex	Calls a Lex bot	Single-turn only; returns `predictedIntent`

Condition Expressions

Condition nodes use relational and logical operators:

Relational: `==`, `!=`, `>`, `>=`, `<`, `<=` **Logical:** `and`, `or`, `not`

```
(retailPrice &gt; 10) and (type == "produce")
```

Conditions are evaluated **in order** -- if multiple match, the first one wins. A **default** path handles unmatched cases.

Sequential Chains and Conditional Branching

Sequential chain example:

```
Input --&gt; Prompt (classify) --&gt; Condition (check category)
                                    |-- "technical" --&gt; KB query --&gt; Prompt (answer) --&gt; Output
                                    |-- "billing"   --&gt; Lambda (lookup account) --&gt; Output
                                    |-- default     --&gt; Prompt (general response) --&gt; Output
```

DoWhile loop example (iterative refinement):

```
Input --&gt; [Loop Start] --&gt; Prompt (generate) --&gt; Inline Code (evaluate quality)
              ^                                          |
              |_________ quality &lt; threshold ____________|
                                                         |
                                        quality &gt;= threshold --&gt; Output
```

Pre/Post-Processing Steps

- **Pre-processing**: Use Lambda or Inline Code nodes before Prompt nodes to clean, transform, or validate input
- **Post-processing**: Use Lambda or Inline Code nodes after Prompt nodes to parse, format, or validate output
- **Guardrails**: Can be attached to Prompt nodes and Knowledge Base nodes (RetrieveAndGenerate only)

Error Handling

- Flow execution stops at the first node failure
- Lambda functions must return responses in the expected format (with `messageVersion`)
- Inline code nodes execute in a sandboxed Python environment with no external packages
- DoWhile loops have mandatory `maxIterations` to prevent infinite loops
- Use condition nodes to handle error paths explicitly

Deployment

1. **Prepare** the flow (applies latest changes to working draft)
2. **Test** with sample inputs in the console or via API
3. **Create a version** (immutable snapshot)
4. **Create an alias** pointing to the version
5. Invoke via `InvokeFlow` against the alias

```
response = bedrock_agent_runtime.invoke_flow(
    flowIdentifier="FLOW_ID",
    flowAliasIdentifier="ALIAS_ID",
    inputs=[{
        "content": {"document": {"genre": "jazz", "count": 10}},
        "nodeName": "FlowInput",
        "nodeOutputName": "document"
    }]
)
```

Async Flow Executions

Flows can run asynchronously for long-running tasks, returning a flow execution ID that you can poll for status.

Exam Tips for Flows

- Every flow has exactly **one Input node** but can have **multiple Output nodes**

- Condition nodes evaluate conditions **in order** (first match wins)
- DoWhile loops: **no nested loops**, max 5 exit conditions, `maxIterations` prevents runaway
- Iterator/Collector pairs handle array processing
- Lambda nodes receive a specific event structure with `messageVersion` field
- Inline Code nodes: Python only, no external packages, last expression = return value
- Guardrails can only be applied to Prompt nodes and KB nodes (with RetrieveAndGenerate)
- **Flows vs. Agents**: Flows are deterministic pipelines; Agents are autonomous with reasoning loops

---

3. Amazon Bedrock AgentCore

What It Is

An agentic platform for building, deploying, and operating AI agents securely at scale. It is **framework-agnostic** (works with Strands, LangGraph, CrewAI, LlamaIndex, Google ADK, OpenAI Agents SDK) and **model-agnostic** (any FM in or outside Bedrock).

Core Services

Service	Purpose
Runtime	Serverless hosting for agents/tools; isolated microVMs per session; supports real-time (<10min) and async (up to 8 hours); MCP and A2A protocol support
Memory	Short-term (within session) and long-term (across sessions) memory; strategies include SessionSummarizer, PreferenceLearner, FactExtractor
Gateway	Converts APIs, Lambda functions, and services into MCP-compatible tools; tool discovery; integrates with API Gateway, Salesforce, Jira, Slack
Identity	Agent identity and access management; works with Cognito, Okta, Azure Entra ID, Auth0; OAuth 2.0 support
Code Interpreter	Sandboxed code execution (Python, JavaScript, TypeScript)
Browser	Serverless browser runtime for agents to interact with web apps (Playwright, BrowserUse)
Observability	Tracing, debugging, monitoring; OpenTelemetry (OTEL)-compatible; integrates with CloudWatch
Evaluations	Automated agent quality assessment on sessions, traces, and spans
Policy	Deterministic control over agent boundaries; Cedar policy language; integrates with Gateway to intercept tool calls

Runtime Architecture

```
User Request --&gt; AgentCore Runtime --&gt; Dedicated microVM (session isolation)
                                          |
                                          +--&gt; Agent code (any framework)
                                          +--&gt; Tools (MCP/A2A)
                                          +--&gt; Memory (short + long term)
                                          +--&gt; Identity (auth)
```

Key properties:

- Each session runs in a **dedicated microVM** with isolated compute, memory, and network

- Supports **bidirectional streaming** for real-time interactions
- Can process payloads up to **100MB**
- **Consumption-based pricing** -- pay only for what you use
- Supports deploying MCP servers, A2A servers, and AGUI servers

Memory Service

Short-term memory:

- Captures turn-by-turn interactions within a single session
- Maintains immediate context so users don't repeat information

Long-term memory:

- Automatically extracts key insights across sessions
- Built-in strategies:
  - **SessionSummarizer**: Summarizes conversation sessions
  - **PreferenceLearner**: Learns user preferences
  - **FactExtractor**: Extracts and retains facts
- Supports **message batching** to reduce API calls in high-throughput scenarios

```
from bedrock_agentcore.memory import MemoryClient
from strands import Agent

memory_client = MemoryClient()
session_manager = memory_client.create_session_manager(
    memory_id="my-memory-store",
    session_id="user-123-session"
)

agent = Agent(
    model="bedrock/anthropic.claude-sonnet-4-20250514-v1:0",
    session_manager=session_manager
)
```

MCP Server Hosting

AgentCore Runtime can host MCP servers in two modes:

Mode	Transport	Use Case
Stateless	Streamable HTTP	Simple request/response tools; scales easily
Stateful	Streamable HTTP with Mcp-Session-Id	Multi-turn conversations, elicitation, sampling, progress notifications

MCP servers must:

- Use `streamable-http` transport
- Listen on host `0.0.0.0`, port `8000`
- Use ARM64 containers
- Expose `/mcp` endpoint for JSON-RPC messages

AgentCore Gateway

Converts existing APIs into MCP-compatible tools without rewriting code:

Input Type	How It Works
OpenAPI spec	Parses spec, creates MCP tools for each endpoint
Lambda functions	Wraps functions as MCP tools
API Gateway	Directly connects REST APIs as a Gateway target
Existing MCP servers	Proxies through for governance and discovery

Supports **natural language tool discovery** -- agents can search for tools using queries rather than knowing exact tool names.

Exam Tips for AgentCore

- AgentCore is **framework-agnostic** and **model-agnostic** -- this is a key differentiator
- Runtime uses **microVMs** for session isolation (not containers, not Lambda)
- Memory has two types: **short-term** (within session) and **long-term** (across sessions)
- Gateway converts APIs to **MCP-compatible tools**
- Policy uses **Cedar** language for fine-grained control
- Supports both **MCP** (Model Context Protocol) and **A2A** (Agent-to-Agent) protocols

---

4. Strands Agents SDK

What It Is

An open-source, model-driven Python SDK for building AI agents. Developed initially for Amazon Q Developer. Uses a "model-driven" approach where the LLM handles reasoning and planning, reducing the need for explicit orchestration code.

Core Architecture

An agent in Strands has three components:

```
from strands import Agent
from strands.tools import tool

@tool
def get_weather(city: str) -&gt; str:
    """Get current weather for a city."""
    return f"Weather in {city}: 72F, sunny"

agent = Agent(
    model="bedrock/anthropic.claude-sonnet-4-20250514-v1:0",  # Model
    tools=[get_weather],                                        # Tools
    system_prompt="You are a helpful weather assistant."         # Prompt
)

result = agent("What's the weather in Seattle?")
```

Component	Role
Model	LLM that handles reasoning, planning, tool selection
Tools	Functions the agent can call; defined via `@tool` decorator or MCP servers

Component	Role
Prompt	System instructions that guide agent behavior

The Agent Loop

```
User input --&gt; LLM (with tools description)
                |
                +--&gt; Natural language response (done)
                +--&gt; Tool call request
                        |
                        v
                     Execute tool --&gt; Return result to LLM --&gt; (repeat loop)
```

The LLM autonomously decides:

- Which tools to use
- What order to use them
- When to ask for clarification
- When to stop

Custom Tools

```
from strands.tools import tool

@tool
def search_database(query: str, limit: int = 10) -&gt; dict:
    """Search the product database.

    Args:
        query: Search query string
        limit: Maximum results to return
    """
    # Implementation here
    return {"results": [...]}
```

Tools can also be loaded from MCP servers:

```
from strands.tools.mcp import MCPClient

mcp_client = MCPClient(server_url="http://my-mcp-server:8000/mcp")
agent = Agent(
    model="bedrock/anthropic.claude-sonnet-4-20250514-v1:0",
    tools=[mcp_client]
)
```

Multi-Agent Orchestration (Strands 1.0)

Strands 1.0 provides four multi-agent primitives:

1. Agents-as-Tools

Agents wrapped as callable tools for hierarchical delegation:

```
research_agent = Agent(
    system_prompt="You research topics thoroughly.",
```

```
    tools=[web_search]
)

writer_agent = Agent(
    system_prompt="You write engaging content.",
    tools=[research_agent.as_tool()]  # Research agent is a tool
)
```

2. Handoffs

Transfer control between agents, preserving conversation context:

```
from strands.multiagent import handoff

support_agent = Agent(system_prompt="Handle general support")
billing_agent = Agent(system_prompt="Handle billing questions")

# support_agent can hand off to billing_agent when needed
```

3. Swarms

Self-organizing teams of agents collaborating on complex tasks:

```
from strands.multiagent import Swarm

swarm = Swarm(
    agents=[analyst_agent, researcher_agent, writer_agent],
    task="Analyze market trends and produce a report"
)
```

4. Graphs

Explicit workflows with conditional routing:

```
from strands.multiagent import GraphBuilder

graph = GraphBuilder()
graph.add_node("research", research_agent)
graph.add_node("analyze", analysis_agent)
graph.add_node("write", writer_agent)
graph.add_edge("research", "analyze")
graph.add_edge("analyze", "write")
```

Memory and State

- **Durable session management**: Automatic persistence and restoration of conversations
- **Native async support**: True concurrent execution and real-time streaming
- Integration with **AgentCore Memory** for managed short/long-term memory

Observability

- Built-in **OpenTelemetry** instrumentation
- Traces every tool call, LLM invocation, and agent decision
- Integrates with AWS X-Ray, CloudWatch, and third-party tools (Langfuse)
- Metrics: latency, token usage, tool call counts, error rates

Deployment Options

Method	Service	Use Case
Serverless	AWS Lambda	Low-latency, event-driven
Containers	ECS/Fargate/EKS	Long-running, complex agents
Managed	AgentCore Runtime	Production-grade with session isolation
Local	Direct execution	Development and testing

Exam Tips for Strands

- **Model-driven** approach: the LLM decides what to do, not explicit code
- Three components: **Model + Tools + Prompt**
- `@tool` decorator is how you define custom tools
- Four multi-agent primitives: **Agents-as-Tools, Handoffs, Swarms, Graphs**
- Supports **MCP** servers as tool sources
- Open-source (Apache 2.0), not an AWS-only service
- Built-in **OpenTelemetry** for observability

---

5. AWS Agent Squad (Multi-Agent Orchestrator)

What It Is

An open-source framework (formerly "Multi-Agent Orchestrator") for building multi-agent systems where specialized agents collaborate to solve complex tasks. Available in Python and TypeScript.

Architecture

```
User Request --&gt; Classifier (intent routing)
                    |
                    +--&gt; Agent A (billing specialist)
                    +--&gt; Agent B (technical support)
                    +--&gt; Agent C (general knowledge)
                    |
                Response aggregation --&gt; User
```

Core Concepts

Concept	Description
Classifier	Routes requests to the most appropriate agent based on intent
Agent	A specialized unit that handles a specific domain
Orchestrator	Coordinates agents, manages context, handles routing
Storage	Manages conversation history and context across agents

Key Differentiators from Bedrock Multi-Agent Collaboration

Feature	Agent Squad	Bedrock Multi-Agent Collaboration
Deployment	Self-managed (open source)	Fully managed by Bedrock
Flexibility	Complete control over behavior	Managed with configuration
Agents	Independent microservices	Bedrock Agent sub-agents
Routing	Custom classifier logic	Supervisor agent with routing
Integration	Any LLM provider	Amazon Bedrock models

Bedrock Native Multi-Agent Collaboration

Amazon Bedrock also has a **built-in multi-agent collaboration** feature:

Mode	Behavior
Supervisor	Supervisor agent breaks down tasks and delegates to sub-agents
Supervisor with Routing	Simple queries routed directly; complex queries delegated

Features:

- **Inline agent support**: Create temporary agents on-the-fly
- **CloudFormation/CDK support**: Infrastructure as code
- **Enhanced tracing**: Full visibility into delegation and execution
- **Payload referencing**: Pass large payloads between agents
- Limits configurable for collaborator count and step count

Exam Tips for Agent Squad / Multi-Agent

- **Agent Squad** = open-source, self-managed, any provider
- **Bedrock Multi-Agent Collaboration** = managed, supervisor pattern, Bedrock-native
- Supervisor mode: supervisor breaks down tasks
- Supervisor with routing: simple queries bypass supervisor
- Know when to use which: open-source flexibility vs. managed simplicity

---

6. Model Context Protocol (MCP)

What It Is

An open standard protocol enabling seamless communication between LLMs and external data sources, tools, and services. Think of it as a "USB-C for AI" -- a universal connector.

Architecture

```
AI Application (MCP Client)  &lt;--MCP Protocol--&gt;  MCP Server (tools/data)
         |                                                  |
    Strands Agent                                   Lambda Function
    LangGraph Agent                                 ECS Container
    Bedrock Agent                                   API Gateway
                                                    Third-party service
```

Three Primitives

Primitive	What It Provides	Example
Tools	Functions models can call	`search_database`, `send_email`
Resources	Data to include in context	Files, database records, API responses
Prompts	Templates for model interaction	Structured queries, report formats

MCP Solves the M x N Problem

Without MCP: M agents x N tools = M*N custom integrations With MCP: M agents + N tools = M+N standardized connections

Transport Types

Transport	Mode	Use Case
stdio	Local	Development, local tools
SSE (Server-Sent Events)	Local/Remote	Streaming responses
Streamable HTTP	Remote	Production deployment (required by AgentCore)

MCP on AWS

Lambda-Based MCP Servers (Stateless)

- Simple request/response tools
- Scales automatically
- No session state between calls
- Good for: database queries, API calls, data transformations

ECS-Based MCP Servers (Stateful/Complex)

- Long-running tools, complex workflows
- Maintains state across calls
- Good for: multi-step processes, browser automation, file operations

AgentCore Gateway

- **Converts existing APIs to MCP** without code changes
- Supports OpenAPI specs, Lambda functions, and API Gateway as input
- Built-in authentication (IAM, OAuth, API keys)
- Natural language tool discovery
- Policy integration for governance

MCP Protocol Details (AgentCore)

Operation	Purpose
`tools/list`	Discover available tools
`tools/call`	Invoke a specific tool with arguments

Stateful MCP (supported by AgentCore Runtime):

- Uses `Mcp-Session-Id` header for session tracking
- **Elicitation**: Server can request user input mid-operation
- **Sampling**: Server can request LLM-generated content
- **Progress notifications**: Real-time updates during long operations

Exam Tips for MCP

- MCP is an **open standard**, not AWS-proprietary
- Three primitives: **Tools, Resources, Prompts**
- AgentCore requires **streamable-http** transport (not stdio)
- Gateway converts **existing APIs to MCP** without rewriting
- MCP versions supported by AgentCore: `2025-06-18` and `2025-03-26`
- Know the difference between stateless (Lambda) and stateful (ECS/AgentCore) MCP servers

---

7. Tool Use / Function Calling

How It Works (Converse API)

The Bedrock Converse API provides a unified interface for tool use across all supported models:

```
1. User sends message + tool definitions to model
2. Model decides if it needs a tool (stopReason = "tool_use")
3. Model returns tool name + parameters (toolUse block)
4. Developer executes the tool
5. Developer sends tool result back (toolResult block)
6. Model generates final response incorporating tool results
```

Tool Definition (ToolSpec)

```
tool_config = {
    "tools": [
        {
            "toolSpec": {
                "name": "get_weather",
                "description": "Get current weather for a location",
                "inputSchema": {
                    "json": {
                        "type": "object",
                        "properties": {
                            "location": {
                                "type": "string",
                                "description": "City name"
                            },
                            "units": {
                                "type": "string",
                                "enum": ["celsius", "fahrenheit"],
                                "description": "Temperature units"
                            }
                        },
                        "required": ["location"]
                    }
```

```
                }
            }
        }
    ]
}
```

Client-Side vs. Server-Side Tool Calling

Mode	Who Executes the Tool	API
Client-side	Your application code	Converse, InvokeModel, Responses API, Chat Completions API
Server-side	Amazon Bedrock	Converse API with AgentCore Gateway integration

The Tool Use Loop (Converse API)

```
import boto3, json

bedrock = boto3.client("bedrock-runtime")

messages = [{"role": "user", "content": [{"text": "What's the weather in Seattle?"}]}]

# Step 1: Send message with tool config
response = bedrock.converse(
    modelId="anthropic.claude-sonnet-4-20250514-v1:0",
    messages=messages,
    toolConfig=tool_config
)

# Step 2: Check if tool use is requested
if response["stopReason"] == "tool_use":
    tool_use_block = response["output"]["message"]["content"]
    messages.append(response["output"]["message"])

    for block in tool_use_block:
        if "toolUse" in block:
            tool = block["toolUse"]
            # Step 3: Execute the tool
            result = get_weather(tool["input"]["location"])

            # Step 4: Send tool result back
            messages.append({
                "role": "user",
                "content": [{
                    "toolResult": {
                        "toolUseId": tool["toolUseId"],
                        "content": [{"json": result}],
                        "status": "success"  # or "error"
                    }
                }]
            })

    # Step 5: Get final response
    final_response = bedrock.converse(
        modelId="anthropic.claude-sonnet-4-20250514-v1:0",
        messages=messages,
        toolConfig=tool_config
```

```
    )
```

Error Handling for Tool Calls

```
tool_result = {
    "toolUseId": tool["toolUseId"],
    "content": [{"text": "Station WXYZ not found in database"}],
    "status": "error"  # Tells the model the tool failed
}
```

The `status` field (`"success"` or `"error"`) is supported by **Amazon Nova** and **Anthropic Claude 3/4** models. The model can then decide to retry, try a different approach, or report the error to the user.

ToolChoice Configuration

Value	Behavior
`auto`	Model decides whether to use tools (default)
`any`	Model must use at least one tool
`tool`	Model must use a specific named tool

Structured Outputs with Tool Use

Tool use can enforce structured JSON output -- define a tool whose input schema matches your desired output format. The model will always return data conforming to that schema.

Exam Tips for Tool Use

- `stopReason == "tool_use"` means the model wants to call a tool
- The developer executes the tool, not the model (client-side)
- `toolUseId` must be passed back in `toolResult` to link request/response
- `status` field in toolResult: `"success"` or `"error"`
- Tool definitions use **JSON Schema** format
- The Converse API is the **recommended** API for tool use (model-agnostic)
- Models can request **multiple tool calls** in a single response

---

8. Agentic Design Patterns

ReAct (Reasoning + Acting)

The core pattern used by Bedrock Agents. Alternates between reasoning about the situation and taking action:

```
Thought: I need to find the user's order status. I should call the order lookup API.
Action: call get_order_status(order_id="12345")
Observation: Order 12345 is "shipped", tracking: UPS1234
Thought: I have the information. I can now respond to the user.
Answer: Your order 12345 has been shipped. Tracking number: UPS1234
```

In Bedrock Agents, this manifests as:

- **Rationale**: The agent's reasoning (Thought)
- **InvocationInput**: The action to take (Action)
- **Observation**: The result from the action

Chain-of-Thought Reasoning

Technique where the model explicitly articulates its reasoning steps. In Bedrock:

- Foundation models like Claude support CoT natively
- Agent orchestration templates can include CoT instructions
- Enhances transparency and debuggability

Human-in-the-Loop

Several implementation patterns:

Pattern	Service	Use Case
Return of Control	Bedrock Agents	Agent pauses and returns parameters to the application for human approval before executing
Step Functions	AWS Step Functions	Workflow pauses at a review/approval step; human approves via callback
Elicitation	MCP (stateful)	MCP server requests additional input from the user mid-operation
Handoffs	Strands Agents	Agent transfers control to a human when it encounters unknown tasks

Return of Control (Bedrock Agents):

```
# Configure action group with RETURN_CONTROL
action_group = {
    "actionGroupExecutor": {
        "customControl": "RETURN_CONTROL"  # Instead of Lambda ARN
    }
}

# Agent returns invocationInputs in the InvokeAgent response
# Developer inspects, approves, executes, and returns results via
# sessionState.returnControlInvocationResults
```

Circuit Breakers and Stopping Conditions

- **Bedrock Agents**: Built-in orchestration limits (max steps per invocation)
- **Bedrock Flows**: `maxIterations` on DoWhile loops (default 10)
- **Strands Agents**: Configurable maximum tool calls and iterations
- **Step Functions**: Timeouts on each state and the overall execution
- **General pattern**: Set maximum iterations, time limits, and budget constraints

Multi-Model Coordination

Pattern	Description	When to Use
Supervisor	One coordinator agent delegates to specialists	Clear hierarchy, well-defined sub-tasks
Swarm	Peer agents collaborate without a leader	Diverse perspectives, creative tasks
Pipeline	Sequential hand-off between specialized agents	Clear stage-wise processing
Router	Classifier routes to the best agent	Intent-based routing

Bedrock Agent Orchestration Stages

```
Pre-Processing --&gt; Orchestration (loop) --&gt; Post-Processing
     |                    |                      |
  Validate input    Reason + Act            Format output
  Categorize        Call tools              (off by default)
                    Query KBs
                    Generate response
```

Each stage has a customizable **prompt template** and optional **Lambda parser**.

Exam Tips for Agentic Patterns

- **ReAct** = Bedrock Agents' native pattern (Thought/Action/Observation)
- **Return of Control** = how Bedrock Agents implement human-in-the-loop without Lambda
- Pre-processing and post-processing are **optional** agent stages
- The orchestration loop continues until the agent has a final answer or hits a limit
- **Trace** reveals the full reasoning chain: rationale, invocation inputs, observations
- Know the four prompt templates: pre-processing, orchestration, KB response generation, post-processing

---

9. Exam Gotchas

Bedrock Agents vs. Prompt Flows vs. Step Functions

Feature	Bedrock Agents	Prompt Flows	Step Functions
Orchestration	Autonomous (LLM decides)	Deterministic (you define)	Deterministic (you define)
Reasoning	Yes (ReAct loop)	No	No
Tool calling	Yes (action groups)	Via Lambda/Agent nodes	Via Lambda/any service
Conditional logic	LLM decides path	Condition nodes	Choice states
Human approval	Return of Control	N/A (build with Lambda)	Callback pattern
Visual builder	Console UI	Visual flow builder	Workflow Studio
Versioning	Agent aliases	Flow versions + aliases	Version ARNs
Use case	Open-ended tasks requiring reasoning	Repeatable AI pipelines	Any workflow orchestration

Common Traps

1. **"Which service for a deterministic AI pipeline?"** -- Prompt Flows (not Agents)
2. **"Which service for open-ended tasks requiring reasoning?"** -- Bedrock Agents (not Flows)
3. **"Which service for general workflow with human approval?"** -- Step Functions
4. **"How to make an agent pause for approval?"** -- Return of Control (not Step Functions)
5. **"Framework-agnostic agent hosting?"** -- AgentCore Runtime (not Bedrock Agents)
6. **"Convert existing APIs to agent-compatible tools?"** -- AgentCore Gateway (not manual MCP)
7. **"Open-source agent SDK by AWS?"** -- Strands Agents (not Bedrock Agents SDK)
8. **"Multi-agent with full control?"** -- Agent Squad or Strands Graphs
9. **"Multi-agent fully managed?"** -- Bedrock Multi-Agent Collaboration
10. **"Immutable prompt for production?"** -- Create a Version (not use the Draft)

Prompt Management Traps

- Draft is **mutable**; versions are **immutable**
- You **cannot** modify a version after creation
- To invoke a prompt version via Converse API, use the version **ARN** as `modelId`
- Prompt Optimization rewrites prompts automatically -- know this exists

Flow Traps

- Iterator produces **individual items** + **arraySize**; Collector produces **array** + **arraySize**
- Inline code: **Python only**, no external packages, last expression = output
- Lex node: **single-turn only**
- Knowledge base node without modelId: returns **raw retrieval results** (array), not generated text

---

10. When to Use Each: Decision Matrix

```
Need autonomous reasoning?
  YES --&gt; Bedrock Agents or Strands Agents
  NO  --&gt; Is it a repeatable AI pipeline?
            YES --&gt; Bedrock Prompt Flows
            NO  --&gt; Is it a general workflow with non-AI steps?
                      YES --&gt; AWS Step Functions
                      NO  --&gt; Direct API call (Converse API)
```

Detailed Decision Matrix

Scenario	Best Choice	Why
Repeatable prompt chain with conditions	**Prompt Flows**	Deterministic, visual, versioned
Open-ended task requiring multi-step reasoning	**Bedrock Agents**	ReAct loop, tool calling, KB queries
Self-managed agent with any LLM provider	**Strands Agents**	Open-source, model-agnostic
Production agent hosting with isolation	**AgentCore Runtime**	microVMs, session isolation, managed

Scenario	Best Choice	Why
Convert REST APIs to agent tools	**AgentCore Gateway**	No code changes, MCP-compatible
Multi-agent with AWS-managed coordination	**Bedrock Multi-Agent Collaboration**	Supervisor pattern, built-in
Multi-agent with full customization	**Strands Graphs or Agent Squad**	Custom routing and orchestration
Workflow with human approval gates	**Step Functions**	Callback pattern, state machine
Agent that pauses for user confirmation	**Bedrock Agents (Return of Control)**	Native agent capability
Reusable prompt across multiple apps	**Prompt Management**	Versioned, shareable, testable
Agent memory across sessions	**AgentCore Memory**	Managed short + long term
Policy enforcement on agent tool use	**AgentCore Policy**	Cedar language, real-time

---

11. TypeScript SDK v3 Usage

Bedrock Converse API with Tool Use

```
import {
  BedrockRuntimeClient,
  ConverseCommand,
  type Message,
  type ToolConfiguration,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

const toolConfig: ToolConfiguration = {
  tools: [
    {
      toolSpec: {
        name: "get_weather",
        description: "Get current weather for a location",
        inputSchema: {
          json: {
            type: "object",
            properties: {
              location: { type: "string", description: "City name" },
            },
            required: ["location"],
          },
        },
      },
    },
  ],
};

async function chat(userMessage: string): Promise&lt;string&gt; {
```

```
  const messages: Message[] = [
    { role: "user", content: [{ text: userMessage }] },
  ];

  // Initial request
  const response = await client.send(
    new ConverseCommand({
      modelId: "anthropic.claude-sonnet-4-20250514-v1:0",
      messages,
      toolConfig,
    })
  );

  // Handle tool use
  if (response.stopReason === "tool_use") {
    const assistantMessage = response.output!.message!;
    messages.push(assistantMessage);

    for (const block of assistantMessage.content!) {
      if (block.toolUse) {
        const toolInput = block.toolUse.input as { location: string };
        const weatherResult = await getWeather(toolInput.location);

        messages.push({
          role: "user",
          content: [
            {
              toolResult: {
                toolUseId: block.toolUse.toolUseId!,
                content: [{ json: weatherResult }],
                status: "success",
              },
            },
          ],
        });
      }
    }

    // Send tool results back
    const finalResponse = await client.send(
      new ConverseCommand({
        modelId: "anthropic.claude-sonnet-4-20250514-v1:0",
        messages,
        toolConfig,
      })
    );

    return finalResponse.output!.message!.content![0].text!;
  }

  return response.output!.message!.content![0].text!;
}
```

Invoking a Prompt from Prompt Management

```
import {
  BedrockRuntimeClient,
  ConverseCommand,
} from "@aws-sdk/client-bedrock-runtime";
```

```
const client = new BedrockRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new ConverseCommand({
    modelId: "arn:aws:bedrock:us-east-1:123456789012:prompt/PROMPT_ID:1",
    messages: [{ role: "user", content: [{ text: "placeholder" }] }],
    promptVariables: {
      genre: { text: "jazz" },
      count: { text: "10" },
    },
  })
);
```

Invoking a Bedrock Flow

```
import {
  BedrockAgentRuntimeClient,
  InvokeFlowCommand,
} from "@aws-sdk/client-bedrock-agent-runtime";

const client = new BedrockAgentRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new InvokeFlowCommand({
    flowIdentifier: "FLOW_ID",
    flowAliasIdentifier: "ALIAS_ID",
    inputs: [
      {
        content: { document: "What is generative AI?" },
        nodeName: "FlowInputNode",
        nodeOutputName: "document",
      },
    ],
  })
);

// Process streaming response
for await (const event of response.responseStream!) {
  if (event.flowOutputEvent) {
    console.log(event.flowOutputEvent.content?.document);
  }
}
```

Invoking a Bedrock Agent

```
import {
  BedrockAgentRuntimeClient,
  InvokeAgentCommand,
} from "@aws-sdk/client-bedrock-agent-runtime";

const client = new BedrockAgentRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new InvokeAgentCommand({
    agentId: "AGENT_ID",
    agentAliasId: "ALIAS_ID",
    sessionId: "unique-session-id",
```

```
    inputText: "What is the status of order 12345?",
    enableTrace: true,
  })
);

for await (const event of response.completion!) {
  if (event.chunk) {
    const text = new TextDecoder().decode(event.chunk.bytes);
    process.stdout.write(text);
  }
  if (event.trace) {
    console.log("Trace:", JSON.stringify(event.trace, null, 2));
  }
}
```

---

12. Key Metrics and Monitoring

Bedrock Agent Tracing

Enable tracing by setting `enableTrace: true` in `InvokeAgent`. The trace includes:

Trace Type	What It Captures
PreProcessingTrace	Input validation and categorization
OrchestrationTrace	Reasoning (rationale), tool invocations, observations
PostProcessingTrace	Response formatting (if enabled)
FailureTrace	Error details when a step fails
GuardrailTrace	Guardrail assessment results
RoutingClassifierTrace	Multi-agent routing decisions

OrchestrationTrace contains:

- `rationale`: The agent's reasoning text (the "Thought" in ReAct)
- `invocationInput`: Which action group/KB the agent chose
- `observation`: Results from the action/query
- `modelInvocationInput`: Full prompt sent to the FM
- `modelInvocationOutput`: FM's raw response

AgentCore Observability

- **OpenTelemetry (OTEL)** compatible traces, metrics, and logs
- Integrates with **Amazon CloudWatch** for dashboards and alarms
- Supports **AWS X-Ray** for distributed tracing
- Visualizes agent execution paths step-by-step
- Enables debugging of performance bottlenecks

Strands Agents Observability

```
# Built-in OpenTelemetry instrumentation
from strands import Agent
```

```
agent = Agent(
    model="bedrock/anthropic.claude-sonnet-4-20250514-v1:0",
    tools=[my_tool],
    trace_attributes={"environment": "production"}
)
```

Key metrics tracked:

- **Agent loop iterations**: How many reasoning cycles
- **Tool call count and latency**: Per-tool performance
- **Token usage**: Input/output tokens per invocation
- **Error rates**: Tool failures, model errors
- **End-to-end latency**: Total time from request to response

CloudWatch Metrics for Bedrock

Metric	Description
`Invocations`	Number of model invocations
`InvocationLatency`	Time per invocation
`InputTokenCount` / `OutputTokenCount`	Token consumption
`InvocationClientErrors` / `InvocationServerErrors`	Error counts
`InvocationThrottles`	Rate limiting events

CloudTrail Integration

All API calls across Prompt Management, Flows, Agents, and AgentCore are logged in CloudTrail:

- `CreatePrompt`, `CreatePromptVersion`, `DeletePrompt`
- `CreateFlow`, `CreateFlowVersion`, `InvokeFlow`
- `CreateAgent`, `InvokeAgent`, `UpdateAgent`
- AgentCore Runtime, Memory, Gateway operations

Quick Reference Card

Service Endpoints to Remember

Service	Client	Build-Time	Runtime
Prompt Management	`bedrock-agent`	`CreatePrompt`, `CreatePromptVersion`	Invoke via `Converse` with prompt ARN
Flows	`bedrock-agent` / `bedrock-agent-runtime`	`CreateFlow`, `CreateFlowVersion`	`InvokeFlow`
Agents	`bedrock-agent` / `bedrock-agent-runtime`	`CreateAgent`, `PrepareAgent`	`InvokeAgent`
AgentCore	`bedrock-agentcore` / Python SDK	Console / CLI / SDK	Runtime SDK

Key ARN Patterns

```
Prompt:      arn:aws:bedrock:{region}:{account}:prompt/{prompt-id}
Prompt Ver:  arn:aws:bedrock:{region}:{account}:prompt/{prompt-id}:{version}
Flow:        arn:aws:bedrock:{region}:{account}:flow/{flow-id}
Agent:       arn:aws:bedrock:{region}:{account}:agent/{agent-id}
Agent Alias: arn:aws:bedrock:{region}:{account}:agent-alias/{agent-id}/{alias-id}
```

Must-Know Limits

Resource	Limit
Prompt variants per prompt	3
Condition node conditions	Up to 5
DoWhile loop max iterations	Configurable (default 10)
Condition expression max length	64 characters
Flow Input nodes per flow	Exactly 1
Flow Output nodes per flow	Multiple allowed
AgentCore Runtime async workload duration	Up to 8 hours
AgentCore Runtime payload size	Up to 100MB
MCP server port (AgentCore)	8000
MCP server host (AgentCore)	0.0.0.0

Tier 2: High-Frequency Services

Each service group is cited five or more times across the five exam domains. Expect multiple questions requiring solid knowledge of configuration, integration patterns, and when to use each service over its alternatives.

Guides in this tier:

- Amazon SageMaker AI Ecosystem
- AI/ML Services (Comprehend, Kendra, Q, Titan, Transcribe, and more)
- Application Integration (Step Functions, EventBridge, SQS, SNS, AppConfig)
- Compute Services (Lambda, EC2, App Runner, Outposts, Wavelength)

Amazon SageMaker AI Ecosystem -- Comprehensive Study Guide (AIP-C01)

The Amazon SageMaker AI ecosystem spans model training, deployment, monitoring, governance, and data preparation. While Bedrock is the primary "fully managed GenAI" service on the exam, SageMaker appears whenever you need **full control** over models, training infrastructure, custom deployment, or MLOps workflows. This guide covers every in-scope SageMaker sub-service for the AIP-C01 exam.

---

1. SageMaker AI Core -- Inference and Deployment

SageMaker AI provides four inference options for deploying models behind endpoints. Knowing which to pick is a common exam scenario.

Inference Options Comparison

Option	Payload Limit	Processing Time Limit	Scaling to Zero	Persistent Endpoint	Best For
Real-Time Inference	25 MB	60s (8 min streaming)	No	Yes	Low-latency, high-throughput production workloads
Serverless Inference	4 MB	60s	Yes	Yes (cold-startable)	Intermittent/unpredictable traffic with tolerance for cold starts
Batch Transform	100 MB per record	Days (3600s timeout per request)	N/A (job-based)	No	Offline predictions on large datasets
Async Inference	1 GB	1 hour	Yes	Yes	Large payloads, long processing times, near-real-time

Real-Time Inference Key Concepts

- **Endpoint Configuration**: Defines instance type, instance count, and model data location
- **Production Variants**: Multiple variants behind one endpoint for A/B testing and canary deployments
- **Model Artifacts**: Must be compressed as `.tar.gz` and stored in S3
- **ContentType / Accept**: MIME types must match what the model container expects
- **Auto-Scaling**: Based on `InvocationsPerInstance` or custom CloudWatch metrics

Advanced Endpoint Architectures

Architecture	Description	Use Case
Multi-Model Endpoints (MME)	Thousands of models behind one endpoint, sharing a container	Cost optimization when models use the same framework

Architecture	Description	Use Case
Multi-Container Endpoints (MCE)	Multiple containers with different frameworks on one endpoint	Models requiring different frameworks
Serial Inference Pipelines	Chain of containers for pre/post-processing + inference	Preprocessing + model inference + postprocessing
Inference Components	Fine-grained resource allocation per model on shared infrastructure	Hosting multiple FMs with different resource requirements

Deployment Strategies

- **Blue/Green**: Create new endpoint, test, switch traffic. SageMaker supports this natively via endpoint updates.
- **Canary**: Route a small percentage of traffic to a new variant, monitor, then shift all traffic.
- **Shadow Testing**: New model receives a copy of production traffic for comparison but does not serve responses to users. Use CloudWatch to compare `ModelLatency`, error rates, etc.

When to Use / When NOT to Use SageMaker Endpoints

Use when:

- You need to host a custom or open-weight model (Llama, Mistral, etc.) with full infrastructure control
- You need multi-model endpoints for cost optimization
- You need GPU-level tuning, custom containers, or specialized inference frameworks
- You need serial inference pipelines with pre/post-processing

Do NOT use when:

- A model is available on Bedrock and you want serverless, zero-ops inference -- use Bedrock
- You just need a one-off prediction on a batch dataset -- consider Bedrock Batch Inference first

---

2. SageMaker JumpStart -- Foundation Models

SageMaker JumpStart is a model hub within SageMaker that provides access to **hundreds of pre-trained foundation models** for one-click deployment and fine-tuning.

What JumpStart Provides

- **Pre-trained FMs**: Llama 2/3, Mistral, Falcon, Bloom, Stable Diffusion, FLAN-T5, GPT-J, Code Llama, and more
- **One-click deployment**: Deploy FMs to SageMaker endpoints directly from Studio
- **Fine-tuning support**: Domain adaptation and instruction-based fine-tuning from Studio or the Python SDK
- **Solution templates**: End-to-end ML solutions for common use cases
- **Pre-optimized containers**: Inference containers with the right dependencies pre-configured

JumpStart Fine-Tuning Approaches

Approach	Description	Data Format
Domain Adaptation	Adapt a model to a specific domain using unlabeled text	CSV, JSON, or TXT (single file, uses `Text` column)
Instruction-Based	Fine-tune using prompt-completion pairs	JSON Lines (`.jsonl`) with `prompt` and `completion` fields

Key JumpStart Hyperparameters for Fine-Tuning

- `epoch`: Number of training passes
- `learning_rate`: Step size for weight updates
- `per_device_train_batch_size`: Training batch size per device
- `lora_r`: LoRA rank (dimension of the low-rank matrices)
- `lora_alpha`: LoRA scaling factor
- `lora_dropout`: Dropout for LoRA layers
- `int8_quantization`: Enable 8-bit quantization to reduce memory
- `enable_fsdp`: Enable Fully Sharded Data Parallel for distributed training
- `instruction_tuned`: Boolean to toggle instruction-based fine-tuning

JumpStart vs Bedrock for Model Access

Dimension	SageMaker JumpStart	Amazon Bedrock
Infrastructure	You manage endpoint instances	Fully managed, serverless
Model Weights	You have access to model artifacts	No access to weights
Customization	Full fine-tuning, LoRA, PEFT, continued pre-training	Fine-tuning and continued pre-training (limited methods)
Cost Model	Pay for instance hours (even when idle)	Pay per token/API call
Open-Weight Models	Full catalog of open-weight models	Subset available via Bedrock and Bedrock Marketplace
Control	Full control over inference infrastructure	Zero infrastructure management

When to Use JumpStart

- You need an **open-weight FM** that is not on Bedrock
- You need **LoRA or full fine-tuning** with custom hyperparameter control
- You want to deploy a model on **specific GPU instance types** (e.g., `ml.g5.xlarge`, `ml.p4d.24xlarge`)
- You need **model weights in S3** for custom pipelines

When NOT to Use JumpStart

- The model is available on Bedrock and you want zero-ops -- use Bedrock
- You need to build a model from scratch -- use SageMaker Training Jobs

---

3. SageMaker Model Registry

The Model Registry is a **central catalog** for managing model versions, approval workflows, and deployment lineage.

Core Concepts

- **Model Group**: A collection of model versions that solve the same problem (e.g., "fraud-detection-model")
- **Model Package (Version)**: A specific trained model version within a group, containing:
  - Model artifacts S3 location

  - Container image URI
  - Inference specification
  - Metadata and tags
- **Model Card**: Documentation attached to each model version (see Section 12)

Lifecycle Stages and Approval Workflow

Model Registry now supports **custom lifecycle stages** such as:

- `Development` -> `Testing` -> `Production`

Each stage has an **approval status**:

- `PendingApproval`
- `Approved`
- `Rejected`

Only personas with the right IAM permissions can update stage status, creating an audit trail.

Key Features

Feature	Description
Version Tracking	Each registered model gets an incremental version number
Approval Workflows	Status tracking (Pending -> Approved -> Rejected) with IAM-based access control
Model Lineage	Track which training job, dataset, and pipeline created each model
Cross-Account Support	Share model groups across AWS accounts
EventBridge Integration	Lifecycle events trigger EventBridge rules for CI/CD automation
Collections	Group related Model Groups together for organization
Model Cards	Each model version links to its corresponding Model Card

Deployment Pipeline Integration

```
Training Job -&gt; Register in Model Registry -&gt; Approval -&gt; Deploy to Endpoint
                                                  |
                                                  v
                                        EventBridge Rule -&gt; Lambda -&gt; Update Endpoint
```

SageMaker Pipelines can automatically register trained models in the Registry, then EventBridge rules can trigger deployment when approval status changes to `Approved`.

Exam-Critical: Model Registry vs Model Cards

Model Registry	Model Cards
Stores **metadata** about models (artifacts location, container image, version)	Stores **documentation** about models (intended use, risk rating, training details)
Used for **MLOps** (versioning, approval, deployment)	Used for **governance and compliance** (audit, risk assessment)
Each model version in the Registry links to its Model Card	Each Model Card is immutable once associated with a model

4. SageMaker Clarify -- Bias, Explainability, and FM Evaluation

SageMaker Clarify provides bias detection, model explainability, and foundation model evaluation capabilities.

Three Primary Functions

1. Pre-Training Bias Detection

Detect bias in your **training data** before model training begins:

- **Class Imbalance (CI)**: Measures whether one group is underrepresented
- **Difference in Proportions of Labels (DPL)**: Compares label distributions across groups
- **Conditional Demographic Disparity (CDD)**: Checks for bias conditioned on other features

2. Post-Training Bias Detection and Explainability

After training, analyze the **model's predictions**:

- **Feature Attribution**: SHAP (Shapley Additive Explanations) values show which features contribute most to predictions
- **Partial Dependence Plots (PDP)**: Show how a feature affects predictions while marginalizing other features
- **Post-training bias metrics**: Disparate Impact, Difference in Conditional Acceptance, Treatment Equality

3. Foundation Model Evaluation (FMEval)

Evaluate LLMs and foundation models across quality and responsibility dimensions:

Automatic Evaluation Dimensions:

- **Accuracy**: Correctness of generated responses
- **Semantic Robustness**: Consistency under input perturbations
- **Factual Knowledge**: Ability to recall facts
- **Prompt Stereotyping**: Detection of stereotypical associations
- **Toxicity**: Presence of harmful or offensive content

Human Evaluation:

- Compare up to 2 models concurrently
- Custom evaluation criteria (helpfulness, tone, brand voice)
- Use your own workforce or AWS-managed evaluation teams

Supported Evaluation Tasks:

- Open-ended generation
- Text summarization
- Question answering
- Text classification
- Custom tasks

How Clarify Runs

Clarify uses **SageMaker Processing Jobs** under the hood:

1. Takes input data and configuration
2. Optionally obtains model predictions

3. Computes bias metrics and feature attributions
4. Generates analysis results (JSON + visual report)

Integration Points

- **SageMaker Experiments**: Tracks feature attribution scores alongside training metrics
- **SageMaker Model Monitor**: Monitors deployed models for bias drift and feature attribution drift in production
- **SageMaker Pipelines**: Integrate Clarify steps into automated ML pipelines

When to Use Clarify

- **Pre-training**: Assess training data for demographic bias
- **Post-training**: Explain model predictions with SHAP values
- **Production monitoring**: Detect bias drift and feature attribution drift (via Model Monitor)
- **FM evaluation**: Compare foundation models before selecting one for production
- **Compliance**: Generate bias reports for auditors and regulators

When NOT to Use Clarify

- You need real-time inference explanations for every request -- Clarify is batch-oriented (though it supports near-real-time for some use cases)
- You only need Bedrock model evaluation -- use Bedrock's built-in evaluation features instead

---

5. SageMaker Ground Truth -- Data Labeling

Ground Truth is a **managed data labeling** service that combines human workers with machine learning to create high-quality training datasets.

Workforce Options

Workforce	Description	Use Case
Amazon Mechanical Turk	Public crowdsourced workforce	High-volume, general labeling (non-sensitive data)
Private Workforce	Your own employees	Sensitive data, domain expertise
Vendor Companies	Third-party labeling providers via AWS Marketplace	Specialized labeling at scale
AWS-Managed (Ground Truth Plus)	AWS expert workforce, fully managed	Turnkey service with quality management

Built-In Task Types

- Image classification, object detection, semantic segmentation
- Video object detection and tracking, video clip classification
- Text classification, named entity recognition
- 3D point cloud object detection and tracking

Active Learning (Automated Data Labeling)

Ground Truth supports **active learning** to reduce labeling costs:

1. A random sample is sent to human labelers
2. A model is trained on the labeled data
3. The model automatically labels high-confidence items
4. Low-confidence items are routed back to human labelers

Active learning is available for: image classification, semantic segmentation, object detection, and text classification. Requires **thousands of data objects** to be effective.

Ground Truth Plus for GenAI

Ground Truth Plus generates datasets for **foundation model customization**:

- **Demonstration Data**: Human annotators create prompt-response pairs (for supervised fine-tuning / SFT)
- **Preference Data**: Human annotators rank model outputs (for Reinforcement Learning from Human Feedback / RLHF)
- Supports LLMs, text-to-image, and text-to-video models
- Includes red teaming capabilities to discover model vulnerabilities

Quality Assurance

- **Annotation Consolidation**: Multiple workers label the same item; a consolidation algorithm produces a single high-fidelity label
- **Approval Workflows**: Review and change annotations
- **Machine Validation**: Automated checks on annotation quality
- **Consensus**: Algorithms route reviews to multiple individuals

When to Use Ground Truth

- You need labeled data for training custom models
- You need human-generated demonstration data or preference data for SFT/RLHF
- You need active learning to reduce labeling costs on large datasets
- You need red teaming for foundation model safety evaluation

When NOT to Use Ground Truth

- You have pre-labeled data that only needs formatting -- use Data Wrangler
- You need synthetic data generation for training -- consider Bedrock for synthetic data generation

---

6. SageMaker Data Wrangler -- Data Preparation

Data Wrangler is a **visual data preparation** tool for importing, transforming, and analyzing data for ML.

Key Capabilities

Capability	Details
Data Import	Amazon S3, Athena, Redshift, Snowflake, Databricks, 40+ third-party sources
300+ Built-In Transforms	Encoding, normalization, imputation, column manipulation, custom PySpark/SQL

Capability	Details
Data Quality Reports	Detect missing values, outliers, target leakage, and anomalies
Quick Model	Estimate a dataset's predictive power without training a full model
Bias Detection	Integrated with Clarify to check for bias during preparation
Natural Language Interface	Describe transforms in plain English
Export Options	S3, SageMaker Pipelines, Feature Store, Python scripts

Data Wrangler Flow

A **data flow** defines a series of ML data preparation steps:

1. Import data from a source
2. Apply transformations (visual or code-based)
3. Analyze and visualize data
4. Export to a destination

Data flows can be exported as:

- SageMaker Processing jobs (for automated execution)
- SageMaker Pipelines steps (for CI/CD integration)
- Python scripts (for manual execution)
- Feature Store ingestion jobs

Data Quality and Insights Report

The report includes:

- **Summary**: Missing values, invalid values, feature types, outlier counts, severity warnings
- **Target Column**: Distribution analysis
- **Quick Model**: Estimated model performance (R2, accuracy, AUC, etc.)
- **Feature Summary**: Features ranked by predictive power
- **Samples**: Anomalous and duplicate sample detection

Exam Relevance (Task 1.3.1)

The exam specifically calls out Data Wrangler for **data validation**. Know that it:

- Generates data quality reports that flag issues before training
- Integrates directly with SageMaker Pipelines for automated data validation
- Can detect bias in data via Clarify integration

When to Use / When NOT to Use

Use when: Visual/low-code data prep, data quality assessment, exploratory analysis, preparing tabular/image/text data for ML

Do NOT use when: Large-scale ETL jobs (use AWS Glue), real-time data streaming (use Kinesis), simple S3 data format conversion

---

7. SageMaker Processing -- Data Processing Jobs

SageMaker Processing provides **fully managed infrastructure** for running data preprocessing, postprocessing, feature engineering, and model evaluation scripts.

How Processing Jobs Work

```
S3 Input -&gt; Processing Container -&gt; S3 Output
            (/opt/ml/processing/)
```

1. You specify input data (S3, Athena, or Redshift)
2. SageMaker provisions compute (CPU or GPU instances)
3. Your script runs inside a container
4. Output is written to S3

Key Characteristics

- Input data is downloaded to `/opt/ml/processing/input/`
- Output data is uploaded from `/opt/ml/processing/output/`
- Supports **scikit-learn, Spark, and custom containers**
- Monitored via CloudWatch logs and metrics (CPU, GPU, memory, disk)
- Infrastructure is fully managed and terminated after the job completes

Common Use Cases

Use Case	Description
Data Preprocessing	Clean, normalize, feature-engineer raw data before training
Feature Engineering	Create new features from existing data
Model Evaluation	Run evaluation scripts on a held-out test set
Bias Analysis	Clarify uses Processing jobs under the hood
Data Validation	Validate data schemas and distributions
Post-Processing	Transform model outputs (e.g., calibrate probabilities)

Exam Relevance (Task 1.3.2)

The exam specifically calls out SageMaker Processing for **data processing**. Distinguish it from:

- **Data Wrangler**: Visual, low-code data preparation
- **Processing Jobs**: Code-first, script-based data processing on managed infrastructure
- **AWS Glue**: Serverless ETL for data integration and cataloging

---

8. SageMaker Model Monitor -- Production Monitoring

Model Monitor **automatically monitors deployed models** in production and alerts when quality issues arise.

Four Monitoring Dimensions

Monitor Type	What It Detects	Baseline Source
Data Quality	Drift in input data distributions (schema violations, missing values, statistical changes)	Training dataset statistics
Model Quality	Degradation in prediction accuracy (accuracy, precision, recall, RMSE)	Ground truth labels (requires labeling)
Bias Drift	Changes in fairness metrics over time	Pre-deployment Clarify bias analysis
Feature Attribution Drift	Changes in SHAP feature importance rankings	Pre-deployment Clarify SHAP analysis

How Model Monitor Works

1. **Enable Data Capture**: Log inputs and outputs of endpoints/batch transforms to S3
2. **Create Baseline**: Generate statistics and constraints from your training dataset
3. **Schedule Monitoring**: Set up periodic monitoring jobs (hourly, daily, etc.)
4. **Detect Violations**: Monitor compares live data against baseline; violations generate reports
5. **Alert**: CloudWatch alarms trigger notifications or automated retraining

Key Details

- Data Capture happens **asynchronously** and does not impact endpoint performance
- Supports **real-time endpoints** and **batch transform jobs** (not multi-model endpoints)
- Works on **tabular data** natively; for NLP/CV, use custom containers (BYOC)
- Keep disk utilization **below 75%** to ensure data capture continues
- Integrates with **SageMaker Model Dashboard** for a centralized monitoring view

Feature Attribution Drift Deep Dive

Uses **Normalized Discounted Cumulative Gain (NDCG)** to compare feature importance rankings:

- Compares SHAP-based feature rankings from training data vs. live data
- NDCG score below **0.90** triggers an alert
- Available via Clarify integration with Model Monitor

When to Use Model Monitor

- You have a deployed SageMaker endpoint and need to detect data drift
- You need to monitor prediction quality over time
- You need to track bias drift for compliance
- You want automated alerts when model performance degrades

When NOT to Use

- You are using Bedrock endpoints -- Bedrock has its own monitoring via CloudWatch
- You need monitoring for multi-model endpoints (not supported)

---

9. SageMaker Neo -- Model Compilation and Edge Optimization

SageMaker Neo **compiles trained models** for optimized inference on cloud instances or edge devices.

How Neo Works

1. **Compiler**: Converts framework-specific operations into a framework-agnostic intermediate representation (IR), optimizes it, and generates binary code for the target platform
2. **Runtime**: A lightweight runtime for each target platform that loads and serves the compiled model

Supported Frameworks

DarkNet, Keras, MXNet, PyTorch, TensorFlow, TensorFlow-Lite, ONNX, XGBoost

Supported Targets

- **Cloud**: SageMaker hosting instances (including Inferentia/Inf1)
- **Edge**: ARM, Intel, NVIDIA processors on Android, Linux, Windows

Key Benefits

- Up to **25x faster inference** with no loss in accuracy
- Less than **1/10th the runtime footprint** (critical for edge)
- Supports FP32, INT8, and FP16 quantization
- Compiled models can be deployed to SageMaker endpoints or edge devices

When to Use Neo

- You need to **optimize inference latency** on specific hardware
- You are deploying models to **edge devices** (IoT, cameras, robots)
- You want to use **AWS Inferentia** (Inf1/Inf2) instances for cost-effective inference
- You need to **reduce model size** through compilation and quantization

When NOT to Use Neo

- You are using Bedrock (fully managed, no compilation needed)
- Your model framework is not supported by Neo
- You need dynamic model architectures that change at inference time

Exam Note

Neo is lower-weight on the AIP-C01 exam compared to Bedrock and JumpStart, but it may appear in questions about **cost optimization** and **edge deployment**.

---

10. SageMaker Unified Studio

SageMaker Unified Studio is a **single development environment** that brings together AWS data, analytics, AI, and ML services.

What Unified Studio Provides

- **Unified interface** for data exploration, SQL analytics, data processing, ML model development, and GenAI app development
- **Integrates tools from**: Amazon EMR, AWS Glue, Amazon Athena, Amazon Redshift, Amazon MWAA, Amazon Bedrock, and Amazon SageMaker AI
- **Project-based collaboration**: Users work in projects to share data, code, and resources
- **Built-in AI assistance**: Amazon Q Developer for code generation, SQL writing, and troubleshooting
- **Governance**: Amazon SageMaker Catalog for data discovery and access control

Key Capabilities

Capability	Tools
SQL Analytics	Amazon Athena, Amazon Redshift query engines
Data Processing	Apache Spark, Trino via AWS Glue / Amazon EMR
ML Development	SageMaker AI notebooks, training, deployment
GenAI Development	Bedrock models, Knowledge Bases, Agents, Guardrails, Flows
Visualization	Amazon QuickSight integration
Data Governance	SageMaker Catalog, data quality monitoring, lineage tracking

Bedrock in Unified Studio

Selected Amazon Bedrock capabilities are available directly in Unified Studio:

- Foundation model access (including Claude, Nova, Llama, DeepSeek-R1)
- Knowledge Bases, Guardrails, Agents, and Flows
- Bedrock IDE for building GenAI applications

Inference Endpoints in Unified Studio

You can create and manage SageMaker inference endpoints directly in Unified Studio:

- Configure instance type, instance count, and auto-scaling
- Deploy models without managing underlying infrastructure
- Search and monitor endpoints

Exam Relevance

Unified Studio questions will likely focus on:

- Understanding that it is a **single pane of glass** for data + analytics + AI
- It integrates **both Bedrock and SageMaker AI** capabilities
- It uses **project-based access control** and **SSO/IAM authentication**

---

11. Model Customization for GenAI

Model customization is critical for the exam. The key is knowing **which technique to use when**.

Customization Techniques Spectrum

```
No Customization (Prompt Engineering)
        |
        v
Retrieval-Augmented Generation (RAG)
        |
        v
Parameter-Efficient Fine-Tuning (PEFT / LoRA)
        |
        v
Full Fine-Tuning (FFT)
        |
        v
Continued Pre-Training
        |
        v
Training from Scratch
```

Technique Comparison

Technique	What It Does	Data Required	Compute Cost	When to Use
Prompt Engineering	Crafts prompts to guide model behavior	None	Zero (inference-time only)	Start here; often sufficient
RAG	Retrieves context from external knowledge at inference time	Knowledge base documents	Low (retrieval + inference)	Need up-to-date factual knowledge without changing model weights
LoRA / PEFT	Adds small adapter layers; freezes base model weights	100s-1000s of labeled examples	Low-Medium	Domain adaptation, task specialization with limited data/budget
Full Fine-Tuning	Updates all model parameters	1000s-10000s+ of labeled examples	High (full GPU training)	Extensive customization with large datasets
Continued Pre-Training	Extends model knowledge with unlabeled domain text	Large corpus of domain text (unlabeled)	Very High	New domain vocabulary, industry jargon, proprietary knowledge
Training from Scratch	Build a model with random weight initialization	Massive dataset (billions of tokens)	Extremely High	Proprietary architecture, unique requirements

LoRA (Low-Rank Adaptation) Deep Dive

LoRA is the **most exam-relevant PEFT technique**:

- **How it works**: Adds small, trainable low-rank matrices to existing model layers (typically attention layers). Only these adapter weights are updated during training.
- **Key hyperparameters**:
  - `lora_r`: Rank of the decomposition (typically 4-64). Higher rank = more parameters = more capacity but more compute.
  - `lora_alpha`: Scaling factor, often set to 2x `lora_r`
  - `lora_dropout`: Dropout applied to LoRA layers for regularization
- **Benefits**:
  - 70%+ reduction in training compute vs full fine-tuning
  - 50%+ reduction in training cost

  - Base model weights remain frozen (can swap adapters at inference time)
  - Multiple LoRA adapters can be loaded on a single base model
- **SageMaker Support**: SageMaker supports deploying LoRA adapters as **adapter inference components** on endpoints, allowing multiple task-specific adapters on a shared base model

Alignment Techniques

Technique	Description	Data Format
SFT (Supervised Fine-Tuning)	Train on prompt-completion pairs	`{"prompt": "...", "completion": "..."}`
DPO (Direct Preference Optimization)	Learn from preferred vs. rejected response pairs	`{"prompt": "...", "chosen": "...", "rejected": "..."}`
PPO (Proximal Policy Optimization)	Reinforcement learning to optimize for reward	Reward model + prompts
RLHF	Human feedback guides model alignment	Human preference rankings

Continued Pre-Training

- Uses **self-supervised learning** on large quantities of unlabeled proprietary data
- Expands the model's foundational knowledge (domain vocabulary, specialized terms)
- Typically followed by SFT and alignment (DPO or PPO)
- Available on both **Bedrock** (for supported models) and **SageMaker AI**

Knowledge Distillation

- Transfers knowledge from a **larger teacher model** to a **smaller student model**
- Useful when you lack adequate training data but have access to a more powerful model
- Produces a smaller, faster, more cost-efficient model
- Available in SageMaker AI (e.g., Amazon Nova customization)

Where to Do Customization

Platform	Techniques	Trade-offs
Amazon Bedrock	Fine-tuning (SFT), continued pre-training, distillation (limited models)	Quickest time-to-value, fully managed, limited model/technique selection
SageMaker Training Jobs	SFT, PEFT/LoRA, FFT, DPO, PPO, continued pre-training, distillation	Balanced ease-of-use and flexibility, managed infrastructure
SageMaker HyperPod	All techniques including large-scale distributed training	Maximum control and scale, requires cluster management

---

12. Model Cards -- Governance and Compliance

Model Cards are a **documentation framework** for machine learning models, designed for governance, compliance, and responsible AI.

What Model Cards Contain

Section	Description
Model Overview	Model name, type, version, source algorithm, framework
Intended Uses	Scenarios where the model is appropriate and inappropriate, assumptions, general purpose
Risk Rating	Unknown, Low, Medium, or High -- impacts production deployment decisions
Business Details	Business problem, expected users, business impact
Training Details	Training data, hyperparameters, training metrics, objective function
Evaluation Details	Evaluation metrics in JSON format (bar charts, matrices, linear graphs)
Additional Information	Custom properties, free-form documentation

Key Properties

- **Immutable**: Once a Model Card is associated with a model version, it cannot be modified (except approval status). Edits create new versions.
- **Versioned**: Each edit creates a new Model Card version for an immutable audit trail
- **Linked to Model Registry**: Each model version in the Registry has a 1:1 link to its Model Card
- **Exportable**: Model Cards can be exported to PDF for sharing with stakeholders and regulators
- **Auto-populated**: When using the SageMaker Python SDK, some fields are auto-populated from training job metadata

Risk Ratings and Compliance

Model Cards support **risk ratings** that help organizations comply with internal policies and external regulations:

- Models with `High` risk may require additional review before production deployment
- Risk ratings are visible in the Model Dashboard
- Organizations can enforce policies that prevent deploying `High` risk models without explicit approval

Exam-Critical Points

- Model Cards address **Task 3.3.1** (model cards for compliance)
- They are NOT the same as Model Registry -- Cards document the "why" and "how", Registry tracks the "what" and "where"
- No native integration with Model Monitor, but you can manually upload Model Monitor metrics to a Model Card
- Model Cards can be created for models not trained in SageMaker, but no information is auto-populated

---

13. Exam Gotchas -- SageMaker vs Bedrock Decision Points

This is one of the most common exam trap areas. The exam tests whether you know when to use SageMaker vs Bedrock.

Decision Framework

```
Do you need to train or build a model from scratch?
  YES -&gt; SageMaker AI
```

```
    NO  -&gt; Continue

  Is the model available on Bedrock?
    YES -&gt; Do you need full control over infrastructure?
      YES -&gt; SageMaker AI (JumpStart or custom endpoint)
      NO  -&gt; Bedrock
    NO  -&gt; SageMaker AI (JumpStart or custom endpoint)

  Do you need fine-tuning?
    Need LoRA with custom hyperparameter control -&gt; SageMaker AI
    Need simple SFT or continued pre-training -&gt; Bedrock (if model supports it) or SageMaker AI
    Need DPO, PPO, RLHF -&gt; SageMaker AI

  Do you need model evaluation?
    Compare Bedrock-hosted models -&gt; Bedrock Evaluation
    Evaluate with custom metrics/benchmarks -&gt; SageMaker Clarify FMEval
    Need human evaluation -&gt; Either (both support it)

  Do you need production monitoring?
    SageMaker endpoint -&gt; Model Monitor
    Bedrock endpoint -&gt; CloudWatch metrics + Bedrock-specific monitoring

  Do you need data labeling for RLHF?
    -&gt; Ground Truth Plus
```

Common Exam Traps

Scenario	Correct Answer	Why
"Deploy a foundation model with zero infrastructure management"	**Bedrock**	Serverless, no endpoint management
"Fine-tune Llama 3 with LoRA and custom hyperparameters"	**SageMaker AI** (JumpStart)	Bedrock has limited fine-tuning control
"Detect bias in training data before model training"	**SageMaker Clarify**	Pre-training bias detection
"Evaluate multiple LLMs for accuracy and toxicity"	**SageMaker Clarify FMEval** or **Bedrock Evaluation**	Both work, but SageMaker Clarify gives more customization
"Monitor a deployed model for data drift"	**SageMaker Model Monitor**	Designed for production monitoring of SageMaker endpoints
"Label data for RLHF fine-tuning"	**Ground Truth Plus**	Generates preference data for RLHF
"Prepare and clean tabular data with no code"	**Data Wrangler**	Visual data preparation tool
"Run a preprocessing script on managed infrastructure"	**SageMaker Processing**	Script-based processing on managed compute
"Track model versions and approval status"	**Model Registry**	Central model version catalog
"Document model risk rating and intended use"	**Model Cards**	Governance and compliance documentation
"Deploy model to edge devices"	**SageMaker Neo** + Edge Manager	Compile and deploy to edge

Scenario	Correct Answer	Why
"Host 1000+ models behind one endpoint"	**Multi-Model Endpoints**	Cost-effective multi-model serving

---

14. When to Use / When NOT to Use -- Decision Matrix

Sub-Service	Use When	Do NOT Use When
SageMaker Endpoints	Custom models, specific GPU requirements, multi-model hosting	Model available on Bedrock and you want serverless
JumpStart	Open-weight FMs, custom fine-tuning, need model artifacts	Model on Bedrock meets your needs with less ops
Model Registry	Tracking model versions, approval workflows, CI/CD pipelines	One-off model experiments (use MLflow/Experiments)
Clarify	Bias detection, SHAP explainability, FM evaluation	Real-time per-request explanations at scale
Ground Truth	Labeled training data, RLHF preference data, active learning	Data already labeled, synthetic data generation
Data Wrangler	Visual data prep, data quality reports, exploratory analysis	Large-scale ETL (use Glue), real-time streaming
Processing	Script-based preprocessing/postprocessing on managed compute	Visual data preparation (use Data Wrangler)
Model Monitor	Production drift detection, bias monitoring, model quality	Bedrock endpoints (not compatible)
Neo	Edge deployment, inference optimization, specific hardware targets	Bedrock models, unsupported frameworks
Unified Studio	Unified data + analytics + AI experience, team collaboration	Single-purpose tasks where Studio/Console suffices
Model Cards	Governance, compliance documentation, risk assessment	Just tracking model artifacts (use Model Registry)

---

15. Key CloudWatch Metrics

Endpoint Instance Metrics (`/aws/sagemaker/Endpoints` namespace)

Metric	Description	Exam Relevance
`CPUUtilization`	Sum of each CPU core's utilization (0%-400% for 4 CPUs)	Capacity planning
`GPUUtilization`	Percentage of GPU units used (0%-400% for 4 GPUs)	GPU sizing and cost optimization
`GPUMemoryUtilization`	GPU memory used (0%-400% for 4 GPUs)	Detect OOM risks
`MemoryUtilization`	Container memory usage (0%-100%)	Detect memory pressure
`DiskUtilization`	Disk space used (0%-100%)	Keep below 75% for data capture

Metric	Description	Exam Relevance
`CPUUtilizationNormalized`	Normalized CPU usage (0%-100%)	Inference components only
`GPUUtilizationNormalized`	Normalized GPU usage (0%-100%)	Inference components only

Endpoint Invocation Metrics (`AWS/SageMaker` namespace)

Metric	Description	Exam Relevance
`Invocations`	Number of InvokeEndpoint requests	Traffic volume
`InvocationsPerInstance`	Invocations normalized by active instances	**Auto-scaling target metric**
`ModelLatency`	Time the model container takes to respond (microseconds)	Performance monitoring
`OverheadLatency`	Time SageMaker adds for overhead (microseconds)	Distinguish model vs infra latency
`Invocation4XXErrors`	Client error count	Request validation issues
`Invocation5XXErrors`	Server error count	Model or infra failures
`ModelSetupTime`	Time to set up the model (serverless)	Cold start measurement

Training Job Metrics (`/aws/sagemaker/TrainingJobs`)

Metric	Description
`CPUUtilization`	CPU usage during training
`GPUUtilization`	GPU usage during training
`GPUMemoryUtilization`	GPU memory during training
`DiskUtilization`	Disk usage during training

Auto-Scaling Key Metrics

- **Primary scaling metric**: `InvocationsPerInstance` -- scales based on request load per instance
- **Alternative**: `ModelLatency` -- scale when latency exceeds threshold
- For serverless endpoints: `ServerlessConcurrentExecutionsUtilization`
- Async inference endpoints can **scale to zero** when queue is empty

Multi-Model Endpoint Specific Metrics

Metric	Description
`ModelLoadingWaitTime`	Time waiting for model to load
`ModelUnloadingTime`	Time to unload a model
`ModelDownloadingTime`	Time to download model from S3
`ModelLoadingTime`	Time to load model into memory
`ModelCacheHit`	Whether the model was already in memory (1 = cache hit)

16. TypeScript SDK v3 Usage

SageMaker Runtime -- Invoke Endpoint

```
import {
  SageMakerRuntimeClient,
  InvokeEndpointCommand,
} from "@aws-sdk/client-sagemaker-runtime";

const client = new SageMakerRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new InvokeEndpointCommand({
    EndpointName: "my-llm-endpoint",
    ContentType: "application/json",
    Accept: "application/json",
    Body: JSON.stringify({
      inputs: "What is machine learning?",
      parameters: {
        max_new_tokens: 256,
        temperature: 0.7,
        top_p: 0.9,
      },
    }),
  })
);

const result = JSON.parse(new TextDecoder().decode(response.Body));
console.log(result);
```

SageMaker Runtime -- Invoke Endpoint with Streaming

```
import {
  SageMakerRuntimeClient,
  InvokeEndpointWithResponseStreamCommand,
} from "@aws-sdk/client-sagemaker-runtime";

const client = new SageMakerRuntimeClient({ region: "us-east-1" });

const response = await client.send(
  new InvokeEndpointWithResponseStreamCommand({
    EndpointName: "my-llm-endpoint",
    ContentType: "application/json",
    Body: JSON.stringify({
      inputs: "Explain quantum computing",
      parameters: { max_new_tokens: 512 },
      stream: true,
    }),
  })
);

if (response.Body) {
  for await (const event of response.Body) {
    if (event.PayloadPart?.Bytes) {
      const chunk = new TextDecoder().decode(event.PayloadPart.Bytes);
```

```
      process.stdout.write(chunk);
    }
  }
}
```

Model Registry -- List and Describe Model Packages

```
import {
  SageMakerClient,
  ListModelPackagesCommand,
  DescribeModelPackageCommand,
} from "@aws-sdk/client-sagemaker";

const client = new SageMakerClient({ region: "us-east-1" });

// List model versions in a group
const listResponse = await client.send(
  new ListModelPackagesCommand({
    ModelPackageGroupName: "my-model-group",
    ModelApprovalStatus: "Approved",
    SortBy: "CreationTime",
    SortOrder: "Descending",
    MaxResults: 10,
  })
);

for (const pkg of listResponse.ModelPackageSummaryList ?? []) {
  console.log(`Version: ${pkg.ModelPackageVersion}, Status: ${pkg.ModelApprovalStatus}`);

  // Describe a specific model version
  const detail = await client.send(
    new DescribeModelPackageCommand({
      ModelPackageName: pkg.ModelPackageArn,
    })
  );
  console.log(`Container: ${detail.InferenceSpecification?.Containers?.[0]?.Image}`);
}
```

Model Registry -- Update Approval Status

```
import {
  SageMakerClient,
  UpdateModelPackageCommand,
} from "@aws-sdk/client-sagemaker";

const client = new SageMakerClient({ region: "us-east-1" });

await client.send(
  new UpdateModelPackageCommand({
    ModelPackageArn: "arn:aws:sagemaker:us-east-1:123456789012:model-package/my-group/1",
    ModelApprovalStatus: "Approved",
    ApprovalDescription: "Passed evaluation metrics thresholds",
  })
);
```

Create a Processing Job

```
import {
  SageMakerClient,
  CreateProcessingJobCommand,
} from "@aws-sdk/client-sagemaker";

const client = new SageMakerClient({ region: "us-east-1" });

await client.send(
  new CreateProcessingJobCommand({
    ProcessingJobName: "data-preprocessing-job",
    RoleArn: "arn:aws:iam::123456789012:role/SageMakerProcessingRole",
    ProcessingResources: {
      ClusterConfig: {
        InstanceCount: 1,
        InstanceType: "ml.m5.xlarge",
        VolumeSizeInGB: 30,
      },
    },
    AppSpecification: {
      ImageUri: "683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:1.2-1-cpu-py3",
      ContainerEntrypoint: ["python3", "/opt/ml/processing/input/code/preprocess.py"],
    },
    ProcessingInputs: [
      {
        InputName: "input-data",
        S3Input: {
          S3Uri: "s3://my-bucket/raw-data/",
          LocalPath: "/opt/ml/processing/input/data",
          S3DataType: "S3Prefix",
          S3InputMode: "File",
        },
      },
      {
        InputName: "code",
        S3Input: {
          S3Uri: "s3://my-bucket/scripts/preprocess.py",
          LocalPath: "/opt/ml/processing/input/code",
          S3DataType: "S3Prefix",
          S3InputMode: "File",
        },
      },
    ],
    ProcessingOutputConfig: {
      Outputs: [
        {
          OutputName: "processed-data",
          S3Output: {
            S3Uri: "s3://my-bucket/processed-data/",
            LocalPath: "/opt/ml/processing/output",
            S3UploadMode: "EndOfJob",
          },
        },
      ],
    },
  })
);
```

17. Cost Considerations

Instance Types for GenAI Workloads

Instance Family	Accelerator	Best For	Key Characteristic
ml.p5.48xlarge	8x NVIDIA H100	Large-scale FM training	Highest GPU performance
ml.p4d.24xlarge	8x NVIDIA A100	FM training and inference	High memory (40GB/80GB per GPU)
ml.g5.xlarge-48xlarge	NVIDIA A10G	Inference and fine-tuning	Cost-effective GPU inference
ml.g4dn.xlarge-16xlarge	NVIDIA T4	Budget inference	Lower cost, suitable for smaller models
ml.inf2.xlarge-48xlarge	AWS Inferentia2	High-throughput inference	Best cost-per-inference for supported models
ml.inf1.xlarge-24xlarge	AWS Inferentia	Cost-effective inference	Lower cost, first-gen Inferentia
ml.trn1.2xlarge-32xlarge	AWS Trainium	Cost-effective FM training	Up to 50% savings vs GPU for training
ml.m5.xlarge-24xlarge	CPU only	Processing, small model inference	No GPU, general purpose

Cost Optimization Strategies

Strategy	Savings	How It Works
Managed Spot Training	Up to 90%	Use Spot Instances for training; SageMaker handles interruptions. Requires checkpointing for long jobs. Set `EnableManagedSpotTraining=True` and `MaxWaitTimeInSeconds > MaxRuntimeInSeconds`
SageMaker Savings Plans	Up to 64%	Commit to consistent usage ($/hour) for 1 or 3 years
AWS Inferentia/Trainium	30-50%	Purpose-built ML chips vs NVIDIA GPUs
Serverless Inference	Variable	Pay only when invoked; scales to zero. Max 6 GB memory
Async Inference	Variable	Scale to zero when queue is empty; process when capacity is available
Multi-Model Endpoints	Variable	Share one endpoint across thousands of models
Inference Components	Variable	Fine-grained resource allocation on shared infrastructure
Right-sizing	Variable	Use Inference Recommender to find the optimal instance type
Neo Compilation	Variable	Compile models for target hardware to reduce latency and compute

Cost Comparison: SageMaker vs Bedrock

Factor	SageMaker Endpoints	Bedrock
Billing Model	Per-instance-hour (pay while endpoint is running)	Per-token (pay per API call) or Provisioned Throughput
Idle Cost	Yes (unless serverless/async with scale-to-zero)	No (on-demand pricing)
Scaling	Manual or auto-scaling configuration	Automatic
Right for	Sustained high-throughput, custom models	Variable traffic, rapid prototyping, multi-model

Exam Tip: Spot Training Checkpointing

For managed spot training, you **must implement checkpointing** for training jobs longer than 1 hour. Without checkpointing:

- If the Spot Instance is interrupted, training restarts from the beginning
- Built-in algorithms that don't support checkpointing are limited to `MaxWaitTimeInSeconds = 3600`
- Use `CheckpointConfig` to specify an S3 location for saving checkpoints

Quick Reference Card

Need	Use This
Deploy a custom/open-weight FM	SageMaker JumpStart + Endpoints
Serverless FM inference	Amazon Bedrock
Fine-tune with LoRA	SageMaker JumpStart or Training Jobs
Track model versions	Model Registry
Document model risk/compliance	Model Cards
Detect training data bias	SageMaker Clarify (pre-training)
Explain predictions with SHAP	SageMaker Clarify (post-training)
Evaluate LLM quality/toxicity	SageMaker Clarify FMEval
Label data for SFT/RLHF	Ground Truth / Ground Truth Plus
Visual data preparation	Data Wrangler
Script-based data processing	SageMaker Processing
Monitor production data drift	Model Monitor
Compile model for edge	SageMaker Neo
Unified data + AI workspace	SageMaker Unified Studio
Cost-effective training	Spot Training + Trainium
Cost-effective inference	Inferentia + Serverless/Async endpoints

AI/ML Individual Services -- Comprehensive Study Guide (AIP-C01)

This guide covers the AWS AI/ML services that appear on the Certified Generative AI Developer - Professional exam **outside of** Amazon Bedrock and SageMaker (which have their own dedicated guides). These services are the specialized, purpose-built AI tools that integrate with and complement generative AI workflows.

Table of Contents

1. [Amazon Comprehend](#1-amazon-comprehend)
2. [Amazon Kendra](#2-amazon-kendra)
3. [Amazon Q Business and Q Apps](#3-amazon-q-business-and-q-apps)
4. [Amazon Q Developer](#4-amazon-q-developer)
5. [Amazon Titan Foundation Models](#5-amazon-titan-foundation-models)
6. [Amazon Lex](#6-amazon-lex)
7. [Amazon Transcribe](#7-amazon-transcribe)
8. [Amazon Textract](#8-amazon-textract)
9. [Amazon Rekognition](#9-amazon-rekognition)
10. [Amazon Augmented AI (A2I)](#10-amazon-augmented-ai-a2i)
11. [Service Comparison Decision Guide](#11-service-comparison-decision-guide)

---

1. Amazon Comprehend

Service Overview

Amazon Comprehend is a **fully managed NLP service** that uses machine learning to extract insights from text. It requires no ML expertise and works through simple API calls.

Core Capabilities:

- **Entity Recognition** -- identifies people, places, organizations, dates, quantities
- **Sentiment Analysis** -- positive, negative, neutral, mixed (document-level and targeted)
- **PII Detection and Redaction** -- locates or redacts 22 universal + 14 country-specific PII types
- **Key Phrase Extraction** -- identifies important phrases
- **Language Detection** -- identifies dominant language (100+ languages)
- **Custom Classification** -- build custom text classifiers without ML expertise
- **Custom Entity Recognition** -- train models to find domain-specific entities
- **Toxicity Detection** -- detects toxic content in text
- **Topic Modeling** -- discovers topics across document collections (async batch only)
- **Syntax Analysis** -- tokenization and Parts of Speech (PoS) labeling
- **Events Extraction** -- extracts structured event data (who-what-when-where)

GenAI Exam Relevance

Exam Task	How Comprehend Applies
Task 3.2.2, 3.2.3	PII detection and redaction in data pipelines
Task 1.3.4	Entity extraction for pre-processing unstructured data
Task 1.6.2	Intent recognition / text classification
Task 3.1.4	Pre-processing filters before sending data to FMs

Key Features for the Exam

PII Detection (High Priority):

- Two operations: `DetectPiiEntities` (locate with offsets) and `ContainsPiiEntities` (label only)
- Supports **English and Spanish** only for PII
- Real-time (synchronous, single document up to 100KB) and batch (asynchronous, S3-based)
- Can **redact** PII in batch mode -- returns copy with PII replaced
- S3 Object Lambda Access Points can filter PII from S3 documents on-the-fly
- 22 universal PII types: NAME, ADDRESS, EMAIL, PHONE, CREDIT_DEBIT_NUMBER, SSN, etc.
- 14 country-specific types: US SSN, UK NHS number, CA Social Insurance Number, Indian Aadhaar, etc.
- Confidence threshold tuning: higher threshold = better precision, lower = better recall

Custom Models:

- Custom Classification: train with labeled documents, auto-splits training/test
- Custom Entity Recognition: train with annotations or entity lists
- Both use Flywheels for ongoing model management and retraining

Real-time vs Async:

- Real-time endpoints for custom models require provisioning (cost while running)
- Async jobs for batch processing through S3

When to Use / When NOT to Use

Use Comprehend when:

- You need standalone PII detection/redaction as a **pre-processing step** before data reaches an FM
- You need entity extraction, sentiment analysis, or classification on text **outside** of a Bedrock pipeline
- You need PII detection on text that is **not** being sent to a foundation model (e.g., data lake governance)
- You need **batch PII redaction** across large document sets in S3
- You need S3 Object Lambda PII filtering for data access control
- You need custom text classification or custom entity recognition

Do NOT use Comprehend when:

- You need PII filtering **inline** during Bedrock model invocation -- use **Bedrock Guardrails** instead
- You need content moderation / toxicity filtering during FM inference -- use **Bedrock Guardrails**
- You need NLP on images or PDFs directly -- use **Textract** first to extract text, then Comprehend

Comprehend PII vs Bedrock Guardrails PII (Critical Exam Topic)

Feature	Comprehend PII	Bedrock Guardrails PII
When to use	Standalone text processing, data pipelines, S3 data governance	Inline with FM invocation (input/output filtering)

Feature	Comprehend PII	Bedrock Guardrails PII
Integration	Standalone API calls or S3 Object Lambda	Built into Bedrock Invoke/Converse pipeline
Languages	English and Spanish only	Varies by configuration
Action	Locate (offsets) or Redact (batch)	Block or Mask
Custom patterns	No custom PII types (fixed taxonomy)	Custom regex patterns supported
Batch support	Yes (async S3 jobs)	No (real-time only)
Use in Bedrock	As a pre/post-processing step outside Bedrock	Natively integrated into inference calls

Exam tip: If the question says "prevent PII from reaching the foundation model" or "filter PII in prompts and responses," the answer is **Bedrock Guardrails**. If the question says "detect PII in documents before ingestion" or "redact PII across an S3 data lake," the answer is **Comprehend**.

Integration with Bedrock/GenAI

- Pre-process text with Comprehend PII detection **before** sending to Bedrock
- Extract entities from documents to enrich metadata for RAG knowledge bases
- Use sentiment analysis to route customer interactions (positive -> upsell, negative -> escalation)
- Custom classification to categorize documents before routing to specialized FM prompts
- Comprehend results can feed into A2I human review workflows

Exam Gotchas

- Comprehend PII detection supports **only English and Spanish** -- not all languages
- `DetectPiiEntities` returns offsets (where PII is); `ContainsPiiEntities` returns labels only (what types exist)
- PII redaction is **batch only** (async) -- you cannot redact in real-time with a single API call
- Custom models require **real-time endpoints** to be provisioned for synchronous inference (cost implication)
- Topic Modeling is **async batch only** -- no real-time topic detection
- Comprehend does NOT support custom PII entity types; if you need custom patterns, use Bedrock Guardrails regex

TypeScript SDK v3 Usage

```
import {
  ComprehendClient,
  DetectPiiEntitiesCommand,
  DetectSentimentCommand,
  DetectEntitiesCommand,
  ContainsPiiEntitiesCommand,
} from "@aws-sdk/client-comprehend";

const client = new ComprehendClient({ region: "us-east-1" });

// Detect PII entities with offsets
const piiResponse = await client.send(
  new DetectPiiEntitiesCommand({
    Text: "John Smith lives at 123 Main St. SSN: 123-45-6789",
    LanguageCode: "en",
  })
);
// Returns: Entities[] with Type, Score, BeginOffset, EndOffset
```

```
// Check if document contains PII (labels only, no offsets)
const containsPii = await client.send(
  new ContainsPiiEntitiesCommand({
    Text: "Call me at 555-0123",
    LanguageCode: "en",
  })
);
// Returns: Labels[] with Name and Score

// Sentiment analysis
const sentiment = await client.send(
  new DetectSentimentCommand({
    Text: "I love this product!",
    LanguageCode: "en",
  })
);
// Returns: Sentiment (POSITIVE|NEGATIVE|NEUTRAL|MIXED), SentimentScore

// Entity extraction
const entities = await client.send(
  new DetectEntitiesCommand({
    Text: "Amazon was founded by Jeff Bezos in Seattle",
    LanguageCode: "en",
  })
);
// Returns: Entities[] with Type (PERSON, ORGANIZATION, LOCATION, etc.)
```

Key Metrics

- `SuccessfulRequestCount`, `ThrottledCount` -- API health
- Custom model metrics: Precision, Recall, F1 during training
- PII detection confidence scores per entity (tunable threshold)

---

2. Amazon Kendra

Service Overview

Amazon Kendra is a **fully managed intelligent search service** that uses NLP and deep learning for semantic, contextual search. It goes beyond keyword matching to understand the meaning and context of queries.

Core Capabilities:

- Semantic search with natural language understanding
- 43+ pre-built data source connectors (S3, SharePoint, Confluence, Salesforce, etc.)
- Retrieval API optimized for RAG use cases
- Access control list (ACL) filtering for user-level document permissions
- Custom document enrichment during ingestion
- Relevance tuning and boosting
- Factoid, descriptive, and keyword query support

GenAI Exam Relevance

Exam Task	How Kendra Applies
Task 2.5.3	Internal knowledge tools, enterprise search
RAG Retriever	Serves as the retrieval layer for RAG architectures

Key Features for the Exam

Index Types (Critical):

Feature	GenAI Enterprise Edition	Enterprise Edition	Developer Edition
Purpose	Highest accuracy for RAG / Retrieve API	Production semantic search	Testing only
Accuracy	Highest (advanced semantic models)	High	Same as Enterprise
Retrieve API	Full support, optimized	Full support	Full support
Query API	Full support	Full support	Full support
Production use	Yes	Yes	No
Regions	US East, US West, Ireland, Sydney	All Kendra regions	All Kendra regions

Retrieve API vs Query API:

- **Retrieve API**: Returns up to 100 passages of up to 200 tokens each. Optimized for RAG. No query suggestions, spell check, or faceting. Use this for feeding passages to an LLM.
- **Query API**: Returns excerpts up to 100 tokens, supports advanced query syntax, spell correction, faceting, query suggestions, incremental learning. Use this for traditional search UX.

Data Source Connectors:

- 43+ connectors: S3, SharePoint, Confluence, Salesforce, ServiceNow, Jira, Google Drive, Slack, etc.
- v2.0 connectors required for GenAI Enterprise Edition
- Automatic crawling, syncing, and incremental updates

Custom Document Enrichment:

- Pre-extraction: modify documents/metadata before indexing
- Post-extraction: enrich with Lambda functions after text extraction
- Inline: basic attribute manipulation during ingestion

When to Use / When NOT to Use

Use Kendra when:

- You need **enterprise search** with ACL-based access control
- You need a **retriever for RAG** with 43+ data source connectors out of the box
- You need **user-level document permissions** (RBAC with ACLs from source systems)
- You already have data in enterprise systems (SharePoint, Confluence, Salesforce, etc.)
- You need **relevance tuning** and search analytics
- You want to use Kendra GenAI Index with **Bedrock Knowledge Bases** as a retriever

Do NOT use Kendra when:

- You only need a **vector database** -- use OpenSearch Serverless, Aurora pgvector, etc.

- Your documents are already in S3 and you just need basic chunking + embeddings -- **Bedrock Knowledge Bases** with a vector store is simpler
- You need sub-millisecond latency for vector search -- use a dedicated vector DB
- Budget is very constrained -- Kendra has a higher starting cost than vector-DB-only solutions

Kendra vs Bedrock Knowledge Bases (Critical Exam Topic)

Feature	Amazon Kendra	Bedrock Knowledge Bases
Primary purpose	Enterprise intelligent search + RAG retriever	Managed RAG pipeline for Bedrock FMs
Data sources	43+ native connectors with ACL support	S3, web crawlers, Confluence, SharePoint, Salesforce (growing)
Access control	Full ACL/RBAC from source systems	Metadata filtering (no native ACL from sources)
Vector storage	Built-in (no separate vector DB needed)	Requires a vector store (OpenSearch Serverless, Aurora pgvector, Pinecone, etc.)
Search capabilities	Semantic + keyword + faceting + spell check	Semantic retrieval only
Can be used together	Yes -- Kendra GenAI Index as retriever for Bedrock KB	Yes
Standalone search UI	Yes (Kendra search experience)	No (designed for FM consumption)
Cost model	Index-based (capacity units)	Per-query + vector DB costs

Exam tip: If the question emphasizes **enterprise data source connectors with access control**, the answer is Kendra. If the question emphasizes a **managed RAG pipeline tightly integrated with Bedrock Agents/FMs**, the answer is Bedrock Knowledge Bases. They can also be used **together** -- Kendra GenAI Index as the retriever backing a Bedrock Knowledge Base.

Integration with Bedrock/GenAI

- **Kendra GenAI Index + Bedrock Knowledge Bases**: Use Kendra as the retriever for Bedrock KB, combining Kendra's 43+ connectors and ACLs with Bedrock's RAG orchestration
- **Kendra + Amazon Q Business**: Q Business can use Kendra as its retriever backend
- **Kendra + Lex**: AMAZON.KendraSearchIntent routes Lex bot queries to Kendra for FAQ-style answers
- **Kendra Retrieve API -> LLM**: Fetch passages from Kendra, inject into FM prompt for custom RAG

Exam Gotchas

- GenAI Enterprise Edition only available in limited regions (US East, US West, Ireland, Sydney)
- GenAI Enterprise Edition only supports **English** language content
- GenAI Edition only supports **v2.0 data source connectors**
- Retrieve API does NOT support query suggestions, spell check, or faceting
- Kendra is not just a vector database -- it has its own built-in retrieval engine
- Kendra does NOT require you to choose an embedding model or manage vectors -- it handles this internally

TypeScript SDK v3 Usage

```
import {
  KendraClient,
  RetrieveCommand,
  QueryCommand,
```

```
} from "@aws-sdk/client-kendra";

const client = new KendraClient({ region: "us-east-1" });

// Retrieve API for RAG (returns passages, optimized for LLM consumption)
const retrieveResponse = await client.send(
  new RetrieveCommand({
    IndexId: "your-index-id",
    QueryText: "What is our company's vacation policy?",
    PageSize: 10, // up to 100
  })
);
// Returns: ResultItems[] with DocumentId, Content (up to 200 tokens), DocumentTitle, DocumentURI

// Query API for traditional search (returns excerpts with highlights)
const queryResponse = await client.send(
  new QueryCommand({
    IndexId: "your-index-id",
    QueryText: "vacation policy",
    PageSize: 10,
    AttributeFilter: {
      EqualsTo: {
        Key: "_category",
        Value: { StringValue: "HR" },
      },
    },
  })
);
// Returns: ResultItems[] with excerpts, highlights, facets, spell corrections
```

Key Metrics

- CloudWatch: `IndexQueryCount`, `IndexQueryLatency`, `DocumentsIndexed`, `DocumentsDeleted`
- Search Analytics dashboard (built-in): query volumes, click-through rates, zero-result queries

---

3. Amazon Q Business and Q Apps

Service Overview

Amazon Q Business is a **fully managed, generative AI-powered enterprise assistant** that connects to your organization's data and systems. It provides conversational Q&A, content generation, summarization, and action execution, all with permissions-aware responses.

Amazon Q Apps is a feature within Q Business that lets end users create **lightweight, purpose-built AI applications** from conversations or prompts.

Core Capabilities:

- Conversational Q&A grounded in enterprise data with source citations
- 40+ pre-built data source connectors
- Built-in RAG pipeline (no need to manage vector stores, embeddings, or chunking)
- Plugins for taking actions in third-party systems (Jira, Salesforce, etc.)
- Admin controls: topic filters, guardrails, PII detection
- IAM Identity Center integration for user authentication

- Amazon Q Apps for user-created lightweight automation
- Agentic RAG with query decomposition and iterative retrieval

GenAI Exam Relevance

Exam Task	How Q Business Applies
Task 2.5.3	Internal knowledge tools for enterprise
RAG	Fully managed RAG solution without infrastructure management

Key Features for the Exam

Architecture:

- Uses foundation models hosted on Amazon Bedrock (no model selection needed -- managed)
- Built-in indexing pipeline with vector database (native index) or can use Kendra as retriever
- Document enrichment with Lambda functions for custom pre-processing
- Supports uploading documents directly in conversations (temporary, session-scoped)

Data Source Connectors (40+):

- Amazon S3, SharePoint, Confluence, Salesforce, Jira, Google Drive, Slack, ServiceNow, Microsoft Exchange, Smartsheet, and more
- Can use Amazon AppFlow to pull from unsupported sources into S3

Plugins:

- Read and write actions to third-party systems
- Custom plugins via OpenAPI schemas with OAuth/Cognito authentication
- Built-in plugins for Jira, Salesforce, etc.

Amazon Q Apps:

- Created by end users (Pro license required) from conversations or prompts
- Input/output cards, plugin cards, custom labels
- Published to company app library; admin-verified apps get priority placement
- Automate repetitive tasks: content generation, ticket creation, document processing
- Q Apps respect the same data governance and permissions as Q Business

Security:

- SAML 2.0 identity provider integration
- Permissions-aware: responses filtered by user's access level
- Admin controls: blocked topics, content restrictions, response scope

Agentic RAG (new capability):

- Query decomposition: breaks complex queries into sub-queries
- Iterative retrieval: re-plans if initial results are insufficient
- Transparent response events showing processing steps
- Multi-turn conversational context

When to Use / When NOT to Use

Use Q Business when:

- You need an **out-of-the-box enterprise AI assistant** without building RAG infrastructure
- You have data across **multiple enterprise systems** (SharePoint, Confluence, Jira, etc.)

- You need **permissions-aware** responses that respect user access controls
- You want end users to **create their own lightweight AI apps** (Q Apps)
- You want **zero ML infrastructure management** -- fully managed end-to-end
- You need **plugin-based actions** (create Jira tickets, query Salesforce, etc.)

Do NOT use Q Business when:

- You need **fine-grained control** over the FM, prompt engineering, or RAG pipeline -- build custom with Bedrock
- You need to use a **specific model** (Q Business manages model selection)
- You need **customer-facing** chatbots -- Q Business is for internal enterprise use
- You need **custom UI/UX** beyond what Q Business web experience provides
- You need to integrate with models **outside of Bedrock**

Q Business vs Custom RAG (Critical Exam Topic)

Feature	Amazon Q Business	Custom RAG (Bedrock KB + FM)
Setup complexity	Low -- managed end-to-end	Medium-High -- configure KB, vector store, FM, prompts
Model choice	Managed (no selection)	Full choice of any Bedrock FM
Data sources	40+ native connectors	S3, web crawlers, select connectors
Permissions	Full ACL from source systems	Metadata filtering only
Customization	Limited (admin controls, plugins)	Full (prompts, chains, agents, guardrails)
UI	Built-in web experience	Build your own
Actions	Plugins (Jira, Salesforce, custom)	Bedrock Agents with Lambda
Pricing	Per-user subscription ($3-20/user/month)	Pay-per-use (API calls + infrastructure)
Target audience	Enterprise employees (internal)	Any (internal or customer-facing)

Exam tip: Q Business is the answer when the question describes an **enterprise** needing to quickly deploy an AI assistant across internal teams with existing enterprise data sources. Custom Bedrock RAG is the answer when the question requires **customization, specific model selection, or customer-facing** applications.

Integration with Bedrock/GenAI

- Q Business uses Bedrock FMs internally (managed)
- Can use Kendra as an alternative retriever backend
- Q Apps extend Q Business with user-created automation
- Integrates with Slack and Microsoft Teams for chat deployment
- Custom plugins connect to any API via OpenAPI schemas

Exam Gotchas

- Q Business is for **internal enterprise** use, not customer-facing chatbots
- You **cannot select** which FM Q Business uses -- it is managed
- Q Apps require a **Pro license** (not Lite)
- Q Apps created by Lite users were **deleted** after August 30, 2024
- Uploaded documents in Q Business conversations are **session-scoped** only (not permanently indexed)
- Q Business is NOT the same as Q Developer -- different products, different purposes

TypeScript SDK v3 Usage

```
import {
  QBusinessClient,
  ChatSyncCommand,
  ListApplicationsCommand,
} from "@aws-sdk/client-qbusiness";

const client = new QBusinessClient({ region: "us-east-1" });

// Chat with Q Business application
const chatResponse = await client.send(
  new ChatSyncCommand({
    applicationId: "your-app-id",
    userMessage: "What is our company's remote work policy?",
  })
);
// Returns: systemMessage (answer), sourceAttributions (citations)

// List Q Business applications
const apps = await client.send(new ListApplicationsCommand({}));
```

---

4. Amazon Q Developer

Service Overview

Amazon Q Developer is an **AI-powered developer assistant** that integrates into IDEs, the AWS console, and CI/CD workflows. It is the evolution of Amazon CodeWhisperer.

Core Capabilities:

- **Code generation**: inline code completion, full function generation
- **Code explanation**: explain selected code blocks
- **Code refactoring**: restructure code for clarity, performance
- **Code transformation**: automated language upgrades (e.g., Java 8 -> 17)
- **Test generation**: unit test scaffolding
- **Bug fixing**: diagnose and suggest fixes
- **Security scanning**: detect vulnerabilities in generated code
- **AWS console assistance**: answer questions about AWS services and diagnose errors
- **CLI assistance**: natural language to bash/CLI translation

GenAI Exam Relevance

Exam Task	How Q Developer Applies
Task 2.5.4	Code generation and refactoring capabilities
Task 2.5.6	Error pattern recognition and debugging

Key Features for the Exam

Code Generation:

- Inline suggestions as you type (auto-completion)
- Full function generation from comments/docstrings
- Context-aware: uses open files, imports, project structure
- Supports 15+ languages: Python, Java, JavaScript, TypeScript, C#, Go, etc.
- Reference tracker shows if suggestions match public code

Agentic Coding:

- Multi-step development: plans, generates code, runs tests, iterates
- Reads/modifies files, executes commands
- Toggle on/off in IDE

Code Transformation:

- Automated Java upgrades (8 -> 17)
- .NET Framework to cross-platform .NET
- Language and OS-level conversions

Security Scanning:

- Detects vulnerabilities in generated and existing code
- AI-powered remediation suggestions
- Integrated into IDE workflow

Customization:

- Code customization: train on your codebase for organization-specific suggestions
- Supports multiple languages for customization
- Usage limits apply for customization creation

IDE Support:

- VS Code, JetBrains IDEs, Visual Studio, Eclipse
- AWS Console (chat and error diagnosis)
- CLI (bash translation)
- Slack and Microsoft Teams integration

When to Use / When NOT to Use

Use Q Developer when:

- Developers need **AI-assisted coding** in their IDE
- You need **automated code transformation** (language upgrades)
- You need **security vulnerability scanning** in generated code
- You want **AWS-specific expertise** in your development workflow
- You need to **debug AWS service errors** in the console

Do NOT use Q Developer when:

- You need to build an **AI-powered application** for end users -- use Bedrock
- You need **enterprise knowledge search** -- use Q Business
- You need to **train custom ML models** -- use SageMaker

Integration with Bedrock/GenAI

- Q Developer uses Bedrock FMs internally for code generation
- Can assist in writing Bedrock API code, CDK/CloudFormation templates
- Helps debug Bedrock application errors

- Not directly programmable -- it is a developer tool, not an API service

Exam Gotchas

- Q Developer is **NOT** the same as Q Business -- different audiences, different capabilities
- Q Developer was formerly called **CodeWhisperer** (exam may reference the rename)
- Q Developer does **not** use your code to train models without explicit consent
- Code customization has **usage limits** on creation/activation
- Q Developer is a **developer productivity tool**, not a service you call via API in your application
- Free tier includes code suggestions; Pro tier adds security scanning, customization

Key Metrics

- Developer productivity: lines of code accepted, suggestion acceptance rate
- Security scanning: vulnerabilities detected, remediation rate
- Transformation: upgrade success rate, manual intervention required

---

5. Amazon Titan Foundation Models

Service Overview

Amazon Titan is a **family of foundation models built by Amazon**, available exclusively through Amazon Bedrock. The family includes text generation, embedding, multimodal embedding, and image generation models.

Model Family:

Model	Model ID	Purpose	Key Specs
Titan Text Embeddings V2	`amazon.titan-embed-text-v2:0`	Text-to-vector embeddings for RAG, search, classification	8,192 tokens / 50K chars, output: 1024/512/256 dims
Titan Multimodal Embeddings G1	`amazon.titan-embed-image-v1`	Text+image-to-vector embeddings	128 text tokens, output: 1024/384/256 dims
Titan Image Generator G1 v2	`amazon.titan-image-generator-v2:0`	Text-to-image, editing, conditioning	512 char prompt, English only
Titan Text Premier	(legacy)	Enterprise text generation	32K token context
Titan Text Express	(legacy)	Balanced cost/performance text	8K token context
Titan Text Lite	(legacy)	Cost-effective text generation	4K token context

> ***Note:*** *Amazon Nova models (Nova Micro, Lite, Pro, Premier, Canvas, Reel, Sonic) are Amazon's newer model family and are gradually superseding Titan Text models for text generation. For the exam, focus on* ***Titan Embeddings*** *and* ***Titan Image Generator*** *as they remain the primary Amazon embedding and image models.*

GenAI Exam Relevance

Exam Task	How Titan Applies
Task 1.5.2	Embedding models for vector representations
RAG	Titan Embeddings V2 is a key embedding model for knowledge bases

Key Features for the Exam

Titan Text Embeddings V2 (High Priority):

- **Dimensions**: 1,024 (default), 512, or 256 -- configurable per request
- **Max input**: 8,192 tokens or 50,000 characters
- **Languages**: Optimized for English; 100+ languages supported (multilingual queries within a single language work well; cross-language queries are sub-optimal)
- **Use cases**: RAG, document search, reranking, semantic similarity, clustering, classification
- **Normalization**: Supported (useful for cosine similarity)
- **Fine-tuning**: Not supported
- **No inference parameters**: Does not accept `maxTokenCount`, `topP`, etc.
- **Delivery modes**: Latency-optimized (real-time) and throughput-optimized (batch)
- **Best practice**: Segment long documents into logical chunks (paragraphs/sections) for retrieval

Titan Multimodal Embeddings G1:

- **Dimensions**: 1,024 (default), 384, or 256
- **Input**: Text (up to 128 tokens) AND/OR images (PNG, JPEG, up to 25MB)
- **Use cases**: Multimodal search, recommendation, personalization
- **Fine-tuning**: Supported (image-text pairs, JSONL format, 1K-500K training samples)
- Generates a single embedding vector from text, image, or both

Titan Image Generator v2:

- Text-to-image generation with photorealistic output
- Image editing, inpainting, outpainting
- Image conditioning (use reference image for structure)
- Background removal
- Subject consistency (via fine-tuning)
- Color palette control
- **Watermark detection**: Built-in invisible watermarking for provenance
- **IP indemnity**: AWS provides intellectual property indemnity for Titan Image Generator outputs
- **Safety filters**: Blocks harmful/inappropriate content generation
- English prompts only, max 512 characters

When to Use / When NOT to Use

Use Titan Embeddings V2 when:

- Building RAG with Bedrock Knowledge Bases (default embedding model option)
- You need **configurable dimensions** (256/512/1024) to balance accuracy vs storage/latency
- You need an embedding model with **multilingual support**
- You need **normalization** for cosine similarity comparisons

Do NOT use Titan Embeddings when:

- You need embeddings for **code** specifically -- consider Cohere Embed v4
- You need **multimodal embeddings** with high text token count -- Titan Multimodal only supports 128 tokens
- You need to **fine-tune** the embedding model -- Titan Text Embeddings V2 does not support fine-tuning

Use Titan Image Generator when:

- You need image generation with **IP indemnity coverage** from AWS

- You need built-in **watermarking** for content provenance
- You need image **editing, conditioning, or background removal**

Integration with Bedrock/GenAI

- **Bedrock Knowledge Bases**: Titan Embeddings V2 is a primary embedding model option
- **Bedrock Batch Inference**: Use throughput-optimized mode for bulk embedding generation
- **Custom RAG**: Generate embeddings via Bedrock InvokeModel, store in your vector database
- **Titan Image Generator**: Called through Bedrock InvokeModel for image generation tasks

Exam Gotchas

- Titan Embeddings V2 dimensions are **1024, 512, 256** (NOT 1536 -- that is Titan V1)
- Titan V1 outputs **1,536** dimensions; V2 outputs **1,024** default
- Titan Multimodal Embeddings max text input is only **128 tokens** (very short vs V2's 8,192)
- Titan Embeddings V2 does **NOT** support fine-tuning
- Titan Multimodal Embeddings G1 **does** support fine-tuning
- Cross-language queries with Titan Embeddings V2 produce **sub-optimal results**
- Titan Text generation models (Premier, Express, Lite) are being superseded by Amazon Nova -- but may still appear on exam
- Amazon provides **IP indemnity** for Titan Image Generator outputs (differentiator from third-party models)

TypeScript SDK v3 Usage

```
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

// Generate text embeddings with Titan Embeddings V2
const embeddingResponse = await client.send(
  new InvokeModelCommand({
    modelId: "amazon.titan-embed-text-v2:0",
    contentType: "application/json",
    accept: "application/json",
    body: JSON.stringify({
      inputText: "What is machine learning?",
      dimensions: 512, // 256, 512, or 1024 (default)
      normalize: true, // for cosine similarity
    }),
  })
);

const embeddingResult = JSON.parse(
  new TextDecoder().decode(embeddingResponse.body)
);
// embeddingResult.embedding -&gt; number[] of length 512

// Generate multimodal embeddings (text + image)
const multimodalResponse = await client.send(
  new InvokeModelCommand({
    modelId: "amazon.titan-embed-image-v1",
```

```
    contentType: "application/json",
    accept: "application/json",
    body: JSON.stringify({
      inputText: "a sunset over the ocean",
      // inputImage: base64EncodedImage, // optional
      embeddingConfig: { outputEmbeddingLength: 384 }, // 256, 384, or 1024
    }),
  })
);

// Generate an image with Titan Image Generator v2
const imageResponse = await client.send(
  new InvokeModelCommand({
    modelId: "amazon.titan-image-generator-v2:0",
    contentType: "application/json",
    accept: "application/json",
    body: JSON.stringify({
      textToImageParams: {
        text: "A futuristic city skyline at sunset, photorealistic",
      },
      imageGenerationConfig: {
        numberOfImages: 1,
        height: 1024,
        width: 1024,
      },
    }),
  })
);
```

Key Metrics

- Bedrock CloudWatch: `InvocationLatency`, `InvocationCount`, `InvocationErrors` (per model)
- Embedding quality: measured by downstream task performance (retrieval accuracy, classification F1)

---

6. Amazon Lex

Service Overview

Amazon Lex is a **fully managed service for building conversational interfaces** (chatbots) using voice and text. It uses the same technology as Amazon Alexa.

Core Capabilities:

- Automatic Speech Recognition (ASR) for voice input
- Natural Language Understanding (NLU) for intent recognition
- Multi-turn dialog management
- Slot filling (extracting parameters from user input)
- Integration with Lambda for fulfillment logic
- Generative AI features: Conversational FAQ (QnAIntent), Assisted Slot Resolution, Descriptive Bot Builder

GenAI Exam Relevance

Exam Task	How Lex Applies
Conversational interfaces	Building chatbot front-ends for GenAI applications
Integration	Routes to Kendra, Bedrock Agents, or Lambda for fulfillment

Key Features for the Exam

Core Concepts:

- **Bot**: The conversational application
- **Intent**: An action the user wants to perform (e.g., OrderPizza, BookHotel)
- **Utterance**: What the user says to trigger an intent ("I want to order a pizza")
- **Slot**: A parameter needed to fulfill an intent (pizza size, topping)
- **Slot Type**: The data type of a slot (built-in: AMAZON.Number, AMAZON.City; custom: PizzaSize)
- **Fulfillment**: Lambda function or return-to-client to process the intent

GenAI Features (V2):

- **AMAZON.QnAIntent**: Conversational FAQ -- routes questions to Kendra or Bedrock Knowledge Base for answers. No sample utterances needed.
- **AMAZON.BedrockAgentIntent**: Routes to Bedrock Agents and Knowledge Bases for advanced conversational AI
- **Assisted NLU**: Uses LLMs to improve intent classification and slot resolution while staying within configured intents/slots
- **Assisted Slot Resolution**: LLM-enhanced slot value extraction
- **Descriptive Bot Builder**: Create intents from natural language descriptions
- **Utterance Generation**: Auto-generate sample utterances from intent descriptions
- **QnA with Bedrock Guardrails**: Apply guardrails to QnAIntent responses

Bot Architecture:

- **Versions**: Immutable snapshots of bot configuration
- **Aliases**: Pointers to versions (e.g., "prod" -> version 5, "staging" -> version 6)
- **Network of Bots**: Combine multiple bots, route to appropriate bot based on input
- **Visual Conversation Builder**: Drag-and-drop conversation flow design
- **Conditional Branching**: Control conversation paths based on conditions

When to Use / When NOT to Use

Use Lex when:

- You need a **structured chatbot** with defined intents, slots, and dialog flows
- You need **voice support** (ASR) for telephony or Alexa-like interfaces
- You need integration with **Amazon Connect** contact centers
- You need a conversational **front-end** that routes to Kendra, Bedrock, or Lambda
- You need **multi-language** chatbot support

Do NOT use Lex when:

- You need a **free-form conversational AI** without structured intents -- use Bedrock Agents directly
- You need an **enterprise knowledge assistant** -- use Q Business
- You only need **text-based** chat without voice -- Bedrock Agents/Converse API may be simpler
- Your bot needs to handle **unbounded** topics without predefined intents -- Bedrock is more flexible

Integration with Bedrock/GenAI

- **AMAZON.QnAIntent + Kendra**: Lex routes FAQ-style questions to Kendra for retrieval
- **AMAZON.QnAIntent + Bedrock KB**: Routes to Bedrock Knowledge Base for RAG answers
- **AMAZON.BedrockAgentIntent**: Routes to Bedrock Agents for complex multi-step tasks
- **Lambda fulfillment**: Intent triggers Lambda which calls Bedrock for generation
- **Amazon Connect**: Lex provides the NLU layer for Connect contact center voice bots
- **Assisted NLU**: Uses Bedrock LLMs to improve intent recognition accuracy

Exam Gotchas

- Lex V2 is the current version -- V1 is legacy
- **AMAZON.QnAIntent** does NOT require sample utterances (it catches unmatched queries)
- Lex is for **structured** conversations with defined intents; Bedrock Agents handle **unstructured** tasks
- Custom vocabulary max **500 phrases** per language
- Lex bots have **versions** (immutable) and **aliases** (mutable pointers)
- Lex integrates with **Polly** for text-to-speech output (voice responses)
- 8 kHz telephony audio is supported (important for contact center use)

TypeScript SDK v3 Usage

```
import {
  LexRuntimeV2Client,
  RecognizeTextCommand,
} from "@aws-sdk/client-lex-runtime-v2";

const client = new LexRuntimeV2Client({ region: "us-east-1" });

const response = await client.send(
  new RecognizeTextCommand({
    botId: "your-bot-id",
    botAliasId: "your-alias-id",
    localeId: "en_US",
    sessionId: "user-session-123",
    text: "I want to book a hotel in Seattle",
  })
);
// Returns: messages[], sessionState (intent, slots, dialogAction)
// sessionState.intent.name -&gt; "BookHotel"
// sessionState.intent.slots -&gt; { city: "Seattle", ... }
```

Key Metrics

- CloudWatch: `MissedUtteranceCount`, `RuntimeSuccessCount`, `RuntimeThrottledCount`
- Analytics dashboard: intent success/failure rates, slot resolution rates, conversation drop-off
- Test Workbench: measure bot performance with test sets

---

7. Amazon Transcribe

Service Overview

Amazon Transcribe is a **fully managed speech-to-text service** using Automatic Speech Recognition (ASR). It converts audio into text with high accuracy, supporting both batch and real-time streaming.

Core Capabilities:

- Batch transcription (from S3 audio/video files)
- Real-time streaming transcription (WebSocket, HTTP/2, SDK)
- Speaker diarization (identify who said what)
- Custom vocabulary and custom language models
- Automatic language identification
- PII redaction in transcripts
- Vocabulary filtering (mask profanity/custom words)
- Punctuation, number normalization, true casing
- Call Analytics (call center specific)
- Medical transcription (specialized medical vocabulary)

GenAI Exam Relevance

Exam Task	How Transcribe Applies
Task 1.3.2	Audio processing for multimodal GenAI pipelines
Pre-processing	Convert audio/video to text for FM consumption

Key Features for the Exam

Transcription Modes:

- **Batch**: Upload audio to S3, get transcript back. Supports all audio/video formats.
- **Streaming**: Real-time via WebSocket or HTTP/2. Supports FLAC, OPUS (in Ogg), PCM. Up to 4 hours. Returns partial results that stabilize.

Accuracy Enhancement:

- **Custom Vocabulary**: Up to 100 entries of domain-specific terms (medical, legal, technical)
- **Custom Language Models**: Train on your text corpus for domain-specific language patterns
- **Vocabulary Filtering**: Mask or remove specific words from output

Privacy and Safety:

- **Automatic Content Redaction**: Redact PII from transcripts (SSN, credit card, etc.)
- **Vocabulary Filtering**: Remove profanity or custom terms
- **Toxic Audio Content Detection**: Identify toxic speech

Streaming Features:

- Partial results with `IsPartial` flag (progresses to complete segments)
- Partial-result stabilization (reduces word changes, slight accuracy trade-off)
- Word-level confidence scores and timestamps
- Speaker diarization in streaming

Call Analytics (Transcribe Call Analytics):

- Call summarization
- Sentiment analysis per speaker turn
- Issue detection
- Action item extraction

- Non-talk time detection

Medical Transcription:

- Specialized medical vocabulary
- Dictation and conversational modes
- Medical specialties support
- HIPAA eligible

When to Use / When NOT to Use

Use Transcribe when:

- You need to convert **audio/video to text** as a pre-processing step for GenAI
- You need **real-time speech-to-text** for live captioning or voice interfaces
- You need to **transcribe call center audio** with analytics
- You need **PII redaction** in audio transcripts
- You need **speaker identification** (diarization)

Do NOT use Transcribe when:

- You need **text-to-speech** -- use Amazon Polly
- You need **real-time conversational speech-to-speech** -- use Amazon Nova Sonic via Bedrock
- Audio is already text -- skip directly to NLP services

Integration with Bedrock/GenAI

- **Audio -> Text -> FM pipeline**: Transcribe converts audio to text, then feed to Bedrock for analysis, summarization, or Q&A
- **Transcribe + Comprehend**: Transcribe audio, then run Comprehend for entity extraction, sentiment, PII detection
- **Transcribe + Lex**: Real-time voice-to-text for Lex chatbot input (Lex has built-in ASR, but Transcribe offers more control)
- **Call Analytics + Bedrock**: Transcribe call audio with analytics, then use Bedrock to generate call summaries, action items

Exam Gotchas

- Streaming supports limited audio formats: **FLAC, OPUS (Ogg), PCM** only (not MP3 in streaming)
- Batch supports **all common formats** (MP3, WAV, FLAC, etc.)
- Streaming has a **4-hour** maximum duration
- Partial-result stabilization trades **accuracy for latency**
- Custom vocabulary is different from Custom Language Models (vocabulary = specific terms, CLM = language patterns)
- Transcribe Call Analytics is a **separate feature** from standard transcription
- Medical transcription is US English only (`en-US`)
- Transcribe processes **audio** -- for document text extraction, use Textract

TypeScript SDK v3 Usage

```
import {
  TranscribeClient,
  StartTranscriptionJobCommand,
```

```
  GetTranscriptionJobCommand,
} from "@aws-sdk/client-transcribe";

const client = new TranscribeClient({ region: "us-east-1" });

// Start a batch transcription job
await client.send(
  new StartTranscriptionJobCommand({
    TranscriptionJobName: "my-job-001",
    LanguageCode: "en-US",
    Media: {
      MediaFileUri: "s3://my-bucket/audio/meeting.mp3",
    },
    OutputBucketName: "my-bucket",
    OutputKey: "transcripts/meeting.json",
    Settings: {
      ShowSpeakerLabels: true,
      MaxSpeakerLabels: 4,
    },
    ContentRedaction: {
      RedactionType: "PII",
      RedactionOutput: "redacted", // or "redacted_and_unredacted"
      PiiEntityTypes: ["NAME", "ADDRESS", "CREDIT_DEBIT_NUMBER"],
    },
  })
);

// Check job status
const job = await client.send(
  new GetTranscriptionJobCommand({
    TranscriptionJobName: "my-job-001",
  })
);
// job.TranscriptionJob.TranscriptionJobStatus -&gt; "IN_PROGRESS" | "COMPLETED" | "FAILED"
```

Key Metrics

- CloudWatch: `TotalRequestCount`, `SuccessfulRequestCount`, `ThrottledCount`
- Per-job: word confidence scores, speaker labels, timestamps

---

8. Amazon Textract

Service Overview

Amazon Textract is a **fully managed document analysis service** that extracts text, forms, tables, and structured data from documents using deep learning. It goes beyond basic OCR.

Core Capabilities:

- **DetectDocumentText**: Raw OCR text extraction (lines and words)
- **AnalyzeDocument**: Extract text + forms (key-value pairs) + tables + signatures + queries + layout
- **AnalyzeExpense**: Extract structured data from invoices and receipts
- **AnalyzeID**: Extract data from identity documents (driver's licenses, passports)
- **AnalyzeLending**: Specialized mortgage document processing
- Custom Adapters: Train custom models for domain-specific document types

GenAI Exam Relevance

Exam Task	How Textract Applies
Document processing	Extract text from documents before sending to FMs
Data pre-processing	Structure unstructured documents for RAG pipelines

Key Features for the Exam

APIs and Features:

API	Purpose	Key Features
`DetectDocumentText`	Basic OCR	Lines, words, confidence scores, bounding boxes
`AnalyzeDocument`	Structured extraction	Forms (key-value), Tables, Queries, Signatures, Layout
`AnalyzeExpense`	Invoice/receipt	Vendor name, line items, totals, tax amounts
`AnalyzeID`	Identity documents	Name, DOB, address, document number, expiration
`AnalyzeLending`	Mortgage packages	Classify and extract from mortgage document types

Processing Modes:

- **Synchronous**: Single-page documents (JPEG, PNG, PDF, TIFF). Near real-time.
- **Asynchronous**: Multi-page documents (PDF, TIFF up to thousands of pages). Uses `StartDocumentTextDetection`/`StartDocumentAnalysis` -> poll with `GetDocumentTextDetection`/`GetDocumentAnalysis`.

AnalyzeDocument Features (FeatureTypes):

- `FORMS`: Extracts key-value pairs (e.g., "Name: John Smith")
- `TABLES`: Extracts table structure with rows, columns, cells, headers, merged cells
- `QUERIES`: Ask specific questions and get targeted answers (e.g., "What is the patient name?")
- `SIGNATURES`: Detects signature locations
- `LAYOUT`: Identifies document structure (paragraphs, headers, footers, lists, figures, page numbers)

Custom Adapters:

- Train on annotated sample documents for domain-specific extraction
- Provide `AdapterId` and `AdapterVersion` when calling AnalyzeDocument

Document Formats:

- Synchronous: JPEG, PNG, PDF (single-page), TIFF (single-page)
- Asynchronous: PDF (multi-page), TIFF (multi-page)
- Max 10MB file size

When to Use / When NOT to Use

Use Textract when:

- You need to **extract text from scanned documents or images** (OCR + structure)
- You need **form field extraction** (key-value pairs)
- You need **table extraction** from documents
- You need to process **invoices, receipts, IDs, or mortgage documents**

- You need to **convert documents to text** before sending to an FM for analysis

Do NOT use Textract when:

- Your documents are already **digital text** (plain text, HTML, etc.) -- no extraction needed
- You need **NLP analysis** on extracted text -- use Comprehend after Textract
- You need **speech-to-text** -- use Transcribe
- You need real-time **handwriting recognition from a stylus** -- Textract works on document images

Integration with Bedrock/GenAI

- **Textract -> Bedrock**: Extract text from documents, then send to FM for summarization, Q&A, analysis
- **Textract -> Comprehend**: Extract text, then detect PII, entities, or sentiment
- **Textract + A2I**: Route low-confidence Textract extractions to human reviewers
- **Document processing pipeline**: S3 -> Textract -> Lambda (post-processing) -> Bedrock Knowledge Base
- **Bedrock Data Automation** may use Textract-like capabilities under the hood for document processing

Exam Gotchas

- Textract is for **documents/images** -- NOT for audio (use Transcribe) or live text (use Comprehend)
- `DetectDocumentText` is just OCR; `AnalyzeDocument` adds structure (forms, tables, queries)
- Queries feature lets you ask **specific questions** about the document -- very useful for targeted extraction
- Synchronous APIs are **single-page only**; multi-page requires async
- Textract returns **Block objects** with hierarchical relationships (PAGE -> LINE -> WORD, TABLE -> CELL)
- AnalyzeExpense is specifically for **invoices and receipts** -- do not use AnalyzeDocument for these
- AnalyzeID is for **government-issued identity documents** only
- HIPAA eligible, PCI DSS compliant

TypeScript SDK v3 Usage

```
import {
  TextractClient,
  AnalyzeDocumentCommand,
  DetectDocumentTextCommand,
  AnalyzeExpenseCommand,
} from "@aws-sdk/client-textract";

const client = new TextractClient({ region: "us-east-1" });

// Analyze document for forms and tables (S3-based)
const analysisResponse = await client.send(
  new AnalyzeDocumentCommand({
    Document: {
      S3Object: {
        Bucket: "my-bucket",
        Name: "documents/tax-form.pdf",
      },
    },
    FeatureTypes: ["FORMS", "TABLES", "QUERIES", "SIGNATURES"],
    QueriesConfig: {
      Queries: [
        { Text: "What is the total amount?" },
        { Text: "What is the customer name?" },
      ],
    },
```

```
  })
);
// Returns: Blocks[] with BlockType (PAGE, LINE, WORD, TABLE, CELL, KEY_VALUE_SET, QUERY, QUERY_RESULT)

// Simple OCR text detection
const textResponse = await client.send(
  new DetectDocumentTextCommand({
    Document: {
      S3Object: {
        Bucket: "my-bucket",
        Name: "documents/letter.png",
      },
    },
  })
);
// Returns: Blocks[] with LINE and WORD blocks, each with Text and Confidence

// Analyze invoice/receipt
const expenseResponse = await client.send(
  new AnalyzeExpenseCommand({
    Document: {
      S3Object: {
        Bucket: "my-bucket",
        Name: "receipts/receipt-001.jpg",
      },
    },
  })
);
// Returns: ExpenseDocuments[] with SummaryFields and LineItemGroups
```

Key Metrics

- CloudWatch: `SuccessfulRequestCount`, `ThrottledCount`, `ServerErrorCount`
- Per-extraction: confidence scores per block/field
- Response time varies by document complexity and page count

---

9. Amazon Rekognition

Service Overview

Amazon Rekognition is a **fully managed image and video analysis service** that uses deep learning for computer vision tasks. No ML expertise required.

Core Capabilities:

- **Label Detection**: Objects, scenes, concepts, activities, dominant colors
- **Content Moderation**: Explicit/inappropriate content detection
- **Facial Analysis**: Face detection, attributes (emotion, age range, landmarks)
- **Face Comparison**: Compare faces between images
- **Face Search**: Match faces against a stored collection
- **Face Liveness**: Verify live person (anti-spoofing)
- **Celebrity Recognition**: Identify celebrities
- **Text Detection**: Extract text from images/videos (OCR for visual media)

- **Personal Protective Equipment (PPE) Detection**: Detect masks, helmets, vests
- **Custom Labels**: Train custom object detection models
- **Custom Moderation**: Train custom content moderation adapters
- **Video Analysis**: Asynchronous analysis of stored videos, real-time streaming

GenAI Exam Relevance

Exam Task	How Rekognition Applies
Content moderation	Pre-filter images/video before GenAI processing
Multimodal preprocessing	Extract labels/text from images for FM context

Key Features for the Exam

Image vs Video Operations:

- **Image operations**: Synchronous. Input from S3 or Base64 bytes. Returns immediately.
- **Video operations (stored)**: Asynchronous. Start -> Get pattern. Videos in S3.
- **Video operations (streaming)**: Real-time via Kinesis Video Streams.

Content Moderation:

- Detects explicit adult content, violence, and other inappropriate material
- **Custom Moderation adapters**: Train on your annotated images to enhance accuracy for your domain
- Returns labels with confidence scores
- Integrates with **A2I** for human review of flagged content
- **Bulk analysis**: `StartMediaAnalysisJob` for processing large image collections

Face Liveness:

- Verifies a live person is present (prevents photo/video spoofing)
- Uses `CreateFaceLivenessSession`, `StartFaceLivenessSession`, `GetFaceLivenessSessionResults`
- Important for identity verification workflows

Custom Labels:

- Train custom object/scene detection models with your images
- Useful for business-specific visual inspection (e.g., product defects)
- Charges for training hours and inference hours

When to Use / When NOT to Use

Use Rekognition when:

- You need **image/video content moderation** (filter before or alongside GenAI)
- You need **facial analysis, comparison, or liveness detection**
- You need to **detect objects, scenes, or text in images/video**
- You need **real-time video stream analysis** (Kinesis Video Streams)
- You need to **moderate user-uploaded images** before processing with an FM

Do NOT use Rekognition when:

- You need to **generate images** -- use Bedrock (Titan Image Generator, Stability AI, Nova Canvas)
- You need **document text extraction** with forms/tables -- use Textract
- You need **text analysis** (sentiment, entities) -- use Comprehend

- You need multimodal **understanding** (describe what's in an image) -- use a multimodal FM via Bedrock (Claude, Nova)

Integration with Bedrock/GenAI

- **Content moderation pipeline**: User uploads image -> Rekognition moderates -> if safe, send to FM for analysis
- **Rekognition + A2I**: Low-confidence moderation results routed to human reviewers
- **Label extraction -> FM context**: Detect labels/objects in images, include as text context for FM prompts
- **Video analysis -> summarization**: Extract scenes/events from video, feed to FM for narrative summarization

Exam Gotchas

- Rekognition is for **analysis**, not generation -- it does not create images
- `DetectModerationLabels` is for content moderation; `DetectLabels` is for general object/scene detection
- Face **collections** store face vectors (not images) for search operations
- Video analysis is **asynchronous** (Start/Get pattern) -- not real-time for stored videos
- Streaming video analysis uses **Kinesis Video Streams** (real-time)
- Custom Labels and Custom Moderation have **training costs** (per-hour)
- Rekognition is a **HIPAA Eligible Service**
- Text detection in images (`DetectText`) is basic OCR -- for forms/tables, use Textract

TypeScript SDK v3 Usage

```
import {
  RekognitionClient,
  DetectLabelsCommand,
  DetectModerationLabelsCommand,
  DetectTextCommand,
} from "@aws-sdk/client-rekognition";

const client = new RekognitionClient({ region: "us-east-1" });

// Detect labels (objects, scenes) in an image
const labels = await client.send(
  new DetectLabelsCommand({
    Image: {
      S3Object: { Bucket: "my-bucket", Name: "images/photo.jpg" },
    },
    MaxLabels: 20,
    MinConfidence: 75,
  })
);
// Returns: Labels[] with Name, Confidence, Instances (bounding boxes), Parents

// Content moderation
const moderation = await client.send(
  new DetectModerationLabelsCommand({
    Image: {
      S3Object: { Bucket: "my-bucket", Name: "uploads/user-image.jpg" },
    },
    MinConfidence: 60,
  })
);
// Returns: ModerationLabels[] with Name, Confidence, ParentName
```

```
// Detect text in images
const textInImage = await client.send(
  new DetectTextCommand({
    Image: {
      S3Object: { Bucket: "my-bucket", Name: "images/sign.jpg" },
    },
  })
);
// Returns: TextDetections[] with DetectedText, Type (LINE|WORD), Confidence, Geometry
```

Key Metrics

- CloudWatch: `SuccessfulRequestCount`, `ThrottledCount`, per-API metrics
- Per-response: confidence scores for labels, moderation labels, faces
- Custom Labels: training loss, F1 score, precision, recall

---

10. Amazon Augmented AI (A2I)

Service Overview

Amazon Augmented AI (A2I) is a service that enables **human-in-the-loop review workflows** for machine learning predictions. It provides the infrastructure to build, manage, and monitor human review processes without building custom review tools.

Core Capabilities:

- Built-in integration with Textract and Rekognition
- Custom task types for any ML workflow
- Configurable confidence thresholds for triggering human review
- Workforce management (private, Mechanical Turk, third-party vendors)
- Worker task templates (customizable UI)
- Results stored in S3
- EventBridge notifications on completion

GenAI Exam Relevance

Exam Task	How A2I Applies
Task 2.1.5	Human review workflows for ML/AI outputs
Quality assurance	Human oversight of FM-generated content

Key Features for the Exam

Three Components:

1. **Worker Task Template**: HTML template defining the review UI presented to human reviewers
2. **Flow Definition**: Specifies conditions for review, workforce, task template, and output location
3. **Human Loop**: An individual review task created when conditions are met

Built-in Task Types:

- **Amazon Textract**: Review key-value pair extractions from documents. Confidence thresholds trigger review automatically.
- **Amazon Rekognition**: Review content moderation results. Confidence thresholds or random sampling trigger review.
- Both built-in types support automatic **Human Loop Activation Conditions** (confidence thresholds).

Custom Task Types:

- For any ML model or workflow (including GenAI/Bedrock outputs)
- You call `StartHumanLoop` explicitly from your application code
- **No automatic activation conditions** -- your application decides when to trigger review
- Supports review of Comprehend, Transcribe, Translate, SageMaker, or any custom output

Workforce Options:

- **Private workforce**: Your organization's employees (via Cognito or OIDC)
- **Amazon Mechanical Turk**: Public crowd workers (not for confidential data)
- **Third-party vendors**: AWS Marketplace review vendors

Human Loop Activation:

Task Type	Activation	Configuration
Textract (built-in)	Automatic based on confidence thresholds	Set in Flow Definition
Rekognition (built-in)	Automatic based on confidence thresholds OR random sampling	Set in Flow Definition
Custom	Manual -- your code calls `StartHumanLoop`	In your application logic

When to Use / When NOT to Use

Use A2I when:

- You need **human review of low-confidence ML predictions**
- You need human verification of **Textract document extractions** or **Rekognition content moderation**
- You need **human-in-the-loop** for GenAI output quality assurance
- You need a **managed review infrastructure** without building custom review tools
- You need **audit trails** for human review decisions (output in S3)

Do NOT use A2I when:

- You need **real-time human approval** in a latency-sensitive pipeline (A2I is async)
- You need **automated content moderation** without humans -- use Rekognition or Bedrock Guardrails alone
- You need **model retraining** -- A2I captures review results, but retraining is a separate process
- You need a full **annotation/labeling platform** -- use SageMaker Ground Truth

Integration with Bedrock/GenAI

- **GenAI output review**: Application calls Bedrock FM -> evaluate output confidence/quality -> if uncertain, call `StartHumanLoop` to route for human review
- **Textract + A2I**: Low-confidence document extractions routed to human reviewers before entering RAG pipeline
- **Rekognition + A2I**: Content moderation with human review for borderline cases
- **Comprehend + A2I**: Route uncertain entity extractions or PII detections for human verification
- **Feedback loop**: A2I review results can be used to improve prompts, fine-tune models, or update guardrails

Common GenAI Pattern:

```
User prompt -&gt; Bedrock FM -&gt; Generate response
                                |
                        Evaluate quality/confidence
                                |
                    High confidence -&gt; Return to user
                    Low confidence  -&gt; A2I Human Loop -&gt; Reviewer corrects -&gt; Return to user
```

Exam Gotchas

- A2I is **asynchronous** -- human review takes time; not suitable for real-time approval flows
- Built-in task types (Textract, Rekognition) have **automatic activation conditions** (confidence thresholds)
- Custom task types do **NOT** have automatic activation -- your code must call `StartHumanLoop`
- A2I is part of the **SageMaker** service family (console access through SageMaker)
- Worker task templates are **HTML-based** -- customizable review UIs
- Results are stored in **S3** as JSON
- A2I sends **EventBridge** events on human loop completion
- Multiple reviewers can be used to increase confidence (consensus)
- A2I is NOT a replacement for Bedrock Guardrails -- Guardrails are automated; A2I involves humans

TypeScript SDK v3 Usage

```typescript
import {
  SageMakerA2IRuntimeClient,
  StartHumanLoopCommand,
  DescribeHumanLoopCommand,
  ListHumanLoopsCommand,
} from "@aws-sdk/client-sagemaker-a2i-runtime";

const client = new SageMakerA2IRuntimeClient({ region: "us-east-1" });

// Start a human review loop (custom task type)
const humanLoop = await client.send(
  new StartHumanLoopCommand({
    HumanLoopName: "review-document-extraction-001",
    FlowDefinitionArn:
      "arn:aws:sagemaker:us-east-1:123456789:flow-definition/my-review-flow",
    HumanLoopInput: {
      InputContent: JSON.stringify({
        extractedText: "Patient Name: J0hn Sm1th", // possibly incorrect OCR
        confidence: 0.62,
        sourceDocument: "s3://my-bucket/docs/form.pdf",
      }),
    },
  })
);
// Returns: HumanLoopArn

// Check status of a human loop
const status = await client.send(
  new DescribeHumanLoopCommand({
    HumanLoopName: "review-document-extraction-001",
  })
);
// status.HumanLoopStatus -&gt; "InProgress" | "Completed" | "Stopped" | "Failed"
// status.HumanLoopOutput?.OutputS3Uri -&gt; S3 path with reviewer's corrections
```

```
// List human loops for a flow definition
const loops = await client.send(
  new ListHumanLoopsCommand({
    FlowDefinitionArn:
      "arn:aws:sagemaker:us-east-1:123456789:flow-definition/my-review-flow",
  })
);
```

Key Metrics

- CloudWatch Events via EventBridge: human loop status changes
- S3 output: reviewer decisions, timestamps, worker IDs
- Metrics to track: review latency, agreement rate between reviewers, override rate vs model predictions

---

11. Service Comparison Decision Guide

"Which service do I use?" Quick Reference

Scenario	Service
Detect/redact PII in text before ingestion to a data lake	**Comprehend**
Filter PII in prompts/responses during Bedrock inference	**Bedrock Guardrails**
Enterprise search with 40+ data source connectors and ACLs	**Kendra**
Managed RAG pipeline integrated with Bedrock Agents	**Bedrock Knowledge Bases**
Out-of-the-box enterprise AI assistant for employees	**Q Business**
Custom GenAI application with model selection and prompt control	**Bedrock (custom RAG)**
AI-assisted code generation in IDE	**Q Developer**
Text-to-vector embeddings for RAG	**Titan Embeddings V2** (or Cohere Embed)
Structured chatbot with intents, slots, and voice	**Lex**
Free-form conversational AI agent	**Bedrock Agents**
Speech to text (audio transcription)	**Transcribe**
Text to speech	**Polly** (not covered here)
Document OCR with forms and tables	**Textract**
Image/video content moderation	**Rekognition**
Image generation	**Bedrock** (Titan Image, Nova Canvas, Stability)
Human review of ML/AI predictions	**A2I**
Extract entities from text	**Comprehend**
Detect objects/labels in images	**Rekognition**

Scenario	Service
Detect text in images	**Rekognition** (basic) or **Textract** (structured)

Common Multi-Service Pipelines

Document Processing for RAG:

```
S3 (documents) -&gt; Textract (extract text/structure) -&gt; Comprehend (PII redaction)
  -&gt; Titan Embeddings V2 (vectorize) -&gt; OpenSearch/Aurora (store)
  -&gt; Bedrock KB (retrieve + generate)
```

Voice-Enabled GenAI Assistant:

```
User speaks -&gt; Transcribe (speech-to-text) -&gt; Lex (intent recognition)
  -&gt; Bedrock Agent (fulfillment) -&gt; Polly (text-to-speech) -&gt; User hears response
```

Content Moderation Pipeline:

```
User uploads image -&gt; Rekognition (content moderation)
  -&gt; If uncertain: A2I (human review)
  -&gt; If safe: Bedrock FM (analyze/describe image)
  -&gt; Bedrock Guardrails (filter response)
```

Enterprise Knowledge Assistant:

```
Q Business (conversational UI) -&gt; Kendra GenAI Index (retrieval)
  -&gt; Bedrock FM (generation, managed by Q Business)
  -&gt; User gets permissions-aware answer with citations
```

Call Center Intelligence:

```
Call audio -&gt; Transcribe Call Analytics (transcribe + sentiment + summarization)
  -&gt; Comprehend (entity extraction, PII detection)
  -&gt; Bedrock FM (generate follow-up recommendations)
  -&gt; A2I (human review for sensitive cases)
```

Embedding Model Comparison (Exam-Relevant)

Model	Dimensions	Max Input	Languages	Fine-tuning	Key Differentiator
Titan Text Embeddings V2	1024/512/256	8,192 tokens	100+ (English optimized)	No	Configurable dimensions, normalization
Titan Text Embeddings V1	1,536	8,192 tokens	25+	No	Legacy, higher dimension
Titan Multimodal Embeddings	1024/384/256	128 text tokens	English	Yes	Text + image joint embedding
Cohere Embed v4	1024	512 tokens	100+	No	Multimodal (text+image), strong multilingual
Cohere Embed Multilingual	1024	512 tokens	100+	No	Strong cross-language search

Cost Considerations

Service	Pricing Model	Free Tier
Comprehend	Per API request + input size	50K units/month (12 months)
Kendra	Index capacity units (hourly)	750 hours Developer Edition (30 days)
Q Business	Per-user subscription ($3-20/month)	None
Q Developer	Free tier + Pro tier ($19/user/month)	Free tier available
Titan Embeddings	Per 1K input tokens via Bedrock	Bedrock free tier varies
Lex	Per text/speech request	10K text, 5K speech/month (12 months)
Transcribe	Per second of audio	60 minutes/month (12 months)
Textract	Per page processed	1K pages/month (3 months)
Rekognition	Per image/minute of video	1K images/month (12 months)
A2I	Per human review task + workforce costs	None

Application Integration Services -- Comprehensive Study Guide (AIP-C01)

Application integration services orchestrate, decouple, and connect the components of generative AI applications. **AWS Step Functions is one of the most heavily tested services** on the exam, referenced in nearly every task statement across both domains. EventBridge, SQS, SNS, AppFlow, and AppConfig appear in targeted scenarios.

---

1. AWS Step Functions (Heavily Tested)

1.1 Overview

AWS Step Functions is a **serverless workflow orchestration service** that coordinates distributed applications and microservices using visual state machines. For GenAI, it is the primary mechanism for building deterministic, auditable AI workflows.

Core Exam Relevance:

- Task 1.2.3: Circuit breaker patterns for resilient AI systems
- Task 1.5.5: Query transformation workflows
- Task 1.6.2: Clarification workflows
- Task 1.6.6: Sequential prompt chains
- Task 2.1.1: Agent orchestration
- Task 2.1.2: ReAct patterns, chain-of-thought reasoning
- Task 2.1.3: Stopping conditions for AI workflows
- Task 2.1.5: Human review/approval processes
- Task 2.4.4: Dynamic content-based routing
- Task 2.5.3: Document processing systems
- Task 2.5.5: Agent design patterns

1.2 Standard vs Express Workflows

Feature	Standard	Express
Max duration	1 year	5 minutes
Execution model	Exactly-once	At-least-once (async), At-most-once (sync)
Pricing	Per state transition	Per execution + duration + memory
Integration patterns	Request Response, Run a Job (.sync), Wait for Callback (.waitForTaskToken)	Request Response only
Execution history	90-day retention in console	CloudWatch Logs only
Max executions/sec	~2,000	~100,000
Use case	Long-running AI workflows, human approval, agent loops	High-volume event processing, real-time inference pipelines

When to use which for GenAI:

- **Standard**: Prompt chaining with human-in-the-loop, agent orchestration with ReAct loops, circuit breaker patterns, any workflow needing `.waitForTaskToken` or `.sync`

- **Express**: High-throughput document classification, real-time inference routing, short AI pipelines under 5 minutes

Exam Gotcha: Express Workflows do NOT support `.sync` or `.waitForTaskToken` integration patterns. If a question involves waiting for human approval or waiting for a Bedrock model customization job to complete, the answer is Standard Workflow.

Exam Gotcha: You can nest Express Workflows inside Standard Workflows to optimize cost for high-throughput sub-steps within a longer orchestration.

1.3 Bedrock Integration (Optimized Integration)

Step Functions has an **optimized integration** with Amazon Bedrock, meaning you can invoke Bedrock directly from a Task state without needing a Lambda function intermediary.

Supported Bedrock APIs:

API	Integration Patterns	Use Case
`InvokeModel`	Request Response, Run a Job (.sync), Wait for Callback (.waitForTaskToken)	Inference on any FM
`CreateModelCustomizationJob`	Request Response	Start fine-tuning job
`CreateModelCustomizationJob.sync`	Run a Job (.sync)	Wait for fine-tuning to complete

Key InvokeModel features specific to Step Functions:

- `Body`: Inline model input payload (up to 256 KiB)
- `Input.S3Uri`: Read input from S3 (for payloads > 256 KiB)
- `Output.S3Uri`: Write output to S3 (for large responses)
- You can specify `Body` OR `Input`, but not both

Task state definition for Bedrock InvokeModel:

```
{
  "Type": "Task",
  "Resource": "arn:aws:states:::bedrock:invokeModel",
  "Arguments": {
    "ModelId": "anthropic.claude-sonnet-4-20250514",
    "Body": {
      "anthropic_version": "bedrock-2023-05-31",
      "max_tokens": 1024,
      "messages": [
        {
          "role": "user",
          "content": "{% $states.input.userPrompt %}"
        }
      ]
    },
    "ContentType": "application/json",
    "Accept": "application/json"
  },
  "Output": {
    "result": "{% $states.result.Body.content[0].text %}"
  },
  "Next": "NextState"
}
```

TypeScript SDK v3 -- Starting a Step Functions execution that invokes Bedrock:

```
import { SFNClient, StartExecutionCommand } from "@aws-sdk/client-sfn";

const sfnClient = new SFNClient({ region: "us-east-1" });

const response = await sfnClient.send(
  new StartExecutionCommand({
    stateMachineArn: "arn:aws:states:us-east-1:123456789012:stateMachine:BedrockPromptChain",
    input: JSON.stringify({
      userPrompt: "Summarize the key benefits of serverless architecture",
      modelId: "anthropic.claude-sonnet-4-20250514",
    }),
  })
);
```

IAM policy for Step Functions to call Bedrock InvokeModel:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["bedrock:InvokeModel"],
      "Resource": [
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-sonnet-4-20250514"
      ]
    }
  ]
}
```

Exam Gotcha: Scope IAM policies to specific model ARNs. Do not use wildcards in production. The exam tests whether you know to restrict `bedrock:InvokeModel` to specific foundation model or provisioned model ARNs.

1.4 Circuit Breaker Pattern for FM Resilience

The circuit breaker pattern prevents cascading failures when a foundation model endpoint is unavailable, throttled, or experiencing high latency. This is **directly tested** in Task 1.2.3.

How it works with Step Functions + DynamoDB:

1. **Caller** starts a Step Functions Express Workflow with the callee service name
2. **Get Circuit Status** step reads from a DynamoDB `CircuitStatus` table
3. If an unexpired record exists for the callee, the circuit is **OPEN** -- return immediate failure
4. If no record exists (or record expired), the circuit is **CLOSED** -- proceed with the call
5. If the call **fails** after retries with exponential backoff, insert a record with `ExpiryTimeStamp` into DynamoDB
6. DynamoDB **TTL** automatically cleans expired records, eventually closing the circuit

State machine logic (Amazon States Language):

```
{
  "Is Circuit Closed": {
    "Type": "Choice",
    "Choices": [
      {
        "Variable": "$.CircuitStatus",
        "StringEquals": "OPEN",
        "Next": "Circuit Open"
      },
```

```
        {
          "Variable": "$.CircuitStatus",
          "StringEquals": "",
          "Next": "Execute Model Call"
        }
      ]
    },
    "Circuit Open": {
      "Type": "Fail",
      "Error": "CircuitBreakerOpen",
      "Cause": "The circuit is open. The callee service is degraded."
    },
    "Execute Model Call": {
      "Type": "Task",
      "Resource": "arn:aws:states:::bedrock:invokeModel",
      "Arguments": {
        "ModelId": "{% $states.input.modelId %}",
        "Body": {
          "prompt": "{% $states.input.prompt %}"
        }
      },
      "Retry": [
        {
          "ErrorEquals": ["States.TaskFailed"],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2,
          "JitterStrategy": "FULL"
        }
      ],
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "Open Circuit"
        }
      ],
      "Next": "Success"
    }
  }
```

GenAI-specific circuit breaker scenarios:

- FM endpoint returns `ThrottlingException` repeatedly -- trip the circuit, fall back to a lighter model
- Bedrock returns `ModelTimeoutException` -- trip the circuit, return cached response
- Model customization job fails -- trip the circuit, continue using base model

Exam Gotcha: The circuit breaker uses DynamoDB TTL for automatic circuit closure. Express Workflows are recommended for the circuit breaker itself because they are cheaper for high-throughput calls. The parent orchestration workflow should be Standard if it needs human approval or long-running coordination.

1.5 ReAct Pattern Implementation

The **ReAct (Reasoning + Acting)** pattern is a fundamental agentic AI design tested in Task 2.1.2. Step Functions can implement this as a deterministic loop.

Pattern: Thought -> Action -> Observation -> Repeat until done

```
■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■
■  Reasoning  ■■■■■&gt;■  Tool/Action ■■■■■&gt;■  Observation    ■
■  (Bedrock)  ■     ■  (Lambda)    ■     ■  (Parse Result) ■
```

```
■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■
        ^                                          ■
        ■              ■■■■■■■■■■■■■■                ■
        ■■■■■■■■■■■■■■■■■■ Continue? ■&lt;■■■■■■■■■■■■■■■■■
                       ■■■■■■■■■■■■■■■
                         ■ Done
                         v
                    ■■■■■■■■■■■■■■■
                    ■  Output    ■
                    ■■■■■■■■■■■■■■■
```

Step Functions implementation approach:

1. **Reasoning State** (Task): Call Bedrock with the current context, ask it to decide the next action
2. **Choice State**: Evaluate the model's decision -- does it want to use a tool, or is it done?
3. **Action State** (Task): Execute the selected tool via Lambda or direct SDK integration
4. **Observation State** (Pass): Append the tool result to the conversation context
5. **Loop back** to Reasoning State

Stopping conditions (Task 2.1.3):

- `TimeoutSeconds` on the overall state machine execution
- `MaxAttempts` limit on the reasoning loop (use a counter variable with a Choice state)
- `HeartbeatSeconds` on individual Task states to detect hung tasks
- Choice state checking iteration count against a configured maximum

Exam Gotcha: The exam distinguishes between Step Functions ReAct (deterministic, auditable, developer-defined loop structure) and Bedrock Agents ReAct (AI-native, model decides the flow). Use Step Functions when you need full control and auditability. Use Bedrock Agents when you need semantic flexibility.

1.6 Human-in-the-Loop Patterns

Human review and approval (Task 2.1.5) use the **Wait for Callback with Task Token** pattern. This is only available in Standard Workflows.

How it works:

1. Task state sends a message containing a **task token** to SQS, SNS, or Lambda
2. Step Functions **pauses** the execution (up to 1 year)
3. An external process (human reviewer via web app, email link, etc.) processes the request
4. The external process calls `SendTaskSuccess` or `SendTaskFailure` with the task token
5. Step Functions **resumes** the execution

Task state with callback token:

```
{
  "Wait for Human Review": {
    "Type": "Task",
    "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",
    "Parameters": {
      "QueueUrl": "https://sqs.us-east-1.amazonaws.com/123456789012/human-review-queue",
      "MessageBody": {
        "taskToken.$": "$$.Task.Token",
        "generatedContent.$": "$.modelOutput",
        "reviewType": "content-safety"
      }
    },
    "HeartbeatSeconds": 3600,
    "TimeoutSeconds": 86400,
```

```
      "Next": "Process Approved Content"
    }
  }
```

TypeScript SDK v3 -- Sending task success from a reviewer application:

```
import { SFNClient, SendTaskSuccessCommand } from "@aws-sdk/client-sfn";

const sfnClient = new SFNClient({ region: "us-east-1" });

await sfnClient.send(
  new SendTaskSuccessCommand({
    taskToken: event.taskToken,
    output: JSON.stringify({
      approved: true,
      reviewerNotes: "Content meets safety guidelines",
    }),
  })
);
```

GenAI use cases for human-in-the-loop:

- Content safety review before publishing AI-generated content
- Fact-checking AI summaries before sending to customers
- Approval of AI-generated code changes before deployment
- Review of AI-extracted data from documents before database updates
- Movie idea pitch workflow: AI generates ideas, human selects favorites, AI refines

Exam Gotcha: The task token is accessed via `$$.Task.Token` (the Context Object, using `$$`), not from the input. `HeartbeatSeconds` sends periodic keep-alive signals; if no heartbeat is received within the interval, the task fails with `States.HeartbeatTimeout`. This is different from `TimeoutSeconds` which is the total allowed wait time.

1.7 Chain-of-Thought Reasoning Workflows

Chain-of-thought (CoT) decomposes complex reasoning into sequential steps, each building on the previous output. Step Functions implements this as a linear sequence of Bedrock InvokeModel calls.

Pattern: Sequential Prompt Chain

```
{
  "Comment": "Chain-of-thought: Analyze, Plan, Execute, Verify",
  "StartAt": "Step 1 - Analyze Problem",
  "States": {
    "Step 1 - Analyze Problem": {
      "Type": "Task",
      "Resource": "arn:aws:states:::bedrock:invokeModel",
      "Arguments": {
        "ModelId": "anthropic.claude-sonnet-4-20250514",
        "Body": {
          "messages": [{"role": "user", "content": "Analyze this problem: {% $states.input.problem %}"}],
          "max_tokens": 1024
        }
      },
      "Output": {
        "analysis": "{% $states.result.Body.content[0].text %}",
        "originalProblem": "{% $states.input.problem %}"
      },
      "Next": "Step 2 - Generate Plan"
    },
```

```
    "Step 2 - Generate Plan": {
      "Type": "Task",
      "Resource": "arn:aws:states:::bedrock:invokeModel",
      "Arguments": {
        "ModelId": "anthropic.claude-sonnet-4-20250514",
        "Body": {
          "messages": [{"role": "user", "content": "Based on this analysis: {% $states.input.analysis %}\nGenerat
          "max_tokens": 1024
        }
      },
      "Output": {
        "plan": "{% $states.result.Body.content[0].text %}",
        "analysis": "{% $states.input.analysis %}"
      },
      "Next": "Step 3 - Execute Plan"
    },
    "Step 3 - Execute Plan": {
      "Type": "Task",
      "Resource": "arn:aws:states:::bedrock:invokeModel",
      "Arguments": {
        "ModelId": "anthropic.claude-sonnet-4-20250514",
        "Body": {
          "messages": [{"role": "user", "content": "Execute this plan: {% $states.input.plan %}"}],
          "max_tokens": 2048
        }
      },
      "Next": "Step 4 - Verify Result"
    },
    "Step 4 - Verify Result": {
      "Type": "Task",
      "Resource": "arn:aws:states:::bedrock:invokeModel",
      "Arguments": {
        "ModelId": "anthropic.claude-sonnet-4-20250514",
        "Body": {
          "messages": [{"role": "user", "content": "Verify and critique: {% $states.input.result %}"}],
          "max_tokens": 1024
        }
      },
      "End": true
    }
  }
}
```

1.8 Prompt Chaining with Step Functions

The AWS Bedrock Serverless Prompt Chaining reference architecture demonstrates six patterns:

Pattern	Description	Step Functions Feature
Sequential chain	Write a novel analysis (prompt 1 output feeds prompt 2)	Linear state sequence
Iterative processing	Generate a short story (process a list of AI-generated items)	Map state
Parallel prompts	Create a vacation itinerary (parallelize distinct prompts)	Parallel state
Backtracking	Pitch movie ideas (go back to a previous step)	Choice state with loop

Pattern	Description	Step Functions Feature
Human input	Movie producer approval (pause for human decision)	`.waitForTaskToken`
Multi-agent chain	Find trending GitHub repos (chain multiple AI agents with APIs)	Sequential tasks with Lambda

Parallel prompt execution:

```
{
  "Generate Multiple Perspectives": {
    "Type": "Parallel",
    "Branches": [
      {
        "StartAt": "Optimistic Analysis",
        "States": {
          "Optimistic Analysis": {
            "Type": "Task",
            "Resource": "arn:aws:states:::bedrock:invokeModel",
            "Arguments": {
              "ModelId": "anthropic.claude-sonnet-4-20250514",
              "Body": {
                "messages": [{"role": "user", "content": "Give an optimistic analysis of: {% $states.input.topic
              }
            },
            "End": true
          }
        }
      },
      {
        "StartAt": "Critical Analysis",
        "States": {
          "Critical Analysis": {
            "Type": "Task",
            "Resource": "arn:aws:states:::bedrock:invokeModel",
            "Arguments": {
              "ModelId": "anthropic.claude-sonnet-4-20250514",
              "Body": {
                "messages": [{"role": "user", "content": "Give a critical analysis of: {% $states.input.topic %}'
              }
            },
            "End": true
          }
        }
      }
    ],
    "Next": "Synthesize Perspectives"
  }
}
```

Exam Gotcha: Step Functions prompt chaining uses the **optimized Bedrock integration** (no Lambda needed). This is different from Bedrock Prompt Flows (which uses a visual flow builder within the Bedrock console). The exam may ask you to choose between them: use Step Functions for complex orchestration with error handling, human approval, and conditional branching; use Prompt Flows for simpler linear/branching flows managed entirely within Bedrock.

1.9 Dynamic Model Routing

Task 2.4.4 tests dynamic content-based routing. Step Functions can route requests to different models based on input characteristics.

```
{
  "Route by Complexity": {
    "Type": "Choice",
    "Choices": [
      {
        "Variable": "$.complexity",
        "StringEquals": "simple",
        "Next": "Invoke Haiku"
      },
      {
        "Variable": "$.complexity",
        "StringEquals": "complex",
        "Next": "Invoke Sonnet"
      },
      {
        "Variable": "$.inputTokenCount",
        "NumericGreaterThan": 100000,
        "Next": "Invoke Long Context Model"
      }
    ],
    "Default": "Invoke Sonnet"
  }
}
```

Dynamic routing strategies for GenAI:

- **Cost-based**: Route simple queries to cheaper models (Haiku/Nova Micro), complex to capable models (Sonnet/Nova Pro)
- **Latency-based**: Route real-time chat to fast models, batch processing to powerful models
- **Content-based**: Route code questions to code-specialized models, creative writing to general models
- **Fallback**: If primary model throttles, fall back to secondary model (combine with circuit breaker)
- **A/B testing**: Route percentage of traffic to different models for evaluation

1.10 Error Handling (Catch, Retry with Backoff)

Step Functions provides sophisticated error handling that is critical for resilient AI systems.

Retry fields:

Field	Default	Description
`ErrorEquals`	Required	Array of error names to match
`IntervalSeconds`	1	Seconds before first retry
`MaxAttempts`	3	Maximum number of retries
`BackoffRate`	2.0	Multiplier for interval between retries
`MaxDelaySeconds`	None	Cap on the exponential backoff delay
`JitterStrategy`	`NONE`	`FULL` randomizes delays to prevent thundering herd

Built-in error names relevant to GenAI workloads:

Error Name	Description
`States.ALL`	Wildcard for any error (except `States.DataLimitExceeded` and `States.Runtime`)
`States.TaskFailed`	Wildcard for task failures (except `States.Timeout`)
`States.Timeout`	Task exceeded `TimeoutSeconds` or missed `HeartbeatSeconds`
`States.HeartbeatTimeout`	Task failed to send heartbeat within `HeartbeatSeconds`
`States.DataLimitExceeded`	Payload exceeded 256 KiB (terminal, cannot be caught by `States.ALL`)
`States.Runtime`	Runtime processing error (terminal, cannot be caught by `States.ALL`)

Retry pattern for Bedrock throttling:

```
{
  "Invoke Model": {
    "Type": "Task",
    "Resource": "arn:aws:states:::bedrock:invokeModel",
    "Arguments": {
      "ModelId": "anthropic.claude-sonnet-4-20250514",
      "Body": { "messages": [{"role": "user", "content": "{% $states.input.prompt %}"}] }
    },
    "Retry": [
      {
        "ErrorEquals": ["Bedrock.ThrottlingException"],
        "IntervalSeconds": 3,
        "MaxAttempts": 5,
        "BackoffRate": 2,
        "JitterStrategy": "FULL",
        "MaxDelaySeconds": 30
      },
      {
        "ErrorEquals": ["States.Timeout"],
        "IntervalSeconds": 5,
        "MaxAttempts": 2,
        "BackoffRate": 1
      },
      {
        "ErrorEquals": ["States.ALL"],
        "IntervalSeconds": 1,
        "MaxAttempts": 2,
        "BackoffRate": 2
      }
    ],
    "Catch": [
      {
        "ErrorEquals": ["Bedrock.ThrottlingException"],
        "Next": "Fallback to Secondary Model"
      },
      {
        "ErrorEquals": ["States.ALL"],
        "Next": "Handle Error"
      }
    ],
    "TimeoutSeconds": 120,
    "Next": "Process Response"
  }
}
```

Exam Gotcha: Retriers are evaluated **in order**. Put specific errors first, `States.ALL` last. `States.ALL` does NOT catch `States.DataLimitExceeded` or `States.Runtime` -- these are terminal errors. Retries count as state transitions and affect billing for Standard Workflows.

Exam Gotcha: `JitterStrategy: "FULL"` randomizes the retry interval between 0 and the calculated backoff delay. This prevents the thundering herd problem when multiple concurrent executions all retry at the same interval.

1.11 Stopping Conditions and Timeout Mechanisms

Task 2.1.3 specifically tests stopping conditions for AI workflows.

Available mechanisms:

Mechanism	Scope	Behavior
`TimeoutSeconds` (state level)	Individual task	Task fails with `States.Timeout` if it exceeds this duration
`TimeoutSeconds` (top level)	Entire state machine	Entire execution fails if total duration exceeds this
`HeartbeatSeconds`	Task with callback	Task fails with `States.HeartbeatTimeout` if no heartbeat received
`MaxAttempts` (in Retry)	Retry loop	Stop retrying after N attempts
Counter + Choice state	Reasoning loop	Manual iteration limit for ReAct/CoT patterns

Iteration limit pattern for ReAct loops:

```
{
  "Check Iteration Limit": {
    "Type": "Choice",
    "Choices": [
      {
        "Variable": "$.iterationCount",
        "NumericGreaterThanEquals": 10,
        "Next": "Force Final Answer"
      }
    ],
    "Default": "Continue Reasoning"
  },
  "Continue Reasoning": {
    "Type": "Task",
    "Resource": "arn:aws:states:::bedrock:invokeModel",
    "Arguments": {
      "ModelId": "anthropic.claude-sonnet-4-20250514",
      "Body": {
        "messages": [{"role": "user", "content": "{% $states.input.context %}"}]
      }
    },
    "Next": "Increment Counter"
  },
  "Increment Counter": {
    "Type": "Pass",
    "Arguments": {
      "iterationCount": "{% $states.input.iterationCount + 1 %}",
      "context": "{% $states.input.context %}"
    },
    "Next": "Check Iteration Limit"
  }
}
```

1.12 Document Processing Workflows

Task 2.5.3 tests document processing systems. Step Functions orchestrates multi-step document pipelines.

Typical document processing workflow:

1. **Event trigger**: S3 upload triggers EventBridge rule, which starts Step Functions
2. **Document classification**: Bedrock classifies document type (invoice, contract, report)
3. **Text extraction**: Amazon Textract extracts text and tables (`.sync` pattern to wait for completion)
4. **Entity extraction**: Bedrock or Comprehend extracts named entities
5. **Summarization**: Bedrock generates a summary
6. **Quality check**: Parallel state runs multiple validation checks
7. **Storage**: Write results to OpenSearch for search, DynamoDB for metadata

Distributed Map for bulk processing:

```
{
  "Process Document Batch": {
    "Type": "Map",
    "ItemProcessor": {
      "ProcessorConfig": {
        "Mode": "DISTRIBUTED",
        "ExecutionType": "EXPRESS"
      },
      "StartAt": "Process Single Document",
      "States": {
        "Process Single Document": {
          "Type": "Task",
          "Resource": "arn:aws:states:::bedrock:invokeModel",
          "Arguments": {
            "ModelId": "anthropic.claude-sonnet-4-20250514",
            "Body": {
              "messages": [{"role": "user", "content": "Extract key data from: {% $states.input.documentText %}"]
            }
          },
          "End": true
        }
      }
    },
    "ItemReader": {
      "Resource": "arn:aws:states:::s3:getObject",
      "Parameters": {
        "Bucket": "my-documents-bucket",
        "Key": "manifest.json"
      }
    },
    "MaxConcurrency": 100,
    "ToleratedFailurePercentage": 5,
    "Next": "Aggregate Results"
  }
}
```

Exam Gotcha: Distributed Map state can launch up to 10,000 parallel child executions (Express Workflows) for bulk document processing. `ToleratedFailurePercentage` allows the overall workflow to succeed even if some documents fail.

1.13 Key Metrics for Step Functions

Metric	Description	Alarm Threshold Guidance
`ExecutionsFailed`	Number of failed executions	> 0 for critical workflows
`ExecutionsTimedOut`	Executions that hit timeout	Monitor for AI workload latency
`ExecutionThrottled`	Executions throttled by quotas	Indicates capacity issues
`ExecutionTime`	Duration of execution	Monitor P99 for SLA compliance
`LambdaFunctionsFailed`	Lambda failures within workflow	Correlate with Bedrock throttling

---

2. Amazon EventBridge

2.1 Event-Driven AI Architectures

Amazon EventBridge is a **serverless event bus** that routes events from sources to targets based on rules. It is the backbone of event-driven AI architectures (Task 2.3.2).

Core concepts:

- **Event bus**: A pipeline that receives events (default bus, custom bus, partner bus)
- **Rules**: Pattern-match on events and route to targets
- **Targets**: What gets invoked when a rule matches (Lambda, Step Functions, SQS, SNS, etc.)
- **Event patterns**: JSON-based filters that match event structure
- **Schema registry**: Automatically discovers and stores event schemas

2.2 Event Patterns for GenAI Applications

S3 upload triggers AI processing:

```
{
  "source": ["aws.s3"],
  "detail-type": ["Object Created"],
  "detail": {
    "bucket": {
      "name": ["documents-for-processing"]
    },
    "object": {
      "key": [{ "suffix": ".pdf" }]
    }
  }
}
```

Bedrock model customization job completion:

```
{
  "source": ["aws.bedrock"],
  "detail-type": ["Bedrock Model Customization Job State Change"],
  "detail": {
    "status": ["Completed", "Failed"]
  }
}
```

SageMaker training job completion (for custom models):

```
{
```

```
  "source": ["aws.sagemaker"],
  "detail-type": ["SageMaker Training Job State Change"],
  "detail": {
    "TrainingJobStatus": ["Completed", "Failed"]
  }
}
```

2.3 Integration with Bedrock, Lambda, Step Functions

Common GenAI event-driven patterns:

Trigger	EventBridge Rule	Target	Use Case
S3 document upload	Match `Object Created`	Step Functions	Document processing pipeline
Bedrock job complete	Match status change	Lambda	Post-processing fine-tuned model
CloudWatch alarm	Match alarm state	SNS + Lambda	Model performance degradation alert
Custom app event	Match custom detail-type	Step Functions	User request triggers AI workflow
Scheduled rule	Cron/rate expression	Lambda	Periodic knowledge base refresh

TypeScript SDK v3 -- Creating an EventBridge rule for S3 -> Step Functions:

```
import {
  EventBridgeClient,
  PutRuleCommand,
  PutTargetsCommand,
} from "@aws-sdk/client-eventbridge";

const ebClient = new EventBridgeClient({ region: "us-east-1" });

await ebClient.send(
  new PutRuleCommand({
    Name: "DocumentUploadRule",
    EventPattern: JSON.stringify({
      source: ["aws.s3"],
      "detail-type": ["Object Created"],
      detail: {
        bucket: { name: ["ai-documents"] },
        object: { key: [{ suffix: ".pdf" }] },
      },
    }),
    State: "ENABLED",
  })
);

await ebClient.send(
  new PutTargetsCommand({
    Rule: "DocumentUploadRule",
    Targets: [
      {
        Id: "StepFunctionsTarget",
        Arn: "arn:aws:states:us-east-1:123456789012:stateMachine:DocProcessing",
        RoleArn: "arn:aws:iam::123456789012:role/EventBridgeStepFunctionsRole",
      },
    ],
  })
);
```

2.4 Exam Gotchas for EventBridge

- EventBridge rules match on **event patterns** (content-based filtering), not message attributes like SNS
- EventBridge can have up to **5 targets per rule** (use multiple rules for more)
- EventBridge **guarantees at-least-once delivery** to targets
- EventBridge has a **built-in archive and replay** feature for reprocessing events
- Step Functions is an **optimized integration** target for EventBridge (can pass the full event as input)
- For cross-account event routing, use **cross-account event bus** patterns

---

3. Amazon SQS

3.1 Async Inference Patterns

Amazon SQS decouples AI request producers from consumers, enabling asynchronous processing (Task 2.4.1).

Standard Queue vs FIFO Queue for AI Workloads:

Feature	Standard	FIFO
Throughput	Nearly unlimited	300 msg/s (3,000 with batching, higher with high throughput mode)
Ordering	Best-effort	Strict ordering within message group
Delivery	At-least-once (possible duplicates)	Exactly-once processing
Use case	High-volume inference, document processing	Ordered conversation turns, sequential processing
AI example	Batch image generation requests	Chat messages that must be processed in order

Async inference pattern with SQS:

```
Producer -&gt; SQS Queue -&gt; Lambda (consumer) -&gt; Bedrock InvokeModel -&gt; Result Store (DynamoDB/S3)
                 |
                 v
           Dead Letter Queue (failed requests)
```

TypeScript SDK v3 -- Sending AI inference request to SQS:

```
import { SQSClient, SendMessageCommand } from "@aws-sdk/client-sqs";

const sqsClient = new SQSClient({ region: "us-east-1" });

await sqsClient.send(
  new SendMessageCommand({
    QueueUrl: "https://sqs.us-east-1.amazonaws.com/123456789012/ai-inference-queue",
    MessageBody: JSON.stringify({
      requestId: "req-12345",
      prompt: "Generate a product description for...",
      modelId: "anthropic.claude-sonnet-4-20250514",
      callbackUrl: "https://api.example.com/results",
    }),
    MessageAttributes: {
      Priority: {
        DataType: "String",
        StringValue: "high",
```

```
        },
      },
    })
  );
```

3.2 Dead Letter Queues for Failed AI Requests

DLQs capture messages that fail processing after a configured number of attempts.

Configuration:

- Set `maxReceiveCount` in the **redrive policy** (e.g., 3 attempts before DLQ)
- DLQ retention period should be **longer** than the source queue retention
- Use CloudWatch alarm on `ApproximateNumberOfMessagesVisible` in the DLQ to alert on failures

Redrive policy setup:

```
import { SQSClient, SetQueueAttributesCommand } from "@aws-sdk/client-sqs";

const sqsClient = new SQSClient({ region: "us-east-1" });

await sqsClient.send(
  new SetQueueAttributesCommand({
    QueueUrl: "https://sqs.us-east-1.amazonaws.com/123456789012/ai-inference-queue",
    Attributes: {
      RedrivePolicy: JSON.stringify({
        deadLetterTargetArn:
          "arn:aws:sqs:us-east-1:123456789012:ai-inference-dlq",
        maxReceiveCount: "3",
      }),
    },
  })
);
```

GenAI-specific DLQ scenarios:

- Bedrock throttling exhausts retries -- message goes to DLQ for later reprocessing
- Malformed prompt causes repeated model errors -- DLQ captures for investigation
- Payload too large for model context window -- DLQ captures for chunking/retry with different approach

Exam Gotcha: DLQ for FIFO queues must also be FIFO. DLQ for Standard queues must be Standard. Using a DLQ with a FIFO queue can break message ordering, so only use when order is not critical for failed messages.

3.3 SQS with Step Functions

SQS has an **optimized integration** with Step Functions supporting both Request Response and Wait for Callback patterns.

- **Request Response**: `arn:aws:states:::sqs:sendMessage` -- sends a message and continues
- **Wait for Callback**: `arn:aws:states:::sqs:sendMessage.waitForTaskToken` -- sends message with task token, pauses until callback

3.4 Key Metrics for SQS

Metric	Description	Alarm Guidance
`ApproximateNumberOfMessagesVisible`	Messages available for processing	High count = consumers can't keep up

Metric	Description	Alarm Guidance
`ApproximateAgeOfOldestMessage`	Age of oldest message	High age = processing backlog
`NumberOfMessagesSent`	Messages sent to queue	Baseline for throughput monitoring
`ApproximateNumberOfMessagesNotVisible`	Messages being processed (in-flight)	Monitor against SQS limits
DLQ `ApproximateNumberOfMessagesVisible`	Failed messages	> 0 triggers investigation

---

4. Amazon SNS

4.1 Notification Patterns for AI Workflows

Amazon SNS is a **pub/sub messaging service** for fan-out notifications and decoupled communication.

GenAI notification patterns:

- **Model completion notification**: SNS topic receives completion event, fans out to multiple subscribers (email, Slack webhook via Lambda, dashboard update)
- **Error alerting**: CloudWatch alarms publish to SNS for AI pipeline failures
- **Human review notification**: Step Functions sends task token + content to SNS, subscriber (reviewer) receives email/SMS

4.2 Fan-Out Patterns

SNS + SQS fan-out for parallel AI processing:

```
                              ■■■ SQS Queue A ■■ Lambda (Summarization)
                              ■
SNS Topic ■■ Subscription ■■■■ SQS Queue B ■■ Lambda (Translation)
                              ■
                              ■■■ SQS Queue C ■■ Lambda (Sentiment Analysis)
```

Each SQS queue subscribes to the same SNS topic. When a document is published to the topic, all three queues receive a copy and process it independently.

Message filtering for targeted processing:

```
import { SNSClient, SubscribeCommand } from "@aws-sdk/client-sns";

const snsClient = new SNSClient({ region: "us-east-1" });

await snsClient.send(
  new SubscribeCommand({
    TopicArn: "arn:aws:sns:us-east-1:123456789012:ai-results",
    Protocol: "sqs",
    Endpoint: "arn:aws:sqs:us-east-1:123456789012:high-priority-results",
    Attributes: {
      FilterPolicy: JSON.stringify({
        priority: ["high"],
        resultType: ["summary", "analysis"],
      }),
    },
```

```
    })
  );
```

4.3 SNS with Step Functions

SNS has an **optimized integration** with Step Functions:

- **Request Response**: `arn:aws:states:::sns:publish` -- publish and continue
- **Wait for Callback**: `arn:aws:states:::sns:publish.waitForTaskToken` -- publish with task token, pause

Exam Gotcha: SNS does NOT support `.sync` (Run a Job) pattern -- only Request Response and Wait for Callback. This makes sense because publishing a notification is not a "job" that completes.

4.4 Key Metrics for SNS

Metric	Description
`NumberOfMessagesPublished`	Messages published to topic
`NumberOfNotificationsDelivered`	Successful deliveries
`NumberOfNotificationsFailed`	Failed deliveries
`PublishSize`	Size of published messages

---

5. Amazon AppFlow

5.1 Data Integration for Knowledge Bases

Amazon AppFlow is a **fully managed data integration service** that securely transfers data between SaaS applications and AWS services. In the GenAI context, it is primarily used to ingest data into S3 for Knowledge Base indexing.

How it fits in GenAI architectures:

```
SaaS Source (Salesforce, Slack, etc.) → AppFlow → S3 Bucket → Bedrock Knowledge Base
```

5.2 Connector Types

Supported sources (80+ connectors): Salesforce, Google Analytics, Google BigQuery, Slack, ServiceNow, Zendesk, Jira Cloud, HubSpot, Marketo, SAP OData, Microsoft Dynamics 365, Microsoft SharePoint Online, GitHub, Snowflake, Datadog, and many more.

Supported destinations: Amazon S3, Amazon Redshift, Amazon Lookout for Metrics, Amazon RDS for PostgreSQL, Salesforce, Snowflake, Upsolver, Zendesk, and others.

Custom connectors: Build your own using the AppFlow Custom Connector SDK (Python/Java) for unsupported SaaS applications.

Flow trigger types:

Type	Description	GenAI Use Case
On demand	Manual trigger	One-time data backfill for knowledge base

Type	Description	GenAI Use Case
Scheduled	Cron-based trigger	Periodic refresh of knowledge base data
Event-driven	Triggered by source events	Real-time sync (Salesforce CDC)

Data transformations within AppFlow:

- Field mapping and renaming
- Data masking and truncation
- Field validation (required, type checking)
- Filtering by field value
- Merging fields

TypeScript SDK v3 -- Creating an AppFlow flow:

```
import { AppflowClient, CreateFlowCommand } from "@aws-sdk/client-appflow";

const appflowClient = new AppflowClient({ region: "us-east-1" });

await appflowClient.send(
  new CreateFlowCommand({
    flowName: "salesforce-to-knowledge-base",
    triggerConfig: {
      triggerType: "Scheduled",
      triggerProperties: {
        Scheduled: {
          scheduleExpression: "rate(1 day)",
          dataPullMode: "Incremental",
        },
      },
    },
    sourceFlowConfig: {
      connectorType: "Salesforce",
      connectorProfileName: "my-salesforce-profile",
      sourceConnectorProperties: {
        Salesforce: { object: "Knowledge__kav" },
      },
    },
    destinationFlowConfigList: [
      {
        connectorType: "S3",
        destinationConnectorProperties: {
          S3: {
            bucketName: "my-knowledge-base-bucket",
            bucketPrefix: "salesforce-articles",
            s3OutputFormatConfig: {
              fileType: "JSON",
            },
          },
        },
      },
    ],
    tasks: [
      {
        sourceFields: ["Title", "Summary", "ArticleBody"],
        taskType: "Map_all",
        connectorOperator: { Salesforce: "NO_OP" },
      },
    ],
  })
```

```
);
```

5.3 Exam Gotchas for AppFlow

- AppFlow transfers data **into AWS** from SaaS (and vice versa) -- it is NOT a general ETL tool (that is AWS Glue)
- AppFlow supports **incremental data pull** for scheduled flows, transferring only changed records
- AppFlow encrypts data in transit and supports **AWS PrivateLink** for private connectivity
- For GenAI, AppFlow is the answer when the question asks about getting **SaaS data into S3** for knowledge base ingestion
- AppFlow does NOT directly integrate with Bedrock -- it writes to S3, then Knowledge Base indexes from S3

---

6. AWS AppConfig

6.1 Feature Flags for Model Selection

AWS AppConfig (part of AWS Systems Manager) enables **dynamic configuration changes** without code deployments. For GenAI, it is specifically tested for dynamic model selection (Task 1.2.2).

Core concepts:

- **Application**: Organizational namespace
- **Environment**: Logical deployment target (dev, staging, prod)
- **Configuration profile**: Contains the configuration data (feature flags or freeform)
- **Deployment strategy**: Controls how configuration changes roll out
- **Hosted configuration store**: AWS-managed storage for config data

6.2 Dynamic Configuration Without Code Changes

Feature flag for model selection:

```
{
  "version": "1",
  "flags": {
    "primaryModel": {
      "name": "Primary Model Selection",
      "description": "Controls which FM is used for primary inference",
      "attributes": {
        "modelId": {
          "constraints": {
            "type": "string",
            "enum": [
              "anthropic.claude-sonnet-4-20250514",
              "anthropic.claude-haiku-4-20250514",
              "amazon.nova-pro-v1:0",
              "amazon.nova-micro-v1:0"
            ]
          }
        },
        "maxTokens": {
          "constraints": {
            "type": "number",
            "minimum": 100,
            "maximum": 4096
```

```
            }
          },
          "temperature": {
            "constraints": {
              "type": "number",
              "minimum": 0,
              "maximum": 1
            }
          }
        }
      },
      "fallbackModel": {
        "name": "Fallback Model",
        "description": "Model to use when primary is unavailable"
      },
      "guardrailsEnabled": {
        "name": "Guardrails Enabled",
        "description": "Toggle Bedrock Guardrails on/off"
      }
    },
    "values": {
      "primaryModel": {
        "enabled": true,
        "modelId": "anthropic.claude-sonnet-4-20250514",
        "maxTokens": 1024,
        "temperature": 0.7
      },
      "fallbackModel": {
        "enabled": true,
        "modelId": "amazon.nova-micro-v1:0"
      },
      "guardrailsEnabled": {
        "enabled": true
      }
    }
  }
```

TypeScript SDK v3 -- Retrieving AppConfig configuration at runtime:

```
import {
  AppConfigDataClient,
  StartConfigurationSessionCommand,
  GetLatestConfigurationCommand,
} from "@aws-sdk/client-appconfigdata";

const appConfigClient = new AppConfigDataClient({ region: "us-east-1" });

// Start a configuration session (do this once per environment)
const session = await appConfigClient.send(
  new StartConfigurationSessionCommand({
    ApplicationIdentifier: "my-genai-app",
    EnvironmentIdentifier: "production",
    ConfigurationProfileIdentifier: "model-selection",
    RequiredMinimumPollIntervalInSeconds: 60,
  })
);

// Periodically poll for latest configuration
const config = await appConfigClient.send(
  new GetLatestConfigurationCommand({
```

```
    ConfigurationToken: session.InitialConfigurationToken,
  })
);

const modelConfig = JSON.parse(
  new TextDecoder().decode(config.Configuration)
);

// Use in your Bedrock call
const modelId = modelConfig.primaryModel.modelId;
const maxTokens = modelConfig.primaryModel.maxTokens;
```

6.3 Deployment Strategies

AppConfig supports controlled rollouts to minimize risk of bad configuration changes.

Predefined strategies:

Strategy	Growth	Duration	Bake Time	Use Case
`AppConfig.AllAtOnce`	100%	0 min	10 min	Instant rollout for non-critical changes
`AppConfig.Linear50PercentEvery30Seconds`	50%	1 min	1 min	Quick rollout with minimal bake
`AppConfig.Linear20PercentEvery6Minutes`	20%	30 min	30 min	Gradual rollout for important changes
`AppConfig.Canary10Percent20Minutes`	10% then 100%	20 min	10 min	Canary deployment for risky changes

Custom deployment strategy:

Field	Description
`GrowthType`	`LINEAR` (even distribution) or `EXPONENTIAL` (percentage-based growth)
`GrowthFactor`	Percentage of targets to include in each step
`DeploymentDurationInMinutes`	Total time for deployment
`FinalBakeTimeInMinutes`	Monitoring period after 100% deployment
`ReplicateTo`	Where to replicate (currently only `NONE` or `SSM_DOCUMENT`)

Automatic rollback: Configure CloudWatch alarms as monitors. If an alarm triggers during deployment, AppConfig automatically rolls back the configuration change.

GenAI scenarios for AppConfig:

- **A/B model testing**: Deploy a new model ID to 10% of traffic (canary), monitor quality metrics, then expand
- **Prompt template updates**: Update system prompts without redeploying application code
- **Guardrail tuning**: Adjust guardrail thresholds dynamically based on observed false positive rates
- **Feature toggles**: Enable/disable RAG, streaming, or specific model capabilities per environment
- **Cost management**: Switch to cheaper models during low-priority hours without deployment

6.4 Exam Gotchas for AppConfig

- AppConfig is NOT the same as AWS Config (which audits resource compliance)
- AppConfig supports **validation** via JSON Schema or Lambda validators before deployment
- Configuration polling uses a session-based model: `StartConfigurationSession` then `GetLatestConfiguration` in a loop
- The `GetLatestConfiguration` call returns an empty body if nothing changed since the last poll (this is normal, not an error)
- AppConfig integrates with **CloudWatch alarms** for automatic rollback during bad deployments
- For the exam, AppConfig is the answer when asked about **changing model parameters or model selection at runtime without code changes**

---

7. Cross-Service Integration Patterns for GenAI

7.1 End-to-End Document Processing Pipeline

```
S3 Upload → EventBridge Rule → Step Functions (Standard) →
  ■■■ Textract (OCR, .sync) →
  ■■■ Bedrock (Classify Document) →
  ■■■ Bedrock (Extract Entities) →
  ■■■ Bedrock (Summarize) →
  ■■■ Choice (Confidence Check) →
  ■     ■■■ High Confidence → Store Results (DynamoDB)
  ■     ■■■ Low Confidence → SQS (Human Review Queue) → .waitForTaskToken
  ■■■ SNS (Notify Completion)
```

7.2 Resilient Multi-Model Inference

```
API Gateway → Lambda →
  ■■■ AppConfig (Get Model Config) →
  ■■■ Step Functions (Express) →
  ■     ■■■ Circuit Breaker Check (DynamoDB) →
  ■     ■■■ Primary Model (Bedrock InvokeModel) →
  ■     ■     ■■■ Retry with backoff + jitter
  ■     ■     ■■■ Catch → Fallback Model (Bedrock InvokeModel)
  ■     ■■■ Response
  ■■■ CloudWatch Metrics
```

7.3 Event-Driven Knowledge Base Refresh

```
AppFlow (Scheduled, Salesforce → S3) →
  EventBridge (S3 Object Created) →
    Step Functions →
      ■■■ Bedrock (StartIngestionJob.sync) →
      ■■■ Wait for Indexing Complete →
      ■■■ SNS (Notify Team)
```

---

8. Quick-Reference Comparison Table

Service	Primary GenAI Role	Exam Weight	Key Integration Pattern
Step Functions	Workflow orchestration, prompt chaining, agent loops, human approval	Very High	Optimized Bedrock integration, `.waitForTaskToken`, Retry/Catch
EventBridge	Event routing, triggering AI pipelines	Medium	Event patterns, rule-based routing to Step Functions
SQS	Async decoupling, buffering AI requests	Medium	DLQ for failed inference, `.waitForTaskToken` with Step Functions
SNS	Notifications, fan-out	Low-Medium	Fan-out to multiple AI processors, human review notifications
AppFlow	SaaS data ingestion for Knowledge Bases	Low	Scheduled/event flows from SaaS to S3
AppConfig	Runtime model/config changes	Low-Medium	Feature flags for model selection, deployment strategies

---

9. Exam Strategy Tips

1. **Step Functions is almost always the answer** when the question involves orchestrating multiple AI steps, error handling, human approval, or deterministic workflows. If you see "audit trail", "exactly-once", or "visual workflow", it is Step Functions.
2. **Step Functions vs Bedrock Agents**: Step Functions for deterministic, developer-defined flows with full auditability. Bedrock Agents for autonomous, LLM-driven orchestration with semantic flexibility. The exam will present scenarios where one is clearly better than the other.
3. **Step Functions vs Bedrock Prompt Flows**: Step Functions for complex flows needing error handling, human-in-the-loop, conditional branching, and integration with non-Bedrock services. Prompt Flows for simpler prompt chains managed entirely within Bedrock.
4. **Circuit breaker pattern**: If the question mentions "resilient AI systems", "prevent cascading failures", or "handle model unavailability gracefully", the answer involves Step Functions + DynamoDB circuit breaker.
5. `.waitForTaskToken`: If the question mentions human review, external approval, or pausing for an async callback, this is the pattern. Only available in Standard Workflows.
6. **AppConfig vs environment variables**: If the question asks about changing model parameters "without redeploying" or "without code changes", AppConfig is the answer (not Lambda environment variables, which require a deployment).
7. **SQS DLQ**: If the question asks about handling failed AI inference requests for later reprocessing, SQS with a dead-letter queue is the answer.
8. **AppFlow**: If the question mentions ingesting data from Salesforce, ServiceNow, or other SaaS into S3 for a knowledge base, AppFlow is the answer.
9. **EventBridge**: If the question mentions reacting to S3 uploads, model training completion events, or building event-driven AI architectures, EventBridge is the answer. It is the "glue" that starts AI pipelines based on events.
10. **Error handling order**: Retriers are evaluated in order (specific errors first, `States.ALL` last). `States.ALL` does NOT catch `States.DataLimitExceeded` or `States.Runtime`. Use `JitterStrategy: "FULL"` for concurrent retry scenarios.

Compute Services for Generative AI -- Study Guide

Exam: AWS Certified Generative AI Developer - Professional (AIP-C01)

Lambda is the single most referenced service on this exam. It appears in 14+ task statements across all three domains -- from data processing to agentic orchestration to guardrail enforcement. EC2 GPU instances, App Runner, Outposts, and Wavelength appear in targeted scenarios around inference hardware, containers, data sovereignty, and edge deployment.

---

1. AWS Lambda (Primary Focus)

1.1 Overview and Key Limits

Lambda runs code without provisioning servers. For the exam, know these hard limits cold:

Resource	Limit	Notes
Timeout	900 seconds (15 min)	Default is 3 seconds. Bedrock calls need 30-120s.
Memory	128 MB - 10,240 MB	CPU scales proportionally with memory.
Ephemeral storage (/tmp)	512 MB - 10,240 MB	Useful for temp files during data processing.
Sync invocation payload	6 MB request / 6 MB response	Key limit for Bedrock responses.
Async invocation payload	256 KB	For event-driven patterns.
Streaming response payload	200 MB	Via function URLs or InvokeWithResponseStream.
Deployment package (.zip)	50 MB zipped / 250 MB unzipped	Container images up to 10 GB.
Layers	5 per function	Combined unzipped size counts toward 250 MB.
Concurrency (account default)	1,000 per Region	Can be increased.
Concurrency scaling rate	1,000 new environments per 10 sec	Per-function burst rate.
Environment variables	4 KB total	Use SSM/Secrets Manager for larger config.

Exam Gotcha: The default timeout of 3 seconds will cause Bedrock InvokeModel calls to fail. Always set timeout to at least 30-60 seconds for FM invocations. For agentic workflows with multiple tool calls, consider 300-900 seconds.

Exam Gotcha: The 6 MB synchronous response limit is bypassed by response streaming (200 MB). If a question mentions "large model responses" or "streaming tokens", response streaming is the answer.

Key CloudWatch Metrics:

- `Invocations` -- total invocation count
- `Duration` -- execution time (p50, p99)
- `Errors` -- function errors
- `Throttles` -- invocations rejected due to concurrency limits
- `ConcurrentExecutions` -- current in-flight executions
- `IteratorAge` -- lag for stream-based event sources (Kinesis, DynamoDB)

1.2 Lambda for GenAI: Invoking Bedrock

Lambda is the primary compute for on-demand FM invocation (Task 2.2.1). There are two API patterns to know:

Pattern 1: InvokeModel (model-specific payloads)

```
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

export const handler = async (event: { prompt: string }) =&gt; {
  const payload = {
    anthropic_version: "bedrock-2023-05-31",
    max_tokens: 1024,
    messages: [
      { role: "user", content: [{ type: "text", text: event.prompt }] },
    ],
  };

  const response = await client.send(
    new InvokeModelCommand({
      modelId: "anthropic.claude-sonnet-4-20250514",
      contentType: "application/json",
      body: JSON.stringify(payload),
    })
  );

  const body = JSON.parse(new TextDecoder().decode(response.body));
  return { statusCode: 200, body: body.content[0].text };
};
```

Pattern 2: Converse API (model-agnostic, preferred)

The Converse API provides a unified interface across all Bedrock models. This is the recommended approach for dynamic model selection (Task 1.2.2) because you swap the `modelId` without changing the request schema.

```
import {
  BedrockRuntimeClient,
  ConverseCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

export const handler = async (event: { prompt: string; modelId: string }) =&gt; {
  const response = await client.send(
    new ConverseCommand({
      modelId: event.modelId,
      messages: [
        {
          role: "user",
          content: [{ text: event.prompt }],
        },
      ],
      inferenceConfig: {
        maxTokens: 1024,
        temperature: 0.7,
      },
    })
  );
```

```
  const text = response.output?.message?.content?.[0]?.text;
  return { statusCode: 200, body: text };
};
```

Exam Gotcha: The Converse API is the correct answer when a question asks about "model-agnostic invocation", "switching between models", or "consistent API across providers". InvokeModel is correct when the question involves model-specific features or a specific payload format.

1.3 Lambda Layers for ML Libraries

Lambda Layers let you package shared code and dependencies separately from function code. Each function can use up to 5 layers, and the total unzipped size (function + layers) must not exceed 250 MB.

GenAI-relevant use cases:

- **Powertools for AWS Lambda** -- available as a managed layer
- **Common SDKs** -- shared Bedrock client configuration
- **Data processing libraries** -- tokenizers, text-processing utilities
- **Validation schemas** -- shared Zod/JSON schemas for input validation

Layer packaging (Python example for ML libraries):

```
mkdir -p python
pip install tiktoken numpy -t python/
zip -r ml-layer.zip python/
aws lambda publish-layer-version \
  --layer-name ml-utilities \
  --zip-file fileb://ml-layer.zip \
  --compatible-runtimes python3.12
```

Exam Gotcha: Lambda Layers do NOT increase the 250 MB deployment limit -- they share it. For large ML frameworks, use container images (up to 10 GB). Layers also cannot be used with Lambda@Edge.

1.4 Lambda Streaming Responses

Response streaming is critical for GenAI workloads because FM responses are generated token-by-token. Streaming reduces Time to First Byte (TTFB) and supports payloads up to 200 MB (vs 6 MB buffered).

Three ways to stream Lambda responses:

1. **Lambda Function URLs** with `InvokeMode: RESPONSE_STREAM`
2. **InvokeWithResponseStream** API via AWS SDK
3. **API Gateway proxy integration** (uses InvokeWithResponseStream under the hood)

Bandwidth limits:

- First 6 MB: uncapped bandwidth
- Remainder: 2 MBps maximum

Native runtime support: Node.js managed runtimes only. For Python and other languages, use a custom runtime or Lambda Web Adapter.

VPC restriction: Lambda function URLs do NOT support response streaming within a VPC. Use `InvokeWithResponseStream` via SDK with a VPC endpoint instead.

```
// Streaming handler using awslambda.streamifyResponse()
import { pipeline } from "node:stream/promises";
```

```
import { Readable } from "node:stream";

export const handler = awslambda.streamifyResponse(
  async (event, responseStream, _context) =&gt; {
    // Write tokens as they arrive from Bedrock
    responseStream.write("Starting generation...\n");

    // Simulate streaming FM output
    for (const chunk of ["Hello", " world", " from", " Lambda!"]) {
      responseStream.write(chunk);
      await new Promise((r) =&gt; setTimeout(r, 100));
    }

    responseStream.end();
  }
);
```

Streaming Bedrock responses through Lambda (real-world pattern):

```
import {
  BedrockRuntimeClient,
  ConverseStreamCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

export const handler = awslambda.streamifyResponse(
  async (event, responseStream, _context) =&gt; {
    const parsed = JSON.parse(event.body || "{}");

    const response = await client.send(
      new ConverseStreamCommand({
        modelId: "anthropic.claude-sonnet-4-20250514",
        messages: [{ role: "user", content: [{ text: parsed.prompt }] }],
        inferenceConfig: { maxTokens: 2048 },
      })
    );

    if (response.stream) {
      for await (const event of response.stream) {
        if (event.contentBlockDelta?.delta?.text) {
          responseStream.write(event.contentBlockDelta.delta.text);
        }
      }
    }

    responseStream.end();
  }
);
```

Exam Gotcha: Streaming responses are still billed for the full function duration, even if the client disconnects. The function is NOT interrupted when the caller drops the connection.

1.5 Lambda as MCP Server (Task 2.1.7)

The Model Context Protocol (MCP) defines a standard interface for AI models to access tools. Lambda is ideal for **stateless MCP servers** because:

- Each invocation is independent (no server to maintain)
- Auto-scales with demand
- IAM provides fine-grained access control per tool
- Pay only when tools are invoked

Architecture pattern:

```
Agent (Bedrock/Strands) --&gt; API Gateway / Function URL --&gt; Lambda (MCP Server)
                                                              |
                                                              +--&gt; DynamoDB
                                                              +--&gt; S3
                                                              +--&gt; External APIs
```

Lambda functions implementing MCP tools receive a standardized request (tool name + parameters) and return structured results. Powertools for AWS Lambda provides a `BedrockAgentFunctionResolver` for handling Bedrock Agent action group invocations, which follows the same tool-calling pattern.

```
// Lambda as a tool endpoint for an agent
export const handler = async (event: {
  tool: string;
  parameters: Record&lt;string, string&gt;;
}) =&gt; {
  switch (event.tool) {
    case "lookup_customer":
      return await lookupCustomer(event.parameters.customerId);
    case "check_inventory":
      return await checkInventory(event.parameters.productId);
    default:
      return { error: `Unknown tool: ${event.tool}` };
  }
};
```

Exam Gotcha: MCP servers on Lambda are *stateless*. If the tool requires maintaining conversation state or long-lived connections (e.g., SSE/WebSocket), use ECS/Fargate or App Runner instead. Lambda is the answer for stateless, on-demand tool execution.

1.6 Lambda for Data Processing

Lambda appears in nearly every data-processing task statement on the exam.

Fixed-size chunking (Task 1.5.1):

```
export function chunkText(text: string, chunkSize: number, overlap: number): string[] {
  const chunks: string[] = [];
  let start = 0;

  while (start &lt; text.length) {
    const end = Math.min(start + chunkSize, text.length);
    chunks.push(text.slice(start, end));
    start += chunkSize - overlap;
  }

  return chunks;
}

export const handler = async (event: { text: string }) =&gt; {
  const chunks = chunkText(event.text, 1000, 200);
```

```
  return { chunks, count: chunks.length };
};
```

Batch embedding generation (Task 1.5.2):

```
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const client = new BedrockRuntimeClient({ region: "us-east-1" });

async function generateEmbedding(text: string): Promise&lt;number[]&gt; {
  const response = await client.send(
    new InvokeModelCommand({
      modelId: "amazon.titan-embed-text-v2:0",
      contentType: "application/json",
      body: JSON.stringify({
        inputText: text,
        dimensions: 1024,
        normalize: true,
      }),
    })
  );

  const body = JSON.parse(new TextDecoder().decode(response.body));
  return body.embedding;
}

export const handler = async (event: { chunks: string[] }) =&gt; {
  // Process chunks in parallel with concurrency control
  const batchSize = 5;
  const embeddings: number[][] = [];

  for (let i = 0; i &lt; event.chunks.length; i += batchSize) {
    const batch = event.chunks.slice(i, i + batchSize);
    const results = await Promise.all(batch.map(generateEmbedding));
    embeddings.push(...results);
  }

  return { embeddings, count: embeddings.length };
};
```

Custom data validation (Task 1.3.1):

```
import { z } from "zod";

const TrainingDataSchema = z.object({
  prompt: z.string().min(1).max(10000),
  completion: z.string().min(1).max(10000),
  category: z.enum(["general", "code", "math", "creative"]),
});

export const handler = async (event: { records: unknown[] }) =&gt; {
  const valid: z.infer&lt;typeof TrainingDataSchema&gt;[] = [];
  const invalid: { index: number; errors: string[] }[] = [];

  event.records.forEach((record, index) =&gt; {
    const result = TrainingDataSchema.safeParse(record);
    if (result.success) {
```

```
      valid.push(result.data);
    } else {
      invalid.push({
        index,
        errors: result.error.errors.map((e) =&gt; e.message),
      });
    }
  });

  return { valid, invalid, validCount: valid.length, invalidCount: invalid.length };
};
```

Data normalization (Task 1.3.4) -- Lambda functions clean, lowercase, strip HTML, remove PII, and standardize formats before data enters the training or RAG pipeline.

1.7 Lambda for Guardrails: Pre/Post-Processing

Prompt verification (Task 1.6.4) -- validate and sanitize user prompts before they reach the FM:

```
export const handler = async (event: { prompt: string; userId: string }) =&gt; {
  // Check prompt length
  if (event.prompt.length &gt; 10000) {
    return { allowed: false, reason: "Prompt exceeds maximum length" };
  }

  // Check for injection patterns
  const injectionPatterns = [
    /ignore previous instructions/i,
    /you are now/i,
    /system:\s/i,
  ];

  for (const pattern of injectionPatterns) {
    if (pattern.test(event.prompt)) {
      return { allowed: false, reason: "Potential prompt injection detected" };
    }
  }

  return { allowed: true, sanitizedPrompt: event.prompt.trim() };
};
```

Post-processing validation (Task 3.1.4) -- validate FM outputs before returning to the user:

```
export const handler = async (event: { response: string; context: string }) =&gt; {
  // Check for hallucinated content
  const containsPII = /\b\d{3}-\d{2}-\d{4}\b/.test(event.response); // SSN pattern
  const containsEmail = /\b[\w.-]+@[\w.-]+\.\w+\b/.test(event.response);

  const issues: string[] = [];
  if (containsPII) issues.push("Response contains potential PII (SSN pattern)");
  if (containsEmail) issues.push("Response contains email addresses");

  if (issues.length &gt; 0) {
    return {
      approved: false,
      issues,
      redactedResponse: event.response
        .replace(/\b\d{3}-\d{2}-\d{4}\b/g, "[REDACTED-SSN]")
```

```
        .replace(/\b[\w.-]+@[\w.-]+\.\w+\b/g, "[REDACTED-EMAIL]"),
      };
    }

    return { approved: true, response: event.response };
  };
```

Automated compliance checks (Task 3.4.3) -- Lambda triggered by EventBridge or Step Functions to audit model outputs, log to CloudWatch, and flag violations.

1.8 Timeout and Concurrency for AI Workloads

Timeout strategy by workload type:

Workload	Recommended Timeout	Rationale
Simple FM invocation	30-60s	Single Converse call
RAG (retrieve + generate)	60-120s	KB lookup + FM call
Agentic workflow	120-300s	Multiple tool calls + FM reasoning
Batch embedding	300-900s	Processing many chunks
Data processing pipeline	300-900s	Large document chunking

Concurrency controls:

Control	Purpose	Use Case
Reserved concurrency	Guarantees AND caps concurrency for a function	Protect downstream Bedrock throttle limits
Provisioned concurrency	Pre-initializes execution environments	Eliminate cold starts for latency-sensitive APIs
Account concurrency limit	Total concurrent executions across all functions	Default 1,000; request increase for production

Reserved concurrency is the exam favorite. Setting reserved concurrency on your Bedrock-invoking Lambda function prevents it from consuming all account concurrency AND protects Bedrock's tokens-per-minute quota from being overwhelmed.

Exam Gotcha: Setting reserved concurrency to 0 effectively disables a function. Lambda sends events to the configured DLQ or on-failure destination instead.

Exam Gotcha: Provisioned concurrency and SnapStart cannot be used together.

1.9 Lambda SnapStart

SnapStart reduces cold start latency by snapshotting the initialized execution environment and restoring it on invocation. Instead of running initialization code on every cold start, Lambda restores from a cached snapshot.

Supported runtimes: Java 11+, Python 3.12+, .NET 8+

How it works:

1. You publish a new function version
2. Lambda runs initialization code and takes an encrypted snapshot
3. Snapshots are stored in S3, divided into 512 KB chunks
4. On invocation, Lambda restores from snapshot instead of re-initializing

5. Tiered caching (L1 on worker nodes, L2 in distributed cache) speeds retrieval

When to use SnapStart vs Provisioned Concurrency:

Dimension	SnapStart	Provisioned Concurrency
Cost	Lower (no pre-warmed environments)	Higher (always-on charge)
Latency	Sub-second cold starts	No cold starts at all
Best for	Moderate latency sensitivity	Strict latency requirements (< 100ms)
Runtimes	Java, Python, .NET	All runtimes
Compatibility	No EFS, no ephemeral > 512 MB	Full feature support

Exam Gotcha: SnapStart requires you to handle uniqueness, network connections, and ephemeral data carefully. Connections established during init are NOT guaranteed to be valid after snapshot restore -- re-establish them in the handler.

1.10 Powertools for AWS Lambda

Powertools is a developer toolkit available for TypeScript, Python, Java, and .NET. Available as individual npm packages or as a Lambda Layer.

Core features for the exam:

Feature	Package	Purpose
Logger	`@aws-lambda-powertools/logger`	Structured JSON logging with correlation IDs
Tracer	`@aws-lambda-powertools/tracer`	X-Ray tracing with automatic segment capture
Metrics	`@aws-lambda-powertools/metrics`	CloudWatch EMF custom metrics
Parameters	`@aws-lambda-powertools/parameters`	SSM, Secrets Manager, AppConfig retrieval with caching
Idempotency	`@aws-lambda-powertools/idempotency`	Prevent duplicate executions (DynamoDB-backed)
Batch	`@aws-lambda-powertools/batch`	Handle SQS/Kinesis/DynamoDB partial failures
Parser	`@aws-lambda-powertools/parser`	Zod-based input validation
Event Handler	`@aws-lambda-powertools/event-handler`	Bedrock Agent function resolver

```
import { Logger } from "@aws-lambda-powertools/logger";
import { Tracer } from "@aws-lambda-powertools/tracer";
import { Metrics, MetricUnit } from "@aws-lambda-powertools/metrics";

const logger = new Logger({ serviceName: "genai-api" });
const tracer = new Tracer({ serviceName: "genai-api" });
const metrics = new Metrics({ namespace: "GenAI", serviceName: "genai-api" });

export const handler = async (event: unknown) =&gt; {
  logger.info("Processing request", { event });

  const segment = tracer.getSegment();
  const subsegment = segment?.addNewSubsegment("BedrockInvocation");

  try {
```

```
    // ... invoke Bedrock ...
    metrics.addMetric("ModelInvocations", MetricUnit.Count, 1);
    metrics.addMetric("InputTokens", MetricUnit.Count, 150);
    return { statusCode: 200, body: "Success" };
  } catch (error) {
    metrics.addMetric("ModelErrors", MetricUnit.Count, 1);
    logger.error("Bedrock invocation failed", { error });
    throw error;
  } finally {
    subsegment?.close();
    metrics.publishStoredMetrics();
  }
};
```

Bedrock Agent Function Resolver (Powertools TypeScript):

```
import { BedrockAgentFunctionResolver } from "@aws-lambda-powertools/event-handler/bedrock-agent";

const resolver = new BedrockAgentFunctionResolver();

resolver.tool("lookup_customer", async (params) =&gt; {
  const customer = await getCustomer(params.customerId);
  return { name: customer.name, plan: customer.plan };
});

resolver.tool("get_order_status", async (params) =&gt; {
  const order = await getOrder(params.orderId);
  return { status: order.status, eta: order.eta };
});

export const handler = resolver.resolve();
```

Exam Gotcha: Idempotency is critical for GenAI workflows. FM invocations are expensive -- Powertools idempotency uses DynamoDB to cache responses and prevent duplicate invocations when retries occur.

1.11 Error Handling Patterns

Synchronous invocation: Caller receives the error directly. No automatic retry. The caller (API Gateway, SDK client) must implement retry logic.

Asynchronous invocation: Lambda retries twice by default (3 total attempts) with increasing backoff. Configurable via:

- `MaximumRetryAttempts` (0-2)
- `MaximumEventAgeInSeconds` (60-21600, i.e., up to 6 hours)

Failed event destinations:

- SQS queue (most common for DLQ pattern)
- SNS topic
- S3 bucket (failure records only)
- Another Lambda function
- EventBridge

Dead Letter Queues (DLQ): Older mechanism; captures discarded events. Use on-failure destinations instead for new designs (richer metadata, more destination options).

Error handling for Bedrock calls:

```
import {
  ThrottlingException,
  ModelTimeoutException,
  ServiceQuotaExceededException,
} from "@aws-sdk/client-bedrock-runtime";

async function invokeWithRetry(
  fn: () =&gt; Promise&lt;unknown&gt;,
  maxRetries = 3
): Promise&lt;unknown&gt; {
  for (let attempt = 0; attempt &lt;= maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      if (
        error instanceof ThrottlingException ||
        error instanceof ServiceQuotaExceededException
      ) {
        // Exponential backoff with jitter
        const delay = Math.min(1000 * 2 ** attempt + Math.random() * 1000, 30000);
        await new Promise((r) =&gt; setTimeout(r, delay));
        continue;
      }
      if (error instanceof ModelTimeoutException) {
        // Model overloaded; retry with backoff
        if (attempt &lt; maxRetries) continue;
      }
      throw error; // Non-retryable error
    }
  }
  throw new Error("Max retries exceeded");
}
```

Exam Gotcha: For event-source mappings (SQS, Kinesis, DynamoDB Streams), Lambda retries the entire batch. Use `bisectBatchOnFunctionError` to isolate the bad record, and `maxRetryAttempts` + `maxRecordAge` to prevent poison pills from blocking the stream.

---

2. Amazon EC2 for GenAI

2.1 GPU Instance Families

Family	Chip	Primary Use	Key Feature
P6/P6e	NVIDIA Blackwell/Blackwell Ultra	Training + Inference at trillion-param scale	Up to 72 GPUs, 3200 Gbps EFA
P5/P5e	NVIDIA H100/H200	Training + Inference (100B+ params)	Up to 8 GPUs, 3200 Gbps EFA, UltraClusters
P4d/P4de	NVIDIA A100	Training + Inference	8 GPUs, 400 Gbps EFA
G6e	NVIDIA L40S	Inference + Spatial computing	Up to 8 GPUs
G5	NVIDIA A10G	Inference + Graphics	Up to 8 GPUs, 3x perf over G4dn
G4dn	NVIDIA T4	Cost-effective inference	Up to 8 GPUs
Trn2	AWS Trainium2	Training + Inference (GenAI)	AWS custom silicon, best price-performance

Family	Chip	Primary Use	Key Feature
Trn1/Trn1n	AWS Trainium	Training	Up to 16 chips, 50% cost savings vs comparable
Inf2	AWS Inferentia2	Inference	Optimized for LLM inference
Inf1	AWS Inferentia	Inference	Previous gen, cost-effective

AWS Custom Silicon:

- **Trainium** (Trn1, Trn2): Designed for training. Use AWS Neuron SDK.
- **Inferentia** (Inf1, Inf2): Designed for inference. Use AWS Neuron SDK.
- These are the cost-optimized choices when the question mentions "lowest cost training"

or "cost-effective inference" on AWS custom hardware.

2.2 When to Use EC2 vs Lambda vs SageMaker

Criteria	Lambda	EC2 GPU	SageMaker Endpoints
Invocation pattern	Event-driven, on-demand	Always-on, persistent	Managed real-time/batch
Max execution time	15 minutes	Unlimited	Unlimited
GPU access	No	Yes (P, G, Trn, Inf)	Yes (managed)
Model hosting	Invoke via Bedrock API	Self-managed	Managed (auto-scaling, A/B)
Scaling	Automatic (ms)	ASG (minutes)	Auto-scaling (minutes)
Cost model	Per-invocation	Per-hour	Per-hour + per-inference
Best for	Bedrock API calls, data processing, tool execution	Custom model hosting, fine-tuning, large batch jobs	Production model endpoints with ML ops

Exam decision tree:

- "Invoke a foundation model" --> Lambda + Bedrock
- "Host a custom model with GPU" --> EC2 or SageMaker
- "Cost-effective training" --> Trn1/Trn2 instances
- "Managed inference endpoint" --> SageMaker
- "Self-managed, full control" --> EC2

2.3 Auto Scaling for AI Workloads

EC2 Auto Scaling Groups with GPU instances use target tracking policies. Key metric:

- **Custom CloudWatch metric** for GPU utilization (reported via CloudWatch agent or

NVIDIA DCGM exporter)

- `CPUUtilization` alone is insufficient for GPU workloads

Step Scaling example: Scale out when average GPU utilization > 70% for 3 minutes; scale in when < 30% for 10 minutes.

Exam Gotcha: EC2 GPU instances are expensive. Always use Spot Instances for fault-tolerant training jobs (up to 90% savings). Use On-Demand or Reserved for inference endpoints requiring availability.

Key CloudWatch Metrics:

- `CPUUtilization`, `NetworkIn`, `NetworkOut` (standard)
- Custom GPU metrics via CloudWatch agent (GPU utilization, GPU memory)

- `StatusCheckFailed` for instance health

---

3. AWS App Runner

3.1 Container Deployment for AI Services

App Runner is a fully managed container service for web applications and APIs. It connects to source code (GitHub) or container images (ECR) and handles build, deploy, scaling, and TLS automatically.

Key characteristics:

- Automatic scaling based on concurrent requests
- Built-in HTTPS, load balancing, health checks
- No infrastructure to manage (no VPC required by default, but VPC connector available)
- Deploys from ECR images or source code
- Min instances can be set to 0 (scale to zero) or higher (provisioned instances)

Auto Scaling configuration:

- `MaxConcurrency`: max concurrent requests per instance (default: 100)
- `MaxSize`: max number of instances (default: 25)
- `MinSize`: min instances (default: 1; set to 0 for scale-to-zero)

3.2 When to Use App Runner vs Lambda vs ECS

Criteria	Lambda	App Runner	ECS/Fargate
Execution time	15 min max	Unlimited	Unlimited
Container support	Container images (no Dockerfile)	Full Dockerfile, ECR	Full container orchestration
Scaling	Per-invocation	Per-request concurrency	Task-based ASG
Complexity	Lowest	Low	Medium-High
GPU support	No	No	Yes (EC2 launch type)
State	Stateless	Stateless (between requests)	Stateful possible
Best for	Short tasks, event-driven	Web APIs, containerized AI services	Complex microservices, GPU workloads

Exam decision tree:

- "Simple containerized API, no infrastructure management" --> App Runner
- "Need GPU containers" --> ECS on EC2 with GPU instances
- "Event-driven, short-lived" --> Lambda
- "Full orchestration control" --> ECS/EKS

Exam Gotcha: App Runner does NOT support GPU instances. If the question mentions GPU inference in a container, the answer is ECS on EC2 (not Fargate, which also lacks GPU).

Key CloudWatch Metrics:

- `RequestCount` -- total HTTP requests
- `2xxStatusCode`, `4xxStatusCode`, `5xxStatusCode` -- response codes
- `RequestLatency` -- response time

- `ActiveInstances` -- currently running instances
- `ConcurrencyUtilization` -- percentage of max concurrency used

---

4. AWS Lambda@Edge

4.1 Edge Processing for AI Applications

Lambda@Edge extends Lambda to CloudFront edge locations worldwide. Functions run in response to CloudFront events: viewer request, viewer response, origin request, origin response.

Key limits (different from standard Lambda):

Resource	Viewer Events	Origin Events
Timeout	5 seconds	30 seconds
Memory	128 MB	128 MB (configurable up to limits)
Package size	1 MB	50 MB
Ephemeral storage	512 MB (fixed)	512 MB (fixed)

Restrictions:

- Must be deployed in **us-east-1** (automatically replicated globally)
- No VPC access
- No environment variables (except reserved Lambda vars)
- No Lambda Layers
- No provisioned concurrency
- No container images
- No arm64 architecture
- No X-Ray tracing

4.2 GenAI Use Cases at the Edge

- **Request routing**: Route AI requests to the nearest Region based on latency
- **Content transformation**: Modify HTML/JSON responses at the edge (e.g., inject

AI-generated summaries, translate content headers)

- **A/B testing**: Route a percentage of users to different model versions
- **Authentication**: Validate JWT tokens before requests reach the origin
- **Prompt caching**: Check CloudFront cache for repeated identical prompts

Lambda@Edge vs CloudFront Functions:

Dimension	Lambda@Edge	CloudFront Functions
Runtime	Node.js, Python	JavaScript only
Execution time	Up to 30s (origin)	< 1ms
Scale	Thousands/sec	Millions/sec
Network access	Yes	No
Use case	Complex logic, API calls	Header manipulation, redirects, URL rewrites

Exam Gotcha: Lambda@Edge is NOT suitable for invoking Bedrock models (too slow for viewer events at 5s, and no direct Bedrock access from edge). Use it for lightweight pre/post-processing, routing, and auth. For actual FM inference at the edge, consider Wavelength or Outposts.

---

5. AWS Outposts

5.1 On-Premises AI Deployment (Task 2.3.4)

AWS Outposts brings AWS infrastructure to on-premises data centers. Two form factors:

- **Outposts Rack**: Full 42U rack with EC2, EBS, S3, ECS, EKS, RDS, etc.
- **Outposts Server**: 1U/2U server for edge locations with limited space

Architecture: Outposts are fully managed by AWS. They connect back to a parent AWS Region for control plane operations, but data processing happens locally.

GenAI on Outposts:

- Deploy Small Language Models (SLMs) locally using EC2 GPU instances on Outposts
- Use Llama.cpp or similar lightweight inference engines
- AWS IoT Greengrass for managing ML components on Outposts Servers
- Anomaly detection, virtual assistants, and time-series forecasting at the edge

5.2 Data Sovereignty and Compliance Use Cases

Outposts is the correct answer when the question mentions:

- **Data residency requirements**: data must not leave a specific geographic location
- **Data sovereignty**: regulatory requirement for data to remain on-premises
- **Low latency to on-premises systems**: factory floor, hospital, trading floor
- **Local data processing**: data too sensitive or too large to send to the cloud
- **Hybrid cloud**: consistent API experience across on-premises and cloud

Exam Gotcha: Outposts requires a network connection back to the parent Region for management. It is NOT an air-gapped solution. For fully disconnected environments, the exam scope does not cover AWS Snow Family, but know that Outposts needs connectivity.

Key CloudWatch Metrics:

- `ConnectedStatus` -- connection to parent Region
- Standard EC2 metrics for instances on Outposts
- S3 on Outposts metrics for local storage

---

6. AWS Wavelength

6.1 Ultra-Low Latency AI at the Edge (Task 2.3.4)

AWS Wavelength embeds AWS compute and storage within telecom providers' 5G networks. Wavelength Zones are extensions of a parent AWS Region but physically located inside carrier data centers.

How it works:

1. Opt in to a Wavelength Zone

2. Extend your VPC with a subnet in the Wavelength Zone
3. Create a **carrier gateway** (like an internet gateway, but for the carrier network)
4. Deploy EC2 instances, EKS pods, or containers in the Wavelength subnet
5. Traffic from 5G devices reaches your app without leaving the telecom network

Supported services in Wavelength Zones:

- Amazon EC2 (including Auto Scaling)
- Amazon ECS/EKS
- Amazon EBS
- Amazon VPC
- Application Load Balancer
- CloudWatch, CloudTrail, CloudFormation, Systems Manager

MTU considerations:

- 9001 bytes between instances in the same Wavelength Zone
- 1500 bytes between carrier gateway and Wavelength Zone
- 1300 bytes between Wavelength Zone and Region (private IP)

6.2 5G Integration Patterns

Use cases on the exam:

- **Real-time inference**: Computer vision, AR/VR, autonomous vehicles
- **Interactive AI**: Voice assistants, real-time translation at the device edge
- **IoT + AI**: Smart factory sensors processed at the 5G edge
- **Gaming**: AI-driven NPCs with ultra-low latency

Architecture pattern:

```
5G Device --&gt; Carrier Network --&gt; Wavelength Zone (EC2/EKS)
                                        |
                                        v
                                  Parent Region (Bedrock, S3, DynamoDB)
```

Wavelength vs Local Zones vs Outposts:

Dimension	Wavelength	Local Zones	Outposts
Location	Inside carrier 5G network	AWS-managed metro facility	Customer data center
Latency target	Single-digit ms (5G)	Single-digit ms (metro)	Sub-ms (on-premises)
Network	Carrier gateway	Internet gateway	Local gateway
Management	AWS-managed	AWS-managed	AWS-managed on-prem
Exam trigger	"5G", "mobile edge", "carrier"	"metro", "city", "end users"	"on-premises", "data center", "sovereignty"

Exam Gotcha: Wavelength is always the answer when the question mentions "5G", "mobile devices at the edge", "carrier network", or "telecom provider". If the question says "on-premises" or "data center", the answer is Outposts. If it says "metro area" without 5G context, consider Local Zones.

Key CloudWatch Metrics:

- Standard EC2 metrics for instances in Wavelength Zones
- VPC Flow Logs for network traffic analysis
- Custom application metrics via CloudWatch agent

Quick Reference: Service Selection Matrix

Scenario	Service
Invoke Bedrock FM on-demand	**Lambda**
Stream FM tokens to client	**Lambda** (response streaming)
Process/chunk documents for RAG	**Lambda**
Generate embeddings in batch	**Lambda** (or Step Functions + Lambda)
Validate prompts before FM call	**Lambda**
Post-process FM output (PII, compliance)	**Lambda**
Stateless MCP tool server	**Lambda**
Handle webhook from external system	**Lambda**
Dynamic model selection	**Lambda** + Converse API
Host custom model with GPU	**EC2** (P5, G5, Inf2, Trn1)
Cost-effective model training	**EC2 Trn1/Trn2** (Trainium)
Cost-effective inference	**EC2 Inf2** (Inferentia2)
Simple containerized AI API	**App Runner**
Containerized AI with GPU	**ECS on EC2** (not App Runner)
Route AI requests at edge	**Lambda@Edge**
AI on-premises (data sovereignty)	**Outposts**
AI at 5G edge (mobile devices)	**Wavelength**

Exam Tips Summary

1. **Lambda timeout**: Default 3s will fail on Bedrock calls. Set 30-120s minimum.
2. **6 MB vs 200 MB**: Buffered responses = 6 MB. Streaming = 200 MB. Streaming is for GenAI.
3. **Converse API**: Model-agnostic. Use it for dynamic model selection questions.
4. **Reserved concurrency**: Protects account concurrency AND Bedrock rate limits.
5. **SnapStart vs Provisioned Concurrency**: SnapStart is cheaper but sub-second, not zero cold start. PC eliminates cold starts entirely.
6. **Layers**: Do NOT increase the 250 MB limit. For large ML packages, use container images (10 GB).
7. **Lambda@Edge limits**: 5s viewer / 30s origin timeout, 1 MB package for viewer events. Cannot invoke Bedrock.
8. **App Runner has no GPU**. GPU containers = ECS on EC2.
9. **Outposts**: "on-premises" + "data sovereignty" + "local data processing".

10. **Wavelength**: "5G" + "carrier" + "mobile edge" + "ultra-low latency".

11. **Trainium for training, Inferentia for inference** -- both are AWS custom silicon, both use the Neuron SDK.

12. **Powertools idempotency**: Prevents duplicate FM invocations during retries. Uses DynamoDB for persistence.

13. **Async error handling**: 2 retries (default), configurable. Use on-failure destinations (SQS/SNS/S3/EventBridge) over DLQs for new designs.

14. **Streaming billing**: You pay for full duration even if the client disconnects.

15. **VPC + streaming**: Function URLs do not support streaming in VPC. Use `InvokeWithResponseStream` via SDK with VPC endpoints.

Tier 3: Important Infrastructure

These services are referenced three to five times across domains. Focus on how they integrate with Bedrock and SageMaker, key configuration options, and the scenarios that distinguish them from similar services.

Guides in this tier:

- Management & Governance (CloudWatch, CloudTrail, Cost Tools, Well-Architected)
- Security, Identity & Compliance (IAM, Cognito, KMS, WAF, Macie)
- Networking & Content Delivery (API Gateway, VPC, CloudFront, AppSync)
- Analytics Services (OpenSearch, Glue, Athena, Kinesis, MSK)
- Database Services (Aurora pgvector, DynamoDB, ElastiCache, Neptune)

Management and Governance Services for the AIP-C01 Exam

This study guide covers the Management and Governance services tested on the AWS Certified Generative AI Developer - Professional (AIP-C01) exam. CloudWatch and CloudWatch Logs are the most heavily tested services in this category -- they appear in nearly every task statement across all five domains.

Table of Contents

1. [Amazon CloudWatch](#1-amazon-cloudwatch)
2. [Amazon CloudWatch Logs](#2-amazon-cloudwatch-logs)
3. [Amazon CloudWatch Synthetics](#3-amazon-cloudwatch-synthetics)
4. [AWS CloudTrail](#4-aws-cloudtrail)
5. [AWS Auto Scaling](#5-aws-auto-scaling)
6. [Cost Tools (Cost Explorer, Cost Anomaly Detection)](#6-cost-tools)
7. [AWS Well-Architected Tool](#7-aws-well-architected-tool)
8. [Other Services](#8-other-services)

---

1. Amazon CloudWatch

CloudWatch is the central observability service for all AWS resources. For the exam, you need to know CloudWatch as the primary monitoring plane for Bedrock model invocations, agents, guardrails, and custom GenAI metrics.

1.1 Core Concepts

Metrics are time-ordered data points published to CloudWatch. Each metric belongs to a **namespace** (e.g., `AWS/Bedrock`) and is identified by a combination of name and **dimensions** (key-value pairs like `ModelId`).

Statistics are aggregations over a time period: `Sum`, `Average`, `Minimum`, `Maximum`, `SampleCount`, and **percentiles** (p50, p90, p99). Percentile statistics are critical for latency monitoring -- the exam tests whether you know to use p99 latency instead of average for tail-latency SLAs.

Periods define the length of time associated with a specific statistic. Minimum period is 1 second (high-resolution metrics) or 60 seconds (standard). Bedrock metrics use standard resolution.

Alarms watch a single metric or a metric math expression and change state based on thresholds. States: `OK`, `ALARM`, `INSUFFICIENT_DATA`. Alarms trigger actions such as SNS notifications, Auto Scaling policies, or EC2 actions.

Dashboards are customizable pages in the CloudWatch console for visualizing metrics. They support cross-account and cross-Region aggregation.

1.2 Bedrock-Specific CloudWatch Metrics

These are the built-in metrics published automatically when you use Amazon Bedrock. No additional configuration is required to start receiving them.

Runtime Metrics (Namespace: `AWS/Bedrock`)

Metric	Unit	Description
`Invocations`	SampleCount	Number of successful requests to `Converse`, `ConverseStream`, `InvokeModel`, `InvokeModelWithResponseStream`
`InvocationLatency`	Milliseconds	End-to-end time from request sent to last token received
`InvocationClientErrors`	SampleCount	Invocations resulting in 4xx client errors
`InvocationServerErrors`	SampleCount	Invocations resulting in 5xx server errors
`InvocationThrottles`	SampleCount	Throttled invocations (do NOT count as Invocations or Errors)
`InputTokenCount`	SampleCount	Number of input tokens
`OutputTokenCount`	SampleCount	Number of output tokens
`OutputImageCount`	SampleCount	Number of output images (image generation models only)
`TimeToFirstToken`	Milliseconds	Time until first token for streaming APIs (`ConverseStream`, `InvokeModelWithResponseStream`)
`LegacyModelInvocations`	SampleCount	Invocations using legacy (deprecated) models
`EstimatedTPMQuotaUsage`	SampleCount	Estimated tokens-per-minute quota consumption

Dimensions for runtime metrics:

- `ModelId` -- available on all metrics
- `ModelId` + `ImageSize` + `BucketedStepSize` -- available on `OutputImageCount`

Log Delivery Metrics (Namespace: `AWS/Bedrock`, Dimension: `Across all model IDs`)

Metric	Description
`ModelInvocationLogsCloudWatchDeliverySuccess`	Successful log deliveries to CloudWatch Logs
`ModelInvocationLogsCloudWatchDeliveryFailure`	Failed log deliveries to CloudWatch Logs
`ModelInvocationLogsS3DeliverySuccess`	Successful log deliveries to S3
`ModelInvocationLogsS3DeliveryFailure`	Failed log deliveries to S3
`ModelInvocationLargeDataS3DeliverySuccess`	Successful large data deliveries to S3
`ModelInvocationLargeDataS3DeliveryFailure`	Failed large data deliveries to S3

Guardrails Metrics (Namespace: `AWS/Bedrock/Guardrails`)

Metric	Unit	Description
`Invocations`	SampleCount	Requests to `ApplyGuardrail` API
`InvocationLatency`	Milliseconds	Guardrail evaluation latency

Metric	Unit	Description
`InvocationClientErrors`	SampleCount	Client-side errors
`InvocationServerErrors`	SampleCount	Server-side errors
`InvocationThrottles`	SampleCount	Throttled requests
`TextUnitCount`	SampleCount	Text units consumed by guardrail policies
`InvocationsIntervened`	SampleCount	Invocations where guardrails intervened (blocked/modified)
`FindingCounts`	SampleCount	Counts per finding type from automated reasoning checks
`TotalFindings`	SampleCount	Total findings per automated reasoning check request
`Latency`	Milliseconds	Automated reasoning policy verification latency

Guardrails Dimensions:

- `Operation` (ApplyGuardrail)
- `GuardrailContentSource` (Input or Output)
- `GuardrailPolicyType` (ContentPolicy, TopicPolicy, WordPolicy, SensitiveInformationPolicy, ContextualGroundingPolicy)
- `GuardrailArn` + `GuardrailVersion`
- `FindingType` + `PolicyArn` + `PolicyVersion`

Agent Metrics (Namespace: `AWS/Bedrock/Agents`)

Metric	Unit	Description
`InvocationCount`	SampleCount	Agent invocation requests
`TotalTime`	Milliseconds	Server-side processing time
`TTFT`	Milliseconds	Time-to-first-token (streaming agent invocations only)
`InvocationThrottles`	SampleCount	Throttled agent invocations
`InvocationServerErrors`	SampleCount	Server errors
`InvocationClientErrors`	SampleCount	Client errors
`ModelLatency`	Milliseconds	Latency of the underlying model call
`ModelInvocationCount`	SampleCount	Model calls made by the agent
`ModelInvocationThrottles`	SampleCount	Model calls throttled by Bedrock
`ModelInvocationClientErrors`	SampleCount	Client errors on model calls
`ModelInvocationServerErrors`	SampleCount	Server errors on model calls
`InputTokenCount`	SampleCount	Tokens input to the model
`OutputTokenCount`	SampleCount	Tokens output from the model

Agent Dimensions:

- `Operation` (InvokeAgent, InvokeInlineAgent)

- `Operation` + `ModelId`
- `Operation` + `AgentAliasArn` + `ModelId`

> ***Exam tip:*** *The agent IAM service role must include* `cloudwatch:PutMetricData` *with a condition restricting to the* `AWS/Bedrock/Agents` *namespace, or metrics will not appear in the dashboard.*

1.3 Custom Metrics for GenAI

Bedrock built-in metrics cover infrastructure-level observability but not application-level quality. For GenAI applications, you often need custom metrics:

Custom Metric	Purpose	How to Compute
Hallucination rate	Track groundedness failures	Count guardrail `ContextualGroundingPolicy` interventions / total invocations
Response quality score	Application-specific quality	Run LLM-as-judge evaluations, publish scores as custom metrics
Token cost per request	Cost attribution	`(InputTokenCount * input_price) + (OutputTokenCount * output_price)`
Prompt confusion rate	Detect unclear prompts	Parse invocation logs for refusal patterns or low-confidence signals
Guardrail intervention rate	Safety monitoring	`InvocationsIntervened` / total `Invocations` per guardrail
RAG retrieval relevance	Knowledge base quality	Score retrieved chunks against the query, publish as metric

Publishing custom metrics uses the `PutMetricData` API:

```
import { CloudWatchClient, PutMetricDataCommand } from "@aws-sdk/client-cloudwatch";

const cw = new CloudWatchClient({ region: "us-east-1" });

await cw.send(new PutMetricDataCommand({
  Namespace: "GenAI/MyApp",
  MetricData: [
    {
      MetricName: "HallucinationRate",
      Dimensions: [
        { Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" },
        { Name: "Environment", Value: "production" },
      ],
      Value: 0.03, // 3% hallucination rate
      Unit: "None",
      Timestamp: new Date(),
    },
    {
      MetricName: "TokenCostUSD",
      Dimensions: [
        { Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" },
      ],
      Value: 0.0042,
      Unit: "None",
      Timestamp: new Date(),
    },
  ],
}));
```

> ***Exam tip:*** *Custom metrics cost $0.30/metric/month. Use dimensions wisely -- each unique combination of namespace + metric name + dimensions creates a separate metric. The exam may test whether you understand the cost implications of high-cardinality dimensions.*

1.4 CloudWatch Alarms for AI Thresholds

Alarms are the primary mechanism for automated response to GenAI quality issues.

Static Threshold Alarms -- compare a metric to a fixed value:

```
import { CloudWatchClient, PutMetricAlarmCommand } from "@aws-sdk/client-cloudwatch";

const cw = new CloudWatchClient({ region: "us-east-1" });

// Alarm when throttling exceeds 10 requests in 5 minutes
await cw.send(new PutMetricAlarmCommand({
  AlarmName: "Bedrock-High-Throttling",
  Namespace: "AWS/Bedrock",
  MetricName: "InvocationThrottles",
  Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
  Statistic: "Sum",
  Period: 300,
  EvaluationPeriods: 1,
  Threshold: 10,
  ComparisonOperator: "GreaterThanThreshold",
  AlarmActions: ["arn:aws:sns:us-east-1:123456789012:GenAI-Alerts"],
  TreatMissingData: "notBreaching",
}));

// Alarm when p99 latency exceeds 10 seconds
await cw.send(new PutMetricAlarmCommand({
  AlarmName: "Bedrock-P99-Latency-SLA",
  Namespace: "AWS/Bedrock",
  MetricName: "InvocationLatency",
  Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
  ExtendedStatistic: "p99",
  Period: 300,
  EvaluationPeriods: 3,
  DatapointsToAlarm: 2,  // 2 out of 3 periods must breach
  Threshold: 10000,
  ComparisonOperator: "GreaterThanThreshold",
  AlarmActions: ["arn:aws:sns:us-east-1:123456789012:GenAI-Alerts"],
}));
```

Composite Alarms combine multiple alarms with AND/OR logic. Use these when a single metric alarm would be too noisy:

```
import { PutCompositeAlarmCommand } from "@aws-sdk/client-cloudwatch";

await cw.send(new PutCompositeAlarmCommand({
  AlarmName: "Bedrock-Combined-Degradation",
  AlarmRule: 'ALARM("Bedrock-High-Throttling") AND ALARM("Bedrock-P99-Latency-SLA")',
  AlarmActions: ["arn:aws:sns:us-east-1:123456789012:GenAI-Critical"],
}));
```

> ***Exam tip:*** *Know the* `TreatMissingData` *options:* `breaching`, `notBreaching`, `ignore`, `missing`. *For intermittent GenAI workloads,* `notBreaching` *prevents false alarms when no invocations occur.*

1.5 Anomaly Detection for Token Patterns

CloudWatch Anomaly Detection uses machine learning to build a model of expected metric behavior, accounting for hourly, daily, and weekly seasonality patterns. It produces an **anomaly detection band** -- a range of expected values.

This is particularly useful for GenAI workloads where static thresholds are hard to set:

- Token usage patterns that grow organically over time
- Latency that varies with prompt length and model load
- Invocation counts that follow business hours

When to use anomaly detection vs. static thresholds:

- Use anomaly detection when the metric has seasonal patterns and you cannot predict a fixed threshold
- Use static thresholds when you have a hard SLA (e.g., p99 latency must be under 10 seconds)

Anomaly detection alarms use the `ANOMALY_DETECTION_BAND` function:

```
await cw.send(new PutMetricAlarmCommand({
  AlarmName: "Bedrock-Token-Anomaly",
  Namespace: "AWS/Bedrock",
  MetricName: "InputTokenCount",
  Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
  Statistic: "Sum",
  Period: 3600,
  EvaluationPeriods: 3,
  ThresholdMetricId: "ad1",
  ComparisonOperator: "GreaterThanUpperThreshold",
  Metrics: [
    {
      Id: "m1",
      MetricStat: {
        Metric: {
          Namespace: "AWS/Bedrock",
          MetricName: "InputTokenCount",
          Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
        },
        Period: 3600,
        Stat: "Sum",
      },
    },
    {
      Id: "ad1",
      Expression: "ANOMALY_DETECTION_BAND(m1, 2)",  // 2 standard deviations
    },
  ],
}));
```

CloudWatch Logs also has its own anomaly detection capability. Log anomaly detectors continuously scan log events in a log group, establish baselines using ML and pattern recognition, and flag significant fluctuations. This complements metric-based anomaly detection by catching novel error patterns in invocation logs.

1.6 CloudWatch Dashboards for AI Observability

Bedrock provides **pre-configured dashboards** under CloudWatch > Generative AI observability

Model Invocations. These dashboards include:

- Invocation count over time

- Invocation latency
- Token counts by model (input vs. output)
- Daily token counts by model ID
- Requests grouped by input token ranges
- Invocation throttles
- Invocation error count

You can create custom dashboards for application-level GenAI observability:

```
import { CloudWatchClient, PutDashboardCommand } from "@aws-sdk/client-cloudwatch";

const cw = new CloudWatchClient({ region: "us-east-1" });

const dashboardBody = {
  widgets: [
    {
      type: "metric",
      x: 0, y: 0, width: 12, height: 6,
      properties: {
        title: "Token Usage by Model",
        metrics: [
          ["AWS/Bedrock", "InputTokenCount", "ModelId", "anthropic.claude-sonnet-4-20250514-v1:0", { stat: "Sum",
          ["AWS/Bedrock", "OutputTokenCount", "ModelId", "anthropic.claude-sonnet-4-20250514-v1:0", { stat: "Sum'
        ],
        period: 3600,
        view: "timeSeries",
        region: "us-east-1",
      },
    },
    {
      type: "metric",
      x: 12, y: 0, width: 12, height: 6,
      properties: {
        title: "Guardrail Interventions",
        metrics: [
          ["AWS/Bedrock/Guardrails", "InvocationsIntervened", "Operation", "ApplyGuardrail", { stat: "Sum" }],
          ["AWS/Bedrock/Guardrails", "Invocations", "Operation", "ApplyGuardrail", { stat: "Sum" }],
        ],
        period: 3600,
        view: "timeSeries",
      },
    },
    {
      type: "metric",
      x: 0, y: 6, width: 12, height: 6,
      properties: {
        title: "P99 Latency &amp; Time to First Token",
        metrics: [
          ["AWS/Bedrock", "InvocationLatency", "ModelId", "anthropic.claude-sonnet-4-20250514-v1:0", { stat: "p99
          ["AWS/Bedrock", "TimeToFirstToken", "ModelId", "anthropic.claude-sonnet-4-20250514-v1:0", { stat: "p99'
        ],
        period: 300,
        view: "timeSeries",
      },
    },
  ],
};

await cw.send(new PutDashboardCommand({
```

```
  DashboardName: "GenAI-Observability",
  DashboardBody: JSON.stringify(dashboardBody),
}));
```

1.7 Metric Math for AI KPIs

Metric math lets you compute derived KPIs without publishing additional custom metrics. Expressions use metric IDs and standard arithmetic.

Common GenAI metric math expressions:

KPI	Expression	Purpose
Error rate	`(m1 + m2) / m3 * 100` where m1=ClientErrors, m2=ServerErrors, m3=Invocations	Overall error percentage
Tokens per invocation (avg)	`m1 / m2` where m1=Sum(InputTokenCount), m2=Sum(Invocations)	Average prompt size
Guardrail intervention %	`m1 / m2 * 100` where m1=InvocationsIntervened, m2=Invocations	Safety trigger rate
Output/Input ratio	`m1 / m2` where m1=OutputTokenCount, m2=InputTokenCount	Verbosity indicator
Estimated cost	`(m1 * input_price_per_token) + (m2 * output_price_per_token)`	Token-based cost approximation

```
// Example: Metric math in a GetMetricData call
import { CloudWatchClient, GetMetricDataCommand } from "@aws-sdk/client-cloudwatch";

const cw = new CloudWatchClient({ region: "us-east-1" });

const end = new Date();
const start = new Date(end.getTime() - 3600 * 1000);

const result = await cw.send(new GetMetricDataCommand({
  StartTime: start,
  EndTime: end,
  MetricDataQueries: [
    {
      Id: "invocations",
      MetricStat: {
        Metric: {
          Namespace: "AWS/Bedrock",
          MetricName: "Invocations",
          Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
        },
        Period: 3600,
        Stat: "Sum",
      },
      ReturnData: false,
    },
    {
      Id: "throttles",
      MetricStat: {
        Metric: {
          Namespace: "AWS/Bedrock",
          MetricName: "InvocationThrottles",
```

```
          Dimensions: [{ Name: "ModelId", Value: "anthropic.claude-sonnet-4-20250514-v1:0" }],
        },
        Period: 3600,
        Stat: "Sum",
      },
      ReturnData: false,
    },
    {
      Id: "throttle_rate",
      Expression: "throttles / (invocations + throttles) * 100",
      Label: "Throttle Rate %",
      ReturnData: true,
    },
  ],
}));
```

> ***Exam tip:*** `METRICS("m1")` *is used when you want to apply a function across all metrics matching a search expression.* `ANOMALY_DETECTION_BAND(m1, N)` *creates an upper and lower band N standard deviations from the model. These are the two most likely metric math functions tested.*

---

2. Amazon CloudWatch Logs

CloudWatch Logs is the centralized log ingestion and analysis service. For the exam, the critical use case is Bedrock model invocation logging and Logs Insights queries for prompt/response analysis.

2.1 Core Concepts

Log Groups are containers for log streams that share the same retention, monitoring, and access control settings. Example: `/aws/bedrock/modelinvocations`.

Log Streams are sequences of log events from the same source. Bedrock creates a stream named `aws/bedrock/modelinvocations` within the configured log group.

Log Events are individual records with a timestamp and raw message (JSON for Bedrock).

Retention Policies control how long logs are kept. Options range from 1 day to 10 years, or indefinite (never expire). The default is indefinite, which can become expensive for high-volume GenAI logging. Always set a retention policy.

Subscription Filters stream log data in real time to destinations:

- AWS Lambda (for real-time processing)
- Amazon Kinesis Data Streams (for analytics pipelines)
- Amazon Data Firehose (for delivery to S3, OpenSearch, etc.)

A single log group can have up to 2 subscription filters.

2.2 Bedrock Model Invocation Logging

Model invocation logging captures the full request and response data for every Bedrock inference call. It is **disabled by default** and must be explicitly enabled.

What gets logged:

- Request metadata (timestamp, request ID, model ID, region, account)
- Input body (prompt text, up to 100 KB in CloudWatch Logs; larger payloads go to S3)
- Output body (response text, up to 100 KB)

- Token counts (input, output)
- Latency
- Error details (if applicable)

Supported API operations:

- `Converse` / `ConverseStream`
- `InvokeModel` / `InvokeModelWithResponseStream`

Configuration options (modalities):

- Text
- Image
- Embedding
- Video

Data is logged for ALL models that support the selected modalities.

Destinations:

- CloudWatch Logs only
- Amazon S3 only
- Both CloudWatch Logs and S3

For large data or binary image outputs, only S3 is supported. CloudWatch Logs handles JSON payloads up to 100 KB.

Setup via SDK:

```
import { BedrockClient, PutModelInvocationLoggingConfigurationCommand } from "@aws-sdk/client-bedrock";

const bedrock = new BedrockClient({ region: "us-east-1" });

await bedrock.send(new PutModelInvocationLoggingConfigurationCommand({
  loggingConfig: {
    cloudWatchConfig: {
      logGroupName: "/aws/bedrock/modelinvocations",
      roleArn: "arn:aws:iam::123456789012:role/BedrockLoggingRole",
      largeDataDeliveryS3Config: {
        bucketName: "my-bedrock-large-data",
        keyPrefix: "large-payloads/",
      },
    },
    textDataDeliveryEnabled: true,
    imageDataDeliveryEnabled: true,
    embeddingDataDeliveryEnabled: false,
    videoDataDeliveryEnabled: false,
  },
}));
```

Required IAM for CloudWatch Logs destination:

- The Bedrock service role must have `logs:CreateLogStream` and `logs:PutLogEvents` on the target log group.
- The trust policy must allow `bedrock.amazonaws.com` to assume the role, with `aws:SourceAccount` and `aws:SourceArn` conditions.

> ***Exam tip:*** *If a question says "we need to analyze the actual prompts and responses sent to Bedrock," the answer is model invocation logging to CloudWatch Logs (or S3), NOT CloudTrail. CloudTrail logs API calls and metadata but does NOT capture prompt/response content.*

2.3 CloudWatch Logs Insights Queries for GenAI

Logs Insights is the interactive query engine for CloudWatch Logs. It uses a purpose-built query language. For the exam, know how to write queries that analyze Bedrock invocation logs.

Common query patterns:

```
# Find the most expensive invocations by token count
fields @timestamp, modelId, inputTokenCount, outputTokenCount
| sort outputTokenCount desc
| limit 20

# Calculate average tokens per model
fields modelId, inputTokenCount, outputTokenCount
| stats avg(inputTokenCount) as avgInput,
        avg(outputTokenCount) as avgOutput,
        count(*) as invocations
  by modelId

# Find invocations with high latency (&gt; 5 seconds)
fields @timestamp, modelId, @message
| filter @message like /InvocationLatency/
| filter InvocationLatency &gt; 5000
| sort InvocationLatency desc

# Search for specific content in prompts (prompt regression testing)
fields @timestamp, modelId, input.inputBodyJson
| filter input.inputBodyJson like /specific-keyword/
| limit 50

# Analyze error patterns
fields @timestamp, modelId, errorCode
| filter ispresent(errorCode)
| stats count(*) as errorCount by errorCode, modelId
| sort errorCount desc

# Token cost analysis by time window
fields @timestamp, modelId, inputTokenCount, outputTokenCount
| stats sum(inputTokenCount) as totalInput,
        sum(outputTokenCount) as totalOutput
  by bin(1h) as timeWindow
| sort timeWindow desc
```

Logs Insights also supports an `anomaly` command that uses ML to detect anomalous log patterns directly within queries. This is useful for ad-hoc investigation:

```
fields @timestamp, @message
| anomaly @message
```

2.4 Sensitive Data Protection

CloudWatch Logs data protection policies can automatically detect and mask sensitive data (PII, PHI, credentials) in log events. This is relevant for GenAI workloads where prompts and responses may contain user PII.

Data protection uses managed data identifiers and can:

- Mask sensitive data in transit (audit findings reported but data masked in log output)
- Mask sensitive data at rest (data is stored in masked form)

> ***Exam tip:*** *If a question describes a requirement to log Bedrock invocations while protecting PII in the logs, the answer involves CloudWatch Logs data protection policies (also referred to as "sensitive data protection" or "log data masking").*

2.5 Log Retention and Cost Optimization

Retention Period	Use Case
1--7 days	Development/testing environments
30 days	Production with short-term debugging needs
90--365 days	Compliance requirements
Indefinite	Audit trail requirements (consider archiving to S3 instead)

To reduce costs on high-volume GenAI logging:

1. Set explicit retention periods instead of using the default (indefinite)
2. Use subscription filters to stream to S3 via Firehose for long-term, cheaper storage
3. Enable only the modalities you need (e.g., skip image logging if you only use text)
4. Use CloudWatch Logs data protection to avoid storing raw PII

---

3. Amazon CloudWatch Synthetics

CloudWatch Synthetics creates **canaries** -- configurable scripts that run on a schedule to monitor endpoints and APIs. Canaries use Lambda functions under the hood and can be written in Node.js, Python, or Java.

3.1 Synthetic Canaries for AI Endpoints

Canaries monitor availability and latency of your GenAI application endpoints from an external perspective. They simulate user interactions and detect problems before real users do.

Key features:

- Run as frequently as once per minute, 24/7
- Capture load time data, screenshots (for web UIs), logs, and metrics
- Publish metrics to the `CloudWatchSynthetics` namespace
- Support API canaries, heartbeat monitors, visual monitoring, and multi-check blueprints
- Can run inside a VPC to monitor private endpoints

Blueprint types relevant to GenAI:

- **API Canary** -- tests REST APIs (your GenAI API endpoints). Supports multiple HTTP

methods, headers, and request bodies. Can validate response codes, latency, headers, and body content.

- **Heartbeat Monitor** -- loads URLs and verifies availability. Useful for monitoring a

GenAI web application front end.

- **Multi-Check Blueprint** -- bundles up to 10 different monitoring steps in a single

canary. Cost-effective for monitoring multiple GenAI endpoints.

3.2 Monitoring AI API Availability

A typical canary for a GenAI endpoint:

```
// Canary script concept (runs as Lambda, uses Synthetics runtime)
// This is the logical flow -- actual canary scripts use the Synthetics
// Node.js/Python library which provides helper functions

// 1. Send a known test prompt to your GenAI API
// 2. Validate the response status code is 200
// 3. Validate the response contains expected structure
// 4. Validate latency is within acceptable bounds
// 5. Optionally validate response quality (e.g., contains expected keywords)
```

Canaries automatically create CloudWatch alarms and can integrate with X-Ray for distributed tracing.

> ***Exam tip:*** *Synthetics canaries are about external availability monitoring of YOUR endpoints (API Gateway, ALB, etc.), not direct monitoring of Bedrock itself. If a question asks about monitoring the availability of a customer-facing GenAI API, the answer is CloudWatch Synthetics. If it asks about monitoring Bedrock model performance, the answer is CloudWatch metrics in the* `AWS/Bedrock` *namespace.*

---

4. AWS CloudTrail

CloudTrail records API calls made in your AWS account as **events**. It is the primary audit and governance tool for Bedrock.

4.1 API Audit Logging for AI Services

CloudTrail is always on and captures the last 90 days of management events in the Event History (free, no trail required). For ongoing retention, create a **trail** that delivers log files to S3.

Every CloudTrail event includes:

- Who made the request (IAM identity, source IP, user agent)
- What API was called and with what parameters
- When the call was made
- Whether it succeeded or failed

4.2 Bedrock API Tracking

Management events (logged by default):

- `InvokeModel`, `InvokeModelWithResponseStream`
- `Converse`, `ConverseStream`
- `ListAsyncInvokes`
- All control-plane operations (CreateAgent, CreateKnowledgeBase, etc.)

Data events (must be explicitly enabled):

- `InvokeAgent` -- configure for resource type `AWS::Bedrock::AgentAlias`
- `InvokeInlineAgent` -- configure for `AWS::Bedrock::InlineAgent`
- `InvokeFlow` -- configure for `AWS::Bedrock::FlowAlias`
- `Retrieve`, `RetrieveAndGenerate` -- configure for `AWS::Bedrock::KnowledgeBase`
- `InvokeModelWithBidirectionalStream` -- configure for `AWS::Bedrock::Model`
- `GetAsyncInvoke`, `StartAsyncInvoke` -- configure for `AWS::Bedrock::Model` and `AWS::Bedrock::AsyncInvoke`
- `ApplyGuardrail` -- configure for `AWS::Bedrock::Guardrail`

To enable data events via CLI:

```
aws cloudtrail put-event-selectors --trail-name myTrail \
  --advanced-event-selectors '[
    {
      "Name": "Log Bedrock agent data events",
      "FieldSelectors": [
        { "Field": "eventCategory", "Equals": ["Data"] },
        { "Field": "resources.type", "Equals": ["AWS::Bedrock::AgentAlias"] }
      ]
    },
    {
      "Name": "Log Bedrock knowledge base data events",
      "FieldSelectors": [
        { "Field": "eventCategory", "Equals": ["Data"] },
        { "Field": "resources.type", "Equals": ["AWS::Bedrock::KnowledgeBase"] }
      ]
    }
  ]'
```

> ***Exam tip:*** *CloudTrail logs WHO called WHAT API and WHEN. It does NOT log the content of prompts and responses -- that is model invocation logging (CloudWatch Logs / S3). This distinction is tested frequently. CloudTrail = audit trail. Invocation logging = content analysis.*

4.3 Governance and Compliance Logging

CloudTrail supports:

- **Multi-region trails** -- log events from all regions into a single S3 bucket
- **Organization trails** -- log events from all accounts in an AWS Organization
- **CloudTrail Lake** -- serverless query engine for analyzing events using SQL-like syntax
- **Log file integrity validation** -- detect whether CloudTrail log files were modified

after delivery

- **KMS encryption** -- encrypt log files at rest

Amazon GuardDuty monitors CloudTrail logs to detect potential security issues in Bedrock APIs, such as unauthorized access patterns or anomalous invocation volumes.

4.4 CloudTrail Insights

CloudTrail Insights automatically detects unusual API activity patterns:

- Unusual volume of write management events
- Unusual volume of specific API calls
- Unusual error rates

When Insights detects anomalous activity, it generates an Insights event. This is useful for detecting sudden spikes in Bedrock API calls that might indicate compromised credentials or runaway automation.

> ***Exam tip:*** *CloudTrail Insights = unusual API ACTIVITY patterns (volume, errors). CloudWatch Anomaly Detection = unusual METRIC patterns (latency, token counts). These serve different purposes and the exam will try to confuse them.*

---

5. AWS Auto Scaling

5.1 Scaling AI Workloads

For Bedrock (serverless), there is no infrastructure to scale -- AWS manages capacity. However, Auto Scaling is relevant when you build GenAI application infrastructure:

- **API Gateway + Lambda** -- Lambda concurrency scales automatically. Use reserved

concurrency to guarantee capacity for GenAI workloads.

- **ECS/Fargate** -- Application Auto Scaling can scale tasks based on CloudWatch metrics.
- **SageMaker Endpoints** -- Application Auto Scaling adjusts the number of instances behind

a SageMaker inference endpoint.

5.2 Target Tracking for AI Metrics

Target tracking policies maintain a target value for a specific metric. For GenAI:

- Scale ECS tasks to maintain average CPU at 70%
- Scale SageMaker endpoint instances to maintain `InvocationsPerInstance` at a target value
- Use custom CloudWatch metrics as scaling signals (e.g., queue depth of pending inference

requests)

```
import {
  ApplicationAutoScalingClient,
  PutScalingPolicyCommand,
} from "@aws-sdk/client-application-auto-scaling";

const aas = new ApplicationAutoScalingClient({ region: "us-east-1" });

// Target tracking for a SageMaker endpoint
await aas.send(new PutScalingPolicyCommand({
  PolicyName: "SageMaker-Invocations-Target",
  ServiceNamespace: "sagemaker",
  ResourceId: "endpoint/my-genai-endpoint/variant/AllTraffic",
  ScalableDimension: "sagemaker:variant:DesiredInstanceCount",
  PolicyType: "TargetTrackingScaling",
  TargetTrackingScalingPolicyConfiguration: {
    TargetValue: 100, // target invocations per instance
    PredefinedMetricSpecification: {
      PredefinedMetricType: "SageMakerVariantInvocationsPerInstance",
    },
    ScaleInCooldown: 600,
    ScaleOutCooldown: 60,
  },
}));
```

> ***Exam tip:*** *Bedrock is serverless -- you do not configure Auto Scaling for it directly. If a question asks about scaling Bedrock, the answer involves Provisioned Throughput (reserved model units) or cross-region inference profiles, NOT Auto Scaling. Auto Scaling applies to SageMaker endpoints, ECS/Fargate tasks, and Lambda concurrency.*

---

6. Cost Tools

6.1 AWS Cost Explorer

Cost Explorer provides visualization of your AWS spending over time. For GenAI workloads:

- Filter by service (Amazon Bedrock) to see total spend
- Group by usage type to see input tokens vs. output tokens vs. Provisioned Throughput
- Use the right-sizing recommendations for SageMaker instances hosting custom models
- Filter by tag to see costs for specific GenAI applications (requires cost allocation tags)

Bedrock cost dimensions visible in Cost Explorer:

- On-Demand token usage (input and output, by model)
- Provisioned Throughput commitment costs
- Model customization (fine-tuning) job costs
- Knowledge base storage and retrieval costs

6.2 Token Cost Tracking

Bedrock pricing is per-token (or per-image for image models). Cost Explorer shows aggregate spend, but for per-request cost attribution you need to compute it from CloudWatch metrics:

```
// Compute estimated cost from CloudWatch metrics
const INPUT_PRICE_PER_1K = 0.003;  // example: Claude Sonnet input
const OUTPUT_PRICE_PER_1K = 0.015; // example: Claude Sonnet output

const inputTokens = 1500;  // from InputTokenCount metric
const outputTokens = 400;  // from OutputTokenCount metric

const estimatedCost = (inputTokens / 1000) * INPUT_PRICE_PER_1K
                    + (outputTokens / 1000) * OUTPUT_PRICE_PER_1K;
// $0.0105 for this single invocation
```

Use **cost allocation tags** in AWS to attribute Bedrock costs to specific teams or projects. Tags applied to Bedrock resources (custom models, Provisioned Throughput) propagate to Cost Explorer.

6.3 AWS Cost Anomaly Detection

Cost Anomaly Detection uses ML to identify unusual spending patterns. It runs approximately three times per day and evaluates spend data from Cost Explorer.

Monitor types:

- **AWS Managed Monitors** -- automatically track costs across all services, accounts, or

cost categories. Automatically include new accounts and tag values.

- **Customer Managed Monitors** -- monitor specific accounts, services, or tag values.

Useful for isolating GenAI workloads.

Key features:

- ML-based thresholds that adapt to organic growth and seasonal trends
- Root cause analysis identifying which service, account, or usage type caused the spike
- Alerts via email, SNS, or AWS Chatbot (Slack/Teams)
- Integration with EventBridge for automated remediation

GenAI-specific cost anomaly patterns:

- Sudden spike in token usage (new feature deployment, prompt engineering regression)
- Unexpected model invocation volume (runaway agent loops, misconfigured retry logic)
- Jump in Provisioned Throughput costs (unintended commitment)

- Knowledge base storage growth (embedding re-indexing)

The Well-Architected FSI Lens recommends tracking custom metrics like `token_in`, `token_out`, and `embedding_ops`, then correlating them with cost anomalies on CloudWatch dashboards.

> ***Exam tip:*** *Cost Anomaly Detection does NOT operate in real time -- there is a delay of several hours between spend occurring and anomaly detection. For real-time cost monitoring, use CloudWatch custom metrics with token counts and alarms.*

---

7. AWS Well-Architected Tool

7.1 The Generative AI Lens

The Well-Architected Generative AI Lens is a structured assessment framework for evaluating GenAI workloads against AWS best practices. It extends the six pillars of the Well-Architected Framework with GenAI-specific considerations.

Six pillars as applied to GenAI:

Pillar	GenAI Focus Areas
Operational Excellence	Model output quality monitoring, operational health, traceability, lifecycle automation, model customization decisions
Security	Endpoint protection, harmful output mitigation, event monitoring/audit, prompt security, model poisoning remediation
Reliability	Throughput handling, component communication, observability, graceful failure handling, artifact versioning, distributed inference
Performance Efficiency	Model performance capture and improvement, acceptable performance maintenance, computation optimization, data retrieval optimization
Cost Optimization	Cost-optimized model selection, inference cost/performance balance, prompt engineering for cost, vector store optimization, agent workflow optimization
Sustainability	Minimize compute for training/customization/hosting, model efficiency techniques, serverless architectures

Generative AI Lifecycle Stages:

1. **Scoping** -- define business problem, assess GenAI relevance, set success metrics
2. **Model Selection** -- evaluate models on modality, accuracy, latency, cost, context window
3. **Model Customization** -- prompt engineering, RAG, fine-tuning, distillation
4. **Development and Integration** -- inference optimization, agent orchestration, UI
5. **Deployment** -- controlled rollout, scaling, infrastructure
6. **Continuous Improvement** -- monitoring, feedback collection, iterative refinement

7.2 Using the Well-Architected Tool

The Generative AI Lens is available as a custom lens in the AWS Well-Architected Tool Lens Catalog. You can:

1. Download the lens JSON from the [AWS Well-Architected custom lens GitHub repository](https://github.com/aws-samples/sample-well-architected-custom-lens)
2. Import it into the Well-Architected Tool
3. Run workload reviews against the lens

4. Get improvement recommendations with links to implementation guidance

The tool generates a prioritized list of improvements and tracks progress over time.

> ***Exam tip:*** *The exam references the Well-Architected Framework and Generative AI Lens in Task 1.1.3. Know the six pillars and how they apply to GenAI. The lens is NOT just a document -- it is a structured assessment tool you can import and run in the WA Tool console. Also know that AWS launched a separate* ***Responsible AI Lens*** *and an updated* ***Machine Learning Lens*** *alongside the GenAI Lens.*

---

8. Other Services

8.1 AWS Chatbot

AWS Chatbot delivers AWS notifications to Slack channels and Microsoft Teams. For GenAI workloads:

- Route CloudWatch alarm notifications to Slack/Teams for real-time team awareness
- Receive Cost Anomaly Detection alerts in chat
- Run read-only AWS CLI commands directly from chat (e.g., describe alarm state)
- Integrate with SNS topics that receive alarm actions

GenAI relevance: When the exam mentions "notify the development team in Slack when Bedrock throttling exceeds a threshold," the answer involves CloudWatch Alarm -> SNS -> AWS Chatbot -> Slack.

8.2 Amazon Managed Grafana

Managed Grafana is a fully managed Grafana service for operational dashboards. It natively integrates with CloudWatch as a data source.

When to use Managed Grafana vs. CloudWatch Dashboards:

- Managed Grafana when you need advanced visualization, cross-data-source correlation

(e.g., Prometheus + CloudWatch), or your team already uses Grafana

- CloudWatch Dashboards when you want zero-setup, native AWS integration, or are building

dashboards programmatically via CloudFormation/CDK

For GenAI: Managed Grafana can pull Bedrock CloudWatch metrics and render them alongside application metrics from other sources, giving a unified operational view.

8.3 AWS Service Catalog

Service Catalog lets administrators create and manage catalogs of approved AWS resources. For GenAI:

- Publish pre-approved Bedrock configurations as Service Catalog products (e.g., a

CloudFormation template that provisions a Bedrock agent with guardrails, logging, and monitoring pre-configured)

- Enforce governance by restricting developers to approved model configurations
- Version control GenAI infrastructure templates

8.4 AWS Systems Manager

Systems Manager provides operational management for AWS resources. GenAI-relevant features:

- **Parameter Store** -- store configuration values like model IDs, prompt templates,

guardrail IDs, and feature flags. Supports encryption via KMS for sensitive values (API keys, connection strings). Use `SecureString` parameter type for secrets.

- **Session Manager** -- secure shell access to EC2 instances running custom model inference servers (no need for SSH keys or bastion hosts).
- **Automation** -- create runbooks for operational tasks like rotating Bedrock logging configurations or updating model versions across environments.
- **Patch Manager** -- keep EC2-based inference infrastructure patched.

> ***Exam tip:*** *If a question asks where to store a model endpoint URL or a prompt template version identifier that Lambda functions need at runtime, the answer is typically Systems Manager Parameter Store (for non-secret config) or Secrets Manager (for credentials). Parameter Store is free for standard parameters.*

Quick-Reference: When to Use What

Scenario	Service
"Monitor model invocation latency and token counts"	CloudWatch metrics (`AWS/Bedrock`)
"Analyze the actual prompts sent to the model"	CloudWatch Logs (model invocation logging)
"Query prompt patterns across thousands of invocations"	CloudWatch Logs Insights
"Detect unusual token usage patterns automatically"	CloudWatch Anomaly Detection
"Audit who called InvokeModel and when"	CloudTrail
"Track which IAM user created a knowledge base"	CloudTrail (management event)
"Log InvokeAgent calls for compliance"	CloudTrail data events (AWS::Bedrock::AgentAlias)
"Monitor availability of our GenAI API from outside"	CloudWatch Synthetics (canaries)
"Alert the team in Slack when latency breaches SLA"	CloudWatch Alarm -> SNS -> AWS Chatbot
"Visualize Bedrock metrics alongside Prometheus data"	Amazon Managed Grafana
"Detect unexpected Bedrock cost spike"	AWS Cost Anomaly Detection
"See Bedrock spend broken down by model"	AWS Cost Explorer
"Assess our GenAI workload against best practices"	Well-Architected Tool (GenAI Lens)
"Provide pre-approved GenAI infrastructure templates"	AWS Service Catalog
"Store model configuration securely"	Systems Manager Parameter Store

Scenario	Service
"Scale SageMaker inference endpoint instances"	Application Auto Scaling
"Detect unusual API call volume to Bedrock"	CloudTrail Insights
"Mask PII in Bedrock invocation logs"	CloudWatch Logs data protection
"Detect anomalous log patterns in invocation logs"	CloudWatch Logs anomaly detection

---

Common Exam Gotchas

1. **CloudTrail vs. Invocation Logging** -- CloudTrail captures API call metadata (who, what, when). Model invocation logging captures prompt/response content. The exam will present scenarios where one is clearly the right answer but the other sounds plausible.
2. **Bedrock metrics are automatic; invocation logging is not** -- Metrics in `AWS/Bedrock` flow to CloudWatch as soon as you make Bedrock API calls. Model invocation logging must be explicitly enabled in Bedrock settings.
3. **Throttles are not counted as Invocations or Errors** -- If you calculate error rates using metric math, you need to include throttles separately: `total_requests = Invocations + InvocationThrottles + InvocationClientErrors + InvocationServerErrors`.
4. **CloudWatch Anomaly Detection vs. CloudTrail Insights** -- Anomaly Detection is for metrics (latency, token counts). CloudTrail Insights is for API activity patterns (call volume, error rates). Different tools, different data sources.
5. **Cost Anomaly Detection has a delay** -- It runs ~3x/day. It will not catch a cost spike in real time. For real-time cost alerting, use CloudWatch custom metrics with alarms.
6. **Bedrock is serverless -- no Auto Scaling** -- You cannot attach Auto Scaling policies to Bedrock. Use Provisioned Throughput for guaranteed capacity, or cross-region inference for load distribution. Auto Scaling applies to SageMaker, ECS, Lambda concurrency.
7. **Three different Bedrock CloudWatch namespaces:**
   - `AWS/Bedrock` -- runtime metrics (invocations, tokens, latency)
   - `AWS/Bedrock/Guardrails` -- guardrail evaluation metrics
   - `AWS/Bedrock/Agents` -- agent invocation metrics
8. **Guardrail** `InvocationsIntervened` -- this is the metric that tells you how often guardrails blocked or modified content. For Task 3.4.2 (pre-defined fairness metrics), this metric combined with `GuardrailPolicyType` dimension gives you per-policy intervention rates.
9. **Well-Architected GenAI Lens is a custom lens** -- it must be downloaded and imported into the WA Tool. It is not a built-in lens. It covers all six pillars and all stages of the GenAI lifecycle.
10. **Log group size limits for CloudWatch Logs** -- Invocation log events are capped at 100 KB in CloudWatch Logs. Anything larger (images, long prompts/responses) must go to S3. If a question mentions large payloads, the answer involves S3 delivery.

Security, Identity, and Compliance -- Comprehensive Study Guide (AIP-C01)

Security services are foundational to the AWS Certified Generative AI Developer - Professional exam. IAM is the single most critical security topic, appearing in tasks across domains 2 and 3. This guide covers IAM (including Access Analyzer and Identity Center), Amazon Cognito, AWS KMS, AWS Secrets Manager, Amazon Macie, AWS WAF, and the AWS Encryption SDK.

---

1. IAM (Identity and Access Management)

IAM is the backbone of AWS security and is **heavily tested** on the exam. You must understand how IAM policies control access to Bedrock foundation models, how service roles enable AI agents, and how to implement least privilege for AI workloads.

1.1 IAM Policies for Amazon Bedrock Access

Amazon Bedrock uses **identity-based policies only** -- it does not support resource-based policies. All access control is done through policies attached to IAM users, groups, or roles.

Key Bedrock IAM Features

IAM Feature	Bedrock Support
Identity-based policies	Yes
Resource-based policies	**No**
Policy actions	Yes
Policy resources (ARN-level)	Yes
Policy condition keys	Yes
ABAC (tags in policies)	Yes
Temporary credentials	Yes
Service roles	Yes
Service-linked roles	**No**

Bedrock Resource ARN Format

```
arn:aws:bedrock:&lt;region&gt;::foundation-model/&lt;model-id&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:agent/&lt;agent-id&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:knowledge-base/&lt;kb-id&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:guardrail/&lt;guardrail-id&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:custom-model/&lt;model-name&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:inference-profile/&lt;profile-id&gt;
arn:aws:bedrock:&lt;region&gt;:&lt;account-id&gt;:flow/&lt;flow-id&gt;
```

> ***Exam Gotcha**: Foundation model ARNs have **no account ID** (they use `::` with an empty account field) because they are AWS-owned. Custom models, agents, knowledge bases, and guardrails use your account ID.*

Restricting Access to Specific Models

You can restrict which foundation models a principal can invoke by specifying model ARNs in the `Resource` element:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSpecificModels",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-5-sonnet*",
        "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-embed-text-v2*"
      ]
    }
  ]
}
```

Denying Access to Specific Models

For organizations that want to allow most models but block specific ones:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenySpecificModels",
      "Effect": "Deny",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:*::foundation-model/meta.llama*"
      ]
    }
  ]
}
```

AWS Managed Policies for Bedrock

Policy	Purpose
`AmazonBedrockFullAccess`	Full admin access to all Bedrock resources. Includes EC2, IAM, KMS, and SageMaker permissions.
`AmazonBedrockReadOnly`	Read-only access to all Bedrock resources.
`AmazonBedrockLimitedAccess`	Basic Bedrock API access, KMS key management, networking, and Marketplace subscriptions. Use when SCPs handle model-level governance.
`AmazonBedrockMarketplaceAccess`	Manage and use Bedrock Marketplace model endpoints with SageMaker integration.

***Exam Gotcha**:* `AmazonBedrockFullAccess` *grants permissions beyond just Bedrock -- it includes EC2 (for VPC config), IAM (for service roles), KMS (for encryption), and SageMaker (for custom models). In a least-privilege*

scenario, create a custom policy instead.

1.2 Bedrock Condition Keys

Condition keys let you add fine-grained restrictions to IAM policies beyond just actions and resources.

Condition Key	Description	Type
`bedrock:GuardrailIdentifier`	Enforce a specific guardrail ARN (or ARN:version) on model invocations	ARN
`bedrock:InferenceProfileArn`	Restrict to a specific inference profile	ARN
`bedrock:PromptRouterArn`	Restrict to a specific prompt router	ARN
`bedrock:BearerTokenType`	Filter by SHORT_TERM or LONG_TERM bearer tokens	String
`bedrock:ServiceTier`	Restrict by service tier	String
`bedrock:ThirdPartyKnowledgeBaseCredentialsSecretArn`	Restrict third-party KB credentials	ARN
`bedrock:InlineAgentName`	Restrict InvokeInlineAgent API by agent name	String
`aws:RequestTag/${TagKey}`	Filter on tags in the create request	String
`aws:ResourceTag/${TagKey}`	Filter on tags attached to the resource	String

Enforcing Guardrails via IAM Policy

This is a **new and exam-relevant** feature. You can force all model invocations to use a specific guardrail:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceGuardrail",
      "Effect": "Deny",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "bedrock:GuardrailIdentifier": "arn:aws:bedrock:us-east-1:123456789012:guardrail/abc123"
        }
      }
    }
  ]
}
```

If the guardrail in the IAM policy does not match the guardrail specified in the API call, the request is rejected.

Controlling Marketplace Models with Product ID Condition Keys

Third-party models from providers like Anthropic, Cohere, and Stability are offered through AWS Marketplace and have product IDs. You can use the `aws-marketplace:ProductId` condition key to control access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowOnlySpecificMarketplaceModels",
      "Effect": "Allow",
      "Action": "aws-marketplace:Subscribe",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws-marketplace:ProductId": [
            "prod-abc123",
            "prod-def456"
          ]
        }
      }
    }
  ]
}
```

> ***Exam Gotcha****: Models from Amazon, DeepSeek, Mistral AI, Meta, Qwen, and OpenAI are* ***not sold through AWS Marketplace*** *and have no product keys. To restrict them, deny the* `bedrock:InvokeModel` *action with their model IDs in the* `Resource` *field instead.*

1.3 Service Roles for AI Services

Amazon Bedrock Agents, Knowledge Bases, and model customization jobs all require IAM service roles. These roles allow the Bedrock service to act on your behalf.

Bedrock Agent Service Role

The trust policy must allow the `bedrock.amazonaws.com` service principal to assume the role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "bedrock.amazonaws.com"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "123456789012"
        },
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:bedrock:us-east-1:123456789012:agent/*"
        }
      }
    }
  ]
}
```

The `aws:SourceAccount` and `AWS:SourceArn` conditions prevent the confused deputy problem. After creating agents, narrow the `SourceArn` from `agent/*` to the specific agent ID.

Permissions the Agent Service Role Needs

Permission	Purpose
`bedrock:InvokeModel` on foundation model ARNs	Run inference with the agent's FM
`s3:GetObject` on schema bucket	Access OpenAPI schemas for action groups
`bedrock:Retrieve` on knowledge base ARNs	Query attached knowledge bases
`bedrock:ApplyGuardrail` (optional)	Apply guardrails to agent responses
`lambda:InvokeFunction`	**Not on the service role** -- use a resource-based policy on the Lambda instead

> ***Exam Gotcha**: Lambda permissions for agent action groups are granted via a **resource-based policy on the Lambda function**, not via the agent's service role. The resource-based policy allows* `bedrock.amazonaws.com` *to invoke the function.*

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowBedrockToInvokeLambda",
      "Effect": "Allow",
      "Principal": {
        "Service": "bedrock.amazonaws.com"
      },
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:my-agent-action",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:bedrock:us-east-1:123456789012:agent/AGENT_ID"
        }
      }
    }
  ]
}
```

Bedrock Does Not Use Service-Linked Roles

Bedrock does **not** create service-linked roles. You must create and manage service roles yourself (or let the console create them). This is different from services like SageMaker or RDS.

1.4 Cross-Account Access for Foundation Models

Since Bedrock does not support resource-based policies, cross-account access requires **IAM role assumption**:

1. **Account B** (the one with the model access) creates an IAM role with Bedrock permissions
2. The trust policy on that role allows **Account A** to assume it
3. **Account A** uses `sts:AssumeRole` to get temporary credentials
4. Account A uses those credentials to call Bedrock APIs in Account B

For cross-account Bedrock Agents or Knowledge Bases, each resource must be in an account with the appropriate model access. Cross-Region inference profiles can span regions within the same account but not across accounts.

1.5 Least Privilege for AI API Access

The exam tests your ability to apply least privilege to AI workloads. Key principles:

1. **Scope model access**: Only allow the specific model IDs the application needs
2. **Scope actions**: Only grant `bedrock:InvokeModel` (not `bedrock:*`) for inference workloads
3. **Use condition keys**: Enforce guardrails, restrict inference profiles, require tags
4. **Separate roles**: Use different IAM roles for different tasks (model invocation vs. model customization vs. agent management)
5. **Time-bound credentials**: Use temporary credentials via STS, not long-term access keys
6. **Regularly review**: Use IAM Access Analyzer to find unused permissions

Example: Minimal Invoke-Only Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeClaudeOnly",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-5-sonnet*"
    }
  ]
}
```

Bedrock API Key Security

Bedrock supports API keys (bearer tokens) as an alternative authentication mechanism. Key points:

- **Short-term keys**: Generated locally, last up to 12 hours or the IAM session duration (whichever is shorter). Cannot be listed or retrieved via API.
- **Long-term keys**: Managed via IAM console, can have configurable expiration (1 day to indefinite). Tracked via CloudTrail.
- **Best practice**: Always prefer STS temporary credentials. If API keys are necessary, use short-term keys. Store long-term keys in Secrets Manager.

Control API key usage with condition keys:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PreventLongTermAPIKeys",
      "Effect": "Deny",
      "Action": "bedrock:CallWithBearerToken",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "bedrock:BearerTokenType": "LONG_TERM"
        }
```

```
      }
    }
  ]
}
```

1.6 Service Control Policies (SCPs) for AI Governance

SCPs are used at the AWS Organizations level to set **permission guardrails** across all accounts. They do not grant permissions -- they only restrict what is allowed.

Key SCP Concepts for the Exam

- SCPs affect all IAM users and roles in member accounts (not the management account)
- SCPs apply to the root user in member accounts
- SCPs do NOT affect service-linked roles
- For a permission to be allowed, every level in the OU hierarchy must allow it
- Deny always wins over Allow

SCP to Restrict Bedrock Models Org-Wide

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyNonApprovedModels",
      "Effect": "Deny",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "NotResource": [
        "arn:aws:bedrock:*::foundation-model/anthropic.claude-3-5-sonnet*",
        "arn:aws:bedrock:*::foundation-model/amazon.titan-embed-text-v2*"
      ]
    }
  ]
}
```

SCP to Block Cross-Region Inference

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyCrossRegionInference",
      "Effect": "Deny",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": "arn:aws:bedrock:*:*:inference-profile/*"
    }
  ]
}
```

SCP to Prevent AI Data Opt-In

AWS lets you opt out of AI services using your content to improve their models. Use an SCP to enforce this org-wide:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAIOptIn",
      "Effect": "Deny",
      "Action": [
        "organizations:DisableAWSServiceAccess"
      ],
      "Resource": "*"
    }
  ]
}
```

> ***Exam Gotcha**: SCPs set maximum permissions. Even if an IAM policy allows* `bedrock:InvokeModel` *on all models, an SCP denying specific models takes precedence. The exam may test whether you choose IAM policies (account-level) vs SCPs (org-level) for different governance scenarios.*

1.7 IAM Access Analyzer for AI Permissions

IAM Access Analyzer helps you achieve least privilege by finding **unused access**, **external access**, and **overly permissive policies**.

Three Types of Analyzers

Analyzer Type	What It Finds	Scope
External access	Resources shared with principals outside your zone of trust	Account or Organization
Internal access	Access paths between IAM principals and resources within your org	Account or Organization
Unused access	Unused roles, access keys, passwords, and service/action-level permissions	Account or Organization

How It Applies to AI Workloads

1. **Find unused Bedrock permissions**: If a role has `bedrock:*` but only uses `bedrock:InvokeModel`, the unused access analyzer flags the excess permissions
2. **Policy validation**: Validates IAM policies against 100+ checks before deployment. Integrates into CI/CD pipelines.
3. **Policy generation**: Generates least-privilege policies based on CloudTrail access activity. Start broad, then refine based on actual usage.
4. **Custom policy checks**: Validate policies against your org's security standards (e.g., "no policy should allow `bedrock:*`")

Key Exam Facts

- External access findings are **free**
- Unused and internal access findings are **charged** per IAM role/user analyzed per month
- Unused access tracking period is configurable: 1 to 365 days

- You can exclude specific accounts, roles, or users from analysis
- Integrates with **Security Hub** and **EventBridge** for automated responses
- Uses **automated reasoning** (provable security), not sampling or heuristics

1.8 TypeScript SDK v3 Examples for IAM

Assuming a Role for Bedrock Access

```
import { STSClient, AssumeRoleCommand } from "@aws-sdk/client-sts";
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const stsClient = new STSClient({ region: "us-east-1" });

const assumeRoleResponse = await stsClient.send(
  new AssumeRoleCommand({
    RoleArn: "arn:aws:iam::123456789012:role/bedrock-invoke-role",
    RoleSessionName: "bedrock-session",
    DurationSeconds: 3600,
  })
);

const bedrockClient = new BedrockRuntimeClient({
  region: "us-east-1",
  credentials: {
    accessKeyId: assumeRoleResponse.Credentials!.AccessKeyId!,
    secretAccessKey: assumeRoleResponse.Credentials!.SecretAccessKey!,
    sessionToken: assumeRoleResponse.Credentials!.SessionToken!,
  },
});
```

---

2. IAM Identity Center

IAM Identity Center (formerly AWS SSO) provides **centralized access management** across multiple AWS accounts and applications using single sign-on.

2.1 Core Concepts

Concept	Description
Identity source	Where user identities are managed: Identity Center directory, Active Directory, or external IdP (Okta, Entra ID, Ping)
Permission sets	Templates defining a collection of IAM policies. Assigned to users/groups for specific accounts.
AWS access portal	Web portal where users sign in once and access all assigned accounts and applications
Delegated administrator	A member account designated to manage Identity Center (recommended over using the management account)

2.2 SSO for AI Applications

IAM Identity Center is relevant to the exam when:

- **Multi-account AI workloads**: Different accounts for dev, staging, and production AI environments. Identity Center provides SSO across all of them.
- **Team-based access**: Data scientists, ML engineers, and app developers get different permission sets
- **Federated access to Bedrock console**: Users sign in with corporate credentials via SAML/OIDC federation to access the Bedrock console

2.3 Permission Sets for AI Teams

Permission sets define what a user can do when they sign into a specific account. They support:

- **AWS managed policies** (e.g., `AmazonBedrockFullAccess`, `AmazonBedrockReadOnly`)
- **Customer managed policies** (policies you create in each target account)
- **Inline policies** (embedded directly in the permission set)
- **Permissions boundaries** (maximum permission limits)

Example Permission Sets for AI Teams

Team	Permission Set	Policies
AI Developers	`BedrockDeveloper`	`AmazonBedrockFullAccess`, `AmazonS3ReadOnlyAccess`
Data Scientists	`BedrockDataScience`	Custom policy with `bedrock:InvokeModel`, `bedrock:CreateModelCustomizationJob`, `s3:GetObject`
AI Operations	`BedrockOps`	Custom policy with `bedrock:Get*`, `bedrock:List*`, `cloudwatch:GetMetricData`
Security/Audit	`BedrockAudit`	`AmazonBedrockReadOnly`, `SecurityAudit` managed policy

> ***Exam Gotcha**: Permission sets create **IAM roles** in each assigned account. When a user signs in via the access portal, they assume one of these roles. The session duration is configurable (1-12 hours). Users should choose the least-privileged permission set when multiple are available.*

2.4 Key Exam Facts

- Identity Center requires **AWS Organizations**
- Creates a service-linked role (`AWSServiceRoleForSSO`) in each account
- Supports **SCIM** for automatic user/group provisioning from external IdPs
- Can be combined with SCPs to restrict what permission sets can do
- Session duration is separate from the permission set session duration
- You can limit identity store access from member accounts using SCPs

---

3. Amazon Cognito

Amazon Cognito provides **authentication, authorization, and user management** for web and mobile applications. It is the primary service for adding user sign-in to AI-powered applications.

3.1 User Pools vs Identity Pools

These are the two core components, and the exam tests your understanding of when to use each.

Feature	User Pools	Identity Pools
Primary purpose	Authentication (who are you?)	Authorization (what can you access?)
Output	JSON Web Tokens (JWTs): ID token, access token, refresh token	Temporary AWS credentials (access key, secret key, session token)
Use case	Sign-up, sign-in, token management for your app	Granting users direct access to AWS services (S3, DynamoDB, etc.)
Supports	Username/password, social login, SAML, OIDC, MFA, passwordless	AWS credentials via IAM roles, ABAC, guest access
Federation	Acts as the identity provider itself	Consumes tokens from user pools, social IdPs, SAML, OIDC

The Combined Flow (Most Common Pattern)

1. User signs in to **User Pool** (via managed login, SDK, or hosted UI)
2. User Pool returns **JWTs** (ID token + access token + refresh token)
3. App sends the ID token to an **Identity Pool**
4. Identity Pool validates the token, maps the user to an **IAM role**
5. Identity Pool returns **temporary AWS credentials**
6. App uses credentials to access AWS services directly

> ***Exam Gotcha**: User pools issue **JWTs** for API authorization. Identity pools issue **AWS credentials** for direct AWS service access. You need user pools for signing users into your AI app. You need identity pools only if the client needs to call AWS services directly (e.g., uploading files to S3 from a mobile app).*

3.2 Token Management

Token	Lifetime	Purpose
ID token	5 min - 1 day (default 30 min)	Contains user identity claims (`sub`, `email`, `cognito:groups`). Used by API Gateway Cognito authorizers.
Access token	5 min - 1 day (default 30 min)	Contains scopes and authorized actions. Used for resource server authorization.
Refresh token	1 hour - 3650 days (default 30 days)	Used to get new ID and access tokens without re-authenticating.

Key points:

- ID and access tokens are **JWTs** that can be verified locally (check signature, issuer, expiration, audience)
- Refresh tokens are **opaque** -- they must be sent to Cognito to get new tokens
- **Token customization**: Use a pre-token-generation Lambda trigger to add custom claims to tokens
- **Token revocation**: Supported for refresh tokens. Revoking a refresh token invalidates all access and ID tokens issued from it.

3.3 Integration with API Gateway for AI Endpoints

This is a high-probability exam topic: securing your AI application's API with Cognito.

Cognito Authorizer on REST API (Most Common Pattern)

1. Create a Cognito User Pool
2. Create a `COGNITO_USER_POOLS` authorizer in API Gateway
3. Enable it on API methods
4. Client gets an ID token from Cognito and passes it in the `Authorization` header
5. API Gateway validates the token and forwards the request to Lambda/Bedrock

```
Client --&gt; [ID Token] --&gt; API Gateway (Cognito Authorizer) --&gt; Lambda --&gt; Bedrock
```

JWT Authorizer on HTTP API

For HTTP APIs (v2), use a JWT authorizer instead:

1. Configure the JWT authorizer with the Cognito User Pool issuer URL and audience (client ID)
2. Client passes an access token or ID token in the `Authorization` header
3. API Gateway validates the JWT signature, issuer, audience, and expiration
4. Optionally require specific scopes for route-level authorization

Cognito + Amazon Verified Permissions

For **fine-grained authorization** (RBAC/ABAC) beyond what scopes provide:

1. Cognito issues tokens with group/attribute claims
2. Application calls Amazon Verified Permissions with the token
3. Verified Permissions evaluates Cedar policies against token claims
4. Returns Allow or Deny decision

> ***Exam Gotcha**: API Gateway Cognito authorizers validate the **ID token**. JWT authorizers validate either ID or access tokens. For scope-based authorization (e.g., "this API requires the `ai:invoke` scope"), use the **access token** with a JWT authorizer.*

3.4 Empowering AI Agents with User Context

A key pattern for the exam: AI agents that act on behalf of users.

1. User signs in to the app, receives tokens from Cognito
2. The AI agent (represented as a Cognito app client) performs an **OAuth 2.0 client credentials grant**
3. The agent passes the user's access token as context using the `aws_client_metadata` parameter
4. A **pre-token-generation Lambda trigger** customizes the agent's access token to include the user's claims
5. The agent uses its customized token to call downstream services on behalf of the user
6. Downstream services verify the token and authorize based on both agent and user claims

3.5 TypeScript SDK v3 Examples

Authenticating a User

```
import {
  CognitoIdentityProviderClient,
  InitiateAuthCommand,
} from "@aws-sdk/client-cognito-identity-provider";

const cognitoClient = new CognitoIdentityProviderClient({
  region: "us-east-1",
});

const authResponse = await cognitoClient.send(
```

```
    new InitiateAuthCommand({
      AuthFlow: "USER_PASSWORD_AUTH",
      ClientId: "your-app-client-id",
      AuthParameters: {
        USERNAME: "user@example.com",
        PASSWORD: "securePassword123!",
      },
    })
  );

  const idToken = authResponse.AuthenticationResult?.IdToken;
  const accessToken = authResponse.AuthenticationResult?.AccessToken;
  const refreshToken = authResponse.AuthenticationResult?.RefreshToken;
```

Getting AWS Credentials from Identity Pool

```
import { CognitoIdentityClient } from "@aws-sdk/client-cognito-identity";
import { fromCognitoIdentityPool } from "@aws-sdk/credential-providers";

const credentials = fromCognitoIdentityPool({
  clientConfig: { region: "us-east-1" },
  identityPoolId: "us-east-1:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  logins: {
    "cognito-idp.us-east-1.amazonaws.com/us-east-1_XXXXXXXXX": idToken!,
  },
});

// Use these credentials with any AWS SDK client
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
const s3Client = new S3Client({ region: "us-east-1", credentials });
```

3.6 Key Exam Facts

- Cognito scales to **millions of users** and **100+ billion authentications per month**
- User pool sign-in method (email, phone, username) **cannot be changed** after creation
- Custom attributes **cannot be removed** after being added
- Cognito does **not** provide built-in backup or export -- you must implement your own
- MFA options: SMS, TOTP (authenticator app), email OTP
- Advanced security features: adaptive authentication, compromised credentials detection
- Cognito supports **passwordless** authentication (passkeys, email/SMS OTP)
- After 5 failed sign-in attempts, lockout durations increase up to ~15 minutes

---

4. AWS KMS (Key Management Service)

KMS provides centralized key management for encrypting data in AI workloads. The exam tests your understanding of key types, key policies, and how KMS integrates with Bedrock and other AI data stores.

4.1 Key Types

Key Type	Managed By	Rotation	Use Case
AWS owned keys	AWS (invisible to you)	AWS-managed	Default encryption for many services. Free. No visibility or control.
AWS managed keys	AWS (visible in KMS)	Every year (automatic)	Default option when you enable encryption. Format: `aws/<service>`. Limited policy control.
Customer managed keys (CMKs)	You	Configurable (annual automatic + on-demand)	Full control over key policy, rotation, grants, and auditing. Costs $1/month/key + API charges.

> ***Exam Gotcha**: When the exam asks about "customer managed KMS key" vs "AWS managed key", the differentiator is **control**. Customer managed keys let you define key policies, enable/disable keys, grant cross-account access, and audit usage via CloudTrail. AWS managed keys are simpler but less flexible.*

4.2 Encryption for AI Data

Bedrock Encryption

Resource	Default Encryption	Customer Managed Key Support
Model customization (training data, output)	AWS owned key	Yes -- specify during job creation
Agents	AWS owned key	Yes -- specify during agent creation
Knowledge Bases	AWS owned key	Yes -- for the KB metadata. Vector store encryption is separate.
Guardrails	AWS owned key	Yes -- specify during creation
Model invocation logging	AWS owned key	Yes -- for CloudWatch Logs and S3 output
Provisioned Throughput	AWS owned key	Yes

Common AI Data Stores

Data Store	Encryption at Rest	KMS Integration
S3 (training data, RAG documents)	SSE-S3 (default), SSE-KMS, SSE-C, or client-side	Full CMK support. Bucket keys reduce KMS costs.
OpenSearch Serverless (vector store)	Always encrypted	CMK support for collection encryption
Amazon Aurora (metadata, structured data)	Encrypted by default	CMK support at cluster creation
DynamoDB (session state, metadata)	Encrypted by default	AWS owned, AWS managed, or CMK
Amazon RDS/Aurora (structured data)	Optional, immutable after creation	CMK support

4.3 Key Policies for AI Services

When Bedrock (or other AI services) need to use your CMK, the key policy must grant the service permission. This is done via **grants** or direct key policy statements.

Key Policy Allowing Bedrock to Use a CMK

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowBedrockServiceRole",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/bedrock-agent-role"
      },
      "Action": [
        "kms:Decrypt",
        "kms:GenerateDataKey",
        "kms:DescribeKey",
        "kms:CreateGrant"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "kms:ViaService": "bedrock.us-east-1.amazonaws.com"
        }
      }
    }
  ]
}
```

The `kms:ViaService` condition key ensures the key can only be used when the request comes through the Bedrock service.

The Lambda Execution Role for Secrets Rotation Needs KMS Permissions

If a secret encrypted with a CMK is being rotated:

```
{
  "Effect": "Allow",
  "Action": [
    "kms:Decrypt",
    "kms:GenerateDataKey"
  ],
  "Resource": "arn:aws:kms:us-east-1:123456789012:key/key-id"
}
```

4.4 Envelope Encryption

This is how KMS protects large amounts of data:

1. KMS generates a **data key** (plaintext + encrypted copy)
2. You use the plaintext data key to encrypt your data locally
3. You discard the plaintext data key
4. You store the **encrypted data key** alongside the encrypted data
5. To decrypt: send the encrypted data key to KMS, get plaintext key back, decrypt data

This is the same pattern used by the AWS Encryption SDK and by services like S3 (SSE-KMS).

> ***Exam Gotcha**: KMS can only directly encrypt/decrypt data up to **4 KB**. For anything larger, you must use envelope encryption (generate a data key, encrypt locally). The AWS Encryption SDK automates this.*

4.5 TypeScript SDK v3 Examples

Encrypting Data with a KMS Key

```
import { KMSClient, GenerateDataKeyCommand, DecryptCommand } from "@aws-sdk/client-kms";
import { createCipheriv, createDecipheriv, randomBytes } from "crypto";

const kmsClient = new KMSClient({ region: "us-east-1" });

// Generate a data key for envelope encryption
const dataKeyResponse = await kmsClient.send(
  new GenerateDataKeyCommand({
    KeyId: "arn:aws:kms:us-east-1:123456789012:key/my-key-id",
    KeySpec: "AES_256",
  })
);

const plaintextKey = dataKeyResponse.Plaintext!; // Use to encrypt data
const encryptedKey = dataKeyResponse.CiphertextBlob!; // Store alongside encrypted data

// Encrypt your data with the plaintext key
const iv = randomBytes(16);
const cipher = createCipheriv("aes-256-gcm", plaintextKey, iv);
const encrypted = Buffer.concat([cipher.update("sensitive AI data"), cipher.final()]);
const authTag = cipher.getAuthTag();

// Store: encryptedKey + iv + authTag + encrypted data
// Discard: plaintextKey (don't persist it)
```

4.6 Key Exam Facts

- KMS keys are **regional** -- a key in us-east-1 cannot be used in eu-west-1 unless you use multi-region keys
- **Multi-region keys**: Replicated keys with the same key material across regions. Useful for cross-region inference or disaster recovery.
- **S3 Bucket Keys**: Reduce KMS API calls (and costs) by generating a bucket-level key that is reused for objects. Reduces costs by up to 99%.
- Key deletion has a **waiting period** of 7-30 days (default 30). Keys cannot be recovered after deletion.
- **Automatic key rotation**: Customer managed symmetric keys only. Rotates every year by default (configurable). Old key material is retained for decryption.
- All KMS API calls are logged in **CloudTrail** -- this is how you audit who decrypted what

---

5. AWS Secrets Manager

Secrets Manager securely stores and rotates credentials, API keys, and other sensitive configuration needed by AI applications.

5.1 API Key Management for AI Services

AI applications often need to manage:

- Third-party AI API keys (OpenAI, Anthropic direct, etc.)
- Database credentials for RAG data sources

- OAuth client secrets for external integrations
- Encryption keys for client-side encryption

Secrets Manager provides:

- **Encryption at rest** using KMS (AWS managed key by default, CMK optional)
- **Fine-grained IAM access control** per secret
- **Versioning** with staging labels (AWSCURRENT, AWSPREVIOUS, AWSPENDING)
- **Cross-region replication** for disaster recovery
- **Automatic rotation** via Lambda functions

5.2 Rotation

Secrets Manager rotation is powered by Lambda functions that execute a four-step workflow:

Step	Lambda Handler	What Happens
1. `createSecret`	Creates a new secret version with `AWSPENDING` label	Generates new credentials
2. `setSecret`	Updates the external service with new credentials	Changes password on the database/service
3. `testSecret`	Validates the new credentials work	Attempts a connection with new credentials
4. `finishSecret`	Moves `AWSCURRENT` to new version, old becomes `AWSPREVIOUS`	Completes the rotation

Rotation Strategies

Strategy	Description	When to Use
Single user	Updates the password for a single user	Simple applications with one set of credentials
Alternating users	Creates two users and alternates between them	Zero-downtime rotation. Requires a superuser secret to create the alternate user.

Lambda Execution Role for Rotation

The Lambda function needs:

- `secretsmanager:DescribeSecret`, `secretsmanager:GetSecretValue`, `secretsmanager:PutSecretValue`, `secretsmanager:UpdateSecretVersionStage`
- `kms:Decrypt`, `kms:GenerateDataKey` (if using CMK)
- Database/service-specific permissions (e.g., `rds:DescribeDBInstances` for RDS)

Network Access

- If the Lambda function runs in a VPC, it needs either a **NAT gateway** or a **Secrets Manager VPC endpoint** to reach the Secrets Manager API
- The Lambda security group must allow outbound access to the database/service

5.3 Integration with Lambda

Two approaches for retrieving secrets in Lambda functions:

Approach	Description	Best For
AWS Parameters and Secrets Lambda Extension	Runtime-agnostic HTTP interface. Caches secrets for 300 seconds by default. Added as a Lambda layer.	Any runtime. Simple setup.
Powertools for AWS Lambda	Code-integrated utility. Supports multiple providers (Secrets Manager, Parameter Store, AppConfig). Built-in transformations and type safety.	Python, TypeScript, Java, .NET.

Both are better than calling the Secrets Manager API directly because they cache secrets locally, reducing API calls and latency.

5.4 TypeScript SDK v3 Examples

Retrieving a Secret

```
import {
  SecretsManagerClient,
  GetSecretValueCommand,
} from "@aws-sdk/client-secrets-manager";

const smClient = new SecretsManagerClient({ region: "us-east-1" });

const secret = await smClient.send(
  new GetSecretValueCommand({
    SecretId: "my-ai-api-key",
    VersionStage: "AWSCURRENT",
  })
);

const apiKey = secret.SecretString;
```

Creating a Secret with Rotation

```
import {
  SecretsManagerClient,
  CreateSecretCommand,
  RotateSecretCommand,
} from "@aws-sdk/client-secrets-manager";

const smClient = new SecretsManagerClient({ region: "us-east-1" });

await smClient.send(
  new CreateSecretCommand({
    Name: "prod/ai-service/api-key",
    SecretString: JSON.stringify({ apiKey: "sk-xxx" }),
    KmsKeyId: "arn:aws:kms:us-east-1:123456789012:key/my-key-id",
    Tags: [{ Key: "Environment", Value: "production" }],
  })
);

await smClient.send(
  new RotateSecretCommand({
    SecretId: "prod/ai-service/api-key",
    RotationLambdaARN: "arn:aws:lambda:us-east-1:123456789012:function:rotate-api-key",
    RotationRules: {
```

```
        AutomaticallyAfterDays: 30,
      },
    })
  );
```

5.5 Key Exam Facts

- Secrets are encrypted at rest with KMS (default: `aws/secretsmanager` managed key)
- Pricing: $0.40/secret/month + $0.05 per 10,000 API calls
- Secrets can store up to **65,536 bytes**
- Secrets Manager supports **resource-based policies** (unlike Bedrock)
- Cross-region replication is automatic for replica secrets
- Secrets Manager is different from Systems Manager Parameter Store:
  - **Secrets Manager**: Built-in rotation, higher cost, designed for credentials
  - **Parameter Store**: No built-in rotation (requires custom Lambda), lower cost (free tier), designed for configuration

---

6. Amazon Macie

Amazon Macie discovers and protects **sensitive data in S3** using machine learning and pattern matching. It is tested in the context of securing AI training data and ensuring PII is not inadvertently used.

6.1 PII Detection in S3

Macie provides two methods for finding sensitive data:

Method	Description	When to Use
Automated sensitive data discovery	Continuously samples and analyzes representative objects across your S3 estate. Daily evaluation. Breadth-first approach.	Ongoing monitoring of all S3 buckets
Sensitive data discovery jobs	Targeted, deeper analysis of specific buckets/prefixes. One-time or scheduled.	Pre-processing AI training data, compliance audits

What Macie Detects

Macie uses **managed data identifiers** (built-in) and **custom data identifiers** (user-defined regex):

Category	Examples
Credentials	AWS secret access keys, SSH private keys, PGP keys, HTTP auth headers
Financial	Credit card numbers, expiration dates, bank account numbers, magnetic stripe data
Personal (PII)	SSNs, passport numbers, driver's license numbers, names, addresses, phone numbers
Health (PHI)	Health insurance IDs, DEA numbers, National Provider Identifiers, HCPCS codes

Custom data identifiers use **regular expressions** to find organization-specific sensitive data (employee IDs, internal project codes, etc.).

Allow Lists

Allow lists tell Macie to **ignore** specific patterns or text. Use them for:

- Known test credit card numbers
- Public phone numbers (company switchboard)
- Internal identifiers that match PII patterns but are not sensitive

6.2 Sensitive Data Discovery for AI Training Data

Before using data to fine-tune or pre-train a model, scan it with Macie:

1. Create a **sensitive data discovery job** targeting the S3 bucket with training data
2. Configure it to use the relevant managed data identifiers (PII, PHI, credentials)
3. Add custom data identifiers for organization-specific sensitive data
4. Review findings to identify objects containing sensitive data
5. Remediate: remove, redact, or exclude flagged objects from the training dataset

Macie produces:

- **Sensitive data findings**: Detailed reports per-object, including data type, count, and location within the file
- **Discovery results**: Records of each analysis, stored for 90 days (configure S3 export for long-term retention)
- **Sensitivity scores**: Per-bucket scores (0-100) based on the amount and type of sensitive data found

6.3 Macie vs Amazon Comprehend PII Detection

This is an exam topic -- understanding when to use each:

Feature	Amazon Macie	Amazon Comprehend
Primary purpose	S3 data security posture	NLP text analysis
Input	S3 objects (files, documents)	Text strings (API input)
Detection method	ML + pattern matching on files	NLP models on text
Scope	Entire S3 estate	Individual text documents/strings
PII capabilities	Discover PII in stored data	Detect and redact PII in text at runtime
Output	Security findings + sensitivity scores	Entity list with confidence scores + redacted text
Use case	"Scan my training data for PII before fine-tuning"	"Redact PII from user prompts before sending to Bedrock"
Integration	S3, Security Hub, EventBridge	Bedrock Guardrails (PII filter), Lambda
Cost model	Per GB scanned + per bucket monitored	Per unit of text analyzed

> ***Exam Gotcha**: Use **Macie** for discovering PII in S3 data stores (bulk, at-rest scanning). Use **Comprehend** (or Bedrock Guardrails PII filters) for detecting/redacting PII in real-time text processing (in-transit). They serve different purposes and are not interchangeable.*

6.4 Key Exam Facts

- Macie is **optimized for S3** only. For other data stores, move data to S3 first.
- Macie can analyze **encrypted** objects if it has access to the decryption key
- 30-day free trial includes automated discovery (up to 150 GB) and bucket inventory
- Macie creates a service-linked role for accessing S3
- Stores discovery results for **90 days** -- configure S3 export for longer retention
- Security Hub controls: `Macie.1` (is Macie enabled?) and `Macie.2` (is automated discovery enabled?)

- Multi-account support: administrator account manages Macie across organization accounts
- Macie findings provide **exact location data**: line numbers, page numbers, cell references, JSON paths (up to 15 occurrences per finding, 1,000 per discovery result)

---

7. AWS WAF (Web Application Firewall)

AWS WAF protects web applications and APIs from common web exploits. For the exam, focus on protecting AI API endpoints from abuse, rate limiting expensive inference calls, and managing AI bot traffic.

7.1 Protecting AI API Endpoints

WAF integrates with **API Gateway**, **CloudFront**, **ALB**, and **AppSync** -- the common front doors for AI applications.

Architecture

```
Client --&gt; CloudFront (WAF) --&gt; API Gateway --&gt; Lambda --&gt; Bedrock
Client --&gt; API Gateway (WAF) --&gt; Lambda --&gt; Bedrock
Client --&gt; ALB (WAF) --&gt; ECS/EKS --&gt; Bedrock
```

WAF is the **first line of defense**, evaluated before other access control features (Cognito authorizers, IAM, etc.).

Web ACL Rule Types

Rule Type	Description
AWS Managed Rules	Pre-built rule groups from AWS. Include Core Rule Set, Known Bad Inputs, SQL injection, Bot Control.
Rate-based rules	Block/count requests that exceed a threshold per time window. Key for AI API protection.
IP-based rules	Allow/block specific IP addresses or CIDR ranges
Geo-match rules	Allow/block based on country of origin
Custom rules	Match on headers, body, query strings, URI path, HTTP method, etc.

7.2 Rate Limiting for AI APIs

AI inference is **expensive** -- rate limiting prevents abuse and controls costs.

Rate-Based Rules

- Minimum threshold: **100 requests per 5-minute window** (changed from the previous 2,000)
- Aggregation keys: IP address, forwarded IP, HTTP method, query string, custom header, label namespace, or combination
- Action when threshold exceeded: Block, Count, CAPTCHA, or Challenge
- Mitigation lag: 30-50 seconds after the threshold is exceeded

Example: Rate Limit AI Inference by API Key

```
{
  "Name": "RateLimitAIInference",
  "Priority": 1,
  "Statement": {
    "RateBasedStatement": {
```

```
        "Limit": 100,
        "AggregateKeyType": "CUSTOM_KEYS",
        "CustomKeys": [
          {
            "Header": {
              "Name": "x-api-key",
              "TextTransformations": [
                {
                  "Priority": 0,
                  "Type": "NONE"
                }
              ]
            }
          }
        ]
      }
    },
    "Action": {
      "Block": {}
    },
    "VisibilityConfig": {
      "SampledRequestsEnabled": true,
      "CloudWatchMetricsEnabled": true,
      "MetricName": "RateLimitAIInference"
    }
  }
```

Rate-Based Rules vs Bot Control

Feature	Rate-Based Rules	Bot Control (Targeted)
Cost	Included in standard WAF fees	Additional subscription fee
Threshold	Configurable (min 100)	Dynamic, based on historical patterns
Mitigation speed	30-50 seconds	Less than 10 seconds
IP tracking limit	10,000 IPs	50,000 IPs + tokens
Best for	Known rate limits for API endpoints	Unknown traffic patterns, bot detection

7.3 Custom Rules for AI-Specific Threats

Managing AI Bot Scrapers

AI bots (crawlers from AI companies scraping content for training data) are a growing concern:

1. **robots.txt**: Basic protection, but bots can ignore it
2. **Bot Control (Common)**: Identify and label AI bots. Set action to Count initially to analyze traffic.
3. **Custom rules**: Block specific User-Agent strings (e.g., `GPTBot`, `ClaudeBot`, `Bytespider`)
4. **Challenge/CAPTCHA**: Force suspected bots to prove they are human
5. **Honeypot**: Create hidden endpoints that only bots would find

Protecting Against Prompt Injection via WAF

While WAF cannot understand prompt injection semantically, you can:

- Block requests with suspicious patterns in the body (regex matching)
- Rate-limit large request bodies (prompt injection often involves large payloads)

- Use custom rules to block known prompt injection signatures
- Combine with Bedrock Guardrails for defense-in-depth

7.4 AWS Solutions Construct: WAF + API Gateway

The `aws-wafwebacl-apigateway` Solutions Construct deploys a WAF web ACL with seven AWS managed rule groups pre-configured and associates it with an API Gateway REST API. Available in CDK (TypeScript, Python, Java).

7.5 Key Exam Facts

- WAF operates at **Layer 7** (application layer)
- Web ACLs are **regional** (except for CloudFront, which requires us-east-1)
- Rule evaluation order is by **priority number** (lower = evaluated first)
- A single web ACL can have up to **5,000 WCUs** (Web ACL Capacity Units)
- WAF logs can be sent to **CloudWatch Logs**, **S3**, or **Kinesis Data Firehose**
- WAF supports **labels** -- rules can add labels to requests, and downstream rules can match on those labels
- WAF precedes all other API Gateway access controls (Cognito, IAM, Lambda authorizers)
- Shield Advanced provides DDoS protection at layers 3, 4, and 7 and includes WAF at no additional cost

---

8. AWS Encryption SDK

The AWS Encryption SDK is a **client-side encryption library** that implements envelope encryption. It is relevant to the exam for encrypting sensitive AI data before storing or transmitting it.

8.1 Core Concepts

Concept	Description
Envelope encryption	Encrypt data with a data key, then encrypt the data key with a wrapping key
Data key	Unique per message. Generated by the SDK. Used to encrypt your data. Never stored in plaintext.
Wrapping key	Protects data keys. Can be a KMS key, raw AES key, or raw RSA key.
Keyring	Specifies which wrapping keys to use. KMS keyring is the most common.
Encryption context	Key-value pairs that provide AAD (Additional Authenticated Data). Bound cryptographically to the ciphertext. Required for decryption.
Encrypted message	Portable format containing encrypted data + encrypted data keys + metadata
Key commitment	Guarantees ciphertext can only be decrypted to a single plaintext (prevents key confusion attacks)

8.2 How It Works

Encryption Flow

1. You specify a **keyring** with one or more wrapping keys (e.g., KMS key ARN)
2. SDK generates a unique **data key**
3. Each wrapping key in the keyring encrypts a copy of the data key

4. SDK uses the plaintext data key to encrypt your data (AES-256-GCM)
5. SDK destroys the plaintext data key from memory
6. SDK returns an **encrypted message** containing: encrypted data + all encrypted data key copies + algorithm info + encryption context

Decryption Flow

1. SDK parses the encrypted message
2. SDK passes the encrypted data keys to the keyring
3. Keyring uses a wrapping key to decrypt one of the data keys
4. SDK verifies key commitment (ensures the correct key)
5. SDK uses the decrypted data key to decrypt the data
6. SDK destroys the data key from memory and returns the plaintext

8.3 Multiple Wrapping Keys

A key feature: you can encrypt a data key under **multiple wrapping keys**. Use cases:

- **Multi-region**: Encrypt under KMS keys in us-east-1 and eu-west-1 so either region can decrypt
- **Multi-party**: Encrypt under keys owned by different teams or organizations
- **Key rotation**: Encrypt under both old and new keys during transition
- Any single wrapping key can decrypt independently

8.4 Encryption Context (AAD)

The encryption context is **not encrypted** but is **cryptographically bound** to the ciphertext. If you encrypt with context `{"purpose": "training-data"}`, you must provide the same context to decrypt.

Best practices:

- Always use an encryption context
- Include meaningful metadata: purpose, owner, classification level
- It appears in CloudTrail logs when KMS keys are used, aiding auditing

8.5 Commitment Policy

Controls whether the SDK uses key commitment:

Policy	Encrypt	Decrypt
`ForbidEncryptAllowDecrypt`	No commitment	Allows with or without commitment
`RequireEncryptAllowDecrypt`	Requires commitment	Allows with or without commitment
`RequireEncryptRequireDecrypt` (default)	Requires commitment	Requires commitment

> ***Exam Gotcha**: Use the default `RequireEncryptRequireDecrypt` unless you need backward compatibility with older ciphertext that lacks key commitment.*

8.6 Language Support

The SDK is available in: **JavaScript/Node.js**, Python, Java, C, .NET, Rust, Go. All implementations are interoperable -- data encrypted in one language can be decrypted in another (assuming the same wrapping keys).

8.7 TypeScript/JavaScript Example

```
import {
  buildClient,
  CommitmentPolicy,
  KmsKeyringNode,
} from "@aws-crypto/client-node";

const { encrypt, decrypt } = buildClient(
  CommitmentPolicy.REQUIRE_ENCRYPT_REQUIRE_DECRYPT
);

const keyring = new KmsKeyringNode({
  generatorKeyId:
    "arn:aws:kms:us-east-1:123456789012:key/my-key-id",
  keyIds: [
    "arn:aws:kms:eu-west-1:123456789012:key/my-eu-key-id",
  ],
});

const encryptionContext = {
  purpose: "ai-training-data",
  dataset: "customer-reviews-v3",
};

// Encrypt
const { result: ciphertext } = await encrypt(keyring, Buffer.from("sensitive training data"), {
  encryptionContext,
});

// Decrypt
const { plaintext, messageHeader } = await decrypt(keyring, ciphertext);

// Verify encryption context matches
for (const [key, value] of Object.entries(encryptionContext)) {
  if (messageHeader.encryptionContext[key] !== value) {
    throw new Error("Encryption context mismatch");
  }
}

console.log(plaintext.toString()); // "sensitive training data"
```

8.8 AWS Encryption SDK vs Other Options

Feature	AWS Encryption SDK	S3 SSE-KMS	S3 Client-Side (S3 Encryption Client)
Where encryption happens	Client-side	Server-side (S3)	Client-side
Works with non-S3 data	Yes (any data)	No (S3 only)	No (S3 only)
Envelope encryption	Yes	Yes	Yes
Portable encrypted format	Yes	No	No

Feature	AWS Encryption SDK	S3 SSE-KMS	S3 Client-Side (S3 Encryption Client)
Supports non-KMS keys	Yes (raw AES, RSA)	No	Yes
Key commitment	Yes	N/A	No

> ***Exam Gotcha**: Use the **AWS Encryption SDK** when you need client-side encryption of data that may be stored anywhere (S3, DynamoDB, local files, transmitted over networks). Use **S3 SSE-KMS** when server-side encryption on S3 is sufficient. Use the **S3 Encryption Client** only if you specifically need client-side encryption for S3 objects.*

8.9 Key Exam Facts

- The Encryption SDK does **not require** any AWS service -- it can use raw AES/RSA keys. But it integrates best with KMS.
- The SDK is **open source**
- Default algorithm: AES-256-GCM with HKDF, ECDSA signing, and key commitment
- Data key caching can improve performance for high-throughput workloads but requires careful security configuration (max age, max bytes, max messages)
- The encrypted message format is **portable** across all language implementations
- Digital signatures (ECDSA) add integrity but **increase decryption cost** significantly

---

9. Cross-Cutting Exam Themes

9.1 Defense in Depth for AI Applications

The exam expects you to know how these services work together:

```
Layer 1: Edge Protection         --&gt; AWS WAF + Shield (DDoS, bot control, rate limiting)
Layer 2: Identity Verification   --&gt; Cognito (user auth) + Identity Center (admin SSO)
Layer 3: API Authorization       --&gt; API Gateway + Cognito Authorizer / IAM Auth
Layer 4: Service Authorization   --&gt; IAM policies (least privilege for Bedrock access)
Layer 5: Org Governance          --&gt; SCPs (model restrictions, region restrictions)
Layer 6: Data Protection (rest)  --&gt; KMS (encryption), Macie (PII detection)
Layer 7: Data Protection (transit) --&gt; TLS everywhere, Encryption SDK (client-side)
Layer 8: Secrets Management      --&gt; Secrets Manager (API keys, credentials, rotation)
Layer 9: Monitoring              --&gt; CloudTrail, Access Analyzer, Security Hub
```

9.2 Common Exam Scenarios

Scenario	Answer
"How do you restrict which FM a developer can invoke?"	IAM policy with specific model ARNs in `Resource`
"How do you enforce model restrictions across all accounts?"	SCP with `Deny` on non-approved model ARNs
"How do you ensure all model calls use a guardrail?"	IAM condition key `bedrock:GuardrailIdentifier`

Scenario	Answer
"How do you add user auth to an AI chatbot API?"	Cognito User Pool + API Gateway authorizer
"How do you scan training data for PII?"	Amazon Macie sensitive data discovery job
"How do you redact PII from prompts in real-time?"	Bedrock Guardrails PII filter (or Comprehend)
"How do you protect an AI API from abuse?"	WAF rate-based rules on API Gateway
"How do you encrypt training data with your own key?"	KMS customer managed key with S3 SSE-KMS or Encryption SDK
"How do you securely store third-party AI API keys?"	Secrets Manager with rotation
"How do you find unused Bedrock permissions?"	IAM Access Analyzer unused access analyzer
"How do you give AI agents temporary access to AWS?"	Cognito Identity Pool with IAM role mapping
"How do you encrypt data client-side before upload?"	AWS Encryption SDK with KMS keyring
"How do you rotate database credentials used by a RAG pipeline?"	Secrets Manager with Lambda rotation function
"How do you manage SSO for AI dev teams across accounts?"	IAM Identity Center with permission sets

9.3 Key Metrics and Limits

Service	Metric/Limit
KMS	4 KB direct encryption limit; symmetric key: 5,500-30,000 requests/sec (varies by region)
Secrets Manager	65,536 byte max secret size; 40,000 API calls/sec per region
WAF	5,000 WCU per web ACL; rate-based rule min threshold: 100/5 min
Cognito User Pools	70 custom attributes max; 5 failed logins before lockout
Cognito Identity Pools	60 identity providers per pool
Macie	10,000 S3 buckets evaluated per account; 90-day finding retention
IAM	5,120 characters max for inline policy; 6,144 characters max for managed policy
SCPs	5,120 characters max; 5 SCPs per OU
Access Analyzer	1-365 day tracking period for unused access

Networking and Content Delivery -- Comprehensive Study Guide (AIP-C01)

Amazon API Gateway is the **most heavily tested** networking service on the AWS Certified Generative AI Developer - Professional exam. It appears in nearly every domain -- from dynamic model selection (Task 1.2.2) to streaming responses (Task 2.4.2) to rate limiting (Task 2.4.3) to API response filtering (Task 3.1.4). VPC endpoints and PrivateLink are tested for network isolation of AI workloads (Task 3.2.1).

---

1. Amazon API Gateway (Heavily Tested)

API Gateway is the central "AI gateway" pattern on the exam. It sits between client applications and AI backends (Lambda + Bedrock), providing authorization, throttling, caching, request validation, and streaming.

1.1 REST API vs HTTP API vs WebSocket API

Feature	REST API	HTTP API	WebSocket API
Protocol	RESTful (HTTP 1.1)	RESTful (HTTP 1.1/2)	WebSocket (persistent)
Latency/Cost	Higher latency, higher cost	~40% lower cost, lower latency	Connection-based pricing
Endpoint Types	Edge-optimized, Regional, Private	Regional only	Regional only
API Keys / Usage Plans	Yes	No	Yes
Per-client throttling	Yes	No	No
Request Validation	Yes (JSON Schema)	No	No
Request Body Transform	Yes (VTL mapping templates)	No	Yes (VTL)
Response Caching	Yes	No	No
AWS WAF	Yes	No	No
Private Endpoints	Yes (via VPC endpoint)	No	No
Lambda Authorizer	Yes	Yes	Yes (on $connect)
Cognito Authorizer	Yes (native)	Yes (via JWT authorizer)	No (use Lambda auth)
JWT Authorizer	No (use Lambda auth)	Yes (native)	No
Response Streaming	Yes (STREAM mode)	No	Bidirectional by nature
Canary Deployments	Yes	No	No
Max Payload	10 MB (buffered) / unlimited (streamed)	10 MB	128 KB (32 KB frame)

When to Use Each for AI Workloads

REST API -- The Default for AI Gateways:

- You need **request validation** (validate prompt JSON schemas before hitting Lambda/Bedrock)
- You need **usage plans and API keys** (per-tenant AI usage tracking)

- You need **per-client throttling** (different rate limits per AI consumer)
- You need **response caching** (cache identical prompt responses)
- You need **response streaming** (stream Bedrock responses back to clients via `STREAM` mode)
- You need **WAF protection** (block prompt injection at the edge)
- You need **VTL mapping templates** (transform requests for model routing)
- You need a **private API** (internal-only AI services)

HTTP API -- Simple, Low-Cost AI Proxy:

- You need a **simple proxy** to Lambda with minimal features
- You want the **lowest cost** and **lowest latency**
- You use **JWT/OIDC** for authorization (native JWT authorizer)
- You do NOT need caching, request validation, or usage plans

WebSocket API -- Real-Time AI Streaming:

- You need **bidirectional communication** (chat interfaces)
- You need to **push partial responses** from server to client (token-by-token streaming)
- You need **long-lived connections** for real-time AI interactions
- You are building a **chat application** with streaming LLM output

Exam Gotcha: REST API Streaming vs WebSocket API

The exam tests whether you know that REST APIs now support response streaming via `STREAM` transfer mode. For many AI use cases, REST API streaming is simpler than WebSocket because you get all REST API features (caching, throttling, WAF) plus streaming. Use WebSocket only when you need true bidirectional communication (the client sends multiple messages during a session, such as a multi-turn chat over a persistent connection).

1.2 Response Streaming (Exam-Critical)

Response streaming is a core exam topic for AI/GenAI use cases. Bedrock's `InvokeModelWithResponseStream` and `ConverseStream` APIs produce streaming output -- API Gateway must relay it to clients.

REST API Response Streaming (STREAM Mode)

Set the integration's `responseTransferMode` to `STREAM`:

- Works with `HTTP_PROXY` and `AWS_PROXY` (Lambda proxy) integration types only
- Lambda functions must be [streaming-enabled](https://docs.aws.amazon.com/lambda/latest/dg/configuration-response-streaming.html) (response streaming Lambda)
- Streams can last **up to 15 minutes**
- **Idle timeouts**: Regional/Private = 5 minutes; Edge-optimized = 30 seconds
- First 10 MB has no bandwidth restriction; beyond 10 MB restricted to 2 MB/s
- **NOT compatible with**: endpoint caching, content encoding, VTL response transformation
- Reduces **time-to-first-byte (TTFB)** dramatically for GenAI applications
- Can exceed the 10 MB response payload limit
- Can exceed the 29-second timeout limit (streams up to 15 minutes)

```
// TypeScript SDK v3: Create REST API with streaming integration
import {
  APIGatewayClient,
  PutIntegrationCommand,
} from "@aws-sdk/client-api-gateway";

const client = new APIGatewayClient({});
```

```
const command = new PutIntegrationCommand({
  restApiId: "abc123",
  resourceId: "xyz789",
  httpMethod: "POST",
  type: "AWS_PROXY",
  integrationHttpMethod: "POST",
  uri: `arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/${lambdaArn}/invocations`,
  // Key setting for streaming
  // Note: responseTransferMode is set via update_integration patchOperations
});

// Then update to enable streaming
import { UpdateIntegrationCommand } from "@aws-sdk/client-api-gateway";

const updateCommand = new UpdateIntegrationCommand({
  restApiId: "abc123",
  resourceId: "xyz789",
  httpMethod: "POST",
  patchOperations: [
    {
      op: "replace",
      path: "/responseTransferMode",
      value: "STREAM",
    },
  ],
});
```

WebSocket API for AI Streaming

WebSocket APIs use `$connect`, `$disconnect`, and `$default` routes plus custom routes:

```
// TypeScript SDK v3: Create WebSocket API
import {
  ApiGatewayV2Client,
  CreateApiCommand,
  CreateRouteCommand,
  CreateIntegrationCommand,
} from "@aws-sdk/client-apigatewayv2";

const client = new ApiGatewayV2Client({});

const api = await client.send(
  new CreateApiCommand({
    Name: "ai-chat-websocket",
    ProtocolType: "WEBSOCKET",
    RouteSelectionExpression: "$request.body.action",
  })
);

// Create Lambda integration for message handling
const integration = await client.send(
  new CreateIntegrationCommand({
    ApiId: api.ApiId,
    IntegrationType: "AWS_PROXY",
    IntegrationUri: `arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/${lambdaArn}/invocations`,
  })
);

// Create custom route for sending prompts
await client.send(
```

```
  new CreateRouteCommand({
    ApiId: api.ApiId,
    RouteKey: "sendPrompt",
    Target: `integrations/${integration.IntegrationId}`,
  })
);
```

Backend Lambda pushes streamed tokens to connected clients using the `@connections` API:

```
// Lambda handler: stream Bedrock response tokens back via WebSocket
import {
  ApiGatewayManagementApiClient,
  PostToConnectionCommand,
} from "@aws-sdk/client-apigatewaymanagementapi";
import {
  BedrockRuntimeClient,
  InvokeModelWithResponseStreamCommand,
} from "@aws-sdk/client-bedrock-runtime";

export async function handler(event: any) {
  const { connectionId, domainName, stage } = event.requestContext;
  const body = JSON.parse(event.body);

  const apigw = new ApiGatewayManagementApiClient({
    endpoint: `https://${domainName}/${stage}`,
  });
  const bedrock = new BedrockRuntimeClient({});

  const response = await bedrock.send(
    new InvokeModelWithResponseStreamCommand({
      modelId: "anthropic.claude-3-sonnet-20240229-v1:0",
      contentType: "application/json",
      body: JSON.stringify({
        anthropic_version: "bedrock-2023-05-31",
        messages: [{ role: "user", content: body.prompt }],
        max_tokens: 1024,
      }),
    })
  );

  // Stream each chunk to the WebSocket client
  if (response.body) {
    for await (const event of response.body) {
      if (event.chunk?.bytes) {
        const chunk = JSON.parse(
          new TextDecoder().decode(event.chunk.bytes)
        );
        if (chunk.type === "content_block_delta") {
          await apigw.send(
            new PostToConnectionCommand({
              ConnectionId: connectionId,
              Data: JSON.stringify({
                type: "token",
                text: chunk.delta?.text,
              }),
            })
          );
        }
      }
    }
```

```
  }

  return { statusCode: 200 };
}
```

Chunked Transfer Encoding and SSE

Chunked Transfer Encoding (`Transfer-Encoding: chunked`) is how HTTP/1.1 streaming works:

- The server does not know the total response size upfront
- Data is sent in chunks, each prefixed with its size
- API Gateway REST API STREAM mode uses chunked transfer encoding under the hood
- The client reads chunks as they arrive (no need to wait for full response)

Server-Sent Events (SSE) are a pattern built on top of chunked responses:

- Content-Type: `text/event-stream`
- One-directional: server pushes events to the client
- Simpler than WebSocket when you only need server-to-client streaming
- Use REST API STREAM mode with SSE formatting in your Lambda response

1.3 Rate Limiting and Throttling (Exam-Critical)

API Gateway uses the **token bucket algorithm** for throttling. This is a core exam topic because AI APIs are expensive and you must protect against abuse.

Four Levels of Throttling (Applied in Order)

Level	Scope	Configurable?
1. Per-client per-method	Usage plan + API key (most specific)	Yes
2. Per-method per-stage	Stage settings or usage plan	Yes
3. Per-account per-Region	All APIs in the account	Yes (request increase)
4. AWS Regional limit	Hard ceiling across all accounts	No

Default account-level limits: 10,000 RPS steady-state, 5,000 burst (varies by Region). Can request an increase.

Token bucket algorithm:

- Tokens are added at the **steady-state rate** (RPS)
- Bucket holds up to **burst** tokens
- Each request consumes one token
- When bucket is empty: `429 Too Many Requests`

Throttling for AI APIs

```
// TypeScript SDK v3: Create a usage plan with throttling for AI API
import {
  APIGatewayClient,
  CreateUsagePlanCommand,
} from "@aws-sdk/client-api-gateway";

const client = new APIGatewayClient({});

const usagePlan = await client.send(
  new CreateUsagePlanCommand({
```

```
    name: "ai-standard-tier",
    description: "Standard tier for AI API consumers",
    throttle: {
      rateLimit: 50, // 50 requests per second steady state
      burstLimit: 100, // burst up to 100
    },
    quota: {
      limit: 10000, // 10,000 requests per month
      period: "MONTH",
    },
    apiStages: [
      {
        apiId: "abc123",
        stage: "prod",
        throttle: {
          // Method-level overrides
          "/inference/POST": {
            rateLimit: 10, // Tighter limit on expensive inference
            burstLimit: 20,
          },
        },
      },
    ],
  })
);
```

Exam Gotcha: Throttling Limits Are Best-Effort

Usage plan throttling and quotas are applied on a **best-effort basis** -- they are targets, not hard ceilings. Clients may occasionally exceed quotas. For hard enforcement, combine with:

- **AWS WAF** rate-based rules (for IP-based rate limiting)
- **Lambda authorizer** logic (for custom enforcement)
- **Bedrock Guardrails** (for content-based limits)

1.4 Request Validation and Transformation

Request Validation (REST API Only)

API Gateway can validate requests **before** they reach your backend, reducing unnecessary Lambda invocations and Bedrock calls:

- Validates **required request parameters** in URI, query string, and headers
- Validates **request body** against a JSON Schema model
- Returns `400 Bad Request` if validation fails (never reaches backend)
- Defined via **request validators** attached to API methods

```
// TypeScript SDK v3: Create a model and request validator
import {
  APIGatewayClient,
  CreateModelCommand,
  CreateRequestValidatorCommand,
  UpdateMethodCommand,
} from "@aws-sdk/client-api-gateway";

const client = new APIGatewayClient({});

// Define a JSON Schema model for AI inference requests
```

```
const model = await client.send(
  new CreateModelCommand({
    restApiId: "abc123",
    name: "InferenceRequest",
    contentType: "application/json",
    schema: JSON.stringify({
      $schema: "http://json-schema.org/draft-04/schema#",
      type: "object",
      required: ["modelId", "prompt"],
      properties: {
        modelId: {
          type: "string",
          enum: [
            "anthropic.claude-3-sonnet-20240229-v1:0",
            "anthropic.claude-3-haiku-20240307-v1:0",
            "amazon.titan-text-lite-v1",
          ],
        },
        prompt: {
          type: "string",
          minLength: 1,
          maxLength: 100000,
        },
        maxTokens: {
          type: "integer",
          minimum: 1,
          maximum: 4096,
        },
        temperature: {
          type: "number",
          minimum: 0,
          maximum: 1,
        },
      },
    }),
  })
);

// Create a request validator
const validator = await client.send(
  new CreateRequestValidatorCommand({
    restApiId: "abc123",
    name: "ValidateBodyAndParams",
    validateRequestBody: true,
    validateRequestParameters: true,
  })
);
```

Request/Response Mapping Templates (VTL)

Mapping templates use **Apache Velocity Template Language (VTL)** to transform requests and responses. This is how API Gateway can route to different Bedrock models based on request content (Task 1.2.2: Dynamic model selection).

```
## Request mapping template: Route to different models based on request
#set($inputRoot = $input.path('$'))
#set($modelId = $inputRoot.modelId)

## Dynamic model routing based on task complexity
```

```
#if($inputRoot.taskType == "simple")
  #set($modelId = "amazon.titan-text-lite-v1")
#elseif($inputRoot.taskType == "complex")
  #set($modelId = "anthropic.claude-3-sonnet-20240229-v1:0")
#end

{
  "modelId": "$modelId",
  "prompt": "$util.escapeJavaScript($inputRoot.prompt)",
  "maxTokens": $inputRoot.maxTokens
}


## Response mapping template: Filter AI response before returning to client
## (Task 3.1.4: API response filtering)
#set($inputRoot = $input.path('$'))
{
  "response": "$util.escapeJavaScript($inputRoot.output.text)",
  "usage": {
    "inputTokens": $inputRoot.usage.inputTokens,
    "outputTokens": $inputRoot.usage.outputTokens
  }
  ## Intentionally omit internal metadata, model internals, etc.
}
```

Exam Gotcha: VTL Not Available with Streaming

When `responseTransferMode` is `STREAM`, VTL response mapping templates are **not supported**. You must perform any response transformation inside the Lambda function or in the client. This means you cannot use API Gateway to filter streaming AI responses -- you must do it in your Lambda handler.

1.5 Integration with Lambda for AI Backends

API Gateway + Lambda is the standard pattern for the AI gateway:

```
Client --&gt; API Gateway --&gt; Lambda --&gt; Amazon Bedrock
                |                        |
                |-- Auth (Cognito/JWT) --|
                |-- Throttling          --|
                |-- Caching             --|
                |-- Validation          --|
```

Lambda Proxy Integration (most common): API Gateway passes the entire request to Lambda as-is. Lambda returns the full response including status code and headers.

Lambda Non-Proxy Integration: API Gateway transforms the request/response using VTL mapping templates. Useful for model routing without changing Lambda code.

1.6 Usage Plans and API Keys

Usage plans are REST API only and provide per-tenant AI usage management:

Concept	Purpose
API Key	Alphanumeric string identifying a client/tenant
Usage Plan	Defines throttle limits + quota for associated API keys
Throttle	Rate (RPS) and burst limits per client

Concept	Purpose
Quota	Total requests allowed per day/week/month

AI use case: Give each customer an API key. Associate it with a usage plan that limits them to, say, 1,000 Bedrock inference calls per month. Premium customers get a higher-tier usage plan.

Exam Gotcha: API Keys Are Not for Authentication

API keys are for **identification and usage tracking**, not for authentication or authorization. For authentication, use:

- **IAM authorization** (for AWS service-to-service)
- **Cognito user pools** (for end-user auth)
- **Lambda authorizers** (for custom auth logic)
- **JWT authorizers** (HTTP API only)

1.7 Custom Authorizers with Cognito

Cognito User Pool Authorizer (REST API):

- Native integration, no Lambda needed
- API Gateway validates the JWT token from Cognito
- Returns `401 Unauthorized` if token is invalid/expired
- Can access user claims in mapping templates via `$context.authorizer.claims`

Lambda Authorizer:

- Custom authorization logic in a Lambda function
- Can validate tokens from any identity provider
- Returns an IAM policy document (allow/deny)
- Two types: **TOKEN** (header-based) and **REQUEST** (parameter-based)
- Cached for up to 3600 seconds (reduces Lambda invocations)

```
// TypeScript SDK v3: Create a Cognito authorizer
import {
  APIGatewayClient,
  CreateAuthorizerCommand,
} from "@aws-sdk/client-api-gateway";

const client = new APIGatewayClient({});

const authorizer = await client.send(
  new CreateAuthorizerCommand({
    restApiId: "abc123",
    name: "CognitoAuth",
    type: "COGNITO_USER_POOLS",
    providerARNs: [
      "arn:aws:cognito-idp:us-east-1:123456789012:userpool/us-east-1_AbCdEf",
    ],
    identitySource: "method.request.header.Authorization",
  })
);
```

1.8 Timeout Considerations (Exam-Critical)

Timeout	Default	Maximum	Notes
REST API integration timeout	29 seconds	29 seconds (can request increase for Regional/Private)	June 2024: can request increase beyond 29s for Regional/Private REST APIs
HTTP API integration timeout	30 seconds	30 seconds	Cannot be changed
WebSocket API	N/A	Idle: 10 minutes, Connection: 2 hours	Connection-based, not request-based
Lambda function timeout	3 seconds	15 minutes	Independent of API Gateway timeout
REST API Streaming	N/A	15 minutes	Streaming bypasses the 29-second limit
REST API idle timeout (streaming, Regional)	5 minutes	5 minutes	Connection drops if no data for 5 min
REST API idle timeout (streaming, Edge)	30 seconds	30 seconds	Much shorter for edge-optimized

Exam Gotcha: API Gateway Timeout vs Lambda Timeout

This is one of the most commonly tested gotchas:

1. **Default**: API Gateway REST API times out at **29 seconds**. Even if your Lambda has a 15-minute timeout, the client gets a `504 Gateway Timeout` after 29 seconds.
2. **Streaming solves this**: With `STREAM` mode, the REST API can stream for up to **15 minutes**, bypassing the 29-second limit entirely.
3. **Async pattern**: For non-streaming long-running jobs, use asynchronous invocation: API Gateway accepts the request, Lambda processes asynchronously, client polls for results via a separate endpoint (or gets notified via WebSocket/SNS).
4. **Timeout increase**: Since June 2024, you can request an integration timeout increase beyond 29 seconds for Regional and Private REST APIs. This is specifically called out as beneficial for "Generative AI use cases with Large Language Models."

1.9 Caching for AI Responses

API Gateway REST API caching stores integration responses to reduce backend calls:

Setting	Value
Cache capacity	0.5 GB to 237 GB
Default TTL	300 seconds (5 minutes)
Max TTL	3600 seconds (1 hour)
Scope	Per-stage; can override per-method
Cache key	Integration request parameters (query strings, headers)
Invalidation	`Cache-Control: max-age=0` header (requires authorization)

AI use case: Cache responses for common/repeated prompts. If many users ask the same question (e.g., "What are your return policies?"), caching avoids redundant Bedrock calls and dramatically reduces cost and latency.

Exam Gotcha: Caching Incompatible with Streaming

When response streaming is enabled (`STREAM` mode), endpoint caching is **not supported**. You must choose between caching and streaming. For GenAI chatbots, streaming is usually more important. For FAQ-style AI, caching may be more valuable.

1.10 Key CloudWatch Metrics

Metric	What It Tells You
`Count`	Total API requests
`4XXError`	Client errors (validation failures, auth failures, throttling)
`5XXError`	Server errors (Lambda failures, integration timeouts)
`Latency`	End-to-end time (API Gateway received request to returned response)
`IntegrationLatency`	Time from API Gateway sending to backend to receiving response
`CacheHitCount`	Requests served from cache
`CacheMissCount`	Requests forwarded to integration

AI-specific monitoring: Watch `IntegrationLatency` closely -- Bedrock inference times vary by model and prompt length. Watch `5XXError` for Lambda timeouts caused by slow model responses.

---

2. AWS AppSync

2.1 GraphQL for AI Applications

AWS AppSync is a managed **GraphQL** service. It can serve as an alternative AI gateway to API Gateway, particularly when you need:

- **A single endpoint** that fetches data from multiple sources (DynamoDB, Lambda, HTTP, Bedrock)
- **Flexible queries** where the client specifies exactly which fields to return (avoids over-fetching)
- **Real-time subscriptions** for live AI output

AppSync + Bedrock Direct Integration

AppSync has a **native data source for Amazon Bedrock**. You can invoke Bedrock models directly from AppSync resolvers without Lambda:

```
// AppSync JS resolver: Invoke Bedrock Converse API directly
import { converse } from "@aws-appsync/utils/ai";

export function request(ctx) {
  return converse({
    modelId: "anthropic.claude-3-5-haiku-20241022-v1:0",
    messages: [
      {
        role: "user",
        content: [{ text: ctx.args.prompt }],
      },
    ],
    system: [{ text: "You are a helpful assistant." }],
  });
}

export function response(ctx) {
```

```
  return ctx.result.output.message.content[0].text;
}
```

Limitations of AppSync Bedrock integration:

- Synchronous only -- invocations must complete within **10 seconds**
- **No streaming APIs** (cannot call `InvokeModelWithResponseStream` or `ConverseStream`)
- Same-region invocations only
- For longer invocations: use a **Lambda data source** + subscriptions for async pattern

2.2 Real-Time Subscriptions for AI

AppSync subscriptions use **WebSockets** to push updates to connected clients. This is the pattern for handling long-running Bedrock invocations:

1. Client sends a GraphQL mutation to start an AI inference job
2. Mutation triggers a Lambda function that calls Bedrock asynchronously
3. Lambda streams tokens back by publishing to AppSync subscription via mutations
4. Client receives real-time updates via WebSocket subscription

```
type Mutation {
  startInference(prompt: String!): InferenceJob
  publishToken(jobId: ID!, token: String!): Token
}

type Subscription {
  onToken(jobId: ID!): Token
    @aws_subscribe(mutations: ["publishToken"])
}

type Token {
  jobId: ID!
  token: String!
}

type InferenceJob {
  jobId: ID!
  status: String!
}
```

2.3 Resolvers with Lambda/DynamoDB

AppSync resolvers map GraphQL operations to data sources:

Resolver Type	Runtime	Use Case
JS resolvers (APPSYNC_JS)	JavaScript	Preferred for new development
VTL resolvers	Velocity Template Language	Legacy, still supported
Direct Lambda resolvers	Lambda function	Complex logic, external API calls
Pipeline resolvers	Chain of functions	Multi-step operations (auth -> validate -> invoke)

AI pattern with Pipeline Resolver:

1. Function 1: Validate and sanitize the prompt (guard against injection)
2. Function 2: Check DynamoDB for cached response
3. Function 3: If no cache hit, invoke Bedrock via Lambda

4. Function 4: Store result in DynamoDB for future cache hits

2.4 When to Use AppSync vs API Gateway for AI

Criterion	AppSync	API Gateway
Query flexibility	Client picks exact fields (GraphQL)	Fixed response shape (REST)
Real-time	Native WebSocket subscriptions	WebSocket API (separate)
Bedrock direct	Yes (10s limit)	No (needs Lambda)
Streaming	Via subscriptions (async pattern)	REST STREAM mode or WebSocket API
Caching	Server-side + client-side	Server-side (REST API)
Rate limiting	No native usage plans	Yes (REST API)
WAF	Yes	Yes (REST API only)
Multiple data sources	Single query, multiple sources	One integration per method

---

3. Amazon CloudFront

3.1 CDN for AI Application Frontends

CloudFront is a global **Content Delivery Network** with 600+ Points of Presence (PoPs). For AI applications, it serves:

- **Static assets** for AI chat UIs (HTML, CSS, JS, images)
- **API acceleration** (CloudFront in front of API Gateway for global reach)
- **Edge compute** (Lambda@Edge / CloudFront Functions for preprocessing)

3.2 Edge Caching Strategies for AI

Strategy	How It Works	AI Use Case
Static asset caching	Cache JS/CSS/images at edge	AI chatbot frontend (React/Angular app)
API response caching	Cache API Gateway responses at edge	Repeated FAQ-style AI queries
Origin Shield	Additional caching layer between edge and origin	Reduce Bedrock calls from global audience
Cache behaviors	Different caching rules per URL path	Cache `/static/*` aggressively, no-cache on `/api/chat`

Cache Key Best Practices for AI:

- Include relevant query parameters in cache key (e.g., `modelId`, `prompt`)
- Exclude auth headers from cache key (they vary per user but shouldn't affect response)
- Use short TTLs for AI responses that may change (model updates)
- Use longer TTLs for static content and embeddings

3.3 Lambda@Edge and CloudFront Functions for AI

Feature	CloudFront Functions	Lambda@Edge
Runtime	JavaScript (lightweight)	Node.js or Python
Execution	Edge location (PoP)	Regional Edge Cache
Max duration	1 ms	5s (viewer) / 30s (origin)
Max memory	2 MB	128-10,240 MB
Network access	No	Yes
Triggers	Viewer request/response	Viewer + Origin request/response

AI Preprocessing Use Cases:

- **Lambda@Edge (Origin Request)**: Inspect request headers, route to different AI backends based on user location, device type, or A/B test group
- **Lambda@Edge (Origin Response)**: Add security headers, mask PII in AI responses before caching
- **CloudFront Functions (Viewer Request)**: Normalize request URLs, add/remove query parameters, simple auth token validation
- **Lambda@Edge for edge inference**: Run lightweight inference at CloudFront edge locations for ultra-low latency personalization

```
// Lambda@Edge: Route AI requests based on user location
// Triggered on Origin Request
export async function handler(event: any) {
  const request = event.Records[0].cf.request;
  const country = request.headers["cloudfront-viewer-country"]?.[0]?.value;

  // Route to region-specific AI endpoint
  if (country === "JP" || country === "KR") {
    request.origin = {
      custom: {
        domainName: "api-apac.example.com",
        port: 443,
        protocol: "https",
        path: "/v1/inference",
      },
    };
    request.headers["host"] = [
      { key: "host", value: "api-apac.example.com" },
    ];
  }

  return request;
}
```

---

4. Amazon VPC

4.1 VPC Endpoints for Bedrock (Exam-Critical)

Interface VPC endpoints (powered by AWS PrivateLink) create private connections from your VPC to Bedrock without traversing the public internet.

Bedrock VPC Endpoint Service Names

Endpoint	Service Name	Purpose
Bedrock Control Plane	`com.amazonaws.{region}.bedrock`	Manage models, fine-tuning jobs
Bedrock Runtime	`com.amazonaws.{region}.bedrock-runtime`	InvokeModel, Converse (inference)
Bedrock Agents	`com.amazonaws.{region}.bedrock-agent`	Build-time agent operations
Bedrock Agents Runtime	`com.amazonaws.{region}.bedrock-agent-runtime`	InvokeAgent, RetrieveAndGenerate

Creating VPC Endpoints for Bedrock

```
// TypeScript SDK v3: Create interface VPC endpoint for Bedrock Runtime
import { EC2Client, CreateVpcEndpointCommand } from "@aws-sdk/client-ec2";

const ec2 = new EC2Client({});

const endpoint = await ec2.send(
  new CreateVpcEndpointCommand({
    VpcId: "vpc-12345",
    VpcEndpointType: "Interface",
    ServiceName: "com.amazonaws.us-east-1.bedrock-runtime",
    SubnetIds: ["subnet-aaa", "subnet-bbb"], // Private subnets in 2+ AZs
    SecurityGroupIds: ["sg-endpoint"],
    PrivateDnsEnabled: true, // Use default DNS name (bedrock-runtime.us-east-1.amazonaws.com)
  })
);
```

VPC Endpoint Policies

Restrict which Bedrock actions can be called through the endpoint:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Principal": "*",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-sonnet*",
        "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-text*"
      ]
    }
  ]
}
```

4.2 Network Isolation for AI Workloads

The standard secure AI architecture:

```
Internet --&gt; CloudFront --&gt; API Gateway --&gt; Lambda (private subnet)
                                                |
```

```
                    VPC Endpoint (PrivateLink)
                              |
                        Amazon Bedrock
```

Key design decisions:

- Lambda in **private subnets** with no internet access (unless via NAT Gateway)
- VPC endpoints for Bedrock, S3, DynamoDB, and other AWS services
- **No public IPs** on any compute resources
- All traffic stays on the AWS backbone network

4.3 Security Groups and NACLs

Security Group for VPC Endpoint:

- Inbound: Allow HTTPS (port 443) from Lambda security group
- Outbound: All traffic (stateful, return traffic auto-allowed)

Security Group for Lambda:

- Outbound: Allow HTTPS (port 443) to VPC endpoint security group
- Inbound: None required (Lambda initiates connections)

NACLs (Network Access Control Lists):

- Stateless (must allow both inbound and outbound)
- Applied at subnet level
- Allow ephemeral ports (1024-65535) for return traffic
- Generally, keep NACLs permissive and use security groups for fine-grained control

```typescript
// TypeScript SDK v3: Create security group for Bedrock VPC endpoint
import {
  EC2Client,
  CreateSecurityGroupCommand,
  AuthorizeSecurityGroupIngressCommand,
} from "@aws-sdk/client-ec2";

const ec2 = new EC2Client({});

const sg = await ec2.send(
  new CreateSecurityGroupCommand({
    GroupName: "bedrock-vpce-sg",
    Description: "Security group for Bedrock VPC endpoint",
    VpcId: "vpc-12345",
  })
);

await ec2.send(
  new AuthorizeSecurityGroupIngressCommand({
    GroupId: sg.GroupId,
    IpPermissions: [
      {
        IpProtocol: "tcp",
        FromPort: 443,
        ToPort: 443,
        UserIdGroupPairs: [
          { GroupId: "sg-lambda-12345" }, // Allow from Lambda SG
        ],
      },
    ],
```

```
    })
  );
```

4.4 Private Subnets for AI Services

Component	Subnet Type	Why
Lambda (AI inference)	Private	No direct internet access; use VPC endpoints
VPC Endpoints	Private	Interface endpoints create ENIs in private subnets
NAT Gateway	Public	Only if Lambda needs internet access (e.g., external APIs)
ALB/NLB	Public or Private	Public for internet-facing, Private for internal

---

5. AWS PrivateLink

5.1 Private Connectivity to AI Services

AWS PrivateLink provides **private connectivity** between VPCs, AWS services, and on-premises networks without exposing traffic to the public internet.

For AI workloads, PrivateLink is used for:

- Connecting to **Bedrock** (via interface VPC endpoints)
- Connecting to **SageMaker** endpoints
- Connecting to **S3** (for training data, model artifacts)
- Exposing your **own AI services** to other accounts/VPCs

Interface Endpoints vs Gateway Endpoints

Type	Powered By	Services	DNS	Cost
Interface Endpoint	PrivateLink (ENI in subnet)	Most AWS services, including Bedrock	Private DNS optional	Per-hour + per-GB
Gateway Endpoint	Route table entry	S3 and DynamoDB only	Prefix list in route table	Free

Exam Gotcha: Bedrock Uses Interface Endpoints, Not Gateway

Bedrock requires **interface VPC endpoints** (PrivateLink). S3 and DynamoDB can use **gateway endpoints** (free). If the exam asks about private Bedrock access, the answer is always interface VPC endpoint / PrivateLink.

5.2 VPC Endpoint Services (Exposing Your AI)

You can create your own PrivateLink-powered service to expose your AI inference endpoint to other VPCs/accounts:

```
Consumer VPC --&gt; Interface Endpoint --&gt; NLB --&gt; Your AI Service (EC2/ECS/EKS)
                  (PrivateLink)
```

Requirements:

- Your service must be behind a **Network Load Balancer (NLB)** or **Gateway Load Balancer (GWLB)**

- Consumer creates an interface endpoint in their VPC pointing to your service
- Traffic never traverses the public internet
- Useful for multi-account AI platform architectures

---

6. Elastic Load Balancing (ELB)

6.1 ALB vs NLB for AI Workloads

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
OSI Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP/TLS)
Routing	Path, host, header, query string, method	Port-based
WebSocket	Yes	Yes (TCP passthrough)
gRPC	Yes	No
SSL termination	Yes	Yes (TLS)
Static IP	No (use Global Accelerator)	Yes
PrivateLink	No (cannot be endpoint service)	Yes (can be endpoint service target)
Lambda targets	Yes	No
Latency	~ms (application-level processing)	~us (ultra-low, network-level)
AI Use Case	API routing, path-based model routing, WebSocket for chat	Extreme throughput, PrivateLink service, SageMaker endpoints

When to Use Each for AI

ALB:

- Route `/v1/chat` to one target group (chat model), `/v1/embed` to another (embedding model)
- Content-based routing for multi-model architectures
- WebSocket support for real-time AI chat
- Lambda as a target (serverless inference)
- Health checks on HTTP endpoints

NLB:

- Required when creating **PrivateLink endpoint services** (ALB cannot back endpoint services)
- Ultra-low latency for high-throughput inference pipelines
- Static IPs for firewall allowlisting
- TCP-level load balancing for custom protocols
- Front-end for EKS/ECS inference containers

6.2 Target Groups for AI Microservices

```
// TypeScript SDK v3: Create ALB target group for AI inference service
import {
  ElasticLoadBalancingV2Client,
  CreateTargetGroupCommand,
} from "@aws-sdk/client-elastic-load-balancing-v2";

const elbv2 = new ElasticLoadBalancingV2Client({});
```

```
const targetGroup = await elbv2.send(
  new CreateTargetGroupCommand({
    Name: "ai-inference-targets",
    Protocol: "HTTP",
    Port: 8080,
    VpcId: "vpc-12345",
    TargetType: "ip", // Use 'ip' for ECS Fargate, 'instance' for EC2
    HealthCheckProtocol: "HTTP",
    HealthCheckPath: "/health",
    HealthCheckIntervalSeconds: 30,
    HealthyThresholdCount: 2,
    UnhealthyThresholdCount: 3,
    Matcher: { HttpCode: "200" },
  })
);
```

6.3 Health Checks for AI Endpoints

AI inference endpoints may have variable response times. Tune health checks accordingly:

Setting	Recommendation for AI
Interval	30 seconds (default) -- not too frequent to avoid load
Timeout	10-30 seconds (inference can be slow)
Healthy threshold	2 (quick recovery)
Unhealthy threshold	3-5 (avoid flapping during slow inference)
Health check path	`/health` (lightweight endpoint, NOT an inference endpoint)

Never use an inference endpoint as a health check path -- it's expensive (invokes Bedrock) and slow.

---

7. AWS Global Accelerator

7.1 Global AI Application Performance

AWS Global Accelerator uses **Anycast IP addresses** to route traffic to the nearest AWS edge location, then over the AWS global backbone to your application endpoints.

Feature	Global Accelerator	CloudFront
Protocol	TCP and UDP	HTTP/HTTPS
IP addresses	2 static Anycast IPv4	Dynamic (DNS-based)
Caching	No	Yes
Use case	Non-HTTP protocols, static IP requirement	HTTP content delivery, caching
AI use case	Multi-region AI inference with failover	CDN for AI app frontends + API caching
DDoS protection	AWS Shield Standard (built-in)	AWS Shield Standard (built-in)
Health checks	Yes (endpoint-level)	Yes (origin-level)

When to use for AI:

- **Multi-region AI deployment**: Run inference in us-east-1 and eu-west-1, Global Accelerator routes users to the closest healthy region
- **Static IP requirement**: Clients that cannot use DNS (IoT devices, on-premises systems) need fixed IPs to reach your AI API
- **Automatic failover**: If Bedrock in one region is degraded, traffic automatically shifts to the next closest healthy region
- **Traffic dials**: Gradually shift traffic between regions during model deployments

```
// TypeScript SDK v3: Create Global Accelerator for multi-region AI API
import {
  GlobalAcceleratorClient,
  CreateAcceleratorCommand,
  CreateListenerCommand,
  CreateEndpointGroupCommand,
} from "@aws-sdk/client-global-accelerator";

const client = new GlobalAcceleratorClient({ region: "us-west-2" });

const accelerator = await client.send(
  new CreateAcceleratorCommand({
    Name: "ai-inference-global",
    IpAddressType: "IPV4",
    Enabled: true,
  })
);

const listener = await client.send(
  new CreateListenerCommand({
    AcceleratorArn: accelerator.Accelerator?.AcceleratorArn,
    PortRanges: [{ FromPort: 443, ToPort: 443 }],
    Protocol: "TCP",
  })
);

// Primary region endpoint group
await client.send(
  new CreateEndpointGroupCommand({
    ListenerArn: listener.Listener?.ListenerArn,
    EndpointGroupRegion: "us-east-1",
    EndpointConfigurations: [
      {
        EndpointId: "arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/ai-api/abc123",
        Weight: 100,
      },
    ],
    TrafficDialPercentage: 100,
    HealthCheckIntervalSeconds: 30,
  })
);
```

---

8. Amazon Route 53

8.1 DNS for AI Applications

Route 53 is a highly available DNS web service. For AI applications, it provides:

- **Domain registration** and DNS management
- **Routing policies** to direct traffic to the right AI endpoint
- **Health checks** for AI service availability
- **DNS failover** for multi-region AI architectures

8.2 Routing Policies (Exam-Referenced)

Latency-based routing is explicitly referenced in the exam appendix.

Policy	How It Works	AI Use Case
Simple	Single resource, no health check	Single-region AI API
Weighted	Percentage-based traffic split	A/B test AI models (80/20 split)
Latency	Route to lowest-latency region	Multi-region Bedrock deployment; user gets fastest region
Failover	Primary/secondary with health check	DR for AI services
Geolocation	Route based on user location	Data sovereignty (EU users to EU Bedrock)
Geoproximity	Route based on geographic distance + bias	Fine-grained geographic AI routing
Multivalue Answer	Return multiple healthy IPs	Simple load distribution
IP-based	Route based on client IP range	Enterprise-specific AI endpoints

Latency-Based Routing for AI (Exam-Critical)

```
// TypeScript SDK v3: Create latency-based routing records
import {
  Route53Client,
  ChangeResourceRecordSetsCommand,
} from "@aws-sdk/client-route-53";

const route53 = new Route53Client({});

await route53.send(
  new ChangeResourceRecordSetsCommand({
    HostedZoneId: "Z1234567890",
    ChangeBatch: {
      Changes: [
        {
          Action: "CREATE",
          ResourceRecordSet: {
            Name: "ai-api.example.com",
            Type: "A",
            SetIdentifier: "us-east-1-ai",
            Region: "us-east-1",
            AliasTarget: {
              HostedZoneId: "Z35SXDOTRQ7X7K", // ALB hosted zone ID
              DNSName: "ai-alb-us-east-1.elb.amazonaws.com",
              EvaluateTargetHealth: true,
            },
          },
        },
        {
          Action: "CREATE",
          ResourceRecordSet: {
            Name: "ai-api.example.com",
```

```
            Type: "A",
            SetIdentifier: "eu-west-1-ai",
            Region: "eu-west-1",
            AliasTarget: {
              HostedZoneId: "Z32O12XQLNTSW2", // ALB hosted zone ID
              DNSName: "ai-alb-eu-west-1.elb.amazonaws.com",
              EvaluateTargetHealth: true,
            },
          },
        },
      ],
    },
  })
);
```

8.3 Health Checks and Failover

Route 53 health checks monitor AI endpoint availability:

Health Check Type	What It Monitors
Endpoint	HTTP/HTTPS/TCP connectivity to a specific URL
Calculated	Combines multiple health checks (AND/OR logic)
CloudWatch alarm	Monitors a CloudWatch alarm state

AI failover pattern:

1. Deploy AI inference in 2+ regions
2. Create health checks for each region's API endpoint
3. Use **failover routing** (primary: us-east-1, secondary: eu-west-1) or **latency routing** with health checks enabled
4. If the primary region's health check fails (e.g., Bedrock latency spikes, Lambda errors), Route 53 automatically routes to the secondary

Exam Gotcha: DNS Propagation Delay

Route 53 failover is DNS-based, so clients may cache the old IP for up to the **TTL** duration. For faster failover:

- Use **short TTLs** (60 seconds) on health-checked records
- Combine with **Global Accelerator** for instant failover (no DNS caching issue)
- Client-side retry logic with exponential backoff

---

9. Exam Cheat Sheet -- Key Facts

API Gateway Numbers to Memorize

Limit	Value
REST API integration timeout (default)	29 seconds
REST API streaming duration	Up to 15 minutes
REST API streaming idle timeout (Regional)	5 minutes

Limit	Value
REST API streaming idle timeout (Edge)	30 seconds
REST API payload (buffered)	10 MB
REST API payload (streamed)	Exceeds 10 MB
HTTP API timeout	30 seconds
WebSocket message payload	128 KB
WebSocket frame size	32 KB
WebSocket idle timeout	10 minutes
WebSocket connection duration	2 hours
Lambda authorizer cache TTL (max)	3600 seconds
Default account throttle (RPS)	10,000
Default account burst	5,000
API Gateway cache TTL (max)	3600 seconds
API Gateway cache size	0.5 GB to 237 GB

Decision Matrix: Which Networking Service?

Scenario	Service
AI API with usage plans, rate limiting, caching, and streaming	**API Gateway REST API**
Simple, low-cost AI proxy with JWT auth	**API Gateway HTTP API**
Real-time bidirectional AI chat	**API Gateway WebSocket API**
GraphQL with multiple data sources + real-time subscriptions	**AWS AppSync**
CDN for AI app frontend + API acceleration	**Amazon CloudFront**
Private access to Bedrock from VPC	**VPC Interface Endpoint (PrivateLink)**
Free private access to S3/DynamoDB from VPC	**VPC Gateway Endpoint**
Multi-region AI with automatic failover (DNS-based)	**Route 53 latency or failover routing**
Multi-region AI with instant failover (static IPs)	**AWS Global Accelerator**
Layer 7 routing to multiple AI models	**ALB with path-based routing**
PrivateLink endpoint service for AI	**NLB** (required for endpoint services)
Ultra-low latency inference load balancing	**NLB**

Exam Gotchas Summary

1. **API Gateway REST vs HTTP**: REST API has caching, WAF, usage plans, request validation, VTL templates, streaming. HTTP API has none of these but is cheaper.
2. **29-second timeout**: REST API default. Streaming bypasses it. Can request increase for Regional/Private.
3. **Streaming disables caching and VTL**: You cannot cache or transform streamed responses in API Gateway.
4. **API keys are not auth**: They are for identification and usage tracking only.
5. **Throttling is best-effort**: Not a hard ceiling. Combine with WAF for enforcement.
6. **AppSync Bedrock limit**: 10-second synchronous limit, no streaming APIs.
7. **Bedrock uses Interface endpoints**: Not Gateway endpoints. Gateway endpoints are S3 and DynamoDB only.
8. **NLB for PrivateLink**: You need NLB (not ALB) to create a VPC endpoint service.
9. **Route 53 TTL and failover**: DNS-based failover is subject to TTL caching. Use short TTLs or Global Accelerator for faster failover.
10. **Lambda@Edge vs CloudFront Functions**: Lambda@Edge can make network calls and run up to 30s. CloudFront Functions are sub-millisecond but no network access.

Analytics Services -- Comprehensive Study Guide (AIP-C01)

Amazon OpenSearch Service is the **most heavily tested analytics service** on the exam, particularly for vector search and RAG architectures. AWS Glue is the second-most important, especially for data quality and data lineage. The remaining services (Athena, EMR, Kinesis, QuickSight, MSK) appear less frequently but are still in-scope.

---

1. Amazon OpenSearch Service

1.1 Overview and Cluster Architecture

Amazon OpenSearch Service is a **fully managed** service for deploying, operating, and scaling OpenSearch clusters. It supports search, analytics, log analysis, and -- critically for this exam -- **vector search** for generative AI applications.

Cluster architecture:

- **Domain**: A managed OpenSearch cluster consisting of nodes
- **Data nodes**: Store data and handle search/indexing requests
- **Dedicated master nodes**: Manage cluster state (recommended for production: 3 dedicated masters)
- **UltraWarm nodes**: Cost-effective warm storage for infrequently accessed data
- **Cold storage**: Lowest-cost tier backed by S3 for rarely accessed data
- **Multi-AZ with Standby**: Production-grade availability with automatic failover

Key sizing consideration for vector workloads: OpenSearch uses half of an instance's RAM for the Java heap (up to 32 GiB). By default, k-NN uses up to 50% of the remaining half. An instance with 32 GiB RAM can accommodate approximately **8 GiB of vector graphs** (32 x 0.5 x 0.5). Monitor `KNNGraphMemoryUsage` to ensure graph memory stays within this budget.

1.2 Vector Search Capabilities (k-NN Plugin)

Vector search converts data (text, images, audio) into high-dimensional numerical vectors (embeddings) that capture semantic meaning. OpenSearch supports the `knn_vector` field type for storing dense vectors.

Core concepts:

- **knn_vector field**: Stores dense vectors with configurable dimensions (up to **16,000** dimensions, 10,000 floats for provisioned domains)
- **k-NN search**: Exact nearest neighbors -- accurate but slow on large datasets
- **Approximate k-NN (ANN)**: Uses algorithms like HNSW for faster searches at scale
- **Maximum k value**: 10,000 neighbors per query

Search engines:

Engine	Algorithms	Best For
FAISS (Facebook)	HNSW, IVF	Large-scale production, quantization support
NMSLIB	HNSW	High-performance HNSW search
Lucene	HNSW	Smaller datasets, integrated with OpenSearch core

Distance metrics:

Metric	Use Case
Euclidean distance (`l2`)	General-purpose, measures absolute distance
Cosine similarity (`cosinesimil`)	Text embeddings -- measures angle between vectors regardless of magnitude
Dot product (`innerproduct`)	Pre-normalized vectors, fastest computation

Creating a k-NN index:

```
PUT my-vector-index
{
  "settings": {
    "index.knn": true
  },
  "mappings": {
    "properties": {
      "embedding": {
        "type": "knn_vector",
        "dimension": 1536,
        "method": {
          "engine": "faiss",
          "name": "hnsw",
          "space_type": "l2",
          "parameters": {
            "m": 16,
            "ef_construction": 256
          }
        }
      },
      "text": { "type": "text" },
      "metadata": { "type": "keyword" }
    }
  }
}
```

Querying a k-NN index:

```
GET my-vector-index/_search
{
  "size": 5,
  "query": {
    "knn": {
      "embedding": {
        "vector": [0.1, 0.2, ...],
        "k": 5
      }
    }
  }
}
```

Filtered k-NN search (important for RAG):

```
GET my-vector-index/_search
{
  "size": 5,
  "query": {
    "knn": {
      "embedding": {
        "vector": [0.1, 0.2, ...],
        "k": 5,
```

```
          "filter": {
            "bool": {
              "must": [
                { "term": { "category": "technical" } },
                { "range": { "date": { "gte": "2024-01-01" } } }
              ]
            }
          }
        }
      }
    }
  }
```

Quantization techniques for cost optimization:

Technique	Compression	Memory Savings	Use Case
Binary quantization	32-bit to 1-bit	Up to 32x	Coarse-grained similarity
Byte quantization	32-bit to 8-bit	75%	Good balance of quality and savings
FP16 quantization	32-bit to 16-bit	50%	Minimal quality loss
Product quantization	Variable	Up to 64x	Maximum compression

Disk-based vector search (OpenSearch 2.17+):

- Keeps compressed vectors in memory, full-precision vectors on disk
- **97% reduction** in memory requirements compared to in-memory mode
- Higher latency (low hundreds of milliseconds) but dramatically lower cost
- Ideal for large-scale production workloads where cost matters more than latency

1.3 Neural Plugin for Semantic Search

The Neural plugin provides end-to-end semantic search by integrating ML models directly into the search pipeline. It automates embedding generation during both ingestion and query time.

Components:

1. **Text embedding ingest pipeline processor**: Generates embeddings during document ingestion
2. **Neural query**: Semantic search that auto-generates query embeddings
3. **Neural sparse query**: Sparse vector search using learned sparse representations

How neural search works:

1. Configure an ML model connector (Bedrock, SageMaker, or remote model)
2. Create an ingest pipeline with a `text_embedding` processor
3. Documents ingested through the pipeline automatically get embeddings generated
4. At query time, use the `neural` query type -- it auto-generates the query embedding

Neural query example:

```
GET my-index/_search
{
  "query": {
    "neural": {
      "embedding_field": {
        "query_text": "What is machine learning?",
        "model_id": "my-bedrock-model-id",
        "k": 10
      }
```

```
      }
    }
  }
```

Neural search with images (multimodal):

```
{
  "neural": {
    "vector_field": {
      "query_text": "query_text",
      "query_image": "base64_encoded_image",
      "model_id": "model_id",
      "k": 100
    }
  }
}
```

Integration with Amazon Bedrock:

- Use CloudFormation templates to create connectors between OpenSearch and Bedrock
- The connector creates an IAM role, a Lambda function, and registers a remote model via the ML Commons plugin
- Supports Amazon Titan Embeddings, Cohere Embed, and Titan Multimodal Embeddings
- Separate templates available for VPC-based domains

1.4 Hybrid Search (Combining Keyword and Vector Search)

Hybrid search blends **lexical search** (BM25 keyword matching) with **semantic search** (vector similarity) for the best of both worlds. This is the recommended approach for production RAG systems.

Why hybrid search matters:

- Keyword search is precise for exact terms, product names, codes
- Semantic search understands intent, synonyms, context
- Combining both yields **10-20% better relevance** than either alone (per Amazon case study)

Implementation requires a search pipeline with normalization:

Step 1 -- Create a search pipeline:

```
PUT _search/pipeline/hybrid-pipeline
{
  "description": "Hybrid search pipeline",
  "phase_results_processors": [
    {
      "normalization-processor": {
        "normalization": {
          "technique": "min_max"
        },
        "combination": {
          "technique": "arithmetic_mean",
          "parameters": {
            "weights": [0.3, 0.7]
          }
        }
      }
    }
  ]
}
```

Step 2 -- Execute a hybrid query:

```
GET my-index/_search?search_pipeline=hybrid-pipeline
{
  "query": {
    "hybrid": {
      "queries": [
        {
          "match": {
            "text": "machine learning algorithms"
          }
        },
        {
          "neural": {
            "embedding_field": {
              "query_text": "machine learning algorithms",
              "model_id": "my-model-id",
              "k": 10
            }
          }
        }
      ]
    }
  }
}
```

Normalization techniques (exam-testable):

Technique	How It Works	When to Use
min_max	Scales scores to [0,1] range	Default choice, works well generally
l2	L2 norm normalization	When score distributions vary widely

Combination techniques:

Technique	How It Works	When to Use
arithmetic_mean	Simple average of normalized scores	Default, balanced approach
geometric_mean	Multiplicative average	Penalizes low scores more
harmonic_mean	Harmonic average	Requires both signals to be strong

Weight tuning: The `weights` array controls relative importance. `[0.3, 0.7]` means 30% keyword, 70% semantic. Tune based on your query distribution.

Sparse vector search (newer approach):

- Reduces corpus terms to approximately 32,000 learned tokens
- Uses mostly-zero weight vectors (sparse representation)
- Can be combined with dense vectors in hybrid queries
- Lower memory footprint than dense vectors

1.5 Sharding Strategies for Vector Databases

Sharding determines how data is distributed across nodes. For vector workloads, shard configuration directly impacts search latency and cost.

Key principles:

- Each shard independently loads k-NN graphs into memory

- More shards = more parallelism but more overhead
- Fewer shards = less overhead but potentially higher per-shard latency
- **k-NN returns k results per shard, per segment** -- always set `size` along with `k`

Shard sizing guidelines for vector workloads:

Factor	Recommendation
Shard size	10-50 GiB per shard (general OpenSearch guidance)
Vector memory	Monitor `KNNGraphMemoryUsage` per data node
Shard count	Number of shards = dataset size / target shard size
Replicas	At least 1 replica for availability; more for read-heavy workloads

Memory planning formula:

```
Required memory per node = (num_vectors x dimensions x 4 bytes x overhead_factor) / num_shards
overhead_factor ~= 1.5 for HNSW graphs
```

HNSW parameters that affect memory and performance:

Parameter	Default	Effect
`m`	16	Connections per node in graph. Higher = better recall, more memory
`ef_construction`	512	Build-time quality. Higher = better graph, slower indexing
`ef_search`	Set at query time	Search-time quality. Higher = better recall, slower search

1.6 Multi-Index Approaches for Specialized Domains

Use multiple indexes when different document types have different embedding models, dimensions, or search requirements.

Common patterns:

Pattern 1 -- Domain-specific indexes:

```
product-descriptions-index  (dimension: 1536, Titan Embeddings)
customer-reviews-index      (dimension: 1024, Cohere Embed)
support-tickets-index       (dimension: 768, custom model)
```

Query across multiple indexes with: `GET product-*,customer-*/_search`

Pattern 2 -- Tiered freshness indexes:

```
recent-docs-index    (hot tier, high-performance instances)
archive-docs-index   (UltraWarm tier, cost-optimized)
```

Pattern 3 -- Language-specific indexes:

```
docs-en-index  (English embedding model)
docs-es-index  (Spanish embedding model)
docs-ja-index  (Japanese embedding model)
```

When to use multi-index vs. single index:

Scenario	Approach
Same embedding model, same dimensions	Single index with metadata filtering
Different embedding models/dimensions	Separate indexes (required)
Different access control needs	Separate indexes
Need independent scaling	Separate indexes
Minimize operational overhead	Single index with metadata fields

1.7 Hierarchical Indexing Techniques

Hierarchical indexing improves retrieval quality by organizing data at multiple levels of granularity.

Approach 1 -- Parent-child document indexing:

- Index both document-level summaries and chunk-level details
- First search at the document level (coarse), then at the chunk level (fine) within matching documents
- Reduces the search space and improves relevance

Approach 2 -- Multi-resolution embeddings:

- Generate embeddings at different granularities: paragraph, page, document
- Coarse search first identifies relevant documents, fine search retrieves exact passages
- Useful for long documents where chunk-level search alone might miss context

Approach 3 -- Metadata-enriched hierarchical filtering:

```
{
  "query": {
    "knn": {
      "chunk_embedding": {
        "vector": [...],
        "k": 20,
        "filter": {
          "bool": {
            "must": [
              { "term": { "document_type": "technical_manual" } },
              { "term": { "section": "troubleshooting" } }
            ]
          }
        }
      }
    }
  }
}
```

1.8 Index Optimization for Performance

Best practices for production vector search:

1. **Force merge** completed indexes to reduce segment count (fewer segments = fewer per-segment k-NN searches)
2. **Warm up** k-NN indexes after deployment with `POST /_plugins/_knn/warmup/my-index`
3. **Use appropriate quantization** -- FP16 or byte quantization for most workloads; binary for very large scale
4. **Set** `ef_search` appropriately at query time (higher = better recall, slower)

5. **Use disk-based search** for cost-sensitive workloads (OpenSearch 2.17+)
6. **Monitor metrics**: `KNNGraphMemoryUsage`, `SearchLatency`, `Searchable Documents`
7. **Use** `_msearch` **API** for bulk queries to reduce per-request overhead

Refresh interval tuning:

- Default is 1 second for provisioned domains
- For write-heavy workloads, increase refresh interval (e.g., 30s) during bulk ingestion
- For Serverless collections, refresh interval is fixed at **60 seconds**

1.9 OpenSearch Serverless vs. Provisioned

Feature	Provisioned	Serverless
Scaling	Manual (add/remove nodes)	Automatic (OCU-based)
Billing	Per-instance-hour + EBS storage	OCU-hours (compute) + S3 storage
Vector engines	FAISS, NMSLIB, Lucene	FAISS only (HNSW algorithm only)
Encryption	Optional	Always required
Access control	IAM + fine-grained access control	Data access policies
Version upgrades	Manual	Automatic
Network	VPC or public	VPC endpoints + network policies
Index refresh	Configurable (default 1s)	Fixed at 60 seconds
Service name (signing)	`es`	`aoss`
Neural search latency	Normal	Up to 15 seconds for newly created pipelines
IVF algorithm	Supported	Not supported
Lucene engine	Supported	Not supported for ANN
Inline/stored scripts	Supported	Not supported
Billion-scale workloads	Need to plan capacity	Auto-scales (contact support for >200 OCUs)

When to use Serverless:

- Unpredictable or bursty workloads
- Want zero infrastructure management
- RAG applications with moderate query volume
- Development/testing environments

When to use Provisioned:

- Predictable, sustained workloads
- Need IVF algorithm or Lucene engine
- Need sub-second index refresh
- Need maximum control over instance types and scaling

1.10 Integration with Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases natively supports OpenSearch Serverless as a vector store. This is the **recommended vector database for Bedrock RAG applications**.

How it works:

1. Bedrock automatically creates an OpenSearch Serverless collection (vector search type)

2. Documents are chunked, embedded using your chosen model (e.g., Titan Embeddings V2), and indexed
3. At query time, Bedrock generates query embeddings and searches the OpenSearch index
4. Retrieved chunks are sent as context to the LLM for answer generation

Configuration in Bedrock:

```
{
  "collectionArn": "arn:aws:aoss:us-east-1:123456789012:collection/my-collection",
  "vectorIndexName": "bedrock-knowledge-base-index",
  "fieldMapping": {
    "vectorField": "embedding",
    "textField": "text",
    "metadataField": "metadata"
  }
}
```

Binary embeddings support:

- Amazon Titan Text Embeddings V2 supports binary embeddings
- When combined with OpenSearch Serverless binary vector stores, reduces memory usage significantly
- Enables cost-effective RAG at scale

1.11 Exam Gotchas -- OpenSearch

1. **k returns per shard**: k-NN returns `k` results per shard, per segment. Always set both `k` and `size` in your query, or you may get more results than expected.
2. **Serverless limitations**: No IVF, no Lucene engine, no inline scripts, 60-second refresh interval, FAISS/HNSW only.
3. **Memory budget**: k-NN uses up to 50% of (RAM - heap). Exceeding this degrades performance.
4. **Neural search on Serverless**: Expect up to 15 seconds latency on newly created pipelines. Only remote models are supported.
5. **Signing difference**: Provisioned uses `es` service name, Serverless uses `aoss`. Must include `x-amz-content-sha256` header for Serverless.
6. **Hybrid search requires a search pipeline**: You cannot just combine match and knn in a bool query and get proper score normalization. You need the normalization processor.
7. **Dimension limits**: Up to 16,000 dimensions for Serverless, 10,000 for provisioned k-NN.
8. **UltraWarm migration**: k-NN indexes from version 2.x+ can migrate to UltraWarm/cold on domain version 2.17+.
9. **Disk-based vector search**: Available on OpenSearch 2.17+. Provides 97% memory reduction but higher latency.
10. **Bedrock integration**: Bedrock Knowledge Bases defaults to OpenSearch Serverless. The connector uses Lambda + ML Commons plugin.

1.12 TypeScript SDK v3 -- OpenSearch

OpenSearch uses its own client library, not the AWS SDK, for data operations. The AWS SDK is used for domain management.

Domain management (AWS SDK):

```
import { OpenSearchClient, DescribeDomainCommand } from "@aws-sdk/client-opensearch";

const client = new OpenSearchClient({ region: "us-east-1" });
const response = await client.send(
```

```
  new DescribeDomainCommand({ DomainName: "my-vector-domain" })
);
console.log(response.DomainStatus?.EngineVersion);
```

Serverless collection management:

```
import {
  OpenSearchServerlessClient,
  CreateCollectionCommand,
} from "@aws-sdk/client-opensearchserverless";

const client = new OpenSearchServerlessClient({ region: "us-east-1" });

const response = await client.send(
  new CreateCollectionCommand({
    name: "my-vector-collection",
    type: "VECTORSEARCH",
    description: "Vector store for RAG application",
  })
);
console.log(response.createCollectionDetail?.id);
```

Data operations (OpenSearch client with SigV4 signing):

```
import { Client } from "@opensearch-project/opensearch";
import { AwsSigv4Signer } from "@opensearch-project/opensearch/aws";
import { defaultProvider } from "@aws-sdk/credential-provider-node";

const client = new Client({
  ...AwsSigv4Signer({
    region: "us-east-1",
    service: "aoss", // "aoss" for Serverless, "es" for provisioned
    getCredentials: () =&gt; defaultProvider()(),
  }),
  node: "https://my-collection-id.us-east-1.aoss.amazonaws.com",
});

// Index a document with vector
await client.index({
  index: "my-vector-index",
  body: {
    embedding: [0.1, 0.2, 0.3 /* ... 1536 dimensions */],
    text: "OpenSearch supports vector search for RAG applications",
    category: "technical",
  },
});

// k-NN search
const results = await client.search({
  index: "my-vector-index",
  body: {
    size: 5,
    query: {
      knn: {
        embedding: {
          vector: [0.1, 0.2, 0.3 /* ... query embedding */],
          k: 5,
        },
      },
    },
```

```
  },
});
```

---

2. AWS Glue

2.1 Glue Data Quality for Validation

AWS Glue Data Quality is a **serverless, ML-powered** capability for measuring and monitoring data quality. It is built on the open-source DeeQu framework and uses the **Data Quality Definition Language (DQDL)** to express rules.

Why it matters for GenAI:

- Garbage in, garbage out -- poor training data quality produces poor model outputs
- Data quality checks are essential before fine-tuning, continued pre-training, or RAG indexing
- Glue Data Quality integrates directly into ETL pipelines to validate data before downstream use

Key capabilities:

- **Rule recommendations**: ML-powered automatic rule suggestions based on data analysis
- **DQDL rules**: 26+ built-in rule types for comprehensive validation
- **Row-level results**: Identify which specific records failed quality checks
- **Anomaly detection**: ML-based detection of data quality drifts over time
- **Dynamic rules**: Compare current metrics against historical values using `last()` operator
- **Labels**: Attach custom metadata to rules (team ownership, criticality, regulatory)
- **Integration**: EventBridge, CloudWatch, S3 output for data quality results

Important DQDL rule types:

Rule Type	What It Checks	Example
Completeness	% of non-null values	`Completeness "email" > 0.95`
Uniqueness	% of unique values	`Uniqueness "user_id" = 1.0`
ColumnValues	Value constraints	`ColumnValues "age" between 0 and 120`
ColumnLength	String length constraints	`ColumnLength "phone" = 10`
ColumnExists	Column presence	`ColumnExists "embedding_vector"`
ColumnCount	Number of columns	`ColumnCount >= 10`
RowCount	Number of rows	`RowCount > 1000`
CustomSql	Custom SQL validation	`CustomSql "SELECT COUNT(*) FROM ..." > 0`
ColumnDataType	Data type compliance	`ColumnDataType "score" = "DOUBLE"`
ColumnCorrelation	Correlation between columns	`ColumnCorrelation "x" "y" > 0.8`
AggregateMatch	Cross-dataset matching	Compares aggregates between datasets

DQDL ruleset example:

```
Rules = [
  # Ensure training data quality
  Completeness "text_content" &gt; 0.99,
  Uniqueness "document_id" = 1.0,
  ColumnValues "label" in ["positive", "negative", "neutral"],
  ColumnLength "text_content" between 10 and 50000,
```

```
    RowCount &gt; 10000,

    # Dynamic rule -- compare to historical baseline
    RowCount &gt; last(5).avg() * 0.9
  ]
```

Integration in a Glue ETL job (visual editor):

1. Add a data source node
2. Add an "Evaluate Data Quality" transform node
3. Define rules using the DQDL builder or text editor
4. Configure actions: publish to CloudWatch, fail job on quality failure
5. Route passing records downstream, failing records to a quarantine location

2.2 Glue Data Catalog for Metadata Management

The Glue Data Catalog is a **persistent, centralized metadata repository** for all data assets. It is a drop-in replacement for the Apache Hive Metastore.

Key concepts:

- **Databases**: Logical groupings of tables
- **Tables**: Metadata definitions (schema, location, format, partitions) -- not the actual data
- **Partitions**: Subdivision of tables for efficient querying (e.g., by date)
- **Connections**: Store parameters for connecting to data stores (JDBC, S3, etc.)
- **Schema Registry**: Manage and enforce schemas for streaming data (Kafka, Kinesis)

For GenAI exam tasks:

- **Data source registration** (Task 3.3.2): Register data sources in the Data Catalog so that Glue ETL, Athena, Redshift Spectrum, and EMR can discover and query them
- Use crawlers to auto-discover schemas or manually define tables via console/API/CLI
- The Data Catalog provides **schema change tracking** and **data access controls**
- Integrates with AWS Lake Formation for fine-grained access control

Best practices:

- Run crawlers regularly (or on schedule) to keep metadata current
- Use incremental crawls for frequently changing data sources
- Establish consistent naming conventions for databases and tables
- Use the Data Catalog as the centralized metadata layer for Athena, Redshift Spectrum, EMR, and Lake Formation

2.3 Data Lineage Tracking for Compliance

Data lineage traces data origins, tracks transformations, and visualizes how data flows through your organization. This is critical for AI governance and compliance.

AWS Glue provides lineage through two mechanisms:

1. AWS Glue Job Lineage (built-in):

- Glue ETL jobs automatically track source-to-target lineage
- Shows which datasets were read, what transformations were applied, where output was written
- Captured at the job run level

2. Amazon DataZone Integration (enhanced lineage):

- DataZone provides a graphical lineage visualization

- Captures lineage from AWS Glue catalogs and Amazon Redshift databases
- Supports column-level lineage
- Provides upstream/downstream navigation
- APIs: `GetLineageNode`, `ListLineageNodeHistory`, `PostLineageEvent`
- Glue jobs and notebooks can send lineage events directly to DataZone

For the exam (Task 3.3.1):

- Know that Glue tracks lineage at the job level automatically
- DataZone extends this with visual, cross-organizational lineage
- Lineage metadata should include: processing stage (raw/filtered/processed/analyzed), transformation steps, software versions, parameter settings
- Store lineage in a centralized catalog -- the Glue Data Catalog + DataZone

Lifecycle metadata tagging pattern:

```
raw-data/          -&gt; stage: "raw"
filtered-data/     -&gt; stage: "filtered", filter_params: {...}
processed-data/    -&gt; stage: "processed", model_version: "v2.1"
analyzed-results/  -&gt; stage: "analyzed", pipeline_run_id: "abc123"
```

2.4 Glue ETL for Data Processing

AWS Glue ETL is a **serverless** data processing engine based on Apache Spark. It is commonly used to prepare data for AI/ML pipelines.

Key features for GenAI:

- **Auto-scaling**: Automatically allocates resources based on workload
- **Flex execution**: Lower-cost execution for non-urgent jobs (uses spare capacity)
- **Streaming ETL**: Process streaming data from Kinesis Data Streams, Kafka, and MSK
- **Built-in transforms**: Clean, enrich, deduplicate, flatten, normalize data
- **Data format conversion**: Convert between JSON, CSV, Parquet, ORC, Avro, Iceberg
- **Worker types**: Standard, G.1X, G.2X, G.4X, G.8X, Z.2X (for memory-intensive jobs)
- **Bookmarking**: Track previously processed data for incremental ETL
- **DPU (Data Processing Unit)**: Unit of billing -- 1 DPU = 4 vCPUs + 16 GB memory

Common GenAI data preparation patterns:

1. **Training data preparation**: Clean, deduplicate, and format text data for fine-tuning
2. **RAG document processing**: Extract text from documents, chunk, and prepare for embedding
3. **Feature engineering**: Transform raw data into features for ML models
4. **Data validation pipeline**: Glue ETL + Data Quality rules before indexing into OpenSearch

2.5 Glue Crawlers

Crawlers automatically discover data schemas and create/update table definitions in the Data Catalog.

How crawlers work:

1. Connect to a data store (S3, JDBC, DynamoDB, etc.)
2. Determine data format using **classifiers** (built-in or custom)
3. Infer schema (column names, types)
4. Create or update table metadata in the Data Catalog
5. Create partitions based on folder structure (e.g., `year=2024/month=01/`)

Supported data stores: Amazon S3, DynamoDB, JDBC databases (RDS, Aurora, Redshift), MongoDB, and others.

Built-in classifiers: JSON, CSV, Parquet, ORC, Avro, XML, and more.

Best practices:

- Use **incremental crawling** for large, frequently updated datasets
- Define **separate include paths** for tables with different schemas in the same S3 prefix
- Schedule crawlers to run after data ingestion completes
- Each Athena table corresponds to an S3 prefix -- avoid multiple tables from the same prefix

2.6 Exam Gotchas -- Glue

1. **DQDL is the language for rules**: Know the name "Data Quality Definition Language" and its basic syntax.
2. **Data Quality works on both ETL jobs and Data Catalog**: Two entry points for the same capability.
3. **Dynamic rules use** `last()`: For comparing current metrics to historical baselines.
4. **Crawlers create metadata, not data**: They do not move or transform data.
5. **Data Catalog is a Hive Metastore replacement**: Compatible with Athena, EMR, Redshift Spectrum.
6. **Lineage is automatic in Glue jobs**: No extra configuration needed for basic lineage.
7. **DataZone provides enhanced lineage**: Visual, cross-org, column-level lineage beyond what Glue provides alone.
8. **Nested/list data types not supported in DQDL**: Rules cannot evaluate nested or list-type data sources.
9. **Row-level results**: Data Quality can output which specific rows failed checks, enabling quarantine patterns.
10. **Glue Streaming ETL**: Can ingest from Kinesis Data Streams, Kafka, and MSK in real time.

2.7 TypeScript SDK v3 -- Glue

```
import {
  GlueClient,
  GetDatabaseCommand,
  GetTableCommand,
  StartJobRunCommand,
  GetDataQualityResultCommand,
  StartCrawlerCommand,
} from "@aws-sdk/client-glue";

const glue = new GlueClient({ region: "us-east-1" });

// Get a database from the Data Catalog
const db = await glue.send(
  new GetDatabaseCommand({ Name: "my_genai_database" })
);
console.log(db.Database?.Name);

// Get table metadata
const table = await glue.send(
  new GetTableCommand({
    DatabaseName: "my_genai_database",
    Name: "training_data",
  })
);
console.log(table.Table?.StorageDescriptor?.Columns);

// Start a Glue ETL job
const jobRun = await glue.send(
  new StartJobRunCommand({
```

```
    JobName: "prepare-training-data",
    Arguments: {
      "--input_path": "s3://my-bucket/raw/",
      "--output_path": "s3://my-bucket/processed/",
    },
  })
);
console.log(jobRun.JobRunId);

// Start a crawler
await glue.send(
  new StartCrawlerCommand({ Name: "my-data-crawler" })
);

// Get data quality results
const dqResult = await glue.send(
  new GetDataQualityResultCommand({ ResultId: "dq-result-abc123" })
);
console.log(dqResult.Score); // Overall data quality score
console.log(dqResult.RuleResults); // Per-rule pass/fail
```

---

3. Amazon Athena

3.1 Overview

Amazon Athena is an **interactive, serverless** query service that analyzes data in Amazon S3 using standard SQL. It is built on Presto/Trino and integrates tightly with the Glue Data Catalog.

Relevance to GenAI:

- Query training datasets stored in S3 without loading them into a database
- Analyze model evaluation results, data quality outputs, and experiment metadata
- Explore and profile data before using it for fine-tuning or RAG
- Query Glue Data Quality results written to S3
- Analyze logs and metrics from ML pipeline runs

3.2 Key Features

- **Serverless**: No infrastructure to manage, pay per query ($5 per TB scanned)
- **S3 data lake native**: Queries data directly in S3 (Parquet, ORC, JSON, CSV, Avro, Iceberg, Hudi, Delta Lake)
- **Glue Data Catalog integration**: Uses Glue tables as the metastore
- **CTAS and INSERT INTO**: Create new tables from query results for downstream processing
- **Federated query**: Query data in other sources (DynamoDB, Redshift, RDS, OpenSearch) via Lambda connectors
- **SPICE integration**: Athena results can feed into QuickSight SPICE for visualization
- **Workgroups**: Separate users/teams, enforce cost controls, configure query result locations
- **Trusted Identity Propagation**: Supports TIP for QuickSight Direct Query

3.3 Athena for AI Pipelines

Common patterns:

1. Profiling training data:

```
SELECT
  COUNT(*) as total_rows,
  COUNT(DISTINCT category) as unique_categories,
  AVG(LENGTH(text_content)) as avg_text_length,
  SUM(CASE WHEN text_content IS NULL THEN 1 ELSE 0 END) as null_count
FROM my_genai_database.training_data
WHERE partition_date = '2024-01-15'
```

2. Analyzing data quality results:

```
SELECT
  rule_name,
  result,
  evaluation_message
FROM my_genai_database.data_quality_results
WHERE result = 'FAIL'
ORDER BY rule_name
```

3. Querying model evaluation outputs:

```
SELECT
  model_id,
  AVG(accuracy) as avg_accuracy,
  AVG(f1_score) as avg_f1,
  COUNT(*) as eval_count
FROM my_genai_database.model_evaluations
GROUP BY model_id
ORDER BY avg_accuracy DESC
```

3.4 Exam Gotchas -- Athena

1. **Cost optimization**: Use columnar formats (Parquet, ORC) and partitioning to reduce data scanned.
2. **Not for real-time**: Athena is for interactive analytics, not sub-second queries. Use OpenSearch for search workloads.
3. **Glue Data Catalog is the metastore**: Athena does not have its own metadata store.
4. **Federated queries**: Can query OpenSearch, DynamoDB, and other sources via Lambda connectors.
5. **$5 per TB scanned**: Optimize by partitioning, using columnar formats, and compressing data.

3.5 TypeScript SDK v3 -- Athena

```
import {
  AthenaClient,
  StartQueryExecutionCommand,
  GetQueryExecutionCommand,
  GetQueryResultsCommand,
} from "@aws-sdk/client-athena";

const athena = new AthenaClient({ region: "us-east-1" });

// Start a query
const execution = await athena.send(
  new StartQueryExecutionCommand({
    QueryString: "SELECT * FROM training_data LIMIT 10",
    QueryExecutionContext: { Database: "my_genai_database" },
    ResultConfiguration: {
      OutputLocation: "s3://my-query-results/athena/",
```

```
    },
  })
);

// Poll for completion
let status = "RUNNING";
while (status === "RUNNING" || status === "QUEUED") {
  const check = await athena.send(
    new GetQueryExecutionCommand({
      QueryExecutionId: execution.QueryExecutionId!,
    })
  );
  status = check.QueryExecution?.Status?.State!;
  if (status === "RUNNING" || status === "QUEUED") {
    await new Promise((r) =&gt; setTimeout(r, 2000));
  }
}

// Get results
const results = await athena.send(
  new GetQueryResultsCommand({
    QueryExecutionId: execution.QueryExecutionId!,
  })
);
console.log(results.ResultSet?.Rows);
```

---

4. Amazon EMR

4.1 Overview

Amazon EMR is a **managed big data platform** for processing vast amounts of data using open-source frameworks including Apache Spark, Hive, Presto, HBase, Flink, and Hudi.

Relevance to GenAI:

- Large-scale data processing and feature engineering for ML
- Distributed text processing and embedding generation
- Processing terabytes of training data that exceeds Glue's practical limits
- Spark MLlib for distributed ML training

4.2 Architecture

Layer	Components
Storage	HDFS, EMRFS (S3), Local file system
Resource Management	YARN
Processing	Spark, MapReduce, Flink, Presto
Applications	Hive, Pig, Spark SQL, MLlib, Spark Streaming

Node types:

- **Primary node**: Manages cluster, coordinates distributed processing
- **Core nodes**: Run tasks + store data in HDFS (if used)
- **Task nodes**: Only run tasks, no HDFS storage -- can use Spot Instances

4.3 EMR for AI Data Preparation

Common patterns:

Spark for distributed text processing:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import length, col

spark = SparkSession.builder.appName("DataPrep").getOrCreate()

# Read raw training data
df = spark.read.parquet("s3://my-bucket/raw-training-data/")

# Clean and filter
cleaned = (df
  .filter(length(col("text")) &gt; 50)
  .filter(col("language") == "en")
  .dropDuplicates(["text"])
  .repartition(100)
)

# Write processed data
cleaned.write.parquet("s3://my-bucket/processed-training-data/")
```

Key integrations:

- **EMRFS**: Direct S3 access, data stays in S3 (no need to copy to HDFS)
- **Glue Data Catalog**: EMR can use Glue as the Hive Metastore
- **SageMaker**: SageMaker Spark connector for distributed training/inference
- **EMR Serverless**: Run Spark/Hive jobs without managing clusters

4.4 EMR Serverless vs. EMR on EC2

Feature	EMR on EC2	EMR Serverless
Management	You manage cluster lifecycle	Serverless, auto-provisions
Scaling	Manual or managed scaling	Automatic
Billing	Per-instance-hour	Per-resource-second
Best for	Long-running, complex workloads	Short-lived, intermittent jobs
Frameworks	Spark, Hive, Presto, HBase, Flink, etc.	Spark, Hive
Customization	Full control over AMIs, bootstrap actions	Limited

4.5 Exam Gotchas -- EMR

1. **EMR vs. Glue**: Both do Spark ETL. Use Glue for simpler, serverless ETL with Data Catalog integration. Use EMR for complex, large-scale processing with multiple frameworks.
2. **EMR Serverless**: Good for intermittent Spark/Hive workloads without cluster management.
3. **Anti-pattern**: Do not use EMR for small datasets that can be processed in memory on a single system.
4. **EMRFS**: Always prefer EMRFS (S3) over HDFS for storage in cloud-native architectures.
5. **Glue Data Catalog integration**: EMR can use Glue as the Hive Metastore for unified metadata.

4.6 TypeScript SDK v3 -- EMR

```
import { EMRClient, RunJobFlowCommand } from "@aws-sdk/client-emr";

const emr = new EMRClient({ region: "us-east-1" });

const cluster = await emr.send(
  new RunJobFlowCommand({
    Name: "data-prep-cluster",
    ReleaseLabel: "emr-7.0.0",
    Instances: {
      MasterInstanceType: "m5.xlarge",
      SlaveInstanceType: "m5.2xlarge",
      InstanceCount: 5,
      KeepJobFlowAliveWhenNoSteps: false,
    },
    Applications: [{ Name: "Spark" }],
    Steps: [
      {
        Name: "Process Training Data",
        ActionOnFailure: "TERMINATE_CLUSTER",
        HadoopJarStep: {
          Jar: "command-runner.jar",
          Args: [
            "spark-submit",
            "--deploy-mode", "cluster",
            "s3://my-bucket/scripts/process_data.py",
          ],
        },
      },
    ],
    JobFlowRole: "EMR_EC2_DefaultRole",
    ServiceRole: "EMR_DefaultRole",
    LogUri: "s3://my-bucket/emr-logs/",
  })
);
console.log(cluster.JobFlowId);
```

---

5. Amazon Kinesis

5.1 Overview

Amazon Kinesis is a platform for collecting, processing, and analyzing **real-time streaming data**. It consists of several services.

Relevance to GenAI:

- Stream real-time data for continuous model inference
- Ingest user interactions for real-time recommendation systems
- Feed streaming data into ML pipelines for anomaly detection
- Process real-time events for RAG applications that need fresh data

5.2 Kinesis Data Streams vs. Data Firehose

Feature	Kinesis Data Streams	Amazon Data Firehose
Purpose	Real-time data streaming with custom consumers	Managed ETL delivery to destinations
Latency	~200ms (real-time)	Near real-time (60-900 second buffering)
Scaling	Manual (provisioned) or auto (on-demand)	Automatic
Consumers	Custom applications, Lambda, KCL	Built-in delivery to S3, Redshift, OpenSearch, Splunk, HTTP
Data retention	1-365 days	No retention (deliver and done)
Ordering	Per-shard ordering via partition key	No ordering guarantee
Replay	Yes (within retention period)	No
Transforms	Custom consumer logic	Lambda transforms, format conversion (Parquet/ORC)
Pricing	Per-shard-hour or on-demand	Per-GB delivered

5.3 Key Concepts

Kinesis Data Streams:

- **Shard**: Unit of capacity. Each shard provides 1 MB/s write, 2 MB/s read
- **Partition key**: Determines which shard a record goes to
- **Sequence number**: Unique per-record identifier within a shard
- **On-demand mode**: Automatically scales shards (default throughput: 4 MB/s write, 8 MB/s read)
- **Enhanced fan-out**: Dedicated 2 MB/s per-consumer throughput (no shared read capacity)

Amazon Data Firehose:

- **Delivery stream**: The core entity -- source to destination pipeline
- **Buffering**: Configurable by size (1-128 MB) and interval (60-900 seconds)
- **Dynamic partitioning**: Partition output data in S3 by key values
- **Format conversion**: Convert JSON to Parquet/ORC using Glue Data Catalog schemas
- **Destinations**: S3, Redshift, OpenSearch Service, Splunk, Datadog, MongoDB, HTTP endpoints

5.4 Streaming Data for AI Patterns

Pattern 1 -- Real-time feature ingestion:

```
User events -&gt; Kinesis Data Streams -&gt; Lambda -&gt; DynamoDB (feature store) -&gt; Model inference
```

Pattern 2 -- Streaming into OpenSearch for RAG:

```
Data source -&gt; Kinesis Data Firehose -&gt; S3 -&gt; Glue ETL -&gt; OpenSearch index
```

Pattern 3 -- Real-time anomaly detection:

```
IoT sensors -&gt; Kinesis Data Streams -&gt; Managed Service for Apache Flink -&gt; Bedrock inference -&gt; Alert
```

Pattern 4 -- Glue Streaming ETL:

```
Kinesis Data Streams -&gt; Glue Streaming ETL -&gt; S3 (Iceberg tables) -&gt; Athena analysis
```

5.5 Exam Gotchas -- Kinesis

1. **Data Streams vs. Firehose**: Streams for custom processing with replay. Firehose for managed delivery to destinations.
2. **Firehose can deliver to OpenSearch Service**: Built-in destination, no custom code needed.
3. **On-demand vs. provisioned**: On-demand auto-scales shards; provisioned requires manual shard management.
4. **Data retention**: Streams retains data 1-365 days. Firehose does not retain (deliver-only).
5. **Enhanced fan-out**: Needed when multiple consumers need dedicated throughput from the same stream.
6. **Kinesis Data Firehose was renamed**: Now called "Amazon Data Firehose" -- exam may use either name.
7. **Lambda integration**: Both Streams and Firehose support Lambda for processing/transformation.

5.6 TypeScript SDK v3 -- Kinesis

```
import {
  KinesisClient,
  PutRecordCommand,
  GetRecordsCommand,
  GetShardIteratorCommand,
} from "@aws-sdk/client-kinesis";

const kinesis = new KinesisClient({ region: "us-east-1" });

// Publish a record to Kinesis Data Streams
await kinesis.send(
  new PutRecordCommand({
    StreamName: "user-interactions",
    Data: Buffer.from(
      JSON.stringify({
        userId: "user-123",
        action: "search",
        query: "machine learning tutorials",
        timestamp: new Date().toISOString(),
      })
    ),
    PartitionKey: "user-123",
  })
);

// Firehose example
import {
  FirehoseClient,
  PutRecordCommand as FirehosePutRecordCommand,
} from "@aws-sdk/client-firehose";

const firehose = new FirehoseClient({ region: "us-east-1" });

await firehose.send(
  new FirehosePutRecordCommand({
    DeliveryStreamName: "events-to-s3",
    Record: {
      Data: Buffer.from(
        JSON.stringify({ event: "page_view", page: "/products" }) + "\n"
      ),
    },
  })
);
```

6. Amazon QuickSight

6.1 Overview

Amazon QuickSight is a **serverless, cloud-native BI service** that delivers interactive dashboards, pixel-perfect reports, natural language queries, and embedded analytics.

Relevance to GenAI:

- Visualize AI/ML metrics, model performance, and data quality results
- Use **Amazon Q in QuickSight** for natural language data exploration
- Embed AI-powered analytics into applications
- Create dashboards for monitoring RAG pipeline health

6.2 Amazon Q in QuickSight (Natural Language Queries)

Amazon Q in QuickSight is the **generative BI** capability that lets users interact with data using natural language.

Capabilities:

- **Multi-visual Q&A**: Ask questions in natural language, get rich visual answers (charts, tables, KPIs)
- **Dashboard authoring**: Build dashboards using natural language prompts
- **Executive summaries**: AI-generated summaries of dashboard insights
- **Data stories**: Generate narrative documents with visuals from natural language prompts
- **Scenario analysis**: Agentic capability for complex business problem analysis
- **Unstructured insights**: Combine insights from documents and structured databases

How it works:

1. User asks a natural language question in the Q interface
2. Amazon Q interprets the question, identifies relevant fields
3. If needed, invokes LLMs on Amazon Bedrock for interpretation
4. Queries the underlying dataset and generates visual answers
5. Results displayed as multi-visual answers (bar charts, KPIs, tables, trend lines)

Topics: Define a "Topic" in QuickSight to configure which datasets and fields are available for Q&A. Topics map business terms to data columns.

6.3 Key Concepts

- **SPICE**: Super-fast Parallel In-memory Calculation Engine. In-memory optimized data store for fast querying. Supports up to 2 billion rows per dataset.
- **Datasets**: Connect to data sources (Athena, Redshift, S3, RDS, etc.)
- **Analyses**: Where you build visuals
- **Dashboards**: Published, read-only versions of analyses
- **Embedded analytics**: Embed dashboards, Q&A, and authoring into applications via SDK

6.4 Exam Gotchas -- QuickSight

1. **Amazon Q in QuickSight**: The generative BI capability. Powered by Bedrock LLMs under the hood.
2. **SPICE**: In-memory engine, not always required (can use direct query mode for real-time data).

3. **Serverless scaling**: Auto-scales to tens of thousands of users.
4. **Pricing**: Per-session pricing for readers (pay only when they view dashboards).
5. **Data sources**: Connects to Athena, Redshift, S3, RDS, OpenSearch, and many more.
6. **Embedding SDK**: JavaScript SDK for embedding dashboards and Q&A into applications.
7. **Not a data processing tool**: QuickSight is for visualization and exploration, not ETL or transformation.

6.5 TypeScript SDK v3 -- QuickSight

```
import {
  QuickSightClient,
  GenerateEmbedUrlForRegisteredUserCommand,
  ListDashboardsCommand,
  DescribeDataSetCommand,
} from "@aws-sdk/client-quicksight";

const qs = new QuickSightClient({ region: "us-east-1" });
const accountId = "123456789012";

// List dashboards
const dashboards = await qs.send(
  new ListDashboardsCommand({ AwsAccountId: accountId })
);
console.log(dashboards.DashboardSummaryList);

// Generate embedding URL for a registered user
const embedUrl = await qs.send(
  new GenerateEmbedUrlForRegisteredUserCommand({
    AwsAccountId: accountId,
    UserArn:
      "arn:aws:quicksight:us-east-1:123456789012:user/default/my-user",
    ExperienceConfiguration: {
      QuickSightConsole: {
        InitialPath: "/start",
      },
    },
    SessionLifetimeInMinutes: 600,
  })
);
console.log(embedUrl.EmbedUrl);
```

---

7. Amazon Managed Streaming for Apache Kafka (MSK)

7.1 Overview

Amazon MSK is a **fully managed Apache Kafka** service for building real-time streaming data pipelines and applications. It manages infrastructure provisioning, cluster operations, patching, and security.

Relevance to GenAI:

- Event-driven architectures for AI applications
- Real-time data ingestion pipelines for continuously updating knowledge bases
- Change data capture (CDC) for keeping RAG indexes current
- High-throughput data streaming for ML feature pipelines

7.2 Key Concepts

Cluster types:

Type	Description
MSK Provisioned	You choose broker types and count. Two broker types: Express (fully managed, faster) and Standard.
MSK Serverless	Auto-scaling, pay-per-use. No broker management.

Core components:

- **Topics**: Logical channels for organizing messages
- **Partitions**: Horizontal scaling within a topic
- **Producers**: Applications that publish messages
- **Consumers**: Applications that read messages
- **Consumer groups**: Coordinate message consumption across multiple consumers
- **MSK Connect**: Managed Kafka Connect for source/sink connectors (e.g., Debezium for CDC)
- **MSK Replicator**: Cross-region data replication

7.3 MSK for AI Architectures

Pattern 1 -- CDC pipeline for RAG freshness:

```
Database -&gt; MSK Connect (Debezium) -&gt; MSK topic -&gt; Glue Streaming ETL -&gt; OpenSearch index
```

Pattern 2 -- Event-driven inference:

```
Application events -&gt; MSK topic -&gt; Lambda (event source mapping) -&gt; Bedrock inference -&gt; Response top
```

Pattern 3 -- Real-time feature pipeline:

```
Clickstream -&gt; MSK topic -&gt; Managed Flink -&gt; Feature Store -&gt; Real-time ML inference
```

MSK + Lambda integration:

- Lambda can consume from MSK topics via event source mapping
- Automatic partition assignment, offset tracking, and batch processing
- Supports event filtering to reduce Lambda invocations
- Supports cross-account event processing

7.4 MSK vs. Kinesis Data Streams

Feature	Amazon MSK	Kinesis Data Streams
Protocol	Apache Kafka (open source)	AWS proprietary
Ecosystem	Kafka ecosystem (Connect, Streams, KSQL)	AWS ecosystem (Lambda, Firehose, Flink)
Ordering	Per-partition	Per-shard
Message size	Up to 10 MB (configurable)	Up to 1 MB
Retention	Unlimited (with tiered storage)	1-365 days
Client libraries	Any Kafka client	AWS SDK, KCL, KPL

Feature	Amazon MSK	Kinesis Data Streams
Best for	Existing Kafka workloads, complex event processing	AWS-native, simpler setup
Pricing	Per-broker-hour (provisioned) or per-GB (serverless)	Per-shard-hour or on-demand

7.5 Exam Gotchas -- MSK

1. **MSK vs. Kinesis**: Choose MSK when you have existing Kafka expertise/code or need the Kafka ecosystem (Connect, Streams). Choose Kinesis for simpler, AWS-native streaming.
2. **MSK Serverless**: Auto-scales, no broker management -- good for variable workloads.
3. **MSK Connect**: Managed Kafka Connect for connecting to external data sources/sinks without managing infrastructure.
4. **Glue Streaming ETL**: Can read from MSK topics directly for real-time processing.
5. **Lambda integration**: Lambda can consume directly from MSK topics with event source mapping.
6. **CDC with Debezium**: Common exam pattern for keeping RAG knowledge bases current with database changes.

7.6 TypeScript SDK v3 -- MSK

```
import {
  KafkaClient,
  ListClustersV2Command,
  GetBootstrapBrokersCommand,
} from "@aws-sdk/client-kafka";

const msk = new KafkaClient({ region: "us-east-1" });

// List MSK clusters
const clusters = await msk.send(new ListClustersV2Command({}));
console.log(clusters.ClusterInfoList);

// Get bootstrap brokers for client connection
const brokers = await msk.send(
  new GetBootstrapBrokersCommand({
    ClusterArn:
      "arn:aws:kafka:us-east-1:123456789012:cluster/my-cluster/abc-123",
  })
);
console.log(brokers.BootstrapBrokerStringSaslIam); // IAM auth endpoint
```

For actual Kafka produce/consume operations, use a Kafka client library (e.g., `kafkajs`) rather than the AWS SDK:

```
import { Kafka } from "kafkajs";

const kafka = new Kafka({
  clientId: "my-genai-app",
  brokers: ["broker1:9098", "broker2:9098"],
  ssl: true,
  sasl: {
    mechanism: "aws",
    authorizationIdentity: "123456789012",
    accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!,
```

```
    sessionToken: process.env.AWS_SESSION_TOKEN,
  },
});

const producer = kafka.producer();
await producer.connect();
await producer.send({
  topic: "document-updates",
  messages: [
    {
      key: "doc-456",
      value: JSON.stringify({
        action: "updated",
        documentId: "doc-456",
        content: "Updated document content for RAG indexing",
      }),
    },
  ],
});
```

---

8. Service Selection Quick Reference

Use this table when an exam question describes a scenario and you need to choose the right service.

Scenario	Service
Vector database for RAG with Bedrock	**OpenSearch Serverless** (vector search collection)
Semantic search + keyword search combined	**OpenSearch Service** (hybrid search)
Validate data quality before fine-tuning	**AWS Glue Data Quality** (DQDL rules)
Register data sources in a metadata catalog	**AWS Glue Data Catalog**
Track data lineage for AI governance	**AWS Glue** (job lineage) + **Amazon DataZone** (visual lineage)
Query S3 data with SQL for exploration	**Amazon Athena**
Process terabytes of training data with Spark	**Amazon EMR** or **Glue ETL**
Real-time streaming into OpenSearch	**Amazon Data Firehose** (built-in OpenSearch destination)
Custom real-time stream processing with replay	**Kinesis Data Streams**
Natural language BI queries for business users	**Amazon Q in QuickSight**
Event-driven AI with Kafka ecosystem	**Amazon MSK**
CDC pipeline for keeping RAG index current	**MSK Connect** (Debezium) -> MSK -> Glue/Lambda -> OpenSearch
Managed delivery of streaming data to S3/Redshift	**Amazon Data Firehose**

Scenario	Service
Visualize model evaluation metrics	**Amazon QuickSight**
Large-scale batch embedding generation	**OpenSearch Ingestion** (ML batch inference) or **EMR**
Cost-optimized vector search at scale	**OpenSearch** disk-based vector search or quantization

---

9. Key Metrics to Monitor

Service	Key Metrics
OpenSearch	`KNNGraphMemoryUsage`, `SearchLatency`, `IndexingLatency`, `SearchableDocuments`, `CPUUtilization`, `JVMMemoryPressure`
OpenSearch Serverless	`SearchOCU`, `IndexingOCU`, `SearchRequestLatency`, `SearchRequestErrors`
Glue	`glue.driver.aggregate.bytesRead`, `glue.driver.aggregate.numCompletedTasks`, DPU utilization, job duration
Glue Data Quality	Data quality score, per-rule pass/fail counts, anomaly detection alerts
Athena	`TotalExecutionTime`, `DataScannedInBytes`, `QueryQueueTime`
EMR	`AppsRunning`, `AppsCompleted`, `HDFSUtilization`, `MemoryAvailableMB`, `YARNMemoryAvailablePercentage`
Kinesis Data Streams	`IncomingRecords`, `IncomingBytes`, `GetRecords.IteratorAgeMilliseconds`, `ReadProvisionedThroughputExceeded`
Data Firehose	`DeliveryToS3.Success`, `IncomingBytes`, `DeliveryToS3.DataFreshness`
QuickSight	SPICE usage, concurrent user sessions, dataset refresh status
MSK	`BytesInPerSec`, `BytesOutPerSec`, `UnderReplicatedPartitions`, `OfflinePartitionsCount`, consumer lag

Database Services -- Comprehensive Study Guide (AIP-C01)

Database services appear across multiple exam domains. Aurora (pgvector) and DynamoDB are the most heavily tested -- Aurora for vector search in RAG architectures, DynamoDB for conversation history and metadata storage. ElastiCache appears in caching/optimization questions, Neptune in knowledge graph scenarios, and DocumentDB as a vector-capable alternative.

Exam Relevance Map

Service	Exam Task	Context
Aurora (pgvector)	1.4.1, 1.5.3	Vector store for RAG, Bedrock Knowledge Bases integration
DynamoDB	1.4.1, 1.6.2	Conversation history, metadata for GenAI apps
DynamoDB Streams	1.4.1	Change detection, event-driven GenAI pipelines
ElastiCache	4.1.4	Semantic caching, response caching for AI
Neptune	1.4.1	Knowledge graphs, GraphRAG
DocumentDB	1.4.1	MongoDB-compatible vector search
RDS (PostgreSQL)	1.4.1	pgvector (non-Aurora alternative)

---

1. Amazon Aurora (pgvector Focus)

1.1 Overview

Amazon Aurora is a **fully managed relational database engine** compatible with MySQL and PostgreSQL. For the exam, focus on **Aurora PostgreSQL-Compatible Edition** because it supports the **pgvector** extension for vector similarity search.

Key Aurora characteristics:

- Up to **5x throughput** of standard PostgreSQL
- **Storage auto-scales** from 10 GB to 128 TB
- Up to **15 read replicas** with sub-10ms replica lag
- **Multi-AZ** with automatic failover (under 30 seconds)
- **Aurora Serverless v2** scales compute capacity automatically
- **Aurora Optimized Reads** improves query performance for vector workloads by using local NVMe storage

1.2 pgvector Extension for Vector Search

pgvector is an **open-source PostgreSQL extension** that adds vector data types and similarity search operations. It allows you to store embeddings directly alongside your relational data.

Core capabilities:

- Store vectors as a native `vector` data type

- Exact and approximate nearest neighbor (ANN) search
- Distance metrics: **L2 (Euclidean)**, **cosine distance**, **inner product**
- Maximum dimensions: **16,000** (as of pgvector 0.8.0)
- Filter with standard SQL `WHERE` clauses during vector search

Enabling pgvector:

```
-- Enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Create a table with a vector column (1536 dims for Titan Embeddings V2)
CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  content TEXT NOT NULL,
  metadata JSONB,
  embedding vector(1536)
);

-- Insert a vector
INSERT INTO documents (content, metadata, embedding)
VALUES (
  'Aurora supports vector search via pgvector',
  '{"source": "docs", "topic": "databases"}',
  '[0.1, 0.2, 0.3, ...]'::vector
);
```

Similarity search queries:

```
-- Cosine similarity search (most common for text embeddings)
SELECT id, content, 1 - (embedding &lt;=&gt; '[0.1, 0.2, ...]'::vector) AS similarity
FROM documents
ORDER BY embedding &lt;=&gt; '[0.1, 0.2, ...]'::vector
LIMIT 10;

-- L2 (Euclidean) distance
SELECT * FROM documents
ORDER BY embedding &lt;-&gt; '[0.1, 0.2, ...]'::vector
LIMIT 10;

-- Inner product (use &lt;#&gt; operator)
SELECT * FROM documents
ORDER BY embedding &lt;#&gt; '[0.1, 0.2, ...]'::vector
LIMIT 10;

-- Filtered vector search (pgvector 0.8.0 iterative scans prevent overfiltering)
SELECT * FROM documents
WHERE metadata-&gt;&gt;'topic' = 'databases'
ORDER BY embedding &lt;=&gt; '[0.1, 0.2, ...]'::vector
LIMIT 10;
```

Distance operator reference:

Operator	Metric	Best For
<->	L2 (Euclidean) distance	Absolute distance comparisons
<=>	Cosine distance	Text embeddings (most common)
<#>	Negative inner product	When vectors are normalized

1.3 Vector Indexing: IVFFlat vs HNSW

Without an index, pgvector performs **exact nearest neighbor** search (scans every row). For production workloads, you need an approximate nearest neighbor (ANN) index.

IVFFlat (Inverted File with Flat Compression):

```
-- Create an IVFFlat index
-- lists = number of clusters (start with rows / 1000, min 100)
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- Tune probes at query time (higher = better recall, slower)
SET ivfflat.probes = 10;
```

- Divides vectors into **clusters (lists)** using k-means
- At query time, searches only the **nearest clusters** (controlled by `probes`)
- **Faster to build** than HNSW
- **Must have data loaded before creating** the index (k-means needs data to cluster)
- Lower recall than HNSW at the same speed
- Best when: dataset is relatively static, cost of index rebuild is acceptable

HNSW (Hierarchical Navigable Small World):

```
-- Create an HNSW index
-- m = connections per layer (default 16), ef_construction = build quality (default 64)
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Tune ef_search at query time (higher = better recall, slower)
SET hnsw.ef_search = 40;
```

- Builds a **multi-layered graph** structure
- **Better recall** than IVFFlat at the same query speed
- **Can be built on an empty table** (incrementally adds vectors)
- **More expensive to build** and uses more memory
- pgvector 0.7.0+: **parallel HNSW index builds** (up to 30x faster)
- Best when: you need highest recall, data changes frequently

Index comparison (exam-critical):

Property	IVFFlat	HNSW
Build speed	Faster	Slower (but parallel since 0.7.0)
Query recall	Good	Better
Memory usage	Lower	Higher
Empty table support	No (needs data first)	Yes
Incremental updates	Requires rebuild	Supports incremental
Tuning knobs	`lists`, `probes`	`m`, `ef_construction`, `ef_search`

1.4 pgvector 0.8.0 Improvements (Latest)

- **Iterative index scans**: Solves the overfiltering problem when combining vector search with SQL `WHERE` clauses. Continues scanning instead of stopping prematurely when filters are strict.
- **Three scan modes**: `off`, `strict_order` (maintains exact ordering), `relaxed_order` (best performance, recommended for production)
- **Improved query planner**: Better decisions about when to use indexes vs sequential scan
- **HNSW performance**: Faster index builds and search

1.5 Quantization (Cost/Performance Optimization)

pgvector 0.7.0+ supports quantization to reduce memory and storage:

Technique	Storage Savings	Recall Impact	Use Case
Scalar (halfvec)	50%	Minimal	Recommended starting point
Binary	~97%	Significant (use re-ranking)	Very large datasets, cost-sensitive

```
-- Scalar quantization: cast to halfvec
CREATE INDEX ON documents
USING hnsw ((embedding::halfvec(1536)) halfvec_cosine_ops);

-- Query with scalar quantization
SELECT * FROM documents
ORDER BY embedding::halfvec(1536) &lt;=&gt; '[0.1, ...]'::halfvec(1536)
LIMIT 10;
```

1.6 When to Use Aurora vs OpenSearch for Vectors

Factor	Aurora (pgvector)	OpenSearch Service
Best for	Existing PostgreSQL workloads, combined SQL + vector queries	Dedicated vector search at scale, full-text + vector hybrid search
Scale	Millions of vectors	Billions of vectors (GPU-accelerated indexing)
Query type	SQL + vector in same query	k-NN, hybrid (BM25 + vector), aggregations
Managed RAG	Bedrock Knowledge Bases supported	Bedrock Knowledge Bases supported (Serverless)
Latency	Single-digit ms for moderate scale	Single-digit ms, optimized for large scale
Additional features	Full SQL, joins, transactions, ACID	Full-text search, dashboards, log analytics
Cost model	Instance-based or Serverless v2	Instance-based or Serverless (OCU-based)

Exam tip: Choose Aurora when the application already uses PostgreSQL and needs both relational queries AND vector search. Choose OpenSearch when vector search is the primary workload or when you need billions of vectors with GPU acceleration.

1.7 Integration with Bedrock Knowledge Bases

Aurora PostgreSQL with pgvector is a **supported vector store** for Amazon Bedrock Knowledge Bases. The managed service handles:

- Chunking documents from S3 data sources
- Generating embeddings using a Bedrock embedding model
- Storing embeddings and metadata in Aurora with pgvector
- Retrieval via the Bedrock Retrieve API

- Metadata filtering for multi-tenant isolation

Required Aurora schema for Bedrock Knowledge Bases:

```
-- Bedrock Knowledge Bases expects this schema structure
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE bedrock_integration.bedrock_kb (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  embedding vector(1536),  -- dimension must match your embedding model
  chunks TEXT,
  metadata JSON
);

CREATE INDEX ON bedrock_integration.bedrock_kb
USING hnsw (embedding vector_cosine_ops);
```

1.8 Performance Tuning for Vector Workloads

- **Aurora Optimized Reads**: Uses local NVMe-based storage to cache data, improving vector query performance for datasets larger than memory
- **Read replicas**: Offload vector search queries to up to 15 read replicas
- **Instance sizing**: Vector workloads are memory-intensive; choose memory-optimized instances (r6g, r7g)
- **Shared memory**: Increase `shared_buffers` and `effective_cache_size` for vector index caching
- **Maintenance**: Run `VACUUM` and `ANALYZE` after bulk vector loads
- **Connection pooling**: Use RDS Proxy or PgBouncer for high-concurrency GenAI workloads

1.9 TypeScript SDK v3 Example

```
import { RDSDataClient, ExecuteStatementCommand } from "@aws-sdk/client-rds-data";

const client = new RDSDataClient({ region: "us-east-1" });

// Vector similarity search via Aurora Data API (Serverless v2)
async function searchSimilarDocuments(
  queryEmbedding: number[],
  limit: number = 10
) {
  const embeddingStr = `[${queryEmbedding.join(",")}]`;

  const command = new ExecuteStatementCommand({
    resourceArn: "arn:aws:rds:us-east-1:123456789012:cluster:my-aurora-cluster",
    secretArn: "arn:aws:secretsmanager:us-east-1:123456789012:secret:my-secret",
    database: "mydb",
    sql: `
      SELECT id, content, metadata,
             1 - (embedding &lt;=&gt; :embedding::vector) AS similarity
      FROM documents
      ORDER BY embedding &lt;=&gt; :embedding::vector
      LIMIT :limit
    `,
    parameters: [
      { name: "embedding", value: { stringValue: embeddingStr } },
      { name: "limit", value: { longValue: limit } },
    ],
  });
```

```
  const response = await client.send(command);
  return response.records;
}
```

1.10 Key Metrics

Metric	Source	What It Tells You
`CPUUtilization`	CloudWatch	Sustained >70% means you need to scale up or add read replicas
`FreeableMemory`	CloudWatch	Low values impact vector index caching
`DatabaseConnections`	CloudWatch	High connection count needs RDS Proxy
`ReadIOPS` / `WriteIOPS`	CloudWatch	Measures storage I/O pressure
`AuroraReplicaLag`	CloudWatch	Monitors read replica freshness
`ServerlessDatabaseCapacity`	CloudWatch	ACU usage for Serverless v2

---

2. Amazon DynamoDB

2.1 Overview and Key Concepts

Amazon DynamoDB is a **fully managed, serverless NoSQL database** that delivers single-digit millisecond performance at any scale. It is the default choice for storing conversation history and metadata in GenAI applications.

Core concepts (exam-critical):

Concept	Description
Table	Collection of items. No fixed schema beyond the primary key.
Item	A single record, max **400 KB**
Partition Key (PK)	Required. Determines which partition stores the item. Must have **high cardinality**.
Sort Key (SK)	Optional. Enables range queries within a partition. Combined with PK to form the primary key.
Global Secondary Index (GSI)	Alternate PK + optional SK. Can query across all partitions. Eventually consistent. Separate throughput. Max **20 per table**.
Local Secondary Index (LSI)	Same PK, different SK. Must be created at table creation time. Shares throughput with base table. Max **5 per table**. Strongly consistent reads supported.

Single-table design: DynamoDB best practice is to store multiple entity types in a single table, using composite sort keys to differentiate them. This is the pattern used for chatbot data models.

2.2 Conversation History Storage Patterns

This is the most exam-relevant DynamoDB pattern for GenAI. The recommended data model uses **vertical partitioning** to store conversation metadata and messages as separate items under the same partition key.

Schema design:

```
Partition Key (PK): UserID
```

```
Sort Key (SK):      CONV#&lt;ConversationID&gt;          -- metadata item
                    CHAT#&lt;ConversationID&gt;#MSG#&lt;ULID&gt; -- message item
```

Metadata item:

```
{
  "PK": "user-123",
  "SK": "CONV#conv-abc",
  "created_at": 1700000000,
  "expires_at": 1707776000,
  "title": "Help with my order",
  "model_id": "anthropic.claude-sonnet-4-20250514"
}
```

Message item:

```
{
  "PK": "user-123",
  "SK": "CHAT#conv-abc#MSG#01HQ3ZZKBKACTAV9WEVGEMMVRY",
  "created_at": 1700000001,
  "expires_at": 1707776001,
  "sender": "user",
  "message": "What is the status of my order #12345?"
}
```

Why this design works:

- Uses **ULID** (Universally Unique Lexicographically Sortable Identifier) for message IDs, providing automatic chronological ordering
- **Vertical partitioning** avoids the 400 KB item size limit -- each message is its own item (1 WCU per small message write vs. potentially hundreds of WCU if messages were stored as a list in a single item)
- `SK` prefix pattern (`CONV#` vs `CHAT#`) enables efficient access patterns: query `begins_with('CONV#')` for listing conversations, `begins_with('CHAT#<convId>#MSG')` for getting messages
- **TTL** on `expires_at` provides automatic cleanup (e.g., 90 days)

Access patterns:

Pattern	Operation	Key Condition
List user conversations	`Query`	`PK = userId AND begins_with(SK, 'CONV#')`
Get messages for conversation	`Query`	`PK = userId AND begins_with(SK, 'CHAT#<convId>#MSG')`
Create conversation	`PutItem`	`PK = userId, SK = 'CONV#<convId>'`
Add message	`PutItem`	`PK = userId, SK = 'CHAT#<convId>#MSG#<ulid>'`
Delete conversation	`BatchWriteItem`	Delete metadata item + all message items

Anti-pattern (exam gotcha): Storing all messages as a list attribute in a single item. This fails because:

1. Items have a 400 KB size limit
2. Every write (even a 2-byte "hi") costs WCU proportional to the full item size
3. No way to query individual messages efficiently

2.3 Metadata Storage for GenAI Applications

DynamoDB is commonly used to store metadata alongside vector databases:

- **Document metadata**: Source, title, date, author, permissions for documents stored in S3 whose embeddings are in a vector store
- **User preferences**: Model preferences, conversation settings, prompt templates
- **Session state**: Active session tracking, model parameters per session
- **Prompt/response logging**: Audit trail of prompts and completions

Example: metadata sidecar for a vector store

```
PK: DOC#&lt;documentId&gt;
SK: META

Attributes:
  source_s3_uri: "s3://my-bucket/docs/report.pdf"
  chunk_count: 42
  embedding_model: "amazon.titan-embed-text-v2:0"
  indexed_at: 1700000000
  permissions: ["role-admin", "role-analyst"]
```

The vector store holds the embeddings and chunk text; DynamoDB holds the rich metadata that supports filtering, access control, and auditing.

2.4 DynamoDB Streams for Change Detection

DynamoDB Streams captures a **time-ordered sequence of item-level modifications** for up to **24 hours**. Each stream record contains information about a single data modification.

Stream view types:

View Type	Content	Use Case
`KEYS_ONLY`	Only the key attributes	Lightweight event notification
`NEW_IMAGE`	Entire item after modification	Sync to downstream systems
`OLD_IMAGE`	Entire item before modification	Audit, rollback
`NEW_AND_OLD_IMAGES`	Both before and after	Diff detection, full audit trail

Key characteristics:

- Stream records appear **exactly once** in the stream
- Records appear in the **same sequence** as actual modifications to the item
- Data retention: **24 hours**
- Maximum **2 simultaneous consumers per shard** (use EventBridge for more)
- Encrypted at rest
- No performance impact on the source table
- Separate endpoint from the DynamoDB data plane

GenAI use cases for DynamoDB Streams:

1. **Trigger re-embedding**: When a document's metadata changes, trigger a Lambda that regenerates embeddings in the vector store
2. **Sync to search index**: Stream changes to OpenSearch via Lambda for full-text search
3. **Real-time analytics**: Stream conversation data to Kinesis Data Streams for analytics
4. **Audit trail**: Capture all modifications to prompts, model configs, or guardrail settings
5. **Cache invalidation**: When source data changes, invalidate cached LLM responses

DynamoDB Streams vs Kinesis Data Streams for DynamoDB:

Feature	DynamoDB Streams	Kinesis Data Streams
Retention	24 hours	Up to 365 days
Consumers per shard	2	Unlimited (with enhanced fan-out)
Integration	Lambda triggers, KCL	Lambda, Firehose, Flink, Spark
Cost	Included with DynamoDB	Separate Kinesis pricing + CDC units
Ordering	Per-item ordering guaranteed	Per-shard ordering

Exam tip: If the question mentions needing more than 2 consumers or retention beyond 24 hours, the answer is Kinesis Data Streams for DynamoDB (or EventBridge integration).

2.5 TTL for Data Lifecycle

Time to Live (TTL) enables **automatic, cost-free deletion** of expired items. The TTL attribute must store a **Unix epoch timestamp in seconds**.

How TTL works:

- DynamoDB scans for expired items in the background
- Items are typically deleted within **a few minutes to 48 hours** after expiration (not instant)
- Deleted items appear in DynamoDB Streams as **service deletions** (distinguishable from user deletions)
- TTL deletes are **free** -- they do not consume write capacity
- Expired items are removed from GSIs and LSIs
- With Global Tables, TTL deletes **replicate to all regions**

GenAI use cases:

- Auto-expire conversation history after 90 days
- Clean up temporary session data
- Remove cached prompt/response pairs after staleness threshold
- Comply with data retention policies (GDPR, etc.)

```
// Setting TTL when creating a conversation
const ttlEpoch = Math.floor(Date.now() / 1000) + (90 * 24 * 60 * 60); // 90 days

const params = {
  TableName: "ChatApp",
  Item: {
    PK: { S: "user-123" },
    SK: { S: "CONV#conv-abc" },
    created_at: { N: String(Math.floor(Date.now() / 1000)) },
    expires_at: { N: String(ttlEpoch) },  // TTL attribute
    title: { S: "New conversation" },
  },
};
```

2.6 On-Demand vs Provisioned Capacity

Mode	Best For	Pricing	Scaling
On-demand	Unpredictable traffic, new apps, spiky GenAI workloads	Per-request pricing	Instant, automatic

Mode	Best For	Pricing	Scaling
Provisioned	Predictable, steady-state workloads	Per-hour capacity units	Auto Scaling available but not instant

On-demand capacity:

- Pay per read/write request
- Scales instantly to accommodate traffic spikes
- No capacity planning needed
- More expensive per-request than provisioned at steady state
- Best for: new GenAI apps with unknown traffic patterns, chatbots with spiky usage

Provisioned capacity:

- Specify Read Capacity Units (RCU) and Write Capacity Units (WCU)
- 1 RCU = 1 strongly consistent read/second for items up to 4 KB
- 1 WCU = 1 write/second for items up to 1 KB
- Auto Scaling adjusts within min/max bounds (but has a ramp-up delay)
- Can use **Reserved Capacity** for up to 77% savings
- Best for: production chatbots with predictable, high-throughput traffic

Exam tip: For GenAI applications that are new or have unpredictable traffic, the answer is almost always on-demand mode. For cost optimization of mature, predictable workloads, switch to provisioned with auto scaling.

2.7 TypeScript SDK v3 Examples

```
import {
  DynamoDBClient,
  PutItemCommand,
  QueryCommand,
} from "@aws-sdk/client-dynamodb";
import { unmarshall, marshall } from "@aws-sdk/util-dynamodb";

const client = new DynamoDBClient({ region: "us-east-1" });

// Store a conversation message
async function putMessage(
  userId: string,
  conversationId: string,
  message: string,
  sender: "user" | "bot"
) {
  // Use a ULID-style sortable ID (timestamp-based for ordering)
  const messageId = `${Date.now()}-${Math.random().toString(36).slice(2, 8)}`;
  const ttl = Math.floor(Date.now() / 1000) + 90 * 24 * 60 * 60;

  const command = new PutItemCommand({
    TableName: "ChatApp",
    Item: marshall({
      PK: userId,
      SK: `CHAT#${conversationId}#MSG#${messageId}`,
      message,
      sender,
      created_at: Math.floor(Date.now() / 1000),
      expires_at: ttl,
    }),
  });
```

```
  await client.send(command);
}

// Get messages for a conversation (ordered by sort key = chronological)
async function getConversationMessages(
  userId: string,
  conversationId: string
) {
  const command = new QueryCommand({
    TableName: "ChatApp",
    KeyConditionExpression: "PK = :pk AND begins_with(SK, :skPrefix)",
    ExpressionAttributeValues: marshall({
      ":pk": userId,
      ":skPrefix": `CHAT#${conversationId}#MSG`,
    }),
  });

  const response = await client.send(command);
  return (response.Items ?? []).map((item) =&gt; unmarshall(item));
}

// List all conversations for a user
async function listConversations(userId: string) {
  const command = new QueryCommand({
    TableName: "ChatApp",
    KeyConditionExpression: "PK = :pk AND begins_with(SK, :skPrefix)",
    ExpressionAttributeValues: marshall({
      ":pk": userId,
      ":skPrefix": "CONV#",
    }),
  });

  const response = await client.send(command);
  return (response.Items ?? []).map((item) =&gt; unmarshall(item));
}
```

2.8 DynamoDB Streams with Lambda (TypeScript)

```
import { DynamoDBStreamHandler } from "aws-lambda";
import { unmarshall } from "@aws-sdk/util-dynamodb";
import { AttributeValue } from "@aws-sdk/client-dynamodb";

// Lambda function triggered by DynamoDB Stream
export const handler: DynamoDBStreamHandler = async (event) =&gt; {
  for (const record of event.Records) {
    const eventName = record.eventName; // INSERT | MODIFY | REMOVE

    if (eventName === "INSERT" &amp;&amp; record.dynamodb?.NewImage) {
      const newItem = unmarshall(
        record.dynamodb.NewImage as Record&lt;string, AttributeValue&gt;
      );

      // Trigger re-embedding when new document metadata is added
      if (newItem.SK?.startsWith("DOC#")) {
        await triggerReembedding(newItem);
      }
    }

    if (eventName === "REMOVE") {
```

```
        // Handle TTL deletions vs user deletions
        const isServiceDeletion =
          record.userIdentity?.type === "Service" &amp;amp;&amp;amp;
          record.userIdentity?.principalId === "dynamodb.amazonaws.com";

        if (isServiceDeletion) {
          console.log("Item expired via TTL");
        }
      }
    }
  };
```

2.9 Key Metrics

Metric	Source	What It Tells You
`ConsumedReadCapacityUnits`	CloudWatch	Actual read throughput consumption
`ConsumedWriteCapacityUnits`	CloudWatch	Actual write throughput consumption
`ThrottledRequests`	CloudWatch	Capacity exceeded -- scale up or switch to on-demand
`SuccessfulRequestLatency`	CloudWatch	P50/P99 latency for operations
`SystemErrors`	CloudWatch	Internal DynamoDB errors (rare, retryable)
`AccountMaxReads/Writes`	CloudWatch	Approaching account-level limits
`TimeToLiveDeletedItemCount`	CloudWatch	Tracks TTL deletion volume
`IteratorAge` (Streams)	CloudWatch	Age of oldest unprocessed stream record -- indicates consumer lag

---

3. Amazon ElastiCache

3.1 Overview: Redis OSS vs Memcached vs Valkey

ElastiCache is a **fully managed in-memory data store** service. For the exam, focus on **ElastiCache for Valkey** (the successor to Redis OSS) because it supports vector search, which enables semantic caching.

Feature	Valkey (Redis OSS successor)	Memcached
Data structures	Strings, hashes, lists, sets, sorted sets, streams, JSON, **vectors**	Simple key-value only
Persistence	Optional (AOF, RDB snapshots)	None
Replication	Yes (primary + replicas)	None
Clustering	Yes (sharding)	Yes (sharding)
Pub/Sub	Yes	No
Lua scripting	Yes	No

Feature	Valkey (Redis OSS successor)	Memcached
Vector search	Yes (HNSW, FLAT)	No
Transactions	Yes (MULTI/EXEC)	No
TTL	Per-key	Per-key

Exam tip: If the question involves caching for GenAI, the answer is always ElastiCache for Valkey (not Memcached) because of vector search support.

3.2 Semantic Caching Patterns for AI

Semantic caching is the most important ElastiCache concept for this exam. Unlike traditional caches that require exact string matches, a semantic cache uses **vector similarity** to match semantically equivalent queries.

How semantic caching works:

1. User sends query to the GenAI application
2. Application generates an **embedding** of the query (e.g., using Amazon Titan Text Embeddings)
3. Application searches ElastiCache for **similar embeddings** (vector similarity search)
4. **Cache hit** (similarity above threshold): Return the cached response directly -- no LLM call needed
5. **Cache miss**: Call the LLM, get the response, then cache both the query embedding and response in ElastiCache for future reuse

Performance impact (from AWS benchmarks):

- Up to **86% reduction** in LLM inference cost
- Up to **88% improvement** in average end-to-end latency
- 95%+ recall rate for vector search in ElastiCache
- Microsecond read latency for cache hits

Architecture:

```
User Query
    |
    v
Generate Query Embedding (Bedrock Titan Embeddings)
    |
    v
Vector Search in ElastiCache ■■■■ Cache Hit? ■■■■ Yes ■■&gt; Return Cached Response
    |                                                 (microseconds)
    No (Cache Miss)
    |
    v
Call LLM (Bedrock) ■■&gt; Get Response ■■&gt; Cache query embedding + response in ElastiCache
    |
    v
Return Response to User
```

Key configuration decisions:

Parameter	Description	Typical Value
Similarity threshold	Minimum similarity score to consider a cache hit	0.90 - 0.98 (tune per use case)
TTL	How long cached responses remain valid	Hours to days (depends on data freshness needs)
Distance metric	Cosine, Euclidean, or inner product	Cosine (most common for text)

Parameter	Description	Typical Value
Index type	HNSW or FLAT	HNSW for large caches, FLAT for small

Similarity threshold tuning (exam-critical):

- **Too high** (e.g., 0.99): Very few cache hits, minimal cost savings
- **Too low** (e.g., 0.80): More cache hits but risk returning incorrect/stale responses
- **Best practice**: Start at 0.95, monitor quality, adjust based on user feedback

3.3 Response Caching Strategies

Beyond semantic caching, ElastiCache supports multiple caching strategies for GenAI:

1. Exact match caching:

- Cache the exact prompt+response pair
- Simplest, zero risk of returning wrong answers
- Low hit rate (queries must be identical)

2. Semantic caching (covered above):

- Cache based on vector similarity
- Higher hit rate, some risk of semantic mismatch
- Best for: FAQ-style queries, customer support, IT helpdesks

3. Conversational memory:

- Store recent conversation context in ElastiCache for fast retrieval
- Agents and chatbots retrieve past interactions from ElastiCache instead of DynamoDB for hot data
- Integration with frameworks: Mem0, LangGraph, LangChain use ElastiCache as a memory backend

4. Tool output caching (for agents):

- Cache the results of tool calls (API lookups, database queries) that agents make
- Agentic AI applications break tasks into steps that may repeatedly look up similar information
- Cache tool outputs to avoid redundant calls

3.4 Cache Invalidation Patterns

Strategy	How It Works	Best For
TTL-based	Entries automatically expire after a set time	Most GenAI caching (set TTL based on data freshness needs)
Proactive invalidation	Application explicitly deletes cache entries when source data changes	When you know exactly when data changes
Proactive update	Preload cache with known hot queries on deployment	Predictable query patterns
Eviction policies	LRU, LFU, or volatile-ttl when memory is full	Automatic memory management

Exam tip: For GenAI caching, TTL-based invalidation is the most common answer. Set TTL to match how frequently your knowledge base updates.

3.5 ElastiCache Serverless

ElastiCache Serverless for Valkey automatically scales capacity based on demand. Relevant for GenAI workloads because:

- No capacity planning needed
- Scales from zero to billions of vectors
- Pay for what you use (ElastiCache Processing Units)
- Up to 30% discount with Database Savings Plans

3.6 TypeScript Example: Semantic Cache

```
// Pseudocode illustrating semantic cache flow with ElastiCache for Valkey
// (Uses the ioredis client for Valkey compatibility)

import Redis from "ioredis";
import {
  BedrockRuntimeClient,
  InvokeModelCommand,
} from "@aws-sdk/client-bedrock-runtime";

const redis = new Redis({
  host: "my-elasticache-cluster.xxxxx.cache.amazonaws.com",
  port: 6379,
  tls: {},
});

const bedrock = new BedrockRuntimeClient({ region: "us-east-1" });

const SIMILARITY_THRESHOLD = 0.95;
const CACHE_TTL_SECONDS = 3600; // 1 hour

// Generate embedding using Titan Embeddings V2
async function getEmbedding(text: string): Promise&lt;number[]&gt; {
  const command = new InvokeModelCommand({
    modelId: "amazon.titan-embed-text-v2:0",
    contentType: "application/json",
    body: JSON.stringify({ inputText: text }),
  });
  const response = await bedrock.send(command);
  const result = JSON.parse(new TextDecoder().decode(response.body));
  return result.embedding;
}

// Search semantic cache, fall back to LLM on miss
async function queryWithSemanticCache(userQuery: string): Promise&lt;string&gt; {
  const queryEmbedding = await getEmbedding(userQuery);

  // Search ElastiCache vector index for similar queries
  // FT.SEARCH is the Valkey vector search command
  const searchResults = await redis.call(
    "FT.SEARCH",
    "semantic_cache_idx",
    `*=&gt;[KNN 1 @embedding $query_vec AS score]`,
    "PARAMS", "2", "query_vec", Buffer.from(new Float32Array(queryEmbedding).buffer),
    "SORTBY", "score",
    "RETURN", "2", "response", "score",
    "DIALECT", "2"
  ) as any[];

  // Check if we got a hit above our similarity threshold
  if (searchResults &amp;&amp; searchResults[0] &gt; 0) {
    const score = parseFloat(searchResults[2]?.[3]); // cosine similarity
```

```
    if (score &gt;= SIMILARITY_THRESHOLD) {
      const cachedResponse = searchResults[2]?.[1];
      console.log(`Cache hit (similarity: ${score})`);
      return cachedResponse;
    }
  }

  // Cache miss: call the LLM
  console.log("Cache miss, calling LLM");
  const llmResponse = await callLLM(userQuery);

  // Store in cache for future use
  const cacheKey = `cache:${Date.now()}`;
  await redis.call(
    "HSET", cacheKey,
    "query", userQuery,
    "response", llmResponse,
    "embedding", Buffer.from(new Float32Array(queryEmbedding).buffer)
  );
  await redis.expire(cacheKey, CACHE_TTL_SECONDS);

  return llmResponse;
}
```

3.7 Key Metrics

Metric	Source	What It Tells You
`CacheHitRate`	CloudWatch	Percentage of successful cache lookups
`EngineCPUUtilization`	CloudWatch	CPU used by Valkey engine (vector search is CPU-intensive)
`DatabaseMemoryUsagePercentage`	CloudWatch	Memory consumption (vectors use significant memory)
`CurrConnections`	CloudWatch	Active connections to cache
`Evictions`	CloudWatch	Items evicted due to memory pressure (may need scaling)
`ReplicationLag`	CloudWatch	Replica freshness
`NetworkBytesIn/Out`	CloudWatch	Network throughput

---

4. Amazon Neptune

4.1 Graph Database Overview

Amazon Neptune is a **fully managed graph database** that supports two graph models:

- **Property Graph** with Apache TinkerPop Gremlin query language
- **RDF (Resource Description Framework)** with SPARQL query language

Neptune Analytics is a separate, memory-optimized engine for graph analytics with built-in vector search.

Key characteristics:

- Up to 15 read replicas

- Multi-AZ with automatic failover
- Continuous backup to S3
- Encryption at rest and in transit
- Supports up to **billions of relationships** with millisecond query latency

4.2 Knowledge Graphs for GenAI (GraphRAG)

This is Neptune's primary exam relevance. **GraphRAG** combines knowledge graphs with RAG to provide more accurate, comprehensive, and explainable AI responses.

Why knowledge graphs improve GenAI:

Traditional RAG	GraphRAG
Retrieves isolated document chunks	Retrieves interconnected entities and relationships
Struggles with multi-hop reasoning	Supports multi-hop reasoning by traversing the graph
Context fragmentation across chunks	Maintains entity relationships across documents
Limited cross-document reasoning	Connects related information across multiple documents

How GraphRAG works:

1. **Document ingestion**: Extract entities, relationships, and structural elements from unstructured text
2. **Graph construction**: Build a knowledge graph of entities and their relationships
3. **Embedding storage**: Store vector embeddings of graph nodes and edges
4. **Retrieval**: Vector search finds relevant nodes, then graph traversal expands context by following relationships
5. **Generation**: LLM generates response using the enriched, relationship-aware context

Amazon Bedrock Knowledge Bases GraphRAG:

- Fully managed: automatically creates and maintains graphs and embeddings
- Uses **Neptune Analytics** as the graph store
- Uses **OpenSearch Serverless** as the vector store
- Claude 3 Haiku for contextual enrichment during graph construction
- Supports only S3 as data source
- Max 1,000 files per data source (can request increase to 10,000)

AWS GraphRAG Toolkit (self-managed):

- Open-source Python toolkit
- Supports Neptune Database or Neptune Analytics as graph store
- Supports Neptune Analytics or OpenSearch Serverless as vector store
- Supports Bring Your Own Knowledge Graph (BYOKG) -- connect existing graphs to LLMs
- More flexible but requires managing the pipeline yourself

4.3 Use Cases for Neptune in GenAI

Use Case	Why Neptune
Fraud detection	Traverse transaction relationships to identify fraud rings
Drug discovery	Map molecular interactions and protein relationships
Recommendation engines	Traverse user-item-feature relationships
IT operations	Map infrastructure dependencies, impact analysis
Compliance/regulatory	Track entity relationships for KYC, AML

Use Case	Why Neptune
Enterprise search	Connect information across departments and documents

4.4 When to Use Neptune vs Other Databases

- **Choose Neptune** when: You need to model and query complex relationships between entities, your queries involve traversing paths/connections, or you need GraphRAG
- **Choose DynamoDB** when: You need simple key-value or document access with single-digit ms latency
- **Choose Aurora** when: You need relational data modeling with SQL queries and vector search
- **Choose OpenSearch** when: You need full-text search combined with vector search at scale

4.5 TypeScript SDK v3 Example

```
// Neptune is typically accessed via HTTP endpoints with Gremlin or SPARQL
// For Neptune Analytics (used with Bedrock Knowledge Bases GraphRAG):

import {
  NeptuneGraphClient,
  ExecuteQueryCommand,
} from "@aws-sdk/client-neptune-graph";

const client = new NeptuneGraphClient({ region: "us-east-1" });

// Query a Neptune Analytics graph
async function queryKnowledgeGraph(graphId: string, query: string) {
  const command = new ExecuteQueryCommand({
    graphIdentifier: graphId,
    queryString: query,
    language: "OPEN_CYPHER", // or GREMLIN
  });

  const response = await client.send(command);
  return response.payload;
}

// Example: Find entities related to a topic within 2 hops
const results = await queryKnowledgeGraph(
  "g-abc123",
  `
  MATCH (e:Entity {name: 'Amazon Bedrock'})-[r*1..2]-(related)
  RETURN e.name, type(r), related.name
  LIMIT 20
  `
);
```

4.6 Key Metrics

Metric	Source	What It Tells You
`GremlinRequestsPerSec`	CloudWatch	Query throughput
`GremlinErrors`	CloudWatch	Query failures
`CPUUtilization`	CloudWatch	Graph traversal is CPU-intensive
`VolumeBytesUsed`	CloudWatch	Graph storage consumption

Metric	Source	What It Tells You
`MainRequestQueuePendingRequests`	CloudWatch	Query queue depth

---

5. Amazon DocumentDB

5.1 Overview

Amazon DocumentDB is a **fully managed, MongoDB-compatible document database**. It supports the MongoDB API, making it a drop-in replacement for applications using MongoDB drivers.

Key characteristics:

- **MongoDB API compatibility** (works with existing MongoDB drivers and tools)
- Storage auto-scales up to 128 TB
- Up to 15 read replicas
- Continuous backup to S3
- Flexible JSON document model with schema-less design

5.2 Vector Search Capabilities

DocumentDB supports vector search starting with version 5.0+:

- **Indexing methods**: HNSW and IVFFlat (same as pgvector)
- **Max dimensions**: 2,000 (indexed), 16,000 (stored without index)
- **Distance metrics**: Euclidean, cosine, dot product
- **Query operators**: `$search` (classic) and `$vectorSearch` (v8.0+, aggregation pipeline)

Creating a vector index:

```
// HNSW index
db.runCommand({
  createIndexes: "documents",
  indexes: [{
    key: { embedding: "cosmosSearch" },
    cosmosSearchOptions: {
      kind: "vector-hnsw",
      m: 16,
      efConstruction: 64,
      similarity: "COS",
      dimensions: 1536
    },
    name: "embedding_hnsw_index"
  }]
});

// IVFFlat index
db.runCommand({
  createIndexes: "documents",
  indexes: [{
    key: { embedding: "cosmosSearch" },
    cosmosSearchOptions: {
      kind: "vector-ivf",
      numLists: 100,
      similarity: "COS",
```

```
      dimensions: 1536
    },
    name: "embedding_ivf_index"
  }]
});
```

Querying vectors (v8.0+ $vectorSearch):

```
db.documents.aggregate([
  {
    $vectorSearch: {
      index: "embedding_hnsw_index",
      path: "embedding",
      queryVector: [0.1, 0.2, 0.3, ...],
      limit: 10,
      numCandidates: 100
    }
  }
]);
```

5.3 When to Use DocumentDB vs DynamoDB

Factor	DocumentDB	DynamoDB
Data model	Rich JSON documents, flexible schema	Key-value with optional document support
Query flexibility	MongoDB query language, aggregation pipeline, ad-hoc queries	Key-based access, limited query patterns without GSIs
Vector search	Native support (HNSW, IVFFlat)	No native vector support
Scale	Vertical + read replicas	Horizontal, virtually unlimited
Latency	Single-digit ms	Single-digit ms
Schema evolution	Excellent (schema-less documents)	Good (schema-less but key design is rigid)
Best for GenAI	Document-heavy RAG with vector search, existing MongoDB apps	Conversation history, metadata, session state
Pricing	Instance-based	Per-request or provisioned capacity

Exam tip: If a question mentions MongoDB compatibility or an existing MongoDB application, DocumentDB is the answer. If it asks about conversation history or metadata at scale, the answer is DynamoDB.

5.4 TypeScript Example

```
import { MongoClient } from "mongodb";

const uri = "mongodb://username:password@docdb-cluster.cluster-xxxxx.us-east-1.docdb.amazonaws.com:27017/?tls=tru

const client = new MongoClient(uri);

async function vectorSearch(queryEmbedding: number[], limit: number = 10) {
  const db = client.db("genai_app");
  const collection = db.collection("documents");

  // DocumentDB 8.0+ $vectorSearch
  const results = await collection.aggregate([
    {
```

```
      $vectorSearch: {
        index: "embedding_hnsw_index",
        path: "embedding",
        queryVector: queryEmbedding,
        limit: limit,
        numCandidates: limit * 10,
      },
    },
    {
      $project: {
        content: 1,
        metadata: 1,
        score: { $meta: "vectorSearchScore" },
      },
    },
  ]).toArray();

  return results;
}
```

---

6. Amazon RDS (PostgreSQL with pgvector)

6.1 Overview

Amazon RDS for PostgreSQL supports the **pgvector** extension, providing the same vector search capabilities as Aurora PostgreSQL but on the standard RDS platform.

6.2 RDS PostgreSQL vs Aurora PostgreSQL for Vectors

Factor	RDS PostgreSQL + pgvector	Aurora PostgreSQL + pgvector
Performance	Standard PostgreSQL performance	Up to 5x PostgreSQL throughput
Storage	Manual provisioning (gp3, io1/io2)	Auto-scaling (10 GB to 128 TB)
Read replicas	Up to 15 (async replication)	Up to 15 (sub-10ms lag via shared storage)
Failover	60-120 seconds	Under 30 seconds
Serverless	No	Aurora Serverless v2
Optimized Reads	No	Yes (NVMe-based local caching)
Bedrock KB integration	Yes (as RDS for PostgreSQL data store)	Yes (preferred option)
Cost	Lower per-instance cost	Higher per-instance, but better price/performance at scale
Best for	Dev/test, small-scale vector workloads, cost-sensitive	Production vector workloads, high-throughput RAG

Exam tip: If a question asks about vector search with PostgreSQL and does not specify a budget constraint, Aurora is the default answer. RDS is the answer when cost is the primary concern or the workload is small-scale.

6.3 TypeScript SDK v3 Example (RDS Data API)

```
import { RDSClient, ExecuteStatementCommand } from "@aws-sdk/client-rds-data";
```

```
const client = new RDSClient({ region: "us-east-1" });

// Same API as Aurora when using Data API
async function vectorSearchRDS(queryEmbedding: number[]) {
  const command = new ExecuteStatementCommand({
    resourceArn: "arn:aws:rds:us-east-1:123456789012:db:my-rds-instance",
    secretArn: "arn:aws:secretsmanager:us-east-1:123456789012:secret:my-secret",
    database: "mydb",
    sql: `
      SELECT id, content, 1 - (embedding &lt;=&gt; :embedding::vector) AS similarity
      FROM documents
      ORDER BY embedding &lt;=&gt; :embedding::vector
      LIMIT 10
    `,
    parameters: [
      { name: "embedding", value: { stringValue: `[${queryEmbedding.join(",")}]` } },
    ],
  });

  return (await client.send(command)).records;
}
```

---

7. Database Selection Decision Guide for GenAI

When the Exam Asks "Which Database?"

```
Need vector search for RAG?
■■■ Already using PostgreSQL? ■■&gt; Aurora PostgreSQL + pgvector
■■■ Already using MongoDB? ■■&gt; DocumentDB
■■■ Need full-text + vector hybrid search? ■■&gt; OpenSearch Service
■■■ Need graph-based relationships (GraphRAG)? ■■&gt; Neptune Analytics
■■■ Need lowest latency vector search (caching)? ■■&gt; ElastiCache for Valkey
■■■ Need billions of vectors, GPU acceleration? ■■&gt; OpenSearch Service

Need conversation history storage?
■■■ Always ■■&gt; DynamoDB (single-table, vertical partitioning)

Need metadata for GenAI apps?
■■■ Always ■■&gt; DynamoDB (key-value access pattern)

Need semantic caching for LLM responses?
■■■ Always ■■&gt; ElastiCache for Valkey

Need knowledge graph?
■■■ Fully managed with Bedrock KB? ■■&gt; Neptune Analytics (GraphRAG)
■■■ Self-managed? ■■&gt; Neptune Database + GraphRAG Toolkit

Need change detection from data store?
■■■ From DynamoDB? ■■&gt; DynamoDB Streams or Kinesis Data Streams
■■■ From relational DB? ■■&gt; Aurora CDC or RDS event notifications
```

Vector Database Comparison Matrix

Feature	Aurora (pgvector)	OpenSearch	DocumentDB	ElastiCache (Valkey)	Neptune Analytics
Max dimensions	16,000	16,000	2,000 (indexed)	Configurable	Configurable
Index types	HNSW, IVFFlat	HNSW	HNSW, IVFFlat	HNSW, FLAT	HNSW
Bedrock KB support	Yes	Yes (Serverless)	No	No	Yes (GraphRAG)
Hybrid search	SQL + vector	BM25 + vector	MongoDB query + vector	Filter + vector	Graph traversal + vector
Quantization	halfvec, binary	Yes	No	No	No
Serverless option	Aurora Serverless v2	OpenSearch Serverless	No	ElastiCache Serverless	Yes (Analytics)
Primary use case	SQL + vector	Dedicated vector search	MongoDB apps + vector	Semantic caching	GraphRAG

---

8. Exam Gotchas and Traps

1. **DynamoDB item size limit is 400 KB**. Never store unbounded lists (like all messages) in a single item. Use vertical partitioning with composite sort keys.
2. **DynamoDB Streams retention is 24 hours**. If you need longer retention or more than 2 consumers, use Kinesis Data Streams for DynamoDB.
3. **IVFFlat requires data before index creation** (needs data to cluster). HNSW can be built on an empty table. If the question says "frequently changing data" or "incremental updates", HNSW is the answer.
4. **pgvector cosine distance operator is** `<=>`, not `<->` (which is L2/Euclidean). For text embeddings, cosine is almost always the right choice.
5. **DocumentDB max indexed vector dimensions is 2,000** (vs 16,000 for pgvector and OpenSearch). If the question mentions high-dimensional vectors (>2,000), DocumentDB is not the answer.
6. **ElastiCache semantic caching uses vector similarity, not exact match**. The similarity threshold must be tuned -- too low risks returning wrong answers, too high misses cache opportunities.
7. **TTL deletions in DynamoDB are not instant** (can take up to 48 hours). If the exam asks about immediate deletion, use `DeleteItem`, not TTL.
8. **DynamoDB on-demand mode** is the default recommendation for new/unpredictable GenAI workloads. Provisioned mode is for cost optimization of mature, predictable workloads.
9. **Aurora Serverless v2** is the answer when the question mentions variable or unpredictable vector search workloads that need to scale to zero.
10. **Neptune Analytics** (not Neptune Database) is what integrates with Bedrock Knowledge Bases for GraphRAG.
11. **GraphRAG is not a replacement for standard RAG** -- it complements it for queries that need multi-hop reasoning across relationships. Standard RAG with vector search is still appropriate for most document Q&A.
12. **ElastiCache for Valkey** (not Redis OSS, not Memcached) is the current recommended choice. AWS has migrated from Redis OSS to Valkey. Exam questions may reference either name.
13. **DynamoDB Streams vs EventBridge**: DynamoDB now integrates directly with EventBridge for more than 2 consumers and advanced filtering/routing. If the question mentions complex event routing, the answer is EventBridge.
14. **Aurora Optimized Reads** specifically helps vector workloads that are larger than memory by using local NVMe storage for caching. This is an Aurora-specific advantage over RDS.

15. **Bedrock Knowledge Bases schema for Aurora**: The vector store must be set up with the correct schema (id, embedding, chunks, metadata columns) before connecting to Bedrock. This is a manual prerequisite, not automatic.

Tier 4: Supporting Services

Referenced once or twice. Know their primary purpose, the specific GenAI scenario each addresses, and the key decision criteria for choosing them over alternatives.

Guides in this tier:

- Developer Tools (X-Ray, Amplify, CDK, CI/CD, SDKs)
- Container Services (ECS, EKS, Fargate, ECR)
- Storage & Migration (S3, EBS, EFS, DataSync)
- Amazon Connect

Developer Tools for GenAI -- Study Guide

Source: AWS Certified Generative AI Developer - Professional (AIP-C01)

This guide covers the developer tools ecosystem relevant to building, deploying, testing, and observing generative AI applications on AWS. These services appear across multiple exam tasks, especially in Domain 2 (Implementation & Integration) and Domain 5 (Governance & Security).

Key Exam Tasks:

- Task 2.3.5: CI/CD pipelines for GenAI (CodePipeline, CodeBuild, automated testing, security scans, rollback)
- Task 2.4.1: Language-specific AWS SDKs
- Task 2.4.3: Observability across service boundaries, SDK exponential backoff
- Task 2.5.2: Declarative UI components for AI interfaces (Amplify AI Kit)
- Task 2.5.6: Foundation Model API call tracing (X-Ray)
- Task 5.2.5: Prompt observability pipelines

---

1. AWS X-Ray

1.1 Distributed Tracing Overview

AWS X-Ray collects data about requests as they flow through distributed applications. It builds a visual **service map** showing how services connect and where bottlenecks occur.

Core Concepts:

Concept	Definition	Exam Relevance
Segment	Data about work done by a single compute resource (hostname, request/response details, timing, errors)	Each service produces one segment per traced request
Subsegment	Granular breakdown within a segment (downstream AWS SDK calls, HTTP calls, SQL queries)	Used to trace Bedrock API calls from Lambda
Trace	Collection of all segments generated by a single request as it crosses services	Trace ID propagated via `X-Amzn-Trace-Id` header
Service Map	Visual graph of nodes (services) and edges (connections) generated from trace data	Shows full topology of your AI application
Sampling	Algorithm controlling which requests are traced (default: 1st request/second + 5% of additional)	Configurable via sampling rules; prevents cost overrun
Annotations	Key-value pairs (string/number/boolean) **indexed** for search and filter expressions	Up to 50 per trace; use for filtering AI traces
Metadata	Key-value pairs of any type, **not indexed**	Store prompt/response payloads, model config
Groups	Named filter expressions that partition traces	Create groups for AI vs. non-AI traffic

Tracing Header Format:

```
X-Amzn-Trace-Id: Root=1-5759e988-bd862e3fe1be46a994272793;Parent=53995c3f42cd8ad8;Sampled=1
```

Service Map Data Retention: 30 days.

Segment Document Size Limit: 64 KB.

1.2 Tracing Bedrock API Calls

X-Ray integrates with Amazon Bedrock AgentCore for distributed tracing of AI agent invocations. When the AWS SDK makes Bedrock API calls from an instrumented service (e.g., Lambda), X-Ray captures these as subsegments automatically.

How it works:

1. Lambda function instrumented with X-Ray creates a root segment
2. AWS SDK call to `bedrock-runtime` (e.g., `InvokeModel`, `Converse`) generates a subsegment
3. Subsegment records: latency, status code, request/response size, errors
4. If the downstream service is instrumented, its segment replaces the inferred segment

Amazon Bedrock AgentCore + X-Ray Integration:

- AgentCore supports trace context propagation via the `X-Amzn-Trace-Id` header
- Custom instrumentation via the AWS Distro for OpenTelemetry (ADOT) SDK
- Requires enabling CloudWatch Transaction Search (one-time account setup)
- Track requests across agent -> tool -> knowledge base -> FM boundaries

Bedrock-Specific Observability (Complementary to X-Ray):

Source	What it Captures
Bedrock Agent Trace	Orchestration steps, tool calls, RAG retrieval
Model Invocation Logging	Full prompt/response, token counts, model ID
CloudWatch Metrics	Invocation count, latency, error rate
X-Ray	End-to-end latency, service dependency map

1.3 Service Maps for AI Applications

A typical GenAI service map shows:

```
Client -&gt; API Gateway -&gt; Lambda (preprocessing)
                           -&gt; Bedrock Runtime (inference)
                           -&gt; DynamoDB (session store)
                           -&gt; S3 (knowledge base source)
```

X-Ray generates **inferred segments** for services that do not send their own segments (e.g., DynamoDB, S3). These appear as nodes on the service map based on the subsegment data from the calling service.

Exam Tip: The service map shows health (error/fault rates) and latency at each node. Use this to identify which component in your AI pipeline is the bottleneck (e.g., Bedrock inference latency vs. vector database retrieval time).

1.4 Segments and Subsegments in Practice

Lambda + X-Ray:

- When active tracing is enabled on a Lambda function, Lambda creates the root segment

automatically

- Your code can only create subsegments within Lambda (not new root segments)
- Lambda adds two nodes to the trace map: one for the Lambda service, one for the function

```
// Node.js X-Ray SDK example in Lambda
```

```
import AWSXRay from 'aws-xray-sdk';
import { BedrockRuntimeClient, ConverseCommand } from '@aws-sdk/client-bedrock-runtime';

// Wrap the AWS SDK client for automatic subsegment creation
const client = AWSXRay.captureAWSv3Client(
  new BedrockRuntimeClient({ region: 'us-east-1' })
);

export const handler = async (event) =&gt; {
  // Bedrock call is automatically captured as a subsegment
  const response = await client.send(new ConverseCommand({
    modelId: 'anthropic.claude-3-haiku-20240307-v1:0',
    messages: [{ role: 'user', content: [{ text: event.prompt }] }],
  }));

  // Add custom annotation (indexed, searchable)
  const segment = AWSXRay.getSegment();
  const subsegment = segment.addNewSubsegment('ProcessResponse');
  subsegment.addAnnotation('modelId', 'claude-3-haiku');
  subsegment.addAnnotation('tokenCount', response.usage?.totalTokens ?? 0);

  // Add metadata (not indexed, stores complex objects)
  subsegment.addMetadata('prompt', event.prompt, 'ai');
  subsegment.addMetadata('response', response.output, 'ai');
  subsegment.close();

  return response.output;
};
```

1.5 Integration with Lambda and API Gateway

Lambda:

- Enable active tracing in function configuration (`TracingConfig: { Mode: Active }`)
- Lambda runs the X-Ray daemon on Java and Node.js runtimes
- Creates facade segment automatically; your code adds subsegments

API Gateway:

- Supports active tracing (enable per stage)
- Adds a node for the gateway stage to the service map
- Propagates trace header downstream to Lambda/other integrations

Exam Gotcha: API Gateway uses X-Ray sampling rules to decide which requests to trace. Lambda uses passive instrumentation when called by API Gateway (respects the upstream sampling decision).

1.6 Annotations and Metadata for AI Traces

Feature	Annotations	Metadata
Indexed	Yes	No
Searchable via filter expressions	Yes	No
Value types	String, Number, Boolean	Any (objects, arrays, etc.)
Limit	50 per trace	No hard limit (within 64 KB segment size)
Key limit	500 alphanumeric characters	No restriction

Feature	Annotations	Metadata
Value limit	1,000 Unicode characters	Serializable to JSON

GenAI-Specific Annotation Strategy:

```
annotation["modelId"] = "anthropic.claude-3-haiku"
annotation["useCase"] = "summarization"
annotation["guardrailApplied"] = true
annotation["tokenBudgetExceeded"] = false
```

GenAI-Specific Metadata Strategy:

```
{
  "namespace": "ai",
  "systemPrompt": "You are a helpful assistant...",
  "userPrompt": "Summarize this document...",
  "modelResponse": "The document covers...",
  "inferenceConfig": { "temperature": 0.7, "maxTokens": 1024 }
}
```

Filter Expression Examples:

```
annotation.modelId = "claude-3-haiku"
annotation.tokenCount &gt; 1000
annotation.useCase = "rag-query" AND responsetime &gt; 5
```

1.7 Prompt Observability Pipelines (Task 5.2.5)

A prompt observability pipeline captures, stores, and analyzes all prompts and responses flowing through your GenAI application.

Architecture:

```
User Request
  -&gt; API Gateway (X-Ray trace starts)
    -&gt; Lambda (preprocessing, prompt assembly)
      -&gt; Bedrock (inference, model invocation logging enabled)
        -&gt; Model invocation logs -&gt; S3 -&gt; Athena (analytics)
        -&gt; CloudWatch Logs (structured logging with embedded metrics)
        -&gt; X-Ray subsegments (latency, error tracking)
    -&gt; Post-processing Lambda
      -&gt; CloudWatch custom metrics (token usage, hallucination rate)
      -&gt; OpenSearch (semantic search over responses)
```

Key Metrics to Monitor:

Metric	Source	Why
Inference latency per model	X-Ray subsegments	Detect slowdowns
Token usage (input + output)	Bedrock invocation logs	Cost control, detect prompt bloat
Error rate / fault rate	X-Ray service map	Pipeline stability
Hallucination rate	Custom metric (post-processing)	Output quality
Guardrail intervention rate	Bedrock guardrail logs	Content safety
Tool selection accuracy	Bedrock agent trace	Agent behavior alignment

Metric	Source	Why
Cost per session	CloudWatch embedded metrics	FinOps

Exam Tip: The exam expects you to know that X-Ray handles cross-service latency and dependency mapping, while Bedrock model invocation logging captures prompt/response content and token usage. These are complementary, not competing, observability mechanisms.

Migration Note: The X-Ray SDK enters maintenance mode on February 25, 2026. AWS recommends migrating to OpenTelemetry via ADOT (AWS Distro for OpenTelemetry). The exam may reference both approaches.

---

2. AWS Amplify

2.1 Overview and Frontend Framework Integration

AWS Amplify is a full-stack development platform for building web and mobile applications. Amplify Gen 2 uses a TypeScript-first, code-based approach for defining backends.

Supported Frameworks:

- React, Next.js, Vue, Angular, React Native, Flutter, Android, Swift

Core Components:

- `@aws-amplify/backend` - Backend definition (data, auth, storage, functions)
- `@aws-amplify/ui-react` - Pre-built UI components
- `amplify sandbox` - Local cloud development environment
- `amplify deploy` - CI/CD deployment

2.2 Amplify AI Kit -- AI-Powered UI Components (Task 2.5.2)

The Amplify AI Kit is the primary exam topic for Amplify. It provides declarative UI components and backend infrastructure for building AI-powered applications with Amazon Bedrock.

Two Route Types:

Route Type	Pattern	API Style	Storage	Use Case
Conversation	`a.conversation()`	Async, multi-turn, streaming	Messages stored in DynamoDB	Chat UIs, conversational search
Generation	`a.generation()`	Sync, single request-response	No automatic storage	Summarization, structured data extraction, alt-text

Cloud Infrastructure Created by AI Kit:

```
AppSync (GraphQL API)
  -&gt; Lambda (conversation handler)
    -&gt; Bedrock (Converse API)
  -&gt; DynamoDB (conversation history)
```

- **AWS AppSync**: Serverless GraphQL API layer; handles authorization and real-time

subscriptions for streaming responses

- **AWS Lambda**: Executes conversation logic; retrieves history and invokes Bedrock
- **Amazon DynamoDB**: Stores conversation and message data; conversations scoped per user
- **Amazon Bedrock**: Foundation model inference via the Converse API

Defining AI Routes in Amplify:

```
// amplify/data/resource.ts
import { a, defineData, type ClientSchema } from '@aws-amplify/backend';

const schema = a.schema({
  // Multi-turn conversation route
  chat: a.conversation({
    aiModel: a.ai.model('Claude 3.5 Haiku'),
    systemPrompt: 'You are a helpful assistant',
  })
  .authorization((allow) =&gt; allow.owner()),

  // Single-shot generation route
  generateRecipe: a.generation({
    aiModel: a.ai.model('Claude 3.5 Haiku'),
    systemPrompt: 'You are a helpful assistant that generates recipes.',
  })
  .arguments({
    description: a.string(),
  })
  .returns(
    a.customType({
      name: a.string(),
      ingredients: a.string().array(),
      instructions: a.string(),
    })
  )
  .authorization((allow) =&gt; allow.authenticated()),
});
```

Amplify AI Kit uses Bedrock's Converse API -- this is the unified API that works across models. The kit handles:

- Conversation history management
- Tool configuration and invocation
- Real-time subscriptions for streaming
- System prompt injection

Pre-built `<AIConversation>` **Component:**

- Themed and extensible React component
- Handles real-time subscriptions, chat history, and UI state
- Manages multi-turn invocations automatically

Supported Models (via Bedrock Converse API):

Provider	Models	Conversation	Generation
Anthropic	Claude 3 Haiku, 3.5 Haiku	Yes	Yes
Amazon	Nova Pro, Nova Lite, Nova Micro	Yes	No
AI21 Labs	Jurassic-2 Large, Mini	Yes	No
Cohere	Command R, Command R+	Yes	No
Meta	Llama 3 models	Yes	No

Prompt Structure in Amplify AI Kit:

1. **System prompt** -- High-level instructions (required for all routes)
2. **Messages** -- Conversation history (managed automatically)

3. **Tool configuration** -- Tools the model can invoke (managed automatically)

2.3 Backend Integration with Bedrock

Beyond the AI Kit, you can connect Amplify backends directly to Bedrock as an HTTP data source using custom resolvers:

```
// amplify/data/resource.ts -- Custom Bedrock data source
const bedrockDataSource = defineData({
  schema,
  authorizationModes: { defaultAuthorizationMode: 'userPool' },
});

// amplify/data/bedrock.js -- Custom resolver for Bedrock
export function request(ctx) {
  return {
    method: 'POST',
    resourcePath: '/model/anthropic.claude-3-haiku-20240307-v1:0/invoke',
    params: {
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        anthropic_version: 'bedrock-2023-05-31',
        messages: [{ role: 'user', content: ctx.args.prompt }],
        max_tokens: 1024,
      }),
    },
  };
}

export function response(ctx) {
  return JSON.parse(ctx.result.body).content[0].text;
}
```

2.4 Authentication for AI Applications

Amplify uses Amazon Cognito for authentication. For AI routes:

- **Conversation routes** use `.authorization((allow) => allow.owner())` -- conversations

are scoped to the authenticated user

- **Generation routes** typically use `.authorization((allow) => allow.authenticated())`
- The user's OIDC access token is passed from the client to AppSync, which authorizes the

request before forwarding it to Lambda/Bedrock

Exam Gotcha: Amplify AI Kit conversation routes store messages in DynamoDB with owner-based authorization. Each user can only see their own conversation history. This is built into the architecture, not something you configure manually.

Exam Gotcha: Amplify AI Kit uses the Bedrock **Converse** API (not InvokeModel). The Converse API provides a unified interface across models, which is why Amplify can support model swapping without code changes.

---

3. CI/CD for GenAI (CodePipeline, CodeBuild, CodeDeploy)

3.1 CI/CD Pipeline Architecture for AI Applications (Task 2.3.5)

GenAI CI/CD differs from traditional software CI/CD because you must version and test **prompts, agent configurations, and model selections** alongside code and infrastructure.

Typical CI/CD Workflow for Serverless AI:

Stage	Actions	Tooling
1. Code + Prompt Commit	Push Lambda code, CDK stacks, prompt files to Git	GitHub, GitLab, CodeCommit
2. Build + Lint	Validate syntax, prompt format, schema alignment	CodeBuild, ESLint, Black, yamllint
3. Unit Tests + Prompt Regression	Run logic tests and golden prompt-response tests	CodeBuild, pytest, promptfoo
4. IaC Validation	Synthesize and validate CDK/CloudFormation templates	`cdk synth`, `cfn-lint`
5. Integration Test	Deploy to staging, invoke full AI workflow	CodeBuild, mocked agents
6. Manual/Auto Approval	Review model cost impact, prompt change checklist	CodePipeline approval gates
7. Deploy to Production	Promote stacks, update Bedrock agent configs	CodeDeploy, CDK, SAM CLI
8. Post-Deploy Smoke Test	Validate agent outputs, log capture, rollback readiness	CloudWatch Synthetics
9. Monitor + Observe	Dashboards, cost alerts, token usage monitors	CloudWatch, X-Ray, Bedrock logs

3.2 AWS CodePipeline

CodePipeline orchestrates the entire release process as a series of stages and actions.

Key Features for GenAI:

- **Pipeline stages**: Source -> Build -> Test -> Approval -> Deploy
- **Source integrations**: GitHub, CodeCommit, S3, ECR
- **Parallel actions**: Run multiple CodeBuild projects in parallel within a stage
- **Manual approval gates**: Require human review before deploying prompt changes to production
- **Declarative templates**: Define pipelines as CloudFormation/CDK

GenAI-Specific Pipeline Pattern:

```
Source (Git)
  -&gt; Build Stage
     -&gt; CodeBuild: Lint + unit tests
     -&gt; CodeBuild: Prompt regression tests (parallel)
     -&gt; CodeBuild: cdk synth + cfn-lint (parallel)
  -&gt; Staging Deploy Stage
     -&gt; CloudFormation deploy to staging
     -&gt; CodeBuild: Integration tests against staging
  -&gt; Approval Gate
     -&gt; Manual review of prompt changes + cost impact
  -&gt; Production Deploy Stage
     -&gt; CodeDeploy: Blue/green or canary deployment
  -&gt; Post-Deploy Stage
     -&gt; CloudWatch Synthetics canary tests
```

3.3 AWS CodeBuild

CodeBuild is a fully managed build service that compiles code, runs tests, and produces deployment artifacts.

Key Features for GenAI:

- **Custom build environments**: Docker images with ML/AI dependencies
- **Parallel test execution**: Shard test suites across multiple build nodes using

`codebuild-tests-run` utility

- **Test report groups**: Consolidate results from parallel test runs
- **Buildspec.yml**: Define build commands, phases, and artifacts

Example Buildspec for GenAI Testing:

```
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 20
    commands:
      - npm ci
  pre_build:
    commands:
      - npm run lint
      - npx cdk synth
      - npx cfn-lint cdk.out/*.template.json
  build:
    commands:
      - npm test                        # Unit tests
      - npm run test:prompts            # Prompt regression tests
      - npm run test:security           # Security scanning
  post_build:
    commands:
      - npm run test:integration        # Integration tests
reports:
  prompt-tests:
    files:
      - 'test-results/prompt-regression.xml'
    file-format: JUNITXML
  security-scan:
    files:
      - 'test-results/security-scan.xml'
    file-format: JUNITXML
artifacts:
  files:
    - 'cdk.out/**/*'
    - 'prompts/**/*'
```

3.4 Automated Testing for GenAI

Prompt Regression Testing:

- Treat prompts as versioned assets in source control (e.g., `/prompts/v1/agent-support-en.yaml`)
- Maintain "golden" test cases: known input -> expected output pairs
- Use tools like `promptfoo` for automated prompt evaluation
- Compare model outputs against expected patterns/scores

Types of Tests in GenAI CI/CD:

Test Type	What It Validates	When
Unit tests	Business logic, data transformations	Build stage
Prompt regression	System prompt + user input -> expected output pattern	Build stage
Schema validation	CDK/CloudFormation template correctness	Build stage
Security scan	Dependency vulnerabilities, IAM over-permissions	Build stage
Integration tests	Full workflow: API -> Lambda -> Bedrock -> response	Staging deploy
Smoke tests	Production endpoint health, basic inference	Post-deploy

Deploy Bedrock Agent Updates Only When:

1. Prompt regression tests pass
2. Tool permissions match IAM templates
3. Confidence thresholds or validation Lambda results meet acceptable criteria

3.5 Security Scanning in AI Pipelines

- **Dependency scanning**: Check AI/ML library versions for known vulnerabilities
- **IAM policy analysis**: Validate least-privilege permissions for Bedrock access
- **Prompt injection testing**: Test for prompt injection vulnerabilities
- **Data leakage checks**: Ensure sensitive data is not embedded in prompts or logs
- **cfn-lint + cfn-guard**: Validate CloudFormation templates against security policies

3.6 AWS CodeDeploy -- Rollback and Deployment Strategies

CodeDeploy automates deployments to EC2, Lambda, and ECS. For GenAI applications, Lambda and ECS deployments are most relevant.

Lambda Deployment Strategies:

Strategy	Behavior	Use Case
`AllAtOnce`	100% traffic shifted immediately	Non-critical, dev environments
`Canary10Percent5Minutes`	10% for 5 min, then 100%	Quick validation
`Canary10Percent30Minutes`	10% for 30 min, then 100%	Production AI endpoints
`Linear10PercentEvery1Minute`	+10% per minute over 10 min	Gradual rollout
`Linear10PercentEvery10Minutes`	+10% every 10 min over ~100 min	Conservative rollout

How Lambda Canary/Linear Works:

1. New Lambda version is published
2. CodeDeploy creates a weighted alias pointing to old + new versions
3. Traffic shifts gradually per the deployment configuration
4. Pre-traffic hook Lambda runs validation before any traffic shifts
5. Post-traffic hook Lambda validates after shift completes
6. CloudWatch alarms trigger automatic rollback if errors spike

ECS Blue/Green Deployments:

- Two task sets behind a load balancer (blue = current, green = new)
- Traffic shifts via target group weights
- Supports canary and linear strategies (same patterns as Lambda)
- Configurable bake period before retiring the blue environment
- Lifecycle hooks can run Lambda functions for synthetic testing

SAM Template for Gradual Lambda Deployment:

```
MyFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: index.handler
    Runtime: nodejs20.x
    AutoPublishAlias: live
    DeploymentPreference:
      Type: Canary10Percent5Minutes
      Alarms:
        - !Ref MyAlarm
      Hooks:
        PreTraffic: !Ref PreTrafficHook
        PostTraffic: !Ref PostTrafficHook
```

Exam Gotcha: For Lambda, `AutoPublishAlias` must be set before you can use `DeploymentPreference`. The first deployment must publish the alias, and the second deployment onward can use gradual traffic shifting.

Exam Gotcha: CodeDeploy for Lambda uses **weighted aliases** to split traffic. This is different from API Gateway canary deployments which use stage canary settings.

---

4. AWS CDK and CloudFormation

4.1 Infrastructure as Code for AI Services

AWS CDK (Cloud Development Kit) lets you define cloud infrastructure in TypeScript, Python, Java, C#, or Go. CDK synthesizes to CloudFormation templates.

CDK Key Concepts:

Concept	Description
App	Root of the construct tree; contains one or more stacks
Stack	Unit of deployment; maps 1:1 to a CloudFormation stack
Construct	Building block; L1 (raw CFN), L2 (opinionated defaults), L3 (patterns)
`cdk synth`	Synthesizes CDK app to CloudFormation template
`cdk deploy`	Deploys the synthesized template
`cdk diff`	Shows what changes would be applied

CDK for GenAI Workflow:

1. Write CDK TypeScript code defining Bedrock agents, Lambda functions, IAM roles
2. `cdk synth` -> generates CloudFormation template
3. `cfn-lint` validates the template

4. `cdk deploy` creates/updates the stack
5. CI/CD pipeline automates steps 2-4

4.2 Bedrock Resources in CloudFormation / CDK

CloudFormation provides L1 (low-level) resource types for all Bedrock resources. CDK wraps these as L1 constructs (prefixed with `Cfn`).

Key CloudFormation Resource Types:

Resource Type	Purpose
`AWS::Bedrock::Agent`	Define a Bedrock agent with model, instructions, action groups, knowledge bases
`AWS::Bedrock::AgentAlias`	Create an alias for agent versioning and deployment
`AWS::Bedrock::KnowledgeBase`	Create a knowledge base for RAG (vector store config, embedding model)
`AWS::Bedrock::DataSource`	Connect S3/other data to a knowledge base for ingestion
`AWS::Bedrock::Guardrail`	Define content filters, denied topics, sensitive info filters, word filters
`AWS::Bedrock::GuardrailVersion`	Version a guardrail configuration
`AWS::Bedrock::Flow`	Define a Bedrock Flow (visual prompt chaining)
`AWS::Bedrock::FlowVersion`	Version a flow
`AWS::Bedrock::FlowAlias`	Alias for a flow version
`AWS::Bedrock::Prompt`	Define a reusable prompt template
`AWS::Bedrock::PromptVersion`	Version a prompt template

CDK Example -- Bedrock Agent with Guardrail:

```
import * as cdk from 'aws-cdk-lib';
import * as bedrock from 'aws-cdk-lib/aws-bedrock';
import * as iam from 'aws-cdk-lib/aws-iam';

const agentRole = new iam.Role(this, 'AgentRole', {
  assumedBy: new iam.ServicePrincipal('bedrock.amazonaws.com'),
  managedPolicies: [
    iam.ManagedPolicy.fromAwsManagedPolicyName('AmazonBedrockFullAccess'),
  ],
});

const guardrail = new bedrock.CfnGuardrail(this, 'ContentGuardrail', {
  name: 'my-ai-guardrail',
  blockedInputMessaging: 'Your input was blocked by content filters.',
  blockedOutputsMessaging: 'The response was blocked by content filters.',
  contentPolicyConfig: {
    filtersConfig: [
      { type: 'SEXUAL', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'VIOLENCE', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'HATE', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'INSULTS', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'MISCONDUCT', inputStrength: 'HIGH', outputStrength: 'HIGH' },
      { type: 'PROMPT_ATTACK', inputStrength: 'HIGH', outputStrength: 'NONE' },
    ],
  },
});
const agent = new bedrock.CfnAgent(this, 'SupportAgent', {
```

```
    agentName: 'customer-support-agent',
    agentResourceRoleArn: agentRole.roleArn,
    foundationModel: 'anthropic.claude-3-haiku-20240307-v1:0',
    instruction: 'You are a customer support agent. Help users with their orders.',
    idleSessionTtlInSeconds: 600,
    guardrailConfiguration: {
      guardrailIdentifier: guardrail.attrGuardrailId,
      guardrailVersion: 'DRAFT',
    },
  });
```

4.3 Custom Resources for AI Provisioning

Some AI provisioning tasks are not natively supported by CloudFormation (e.g., triggering a knowledge base sync, running a custom model evaluation). Use CloudFormation Custom Resources backed by Lambda to fill these gaps.

```
import * as cr from 'aws-cdk-lib/custom-resources';

// Custom resource to trigger knowledge base sync after data source creation
const kbSync = new cr.AwsCustomResource(this, 'KBSync', {
  onCreate: {
    service: 'BedrockAgent',
    action: 'startIngestionJob',
    parameters: {
      knowledgeBaseId: knowledgeBase.attrKnowledgeBaseId,
      dataSourceId: dataSource.attrDataSourceId,
    },
    physicalResourceId: cr.PhysicalResourceId.of('kb-sync-trigger'),
  },
  policy: cr.AwsCustomResourcePolicy.fromSdkCalls({
    resources: cr.AwsCustomResourcePolicy.ANY_RESOURCE,
  }),
});
```

Exam Tip: `cdk synth` + `cfn-lint` is the standard validation step in CI/CD for CDK-based GenAI projects. The exam may ask about validating IaC before deployment -- the answer is always synthesize first, then validate the generated template.

---

5. AWS CodeArtifact

5.1 Package Management for AI Dependencies

CodeArtifact is a fully managed artifact repository service that stores and shares software packages.

Key Concepts:

Concept	Description
Domain	Top-level organizational unit; all repositories in a domain share a common policy
Repository	Stores packages; can have upstream repositories and external connections
External Connection	Link to a public repository (npmjs.com, PyPI, Maven Central, etc.)
Upstream Repository	Another CodeArtifact repository that this one can pull from

Supported Package Formats: npm, PyPI, Maven, NuGet, Ruby (gems), Swift, Cargo, Generic

Architecture Pattern for AI Teams:

```
External Connections           CodeArtifact Domain
  npmjs.com  ■■■■■&gt; npm-store (repo)
  pypi.org   ■■■■■&gt; pypi-store (repo)
                      |
                  shared-ai-packages (upstream repo)
                      |
              ■■■■■■■■■■■■■■■■■■
          team-a-repo  team-b-repo  team-c-repo
```

Why CodeArtifact Matters for GenAI:

- **Dependency control**: Pin and audit AI/ML library versions (LangChain, boto3, transformers)
- **Security**: Scan packages before they enter your repositories
- **Availability**: Cached packages remain available even if public registries go down
- **Cross-team sharing**: Publish internal AI utility packages (prompt templates, custom evaluators) to a shared repository
- **One external connection per repo**: Each repository can connect to only one external public repository; use upstream repos to aggregate

Authentication:

```
# Authenticate npm with CodeArtifact
aws codeartifact login --tool npm \
  --repository my-ai-repo \
  --domain my-domain \
  --domain-owner 123456789012

# Authenticate pip with CodeArtifact
aws codeartifact login --tool pip \
  --repository my-ai-repo \
  --domain my-domain \
  --domain-owner 123456789012
```

Exam Tip: CodeArtifact is the answer when the exam asks about managing private AI/ML package dependencies in an enterprise environment, especially when teams need to share internal packages while controlling access to public registries.

---

6. AWS CLI for Bedrock

6.1 Key Bedrock CLI Commands

The AWS CLI provides two Bedrock service prefixes:

- `aws bedrock` -- Control plane (manage models, agents, knowledge bases)
- `aws bedrock-runtime` -- Data plane (invoke models, run inference)

Essential Commands:

```
# List available foundation models
aws bedrock list-foundation-models \
  --region us-east-1

# List foundation models by provider
```

```
aws bedrock list-foundation-models \
  --by-provider anthropic

# Get details about a specific model
aws bedrock get-foundation-model \
  --model-identifier anthropic.claude-3-haiku-20240307-v1:0

# Invoke a model (InvokeModel API)
aws bedrock-runtime invoke-model \
  --model-id amazon.titan-text-lite-v1 \
  --content-type application/json \
  --body '{"inputText": "What is generative AI?"}' \
  output.json

# Use the Converse API (recommended for multi-turn)
aws bedrock-runtime converse \
  --model-id anthropic.claude-3-haiku-20240307-v1:0 \
  --messages '[{"role":"user","content":[{"text":"What is GenAI?"}]}]' \
  --inference-config '{"maxTokens":512,"temperature":0.7}'

# List agents
aws bedrock-agent list-agents

# List knowledge bases
aws bedrock-agent list-knowledge-bases

# Start a knowledge base ingestion job
aws bedrock-agent start-ingestion-job \
  --knowledge-base-id KB123456 \
  --data-source-id DS789012
```

6.2 Scripting AI Workflows

Batch Model Evaluation Script:

```
#!/bin/bash
# Test multiple models with the same prompt
MODELS=("anthropic.claude-3-haiku-20240307-v1:0" "amazon.titan-text-lite-v1")
PROMPT="Summarize the benefits of serverless architecture in 3 bullets."

for MODEL in "${MODELS[@]}"; do
  echo "Testing model: $MODEL"
  aws bedrock-runtime converse \
    --model-id "$MODEL" \
    --messages "[{\"role\":\"user\",\"content\":[{\"text\":\"$PROMPT\"}]}]" \
    --inference-config '{"maxTokens":256}' \
    --query 'output.message.content[0].text' \
    --output text
  echo "---"
done
```

Exam Tip: The exam may ask which CLI command to use for real-time inference. The answer is `aws bedrock-runtime invoke-model` or `aws bedrock-runtime converse`. The `converse` command is recommended when the model supports it because it provides a unified interface across model providers.

---

7. AWS Tools and SDKs

7.1 AWS SDK v3 Architecture (Modular Clients)

AWS SDK for JavaScript v3 is the standard for TypeScript/Node.js GenAI development.

Key Architectural Differences from v2:

Feature	SDK v2	SDK v3
Package structure	Monolithic `aws-sdk`	Modular per-service `@aws-sdk/client-*`
Tree-shaking	Not possible	Fully tree-shakeable
Middleware	Not customizable	Pluggable middleware stack
TypeScript	Community types (`@types/aws-sdk`)	First-class TypeScript support
Commands	Method calls (`s3.putObject()`)	Command objects (`client.send(new PutObjectCommand())`)
Bundle size	~80MB (all services)	~few KB per service

SDK v3 Command Pattern:

```
import { BedrockRuntimeClient, ConverseCommand } from '@aws-sdk/client-bedrock-runtime';

const client = new BedrockRuntimeClient({ region: 'us-east-1' });
const response = await client.send(new ConverseCommand({
  modelId: 'anthropic.claude-3-haiku-20240307-v1:0',
  messages: [{ role: 'user', content: [{ text: 'Hello' }] }],
}));
```

7.2 Key SDK Clients for GenAI

Package	Purpose	Key Commands
`@aws-sdk/client-bedrock`	Control plane: manage models, agents, guardrails	`ListFoundationModelsCommand`, `GetFoundationModelCommand`
`@aws-sdk/client-bedrock-runtime`	Data plane: run inference	`InvokeModelCommand`, `ConverseCommand`, `ConverseStreamCommand`, `InvokeModelWithResponseStreamCommand`
`@aws-sdk/client-bedrock-agent`	Manage agents, knowledge bases, flows	`CreateAgentCommand`, `CreateKnowledgeBaseCommand`, `StartIngestionJobCommand`
`@aws-sdk/client-bedrock-agent-runtime`	Invoke agents, query knowledge bases	`InvokeAgentCommand`, `RetrieveCommand`, `RetrieveAndGenerateCommand`
`@aws-sdk/client-s3`	Store/retrieve documents for RAG	`PutObjectCommand`, `GetObjectCommand`
`@aws-sdk/client-lambda`	Invoke Lambda functions	`InvokeCommand`
`@aws-sdk/client-dynamodb`	Session/conversation storage	`PutItemCommand`, `QueryCommand`

Streaming Inference Example:

```
import {
  BedrockRuntimeClient,
  ConverseStreamCommand,
} from '@aws-sdk/client-bedrock-runtime';

const client = new BedrockRuntimeClient({ region: 'us-east-1' });
```

```
const response = await client.send(new ConverseStreamCommand({
  modelId: 'anthropic.claude-3-haiku-20240307-v1:0',
  messages: [{ role: 'user', content: [{ text: 'Write a haiku about cloud computing' }] }],
  inferenceConfig: { maxTokens: 256, temperature: 0.7 },
}));

// Process the stream
if (response.stream) {
  for await (const event of response.stream) {
    if (event.contentBlockDelta?.delta?.text) {
      process.stdout.write(event.contentBlockDelta.delta.text);
    }
    if (event.messageStop) {
      console.log('\n--- Stream complete ---');
    }
    if (event.metadata) {
      console.log('Usage:', event.metadata.usage);
    }
  }
}
```

7.3 Retry Strategies and Exponential Backoff (Task 2.4.3)

AWS SDKs implement automatic retry with exponential backoff and jitter for transient errors (throttling, service unavailable, timeouts).

How Exponential Backoff Works:

```
Attempt 1: wait 0ms (immediate)
Attempt 2: wait base * 2^1 + jitter  (e.g., ~200ms)
Attempt 3: wait base * 2^2 + jitter  (e.g., ~400ms)
Attempt 4: wait base * 2^3 + jitter  (e.g., ~800ms)
... up to maxAttempts or maxDelay
```

Jitter adds randomness to prevent thundering herd: all clients retrying at the same time after a shared failure.

SDK v3 Retry Configuration:

```
import { BedrockRuntimeClient } from '@aws-sdk/client-bedrock-runtime';

const client = new BedrockRuntimeClient({
  region: 'us-east-1',
  // Standard retry mode (default)
  maxAttempts: 3,
});

// Or use adaptive retry mode for high-throughput
const adaptiveClient = new BedrockRuntimeClient({
  region: 'us-east-1',
  retryMode: 'adaptive',  // Adjusts retry rate based on throttling signals
  maxAttempts: 5,
});
```

Three Retry Modes in SDK v3:

Mode	Behavior	When to Use
`standard`	Exponential backoff with jitter, max 3 attempts	Default; most use cases

Mode	Behavior	When to Use
`adaptive`	Token bucket rate limiting + exponential backoff	High-throughput APIs with throttling
`legacy`	Basic retry without jitter	Backward compatibility only

Retryable Errors:

- `ThrottlingException` / HTTP 429 -- Always retry with backoff
- `ServiceUnavailableException` / HTTP 503 -- Retry with backoff
- `InternalServerError` / HTTP 500 -- Retry with backoff
- Network timeouts -- Retry with backoff
- `ModelNotReadyException` -- Retry (model is warming up)

Non-Retryable Errors:

- `ValidationException` / HTTP 400 -- Fix the request
- `AccessDeniedException` / HTTP 403 -- Fix IAM permissions
- `ResourceNotFoundException` / HTTP 404 -- Fix the resource ID

Exam Gotcha: The exam will test your knowledge of when to use exponential backoff vs. when to fix the request. Retryable = transient server errors and throttling. Non-retryable = client errors (4xx except 429).

Exam Gotcha: Adaptive retry mode is appropriate when your application makes many concurrent Bedrock API calls and needs to automatically adjust its request rate to avoid throttling. Standard mode is sufficient for most single-request patterns.

7.4 Middleware Stack

The SDK v3 middleware stack lets you intercept and modify requests/responses at various stages of the request lifecycle.

Middleware Stages (in order):

```
initialize -&gt; serialize -&gt; build -&gt; finalizeRequest -&gt; deserialize
```

Stage	Purpose	Example Use
`initialize`	Set defaults, validate input	Add default inference config
`serialize`	Convert input to HTTP request	Custom serialization
`build`	Add headers, sign request	Add custom tracing headers
`finalizeRequest`	Last chance before sending	Logging, metrics emission
`deserialize`	Parse HTTP response	Custom response processing

Custom Middleware Example -- Logging All Bedrock Calls:

```
import { BedrockRuntimeClient, ConverseCommand } from '@aws-sdk/client-bedrock-runtime';

const client = new BedrockRuntimeClient({ region: 'us-east-1' });

// Add middleware to log all requests and responses
client.middlewareStack.add(
  (next, context) =&gt; async (args) =&gt; {
    const startTime = Date.now();
    console.log('Request:', {
      commandName: context.commandName,
      input: args.input,
```

```
    });

    const result = await next(args);

    console.log('Response:', {
      commandName: context.commandName,
      duration: Date.now() - startTime,
      metadata: result.output.$metadata,
    });

    return result;
  },
  {
    step: 'initialize',
    name: 'bedrockLogger',
    tags: ['LOGGING'],
  }
);
```

Middleware for Token Usage Tracking:

```
client.middlewareStack.add(
  (next, context) =&gt; async (args) =&gt; {
    const result = await next(args);

    if (context.commandName === 'ConverseCommand') {
      const usage = result.output.usage;
      if (usage) {
        // Emit CloudWatch custom metric
        console.log(JSON.stringify({
          _aws: { Timestamp: Date.now(), CloudWatchMetrics: [{
            Namespace: 'GenAI/TokenUsage',
            Dimensions: [['ModelId']],
            Metrics: [
              { Name: 'InputTokens', Unit: 'Count' },
              { Name: 'OutputTokens', Unit: 'Count' },
            ],
          }]},
          ModelId: args.input.modelId,
          InputTokens: usage.inputTokens,
          OutputTokens: usage.outputTokens,
        }));
      }
    }

    return result;
  },
  { step: 'deserialize', name: 'tokenTracker' }
);
```

Exam Tip: The middleware stack is how you add cross-cutting concerns (logging, metrics, tracing, retry customization) without modifying business logic. The exam may describe a scenario where you need to add observability to all Bedrock calls -- middleware is the answer.

Quick Reference: When to Use Each Service

Scenario	Service
Trace requests across Lambda -> Bedrock -> DynamoDB	**X-Ray**
Build a chat UI with Bedrock in React/Next.js	**Amplify AI Kit**
Automate prompt testing + deployment pipeline	**CodePipeline + CodeBuild**
Gradually shift traffic to new model version	**CodeDeploy** (canary/linear)
Define Bedrock agents, guardrails, KBs as IaC	**CDK / CloudFormation**
Manage AI/ML package dependencies across teams	**CodeArtifact**
Script batch model evaluations	**AWS CLI** (`bedrock-runtime converse`)
Add retry logic to high-throughput Bedrock calls	**SDK v3** (adaptive retry mode)
Add logging/metrics to all SDK calls	**SDK v3 middleware stack**
Monitor token usage and inference cost	**CloudWatch + Bedrock model invocation logging**
Find performance bottleneck in AI pipeline	**X-Ray service map**
Search traces by model ID or use case	**X-Ray annotations + filter expressions**

Exam Gotchas Summary

1. **X-Ray annotations vs. metadata**: Annotations are indexed and searchable (max 50 per trace). Metadata is not indexed but can store any JSON object. For AI traces, put `modelId` and `useCase` in annotations; put prompts and responses in metadata.
2. **X-Ray in Lambda**: Lambda creates the root segment. Your code creates subsegments only. You cannot create new root segments in Lambda.
3. **X-Ray SDK -> OpenTelemetry migration**: X-Ray SDK enters maintenance mode February 2026. ADOT (AWS Distro for OpenTelemetry) is the recommended replacement. Bedrock AgentCore uses ADOT natively.
4. **Amplify AI Kit route types**: `a.conversation()` = multi-turn, async, streaming, DynamoDB-backed. `a.generation()` = single-turn, sync, no storage. Both use Bedrock's Converse API.
5. **Amplify AI Kit authorization**: Conversation routes use owner-based auth (each user sees only their conversations). This is architectural, not a configuration choice.
6. **CI/CD for prompts**: Prompts are versioned assets in source control. They go through regression testing (golden test cases) before deployment. CodePipeline approval gates are used for human review of prompt changes.
7. **CodeDeploy canary for Lambda**: Requires `AutoPublishAlias`. First deploy creates the alias; second deploy onward uses gradual traffic shifting. Uses weighted aliases, not API Gateway canaries.
8. **CloudFormation Bedrock resources**: `AWS::Bedrock::Agent`, `AWS::Bedrock::KnowledgeBase`, `AWS::Bedrock::Guardrail`, `AWS::Bedrock::DataSource`, `AWS::Bedrock::Flow` are all available as native CFN resources.

9. **SDK v3 retry modes**: `standard` (default, 3 attempts), `adaptive` (token bucket for high throughput), `legacy` (backward compat). Exponential backoff with jitter prevents thundering herd.
10. **SDK v3 retryable vs. non-retryable**: Retry 429 (throttle), 500, 503. Do NOT retry 400 (validation), 403 (access denied), 404 (not found).
11. **CodeArtifact**: One external connection per repository. Use upstream repos to aggregate multiple public sources. This is the answer for enterprise AI/ML dependency management.
12. `cdk synth` **+** `cfn-lint`: Standard IaC validation step in CI/CD. Always validate synthesized templates before deployment.

Container Services for GenAI -- Study Guide

Exam: AWS Certified Generative AI Developer - Professional (AIP-C01)

This guide covers container services relevant to the AIP-C01 exam: Amazon ECS, Amazon EKS, AWS Fargate, and Amazon ECR. Containers appear in the exam primarily under:

- **Task 2.1.7**: ECS for hosting MCP servers that provide complex tools to AI agents
- **Task 2.2.2**: Container-based deployment for LLMs (memory requirements, GPU utilization, token processing)
- **Task 2.2.3**: Container optimization for Foundation Model deployment
- General knowledge: hosting custom models, AI microservices, and inference endpoints

---

1. Amazon ECS (Elastic Container Service)

Overview

Amazon ECS is a fully managed container orchestration service that runs Docker containers at scale. It eliminates the need to install and operate your own container orchestration software.

Core Concepts:

Concept	Description
Cluster	Logical grouping of tasks or services. Can span multiple AZs.
Task Definition	JSON blueprint for your application: container images, CPU/memory, ports, volumes, IAM roles, network mode. Versioned with revision numbers.
Task	A running instantiation of a task definition. Can be a standalone task or part of a service.
Service	Maintains a desired count of tasks, handles rolling deployments, integrates with load balancers, and supports auto scaling.
Capacity Provider	Determines the infrastructure (Fargate, Fargate Spot, EC2 Auto Scaling groups) that tasks run on.

Launch Types:

- **Fargate**: Serverless -- AWS manages the underlying compute. You specify CPU and memory at the task level.
- **EC2**: You manage EC2 instances registered to the cluster. Required for GPU workloads.
- **External (ECS Anywhere)**: Register on-premises or other servers to an ECS cluster.

Network Modes:

- `awsvpc` (required for Fargate): Each task gets its own ENI with a private IP. Best isolation.
- `bridge`: Default Docker bridge networking on EC2. Tasks share the host network stack via port mappings.
- `host`: Task uses the host's network directly. No port mapping overhead.
- `none`: No external network connectivity.

GPU Support for AI Inference

ECS supports GPU workloads through EC2 instances with NVIDIA GPUs. This is the primary way to run self-managed AI inference on containers in AWS.

GPU-Optimized AMI:

- Amazon provides an ECS GPU-optimized AMI (Amazon Linux 2023) with pre-installed NVIDIA kernel

drivers, NVIDIA Fabric Manager, NVIDIA Container Toolkit, and Docker GPU runtime.

- Supports GPU architectures: Ampere, Turing, Volta, Maxwell, Hopper, and Ada Lovelace.
- No additional configuration required -- works out of the box.

Supported GPU Instance Types:

Instance Family	GPU	Use Case
p3 (V100)	1-8 GPUs, 16-128 GiB GPU mem	Training and inference
p4d (A100)	8 GPUs, 320 GiB GPU mem	Large model training
p5 (H100)	8 GPUs, 640 GiB GPU mem	Frontier model training
g4dn (T4)	1-4 GPUs, 16-64 GiB GPU mem	Cost-effective inference
g5 (A10G)	1-8 GPUs, 24-192 GiB GPU mem	Inference and fine-tuning
g6/g6e (L40S)	1-8 GPUs, 24-384 GiB GPU mem	LLM inference, best price/performance

Specifying GPUs in a Task Definition:

```
{
  "containerDefinitions": [
    {
      "name": "inference-container",
      "image": "my-model-server:latest",
      "resourceRequirements": [
        {
          "type": "GPU",
          "value": "1"
        }
      ],
      "memory": 16384,
      "cpu": 4096
    }
  ],
  "requiresCompatibilities": ["EC2"],
  "family": "ai-inference-task"
}
```

Key Considerations for GPU Tasks:

- GPUs are **not supported on Fargate** -- you must use the EC2 launch type.
- GPUs are **not supported on Windows** containers.
- Set `ECS_ENABLE_GPU_SUPPORT` to `true` in the container agent configuration.
- Use placement constraints to target specific GPU instance types:

`attribute:ecs.instance-type == g6e.xlarge`

- The ECS container agent pins physical GPUs to containers. The total GPU count across all containers in a task cannot exceed the instance's available GPUs.
- Container images must include the necessary GPU libraries (CUDA, cuDNN). Use NVIDIA base images like `nvidia/cuda` or framework images like `pytorch/pytorch:latest-cuda`.
- NVIDIA GPU Time-slicing is supported on Bottlerocket, allowing multiple tasks to share a single GPU concurrently.

MCP Server Hosting on ECS (Task 2.1.7)

The exam calls out ECS specifically for hosting MCP (Model Context Protocol) servers that provide complex tools to AI agents.

Why ECS for MCP Servers:

- MCP servers that provide complex, long-running, or stateful tools need persistent

containers (not Lambda's 15-minute timeout).

- ECS provides reliable service orchestration, health checks, and auto-recovery.
- Service Connect enables service-to-service communication between MCP servers and agents.
- Supports WebSocket/Streamable HTTP transport for the MCP protocol.

Amazon ECS MCP Server (AWS-managed):

- AWS provides a fully managed, remote MCP server for ECS operations (preview as of late 2025).
- Provides tools for containerizing apps (`containerize_app`), deploying to ECS

(`create_ecs_infrastructure`), troubleshooting (`ecs_troubleshooting_tool`), and managing resources (`ecs_resource_management`).

- Uses CloudFormation for infrastructure provisioning.
- Works with Amazon Q Developer, Kiro CLI, Cursor, and any MCP-compatible AI assistant.

Deploying Your Own MCP Server on ECS:

```
MCP Client (Agent) --&gt; ALB --&gt; ECS Service --&gt; MCP Server Container
                                  |
                                  +--&gt; ECR (image source)
                                  +--&gt; CloudWatch (logs/metrics)
```

1. Build MCP server as a Docker image and push to ECR.
2. Create ECS task definition with appropriate CPU/memory.
3. Deploy as an ECS service behind an ALB.
4. Secure with OAuth 2.0 authentication, WAF, and security groups.
5. Use Service Connect for inter-service discovery if running multiple MCP servers.

Guidance for Deploying MCP Servers on AWS (AWS Solutions):

- Containerized architecture with OAuth 2.0 authentication.
- Content delivery network (CloudFront) and WAF for security layers.
- Container orchestration for high availability.
- Centralized logging for monitoring server behavior.

Auto Scaling for AI Workloads

ECS provides two levels of auto scaling, both important for AI inference:

1. Service Auto Scaling (Task-Level) Adjusts the number of running tasks in a service using Application Auto Scaling.

Policy Type	How It Works	AI Use Case
Target Tracking	Keeps a metric at a target value (e.g., CPU at 70%)	Steady-state inference load
Step Scaling	Scales by specific increments at alarm thresholds	Bursty inference with known patterns
Scheduled Scaling	Scales at specific times	Predictable daily inference peaks

Built-in metrics for target tracking:

- `ECSServiceAverageCPUUtilization`
- `ECSServiceAverageMemoryUtilization`
- `ALBRequestCountPerTarget`

For AI inference, custom metrics like queue depth, tokens-per-second, or GPU utilization (via Container Insights) are often more meaningful than CPU/memory alone.

2. Cluster Auto Scaling (Instance-Level) Adjusts the number of EC2 instances in the cluster using capacity providers.

- Set a `targetCapacity` percentage (e.g., 100% means instances are fully utilized before

adding more).

- ECS publishes `CapacityProviderReservation` to CloudWatch and manages a target tracking

scaling policy on the Auto Scaling group.

- For GPU workloads, cluster auto scaling ensures GPU instances are provisioned when tasks

need them.

Best Practice for AI Workloads:

- Use `binpack` placement strategy to maximize GPU utilization per instance before scaling out.
- Maintain headroom (over-provision slightly) because scaling GPU instances takes longer than

scaling standard instances.

- Use step scaling over target tracking when you need faster reaction to inference queue

backlogs.

Service Connect for AI Microservices

Service Connect provides built-in service discovery and service mesh capabilities for ECS services, eliminating the need for manual DNS configuration or external service meshes.

How It Works:

- Creates a managed Envoy proxy sidecar in each task.
- Services discover each other via short DNS names within a Cloud Map namespace.
- Round-robin load balancing with passive health checks, outlier detection, and retries.

Key Concepts:

- **Namespace**: A logical grouping (Cloud Map) where services discover each other. Can span

clusters within the same Region.

- **Client Service**: Consumes endpoints in the namespace (needs the namespace to discover others).
- **Client-Server Service**: Both provides and consumes endpoints.
- **Port Name**: Named port mapping in the task definition used for endpoint configuration.

AI Architecture Pattern -- MCP Server Mesh:

```
Agent Service (client)
    |-- connects to --&gt; Inference MCP Server (client-server)
    |-- connects to --&gt; RAG MCP Server (client-server)
    |-- connects to --&gt; Code Execution MCP Server (client-server)
```

All services share a namespace. The agent discovers MCP servers by short name (e.g., `inference-server:8080`). Service Connect handles load balancing across task replicas.

Resource Overhead:

- Add 256 CPU units and at least 64 MiB memory for the Service Connect proxy container.

- If a service receives > 500 requests/second, add 512 CPU units for the proxy.
- If the namespace has > 100 services or > 2000 tasks, add 128 MiB memory for the proxy.

Limitations:

- Does not support Windows containers.
- Only interconnects ECS services within the same namespace (external consumers need an ALB or other mechanism).
- Does not support HTTP 1.0.

---

2. Amazon EKS (Elastic Kubernetes Service)

Kubernetes for AI Workloads

Amazon EKS is a managed Kubernetes service. It runs the Kubernetes control plane across multiple AZs and integrates with AWS services for networking, security, and observability.

When EKS matters for the exam:

- Organizations with existing Kubernetes expertise or multi-cloud strategies.
- Complex AI/ML pipelines that need Kubernetes-native tooling (KubeFlow, Ray, Argo Workflows).
- GPU scheduling with fine-grained control (DRA, device plugins, node affinity).
- Distributed training across many GPU nodes using Elastic Fabric Adapter (EFA).

GPU Node Groups

EKS-Optimized Accelerated AMIs:

- Pre-built AMIs with NVIDIA drivers and the NVIDIA Kubernetes device plugin.
- `eksctl` automatically detects GPU instance types and installs the device plugin.

GPU Resource Scheduling:

- Kubernetes uses the `nvidia.com/gpu` resource type in pod specs.
- The NVIDIA device plugin advertises GPU resources to the Kubernetes scheduler.
- Pods request GPUs via `resources.limits`:

```
apiVersion: v1
kind: Pod
metadata:
  name: inference-pod
spec:
  containers:
    - name: model-server
      image: my-inference-server:latest
      resources:
        limits:
          nvidia.com/gpu: 1
          memory: "16Gi"
          cpu: "4"
```

Capacity Blocks for ML:

- Reserve GPU instances for a specific future time window.
- Useful for short-duration ML training or batch inference jobs.
- Supported for both managed node groups and self-managed nodes.

- Launch template requires `InstanceMarketOptions.MarketType = capacity-block`.

Elastic Fabric Adapter (EFA) for Distributed Training:

- EFA provides low-latency, high-throughput networking between GPU instances.
- Required for large-scale distributed training with NCCL (NVIDIA Collective Communications Library).
- Supported on p3dn, p4d, p5 instance types.
- Requires VPC CNI version 1.18.5+ for EFA-only interfaces.

Dynamic Resource Allocation (DRA):

- Kubernetes-native mechanism for fine-grained GPU allocation.
- Supports NVIDIA IMEX (Internode Memory Exchange) for cross-node GPU communication.
- Relevant for P6e-GB200 UltraServers with NVLink domains.

EKS vs ECS Decision (Exam Perspective)

Factor	Choose ECS	Choose EKS
Team expertise	AWS-native, no Kubernetes knowledge	Existing Kubernetes skills
Simplicity	Simpler task definitions and services	More complex but more flexible
GPU scheduling	Good enough for single-node GPU tasks	Better for multi-node distributed training
ML tooling	Basic container orchestration	KubeFlow, Ray, Argo, Spark on K8s
Multi-cloud	AWS-only	Portable across clouds
MCP servers	Preferred (exam calls out ECS)	Works but not specifically called out
Managed overhead	Lower (no etcd, API server concerns)	Control plane managed, but more K8s concepts

---

3. AWS Fargate

Serverless Containers for AI

Fargate is a serverless compute engine for ECS and EKS. AWS manages all underlying infrastructure. You define CPU and memory at the task (ECS) or pod (EKS) level.

Task CPU and Memory Combinations:

vCPU	Memory Range	Notes
0.25	0.5 GB - 2 GB	Lightweight sidecars
0.5	1 - 4 GB	Small services
1	2 - 8 GB	General purpose
2	4 - 16 GB	Medium workloads
4	8 - 30 GB	Larger workloads
8	16 - 60 GB	CPU-intensive inference (Linux, platform 1.4.0+)
16	32 - 120 GB	Largest Fargate tasks (Linux, platform 1.4.0+)

Key Characteristics:

- Always uses `awsvpc` network mode (each task gets its own ENI).
- Pricing is per-second (1-minute minimum for Linux, 5-minute minimum for Windows).

- Ephemeral storage: 20 GB default, configurable up to 200 GB.
- Supports EBS volumes, EFS volumes, and bind mounts.
- Supports Seekable OCI (SOCI) for lazy loading -- containers start faster by downloading

the image in the background while the application initializes.

Fargate Limitations for AI/ML (Exam-Critical)

Limitation	Impact on AI Workloads
No GPU support	Cannot run GPU-accelerated inference or training
Max 16 vCPU / 120 GB memory	Insufficient for large Foundation Models that need hundreds of GBs
No `privileged` mode	Cannot access host-level GPU drivers or specialized hardware
No `ipcMode` shared	Cannot use shared memory for inter-process communication (limits some ML frameworks)
Limited Linux capabilities	Only `CAP_SYS_PTRACE` can be added
No placement constraints	Cannot target specific instance types or hardware

When to Use Fargate vs EC2 Launch Type

Use Case	Fargate	EC2 Launch Type
MCP servers (CPU-only tools)	Good fit -- scales to zero, simple	Viable but over-provisioned
Text preprocessing/embedding	Good fit if within resource limits	Better for high-throughput batch
GPU inference (LLMs)	Not supported	Required
Small model inference (CPU)	Good fit (e.g., sklearn, small transformers)	Unnecessary overhead
Batch AI processing	Fargate Spot (up to 70% discount)	EC2 Spot for GPU workloads
Real-time API serving	Good for moderate load	Better for high-throughput with GPU

Fargate Spot:

- Up to 70% discount for interrupt-tolerant tasks.
- Fargate may reclaim capacity with a 30-second warning (SIGTERM).
- Good for: batch preprocessing, non-critical inference, data pipelines.
- Not suitable for: real-time user-facing inference, stateful MCP servers.

---

4. Amazon ECR (Elastic Container Registry)

Container Image Management

Amazon ECR is a managed Docker container registry for storing, managing, and deploying container images. It supports both private and public repositories.

Core Features:

Feature	Description
Private Registry	One per account per Region. Stores Docker and OCI images.
IAM Integration	Fine-grained access control via IAM policies and repository policies.

Feature	Description
Encryption	Images encrypted at rest using AWS KMS (default AWS-managed key or customer-managed CMK).
Tag Immutability	Prevents image tags from being overwritten. Recommended for production.
Pull-Through Cache	Cache images from public registries (Docker Hub, GitHub, etc.) in your private registry.
Managed Signing	Automatic cryptographic signatures when images are pushed.

Image Lifecycle for AI/ML:

- ML model serving images are often large (5-20+ GB with CUDA, PyTorch, model weights).
- Use multi-stage Docker builds to minimize image size.
- Use ECR lifecycle policies to automatically clean up old images.
- Use SOCI (Seekable OCI) indexes for faster container startup on Fargate.

Image Scanning

ECR provides two scanning modes:

Basic Scanning:

- Uses AWS-native technology with the CVE database.
- Scans for operating system package vulnerabilities.
- Two frequencies: scan on push or manual scan.
- One scan per image per 24 hours, max 100,000 images per registry per day.
- Findings sent to EventBridge.

Enhanced Scanning (Amazon Inspector):

- Integrates with Amazon Inspector for automated, continuous scanning.
- Scans both OS package vulnerabilities AND programming language package vulnerabilities (Python, Node.js, Java, etc.).
- Two frequencies: scan on push or continuous (re-scans when new CVEs are published).
- Maps vulnerabilities to running containers in ECS and EKS.

Security Hub Controls for ECR:

Control	Requirement
ECR.1	Image scanning configured
ECR.2	Tag immutability enabled
ECR.3	At least one lifecycle policy
ECR.4	Public repos tagged
ECR.5	Encrypted with customer-managed KMS key

Cross-Region and Cross-Account Replication

ECR supports private image replication across Regions and accounts:

- **Cross-Region replication**: Replicate images to other AWS Regions for multi-Region deployments. Reduces pull latency for globally distributed inference services.
- **Cross-Account replication**: Share images with other AWS accounts. The destination account must configure a registry permissions policy granting `ecr:ReplicateImage` and `ecr:CreateRepository`.

- Up to 25 unique destinations across all rules, max 10 rules, max 100 filters per rule.
- Replication occurs once per image push/restore. Most images replicate in under 30 minutes.
- Repository policies, IAM policies, and lifecycle policies are NOT replicated (use repository creation templates to replicate settings like tag immutability and encryption).
- Replication does not propagate deletes or archive actions.

Lifecycle Policies

Lifecycle policies automate cleanup of unused images:

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep only last 10 tagged images",
      "selection": {
        "tagStatus": "tagged",
        "tagPatternList": ["v*"],
        "countType": "imageCountMoreThan",
        "countNumber": 10
      },
      "action": {
        "type": "expire"
      }
    },
    {
      "rulePriority": 2,
      "description": "Expire untagged images older than 7 days",
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 7
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

Archive Storage:

- Images can be transitioned to archive storage (cheaper) instead of expired.
- Minimum storage duration of 90 days for archived images.
- Archived images cannot be scanned; must be restored first.

---

5. Container Deployment Patterns for GenAI

Pattern 1: Custom Model Serving Container

Deploy a custom inference server (vLLM, TGI, Triton) on ECS with GPU instances.

```
Client Request
```

```
    |
    v
ALB (Application Load Balancer)
    |
    v
ECS Service (g6e instances, EC2 launch type)
    |-- Task: vLLM server container (GPU pinned)
    |-- Task: vLLM server container (GPU pinned)
    |
    v
ECR (model + inference server image)
S3 (model weights, loaded at startup or via EFS)
```

Task Definition Considerations:

- Use `resourceRequirements` with `type: GPU` to request GPUs.
- Set memory to accommodate the model weights + runtime overhead. A 7B parameter model

in FP16 needs approximately 14 GB GPU memory; a 70B model needs approximately 140 GB.

- Use placement constraints to target appropriate GPU instances.
- Mount EFS or use S3 for model weight storage (avoids baking weights into the image).
- Set health checks with longer intervals for model loading time.

Pattern 2: MCP Server Deployment

Deploy MCP servers on ECS for AI agents to use as tools.

```
AI Agent (Bedrock Agent / Strands Agent)
    |
    v (MCP protocol over Streamable HTTP)
    |
ECS Service (Fargate or EC2)
    |-- MCP Server: Database Query Tool
    |-- MCP Server: Web Search Tool
    |-- MCP Server: Code Execution Tool
    |
Service Connect Namespace (short-name discovery)
```

Fargate for CPU-Only MCP Servers:

- Most MCP servers (database tools, API integrations, search) are CPU-bound.
- Fargate simplifies operations -- no instance management.
- Scale to zero with Fargate Spot for cost optimization on non-critical tools.

EC2 for GPU-Enhanced MCP Servers:

- Some MCP servers may need GPUs (e.g., local embedding generation, image processing tools).
- Use EC2 launch type with GPU instances.

Pattern 3: AI Microservices Architecture

Decompose an AI application into independently scalable container services.

```
API Gateway
    |
    v
[Orchestrator Service] (ECS/Fargate)
    |
    +---&gt; [Embedding Service] (ECS/Fargate, scales independently)
```

```
+---&gt; [Vector Search Service] (ECS/Fargate)
+---&gt; [Inference Service] (ECS/EC2 with GPU)
+---&gt; [Post-Processing Service] (ECS/Fargate)
|
All connected via Service Connect namespace
```

Benefits:

- Each service scales based on its own bottleneck (GPU utilization for inference,

CPU for preprocessing, memory for vector search).

- Independent deployment and versioning.
- Fault isolation -- a failure in one service does not cascade.

Pattern 4: Sidecar Patterns for AI

Inference Sidecar: A lightweight model runs as a sidecar container alongside the main application for low-latency local inference (e.g., content classification, PII detection).

```
{
  "containerDefinitions": [
    {
      "name": "main-app",
      "image": "my-app:latest",
      "essential": true,
      "portMappings": [{ "containerPort": 8080 }]
    },
    {
      "name": "pii-detector",
      "image": "my-pii-model:latest",
      "essential": false,
      "portMappings": [{ "containerPort": 9090 }]
    }
  ]
}
```

The main application calls `localhost:9090` for PII detection with no network hop.

Observability Sidecar: AWS Distro for OpenTelemetry (ADOT) collector runs as a sidecar to collect traces, metrics, and logs from AI inference containers.

Service Connect Proxy: Automatically injected by ECS when using Service Connect. Handles service discovery, load balancing, retries, and metrics collection.

---

6. Exam Gotchas

ECS vs EKS vs Lambda for AI Workloads

Scenario	Best Choice	Why
Simple MCP server (stateless, fast)	**Lambda**	15-min max, scales to zero, cheapest for sporadic use
Complex MCP server (stateful, long-running)	**ECS**	Persistent containers, Service Connect, exam specifically calls this out
GPU inference (single node)	**ECS on EC2**	Simpler GPU task definitions, managed scaling

Scenario	Best Choice	Why
Distributed training (multi-node GPU)	**EKS**	Kubernetes-native GPU scheduling, EFA, DRA
Existing Kubernetes platform	**EKS**	Organizational consistency
Batch preprocessing (no GPU)	**Fargate** or **Lambda**	Serverless, pay per use

Common Exam Traps

1. **"Fargate for GPU inference"** -- Wrong. Fargate does not support GPUs. If a question mentions GPU requirements, the answer must include EC2 launch type or EKS with GPU nodes.

2. **"ECS vs SageMaker for model hosting"** -- SageMaker endpoints are managed and provide built-in model optimization, A/B testing, and auto scaling out of the box. ECS gives you full control but requires you to manage the inference server, scaling, and health checks. Choose ECS when you need custom inference logic, non-standard model formats, or tight integration with other containerized services.

3. **"Fargate for large models"** -- Fargate maxes out at 16 vCPU and 120 GB memory with no GPU. Most LLMs above 7B parameters in FP16 need GPU memory, not just system RAM. Even CPU-only inference of large models would exceed Fargate's resource limits.

4. **"ECR replication copies everything"** -- Only images pushed or restored AFTER replication is configured get replicated. Pre-existing images are not retroactively replicated. Deletes and lifecycle policy expirations are also not replicated.

5. **"Service Connect replaces ALB"** -- Service Connect only handles service-to-service communication within a namespace. External traffic still needs an ALB or NLB.

6. **"ECS auto scaling is instant"** -- Scaling GPU instances takes several minutes. Over-provision (maintain headroom) and use step scaling with aggressive thresholds for latency-sensitive AI workloads.

---

7. When to Use Each -- Decision Matrix

```
Need GPU? ■■Yes■■&gt; Need multi-node training? ■■Yes■■&gt; Amazon EKS (GPU node groups + EFA)
    ■                        ■
    ■                        No
    ■                        ■
    ■                        v
    ■                Amazon ECS on EC2 (GPU tasks)
    ■
    No
    ■
    v
Need persistent containers? ■■Yes■■&gt; Memory &gt; 120 GB? ■■Yes■■&gt; ECS on EC2
    ■                                        ■
    ■                                        No
    ■                                        ■
    ■                                        v
    ■                                  AWS Fargate
    No
    ■
    v
Short-lived / event-driven? ■■Yes■■&gt; AWS Lambda
    ■
```

```
    No
    ■
    v
AWS Fargate (default serverless container choice)
```

Quick Reference Table:

Workload	Service	Launch Type	Why
LLM inference (7B+ params)	ECS	EC2 (GPU)	GPU required for practical latency
Small model inference (CPU)	ECS	Fargate	Serverless, cost-effective
MCP server (complex tools)	ECS	Fargate or EC2	Exam calls out ECS specifically
Distributed training	EKS	EC2 (GPU + EFA)	Multi-node GPU coordination
Data preprocessing pipeline	ECS	Fargate / Fargate Spot	No GPU needed, cost-sensitive
Real-time embedding service	ECS	Fargate (if CPU) or EC2 (if GPU)	Depends on model size
Batch inference	ECS or AWS Batch	Fargate Spot or EC2 Spot	Cost optimization
Container image storage	ECR	N/A	Managed registry, integrates with ECS/EKS

---

8. TypeScript SDK v3 Usage

Register a Task Definition (GPU Inference)

```
import {
  ECSClient,
  RegisterTaskDefinitionCommand,
} from "@aws-sdk/client-ecs";

const ecs = new ECSClient({ region: "us-east-1" });

const command = new RegisterTaskDefinitionCommand({
  family: "gpu-inference-task",
  requiresCompatibilities: ["EC2"],
  networkMode: "awsvpc",
  cpu: "4096",
  memory: "16384",
  taskRoleArn: "arn:aws:iam::123456789012:role/ecsTaskRole",
  executionRoleArn: "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  containerDefinitions: [
    {
      name: "inference-server",
      image: "123456789012.dkr.ecr.us-east-1.amazonaws.com/my-model:latest",
      essential: true,
      resourceRequirements: [
        { type: "GPU", value: "1" },
      ],
      portMappings: [
```

```
        { containerPort: 8080, protocol: "tcp" },
      ],
      logConfiguration: {
        logDriver: "awslogs",
        options: {
          "awslogs-group": "/ecs/gpu-inference",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "inference",
        },
      },
      environment: [
        { name: "MODEL_ID", value: "my-fine-tuned-model" },
        { name: "NVIDIA_VISIBLE_DEVICES", value: "all" },
      ],
    },
  ],
});

const response = await ecs.send(command);
console.log("Task definition ARN:", response.taskDefinition?.taskDefinitionArn);
```

Create an ECS Service with Auto Scaling

```
import { ECSClient, CreateServiceCommand } from "@aws-sdk/client-ecs";

const ecs = new ECSClient({ region: "us-east-1" });

const createService = new CreateServiceCommand({
  cluster: "ai-inference-cluster",
  serviceName: "inference-service",
  taskDefinition: "gpu-inference-task",
  desiredCount: 2,
  launchType: "EC2",
  networkConfiguration: {
    awsvpcConfiguration: {
      subnets: ["subnet-abc123", "subnet-def456"],
      securityGroups: ["sg-inference"],
      assignPublicIp: "DISABLED",
    },
  },
  loadBalancers: [
    {
      targetGroupArn: "arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/inference-tg/abc123",
      containerName: "inference-server",
      containerPort: 8080,
    },
  ],
  placementConstraints: [
    {
      type: "memberOf",
      expression: "attribute:ecs.instance-type =~ g6e.*",
    },
  ],
  serviceConnectConfiguration: {
    enabled: true,
    namespace: "ai-services",
    services: [
      {
        portName: "inference-port",
```

```
        discoveryName: "inference-server",
        clientAliases: [
          { port: 8080, dnsName: "inference-server" },
        ],
      },
    ],
  },
});

const serviceResponse = await ecs.send(createService);
```

Register Auto Scaling Target

```
import {
  ApplicationAutoScalingClient,
  RegisterScalableTargetCommand,
  PutScalingPolicyCommand,
} from "@aws-sdk/client-application-auto-scaling";

const autoScaling = new ApplicationAutoScalingClient({ region: "us-east-1" });

await autoScaling.send(new RegisterScalableTargetCommand({
  ServiceNamespace: "ecs",
  ResourceId: "service/ai-inference-cluster/inference-service",
  ScalableDimension: "ecs:service:DesiredCount",
  MinCapacity: 1,
  MaxCapacity: 10,
}));

await autoScaling.send(new PutScalingPolicyCommand({
  ServiceNamespace: "ecs",
  ResourceId: "service/ai-inference-cluster/inference-service",
  ScalableDimension: "ecs:service:DesiredCount",
  PolicyName: "cpu-target-tracking",
  PolicyType: "TargetTrackingScaling",
  TargetTrackingScalingPolicyConfiguration: {
    TargetValue: 70.0,
    PredefinedMetricSpecification: {
      PredefinedMetricType: "ECSServiceAverageCPUUtilization",
    },
    ScaleInCooldown: 300,
    ScaleOutCooldown: 60,
  },
}));
```

Push Image to ECR

```
import {
  ECRClient,
  GetAuthorizationTokenCommand,
  CreateRepositoryCommand,
  DescribeRepositoriesCommand,
} from "@aws-sdk/client-ecr";

const ecr = new ECRClient({ region: "us-east-1" });

// Create repository with scanning and immutability
```

```
  await ecr.send(new CreateRepositoryCommand({
    repositoryName: "my-inference-server",
    imageScanningConfiguration: { scanOnPush: true },
    imageTagMutability: "IMMUTABLE",
    encryptionConfiguration: {
      encryptionType: "KMS",
      kmsKey: "arn:aws:kms:us-east-1:123456789012:key/my-key-id",
    },
  }));

  // Get auth token for Docker login
  const authResponse = await ecr.send(new GetAuthorizationTokenCommand({}));
  const token = authResponse.authorizationData?.[0]?.authorizationToken;
  // Decode and use with `docker login`
```

---

9. Key Metrics -- CloudWatch

Built-in ECS Metrics (AWS/ECS namespace)

Metric	Dimensions	Description
`CPUUtilization`	ClusterName, ServiceName	Percentage of CPU units used by tasks vs reserved
`MemoryUtilization`	ClusterName, ServiceName	Percentage of memory used by tasks vs reserved
`CPUReservation`	ClusterName	Percentage of CPU reserved by tasks vs total registered (EC2 only)
`MemoryReservation`	ClusterName	Percentage of memory reserved by tasks vs total registered (EC2 only)
`GPUReservation`	ClusterName	Percentage of GPUs reserved by tasks vs total available (EC2 only)
`EBSFilesystemUtilization`	ClusterName, ServiceName	EBS storage used vs allocated

Container Insights Metrics (ECS/ContainerInsights namespace)

Requires enabling Container Insights on the cluster. Additional cost but provides per-task and per-container visibility.

Metric	Description
`CpuUtilized` / `CpuReserved`	Absolute CPU units used/reserved
`MemoryUtilized` / `MemoryReserved`	Absolute memory bytes used/reserved
`NetworkRxBytes` / `NetworkTxBytes`	Network I/O per task
`StorageReadBytes` / `StorageWriteBytes`	Storage I/O per task
`RunningTaskCount` / `PendingTaskCount`	Task status counts
`DesiredTaskCount`	Target task count for the service
`ContainerInstanceCount`	Number of EC2 instances in the cluster

Service Connect Metrics

When Service Connect is enabled, additional metrics are published:

Metric	Description
`ActiveConnectionCount`	Current active connections to the service
`NewConnectionCount`	New connections per period
`RequestCount` / `GrpcRequestCount`	Total HTTP/gRPC requests
`HTTPCode_Target_2XX/4XX/5XX_Count`	Response code distribution
`TargetResponseTime`	Latency from the proxy to the service
`RequestCountPerTarget`	Requests per individual task

Cluster Auto Scaling Metric

Metric	Namespace	Description
`CapacityProviderReservation`	AWS/ECS/ManagedScaling	Percent of container instances in use for a capacity provider. Drives the auto scaling policy.

AI-Relevant Custom Metrics

For GPU inference workloads, publish custom metrics via CloudWatch agent or Container Insights:

Custom Metric	What It Measures	Scaling Signal
GPU Utilization (%)	How busy the GPU compute cores are	Scale out if consistently > 80%
GPU Memory Utilization (%)	VRAM usage	Indicates model size vs instance capacity
Inference Latency (p50/p95/p99)	End-to-end inference time	Scale out if p99 exceeds SLA
Tokens Per Second	Throughput of LLM inference	Core performance metric
Queue Depth	Pending inference requests	Leading indicator for scale-out
Batch Size	Requests batched per inference call	Tuning metric for throughput vs latency

---

Summary -- Key Facts for Exam Day

1. **GPU inference requires EC2 launch type** (ECS) or GPU node groups (EKS). Fargate has no GPU support.
2. **ECS is specifically called out for MCP servers** providing complex tools to AI agents (Task 2.1.7).
3. **Fargate maxes out at 16 vCPU / 120 GB memory** -- insufficient for large Foundation Models.
4. **Service Connect** provides service mesh capabilities without external tools -- uses Cloud Map namespaces for short-name discovery between AI microservices.
5. **ECR Enhanced Scanning** (via Inspector) scans both OS and language package vulnerabilities. Basic scanning only covers OS packages.

6. **ECR replication** is push-only (not retroactive), does not replicate deletes, and does not replicate policies.

7. **ECS GPU task definitions** use `resourceRequirements` with `type: GPU` and a `value` string.

8. **Cluster auto scaling** uses `CapacityProviderReservation` metric with target tracking. Use `binpack` strategy to maximize GPU utilization per instance.

9. **Fargate Spot** offers up to 70% savings for interrupt-tolerant AI batch workloads.

10. **SOCI (Seekable OCI)** enables lazy loading of large container images on Fargate, reducing cold start time for AI containers with large images.

Storage and Migration Services for GenAI

Context: AWS Certified AI Practitioner (AIP-C01) Exam

S3 is the most exam-relevant storage service. It is the primary data source for Bedrock Knowledge Bases, the storage layer for training data, model artifacts, prompt templates, and evaluation datasets. EBS and EFS appear in the context of GPU instance storage and SageMaker shared file systems. DataSync and Transfer Family cover data migration into S3 for AI workloads.

Exam Relevance by Task:

- Task 1.4.1: S3 document repositories for vector architectures
- Task 1.4.2: S3 object metadata for document timestamps, custom attributes
- Task 1.6.3: S3 for template repositories (prompt management)
- Task 3.2.2: S3 Lifecycle configurations for data retention

Amazon S3

1. Overview and Key Concepts

Amazon S3 is an object storage service with effectively unlimited capacity, 99.999999999% (11 nines) durability, and 99.99% availability for Standard storage class.

Core primitives:

Concept	Description
Bucket	Top-level container. Globally unique name. Region-specific.
Object	A file + metadata. Identified by a key (full path including prefixes).
Key	The full "path" to an object, e.g. `training-data/v2/corpus.jsonl`
Prefix	A logical grouping using the key path before the object name. Not a real folder.
Version ID	Unique identifier for each version of an object when versioning is enabled.

Key limits:

- Object size: 0 bytes to 5 TB. Use multipart upload for objects > 100 MB (required > 5 GB).
- Bucket limit: 100 buckets per account by default (can request increase).
- No limit on the number of objects per bucket.
- Metadata per object: 2 KB of user-defined metadata in the PUT request header.
- Object tags: up to 10 key-value pairs per object.

Storage classes (ranked by cost, highest to lowest):

1. S3 Standard -- frequent access, low latency
2. S3 Intelligent-Tiering -- auto-tiering based on access patterns
3. S3 Standard-IA -- infrequent access, per-GB retrieval fee
4. S3 One Zone-IA -- single AZ, lower cost, lower durability
5. S3 Glacier Instant Retrieval -- archive, millisecond retrieval
6. S3 Glacier Flexible Retrieval -- archive, minutes to hours retrieval
7. S3 Glacier Deep Archive -- lowest cost, 12-48 hour retrieval

> ***Exam tip:*** *S3 Standard is the default. All new objects are encrypted at rest with SSE-S3 by default since January 5, 2023. You do not need to enable encryption -- it is always on.*

---

2. S3 as Data Source for Bedrock Knowledge Bases

S3 is the primary data source connector for Amazon Bedrock Knowledge Bases. When you create a Knowledge Base, you point it at an S3 bucket (or prefix within a bucket) containing your source documents.

How it works:

1. You upload documents (PDF, TXT, DOCX, HTML, CSV, MD, XLS, images, audio, video) to an S3 bucket.
2. You create a Knowledge Base and configure the S3 data source connector with the bucket URI.
3. Bedrock crawls the bucket, parses documents, chunks them, generates embeddings, and stores vectors in the configured vector store.
4. When you call `RetrieveAndGenerate` or `Retrieve`, Bedrock queries the vector store and returns relevant chunks with citations to the original S3 objects.

Key configuration details:

- **Inclusion prefixes**: Scope the data source to a specific S3 path prefix instead of the entire bucket. Use this to separate training data from evaluation data from production corpora.
- **Incremental syncs**: Bedrock tracks changes since the last sync and only processes new, modified, or deleted content. Call `StartIngestionJob` to trigger a sync.
- **Multimodal content**: S3 data sources support images, audio, and video files for multimodal Knowledge Bases.
- **Only General Purpose buckets** are supported (not Directory buckets / S3 Express One Zone).
- **Bucket must be in the same Region** as the Knowledge Base.
- **DataSync integration**: You can use AWS DataSync to continuously transfer files from on-premises storage to the S3 bucket on a schedule.

S3 Vectors (new feature): S3 can also serve as the vector store itself via S3 Vectors, an integration that lets Bedrock store embeddings directly in S3 table buckets (Apache Iceberg tables). This reduces cost for RAG applications by eliminating the need for a separate vector database.

Required IAM permissions on the Knowledge Base role:

- `s3:GetObject` on the bucket/prefix
- `s3:ListBucket` on the bucket
- If the bucket is encrypted with KMS: `kms:Decrypt` on the KMS key

```
// TypeScript SDK v3 -- Create a Bedrock Knowledge Base S3 data source
import {
  BedrockAgentClient,
  CreateDataSourceCommand,
} from "@aws-sdk/client-bedrock-agent";

const client = new BedrockAgentClient({ region: "us-east-1" });

const response = await client.send(
  new CreateDataSourceCommand({
    knowledgeBaseId: "KB_ID",
    name: "product-docs-source",
```

```
    dataSourceConfiguration: {
      type: "S3",
      s3Configuration: {
        bucketArn: "arn:aws:s3:::my-knowledge-base-docs",
        inclusionPrefixes: ["product-manuals/"],
      },
    },
  })
);
```

> ***Exam tip:*** *If a question asks about the most cost-effective way to keep a Knowledge Base current with on-premises document changes, the answer is DataSync to S3 + incremental sync.*

---

3. Object Metadata for GenAI (Timestamps, Custom Attributes, Tagging)

S3 objects have two types of metadata that are directly relevant to GenAI workloads:

System-Defined Metadata

Automatically maintained by S3. Key headers include:

- `Content-Type` -- MIME type (e.g., `application/pdf`)
- `Content-Length` -- size in bytes
- `Last-Modified` -- timestamp of last modification
- `ETag` -- entity tag (hash of the object)
- `x-amz-storage-class` -- current storage class
- `x-amz-server-side-encryption` -- encryption algorithm used
- `x-amz-version-id` -- version identifier (if versioning enabled)

User-Defined Metadata

Custom key-value pairs set at upload time. Must be prefixed with `x-amz-meta-`. Limited to 2 KB total in the PUT request header. Cannot be modified after upload -- you must copy the object to change metadata.

Object Tags

Key-value pairs (up to 10 per object) that can be added, modified, or removed after upload. Tags are case-sensitive. Max key length: 128 Unicode chars. Max value length: 256 Unicode chars.

GenAI use cases for metadata and tags:

Use Case	Mechanism
Filter Knowledge Base results by date	`.metadata.json` file with `created_date` attribute
Track document source/author	`.metadata.json` file with custom attributes
Control Lifecycle transitions	Tags + Lifecycle rules based on tag filters
Cost allocation for AI projects	Tags for cost tracking
ABAC (attribute-based access control)	Tags in IAM policy conditions
Data governance classification	Tags for sensitivity levels

Bedrock Knowledge Base Metadata Files

This is critical for the exam. For each document in your S3 data source, you can create a companion metadata file that Bedrock uses for filtering during retrieval.

File naming convention: `<document-name>.<extension>.metadata.json`

For example, for `product-manual.pdf`, the metadata file is `product-manual.pdf.metadata.json`.

File location: Same folder as the source document.

File format:

```
{
  "metadataAttributes": {
    "company": {
      "value": {
        "type": "STRING",
        "stringValue": "Acme Corp"
      },
      "includeForEmbedding": true
    },
    "created_date": {
      "value": {
        "type": "NUMBER",
        "numberValue": 20240115
      },
      "includeForEmbedding": true
    },
    "category": {
      "value": {
        "type": "STRING",
        "stringValue": "user-guide"
      },
      "includeForEmbedding": false
    }
  }
}
```

Key rules:

- Max metadata file size: 10 KB
- Supported types: STRING, NUMBER, BOOLEAN
- `includeForEmbedding: true` means the attribute value is included in the text chunk before

embedding, which influences semantic search relevance

- `includeForEmbedding: false` means the attribute is only available for explicit filtering in

retrieval queries, not embedded

S3 Metadata (general availability): A newer feature that automatically captures and stores metadata from objects as they are uploaded, making it queryable in Apache Iceberg tables stored in S3 Tables. This supports both system-defined and custom metadata and integrates with Athena, Redshift, and QuickSight for analytics.

```
// TypeScript SDK v3 -- Upload an object with user-defined metadata and tags
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutObjectCommand({
    Bucket: "my-knowledge-base-docs",
    Key: "product-manuals/manual-v2.pdf",
    Body: fileBuffer,
```

```
    ContentType: "application/pdf",
    Metadata: {
      // These become x-amz-meta-* headers
      "document-version": "2.0",
      author: "engineering-team",
    },
    Tagging: "project=genai-chatbot&amp;environment=production",
  })
);
```

> ***Exam tip:*** *User-defined metadata (`x-amz-meta-*`) is immutable after upload. Tags are mutable. Bedrock Knowledge Base metadata uses a separate `.metadata.json` sidecar file, not S3 user-defined metadata or tags directly.*

---

4. S3 Lifecycle Policies for AI Data Retention

S3 Lifecycle configurations automate the transition of objects between storage classes and their eventual deletion. This is directly referenced in Task 3.2.2 for data retention compliance.

Lifecycle rule actions:

1. **Transition actions** -- move objects to a cheaper storage class after N days
2. **Expiration actions** -- delete objects after N days
3. **Noncurrent version expiration** -- delete old versions after N days (requires versioning)
4. **Abort incomplete multipart uploads** -- clean up partial uploads after N days

Lifecycle rule filters:

- **Prefix filter** -- apply to objects matching a key prefix
- **Tag filter** -- apply to objects matching specific tags
- **Object size filter** -- apply to objects within a size range
- **AND combination** -- combine prefix, tags, and size filters

AI data retention patterns:

Data Type	Lifecycle Strategy
Raw training data	Transition to S3-IA after 90 days, Glacier after 365 days
Model artifacts	Keep in Standard for active deployment, S3-IA after model is retired
Evaluation datasets	Retain in Standard for 30 days, then S3-IA, expire after compliance period
Inference logs	Transition to S3-IA after 30 days, Glacier after 90 days, expire per policy
Prompt templates (active)	Keep in Standard (frequently accessed)
Knowledge Base source docs	Match retention to document refresh cadence
Intermediate embeddings	Expire after reprocessing or retain for audit

Transition waterfall constraints:

- Cannot transition from S3 Standard-IA to S3 One Zone-IA
- Minimum 30 days before transitioning from Standard to Standard-IA or One Zone-IA
- Minimum 30 days in Standard-IA or One Zone-IA before transitioning to Glacier classes
- Minimum storage duration charges: Standard-IA (30 days), Glacier Instant (90 days), Glacier

Flexible (90 days), Glacier Deep Archive (180 days)

```
// TypeScript SDK v3 -- Set a Lifecycle configuration on a bucket
```

```
import {
  S3Client,
  PutBucketLifecycleConfigurationCommand,
} from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutBucketLifecycleConfigurationCommand({
    Bucket: "my-ai-data-bucket",
    LifecycleConfiguration: {
      Rules: [
        {
          ID: "archive-training-data",
          Status: "Enabled",
          Filter: {
            Prefix: "training-data/",
          },
          Transitions: [
            { Days: 90, StorageClass: "STANDARD_IA" },
            { Days: 365, StorageClass: "GLACIER" },
          ],
        },
        {
          ID: "expire-inference-logs",
          Status: "Enabled",
          Filter: {
            Tag: { Key: "data-type", Value: "inference-log" },
          },
          Transitions: [
            { Days: 30, StorageClass: "STANDARD_IA" },
            { Days: 90, StorageClass: "GLACIER" },
          ],
          Expiration: { Days: 730 },
        },
        {
          ID: "cleanup-multipart-uploads",
          Status: "Enabled",
          Filter: { Prefix: "" },
          AbortIncompleteMultipartUpload: { DaysAfterInitiation: 7 },
        },
      ],
    },
  })
);
```

Exam tip: *Lifecycle rules use the object creation date (or last modified date for versioned objects) as the reference point for transition timing. Objects replicated via CRR honor the original creation time, not the replication time.*

---

5. S3 Intelligent-Tiering for AI Data Cost Optimization

S3 Intelligent-Tiering automatically moves objects between access tiers based on access patterns, with no retrieval charges and no operational overhead. This is the recommended storage class for data with unpredictable or changing access patterns.

Access tiers (automatic):

Tier	Trigger	Savings
Frequent Access (FA)	Default tier on upload	Baseline
Infrequent Access (IA)	No access for 30 consecutive days	~40%
Archive Instant Access (AIA)	No access for 90 consecutive days	~68%

Optional archive tiers (must be enabled):

Tier	Trigger	Savings	Retrieval Time
Archive Access	No access for 90+ days (configurable, min 90)	~71%	3-5 hours
Deep Archive Access	No access for 180+ days (configurable, min 180)	~95%	12 hours

How it works:

- A small monthly monitoring and automation charge per object (no minimum object size).
- Objects are automatically moved to lower-cost tiers when not accessed.
- When accessed, objects move back to Frequent Access at no additional charge.
- No retrieval fees for Frequent, Infrequent, and Archive Instant tiers.
- Objects in Archive or Deep Archive tiers require a restore before access (async).

AI workload fit:

- Training datasets accessed heavily during training, then rarely afterward
- Evaluation datasets used periodically during model iteration
- Knowledge Base source documents that may not be re-crawled frequently
- Historical inference logs needed for occasional auditing

Monitoring:

- S3 Inventory reports include the Intelligent-Tiering access tier for each object.
- S3 Event Notifications fire when objects move to Archive or Deep Archive tiers.
- HEAD requests return the `x-amz-archive-status` header for archived objects.

```
// TypeScript SDK v3 -- Upload an object directly to Intelligent-Tiering
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutObjectCommand({
    Bucket: "my-ai-data-bucket",
    Key: "datasets/evaluation-set-q4.jsonl",
    Body: dataBuffer,
    StorageClass: "INTELLIGENT_TIERING",
  })
);


// TypeScript SDK v3 -- Enable optional archive tiers via Intelligent-Tiering configuration
import {
  S3Client,
  PutBucketIntelligentTieringConfigurationCommand,
} from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
```

```
    new PutBucketIntelligentTieringConfigurationCommand({
      Bucket: "my-ai-data-bucket",
      Id: "archive-old-training-data",
      IntelligentTieringConfiguration: {
        Id: "archive-old-training-data",
        Status: "Enabled",
        Tierings: [
          { AccessTier: "ARCHIVE_ACCESS", Days: 90 },
          { AccessTier: "DEEP_ARCHIVE_ACCESS", Days: 180 },
        ],
        Filter: {
          Prefix: "training-data/",
        },
      },
    })
  );
```

> ***Exam tip:*** *Intelligent-Tiering is the best answer when a question describes "unpredictable access patterns" or "optimize costs without managing storage classes manually." It has no retrieval fees for the three automatic tiers. If the question mentions "known infrequent access," Standard-IA is cheaper (no monitoring fee).*

---

6. S3 Cross-Region Replication (CRR) for Global AI Deployments

CRR automatically replicates objects from a source bucket to one or more destination buckets in different AWS Regions. This supports global AI deployments, disaster recovery, and compliance.

Requirements:

- Versioning must be enabled on both source and destination buckets.
- Source and destination must be in different Regions (same Region = SRR).
- An IAM role must grant S3 permission to replicate objects.
- In cross-account scenarios, the destination bucket policy must allow the source role.

Replication scope:

- **Entire bucket** or **filtered by prefix** or **filtered by object tags**
- New objects only (by default) -- use S3 Batch Replication for existing objects.
- Metadata, tags, and ACLs are replicated.
- Encrypted objects (SSE-S3, SSE-KMS) can be replicated with proper KMS configuration.

Replication Time Control (S3 RTC):

- SLA: 99.99% of objects replicated within 15 minutes.
- Includes CloudWatch metrics for monitoring replication progress.
- Adds cost but guarantees compliance with replication time requirements.

Bi-directional (two-way) replication:

- Both buckets are source and destination.
- Enables shared datasets across Regions.
- Requires Replica Modification Sync to keep metadata changes in sync.

AI deployment patterns with CRR:

Pattern	Description
Global Knowledge Base	Replicate source docs to buckets in each Region where Bedrock runs
DR for training data	Replicate training corpora to a backup Region

Pattern	Description
Multi-region inference	Replicate model artifacts and prompt templates closer to users
Data sovereignty	Replicate data to Regions that meet regulatory requirements
Latency reduction	Keep data co-located with the compute that processes it

Key considerations:

- CRR does not replicate objects that existed before the replication rule was created. Use S3 Batch Replication to backfill.
- Delete markers are replicated by default (can be disabled).
- Lifecycle actions on the source are NOT replicated. Each bucket needs its own Lifecycle rules.
- There are inter-Region data transfer charges for CRR.
- Replicating objects in Intelligent-Tiering with Batch Replication or CopyObject causes tier-up of the source object (it gets accessed).

```
// TypeScript SDK v3 -- Configure Cross-Region Replication
import { S3Client, PutBucketReplicationCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutBucketReplicationCommand({
    Bucket: "my-source-bucket",
    ReplicationConfiguration: {
      Role: "arn:aws:iam::123456789012:role/S3ReplicationRole",
      Rules: [
        {
          ID: "replicate-knowledge-base-docs",
          Status: "Enabled",
          Filter: {
            Prefix: "knowledge-base/",
          },
          Destination: {
            Bucket: "arn:aws:s3:::my-destination-bucket-eu-west-1",
            StorageClass: "STANDARD",
            EncryptionConfiguration: {
              ReplicaKmsKeyID:
                "arn:aws:kms:eu-west-1:123456789012:key/dest-key-id",
            },
          },
          SourceSelectionCriteria: {
            SseKmsEncryptedObjects: {
              Status: "Enabled",
            },
          },
          DeleteMarkerReplication: {
            Status: "Enabled",
          },
        },
      ],
    },
  })
);
```

Exam tip: *If a question asks about serving a Knowledge Base to users in multiple Regions, the answer involves CRR to replicate source documents + a Knowledge Base in each Region. Multi-Region Access Points can*

simplify the endpoint but still require CRR underneath.

---

7. S3 Encryption

All S3 objects are encrypted at rest by default. There are four server-side encryption options and one client-side option.

Server-Side Encryption Options

Method	Key Management	When to Use
SSE-S3 (AES-256)	AWS manages keys entirely	Default. No additional cost. Good for most workloads.
SSE-KMS	AWS KMS manages keys (AWS managed or customer managed)	Need audit trail, key rotation control, or cross-account access control
DSSE-KMS	Dual-layer encryption with KMS	Compliance requiring two layers of encryption
SSE-C	Customer provides keys per request	You manage your own key material. Must use HTTPS.

SSE-S3 (default):

- Applied automatically since January 5, 2023.
- Uses AES-256.
- No additional cost, no configuration needed.

SSE-KMS:

- Uses AWS KMS for key management.
- **AWS managed key** (`aws/s3`): Automatic, no per-key cost, but cannot be shared cross-account.
- **Customer managed key (CMK)**: Full control over key policy, rotation, and audit via

CloudTrail. Required for cross-account access to encrypted objects.

- **S3 Bucket Keys**: Reduces KMS API call costs by up to 99%. Generates a bucket-level key from

the KMS key, which then creates data keys for individual objects. Enable this when using SSE-KMS at scale.

- Every GET/PUT for SSE-KMS objects must use HTTPS and valid AWS credentials.
- Encryption context: defaults to bucket ARN, can be customized.

SSE-C:

- You provide the encryption key with each PUT/GET request via HTTPS headers.
- S3 does not store your key. If you lose it, you lose the data.
- Being deprecated for new buckets starting April 2026.

Client-Side Encryption:

- You encrypt data before uploading. S3 stores the ciphertext.
- You manage all key material and encryption/decryption logic.
- Use the AWS Encryption SDK or S3 Encryption Client.

Encryption for Bedrock Knowledge Bases:

- S3 data sources can be encrypted with SSE-S3 or SSE-KMS.
- If using SSE-KMS with a customer managed key, the Knowledge Base IAM role needs `kms:Decrypt`

permission on that key.

- The Knowledge Base itself can have a separate KMS key for server-side encryption of its

configuration.

```
// TypeScript SDK v3 -- Upload with SSE-KMS using a customer managed key
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutObjectCommand({
    Bucket: "my-secure-bucket",
    Key: "training-data/sensitive-corpus.jsonl",
    Body: dataBuffer,
    ServerSideEncryption: "aws:kms",
    SSEKMSKeyId: "arn:aws:kms:us-east-1:123456789012:key/my-cmk-id",
    BucketKeyEnabled: true,
  })
);
```

> ***Exam tip:*** *SSE-S3 is always on by default. Use SSE-KMS with a customer managed key when you need: (1) cross-account sharing of encrypted objects, (2) key rotation control, (3) CloudTrail audit of key usage, or (4) separation of duties between key admins and data admins. Enable S3 Bucket Keys to reduce KMS costs at scale.*

---

8. S3 Event Notifications for AI Pipelines

S3 event notifications trigger downstream processing when objects are created, deleted, or transitioned. This is the foundation of event-driven AI pipelines.

Supported event types (key subset):

- `s3:ObjectCreated:*` -- any object creation (PUT, POST, COPY, multipart)
- `s3:ObjectRemoved:*` -- any object deletion
- `s3:ObjectRestore:*` -- object restored from Glacier
- `s3:LifecycleExpiration:*` -- object expired by Lifecycle rule
- `s3:LifecycleTransition` -- object transitioned to another storage class
- `s3:IntelligentTiering` -- object moved to Archive or Deep Archive tier
- `s3:ObjectTagging:*` -- tags added or deleted
- `s3:Replication:*` -- replication events

Notification destinations:

1. **Amazon SNS** -- fan-out to multiple subscribers
2. **Amazon SQS** -- queue for decoupled processing (FIFO not supported)
3. **AWS Lambda** -- direct function invocation
4. **Amazon EventBridge** -- advanced filtering, routing, and transformation

EventBridge integration (recommended for new designs):

- Must be explicitly enabled on the bucket.
- Provides richer filtering (by prefix, suffix, object size, metadata).
- Supports rule-based routing to 20+ target services.
- Supports archive and replay of events.
- Supports schema discovery.

AI pipeline patterns:

```
S3 upload --&gt; EventBridge --&gt; Step Functions --&gt; [Parse, Chunk, Embed, Index]
S3 upload --&gt; Lambda --&gt; StartIngestionJob (Bedrock KB sync)
S3 upload --&gt; SQS --&gt; SageMaker Processing Job
S3 upload --&gt; SNS --&gt; Lambda (thumbnail) + SQS (metadata extraction)
```

> ***Exam tip:*** *Use EventBridge (not direct SNS/SQS/Lambda) when you need advanced filtering, multiple targets, or event replay. EventBridge must be explicitly enabled per bucket. SQS FIFO queues are NOT supported as S3 event notification destinations.*

---

9. S3 Select for Efficient Data Retrieval

S3 Select lets you use SQL expressions to retrieve a subset of data from an S3 object without downloading the entire object. This reduces data transfer and speeds up retrieval for large files.

Supported formats: CSV, JSON, Apache Parquet

Key limits:

- Max object size: 5 TB
- Max SQL expression: 256 KB
- Max record length: 1 MB
- Console limit: 40 MB per query (no limit via API/CLI)
- Not available for objects in Glacier storage classes
- Not available for SSE-C encrypted objects

Use cases in GenAI:

- Extract specific columns from large CSV training datasets
- Filter JSON inference logs by timestamp or model ID
- Sample rows from Parquet evaluation datasets
- Pre-filter data before feeding into SageMaker Processing jobs

Note: S3 Select is no longer available to new customers as of 2024. Existing customers can continue using it. For new workloads, consider Amazon Athena or S3 Object Lambda as alternatives.

> ***Exam tip:*** *If a question asks about querying data "in place" in S3 without downloading, the modern answer is Amazon Athena. S3 Select may still appear in legacy-oriented questions.*

---

10. Versioning for AI Artifacts

S3 versioning preserves every version of every object in a bucket. Once enabled, it cannot be disabled -- only suspended.

States:

- **Unversioned** (default) -- objects have a null version ID
- **Versioning-enabled** -- every PUT creates a new version with a unique version ID
- **Versioning-suspended** -- new PUTs get a null version ID, existing versions are preserved

AI artifact versioning patterns:

- **Model artifacts**: Version model weights, configuration files, and tokenizers. Roll back to a

previous model version if the new one degrades.

- **Training datasets**: Track dataset versions. Reproduce training runs by referencing specific

version IDs.

- **Prompt templates**: Version prompt templates for A/B testing and rollback.
- **Evaluation datasets**: Maintain versioned benchmark datasets for consistent model comparison.

Key behaviors:

- DELETE on a versioned object creates a delete marker (soft delete). The object is still recoverable by version ID.
- MFA Delete adds another layer of protection -- requires MFA to permanently delete versions or change versioning state. Only the root account can enable MFA Delete.
- Lifecycle rules can target noncurrent versions for expiration (clean up old versions after N days).

Versioning is required for CRR and SRR.

```
// TypeScript SDK v3 -- Enable versioning on a bucket
import {
  S3Client,
  PutBucketVersioningCommand,
} from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

await s3.send(
  new PutBucketVersioningCommand({
    Bucket: "my-ai-artifacts-bucket",
    VersioningConfiguration: {
      Status: "Enabled",
    },
  })
);
```

> ***Exam tip:*** *Versioning cannot be disabled, only suspended. Suspending versioning does not delete existing versions. CRR requires versioning on both source and destination buckets.*

---

11. Access Control

S3 provides multiple layers of access control. The exam expects you to know when to use each.

Bucket Policies (most common)

JSON resource-based policies attached to a bucket. Use for:

- Granting cross-account access
- Enforcing encryption requirements
- Restricting access by IP, VPC endpoint, or source ARN
- Requiring HTTPS/TLS

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequireKMSEncryption",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::my-bucket/*",
```

```
        "Condition": {
          "StringNotEquals": {
            "s3:x-amz-server-side-encryption": "aws:kms"
          }
        }
      }
    ]
  }
```

IAM Policies

Identity-based policies attached to IAM users, roles, or groups. Use for:

- Controlling what actions a specific role (e.g., a Bedrock Knowledge Base role) can perform
- Implementing least-privilege access per service or application

S3 Access Points

Named network endpoints with distinct permissions policies. Use for:

- Simplifying access management for shared datasets
- Providing different access policies for different teams (data science, ML engineering, audit)
- VPC-restricted access points

S3 Block Public Access

Account-level and bucket-level settings that override any policy granting public access. Always enable this for AI workloads.

ACLs (legacy)

S3 ACLs are legacy. AWS recommends disabling ACLs (bucket owner enforced) for all new buckets. This makes the bucket owner the sole owner of all objects regardless of who uploaded them.

S3 Object Lambda

Transforms data as it is retrieved from S3. Use cases: PII redaction before returning data to a model, format conversion, watermarking AI-generated content.

> ***Exam tip:*** *For Bedrock Knowledge Bases, the most common access pattern is an IAM role with* `s3:GetObject` *and* `s3:ListBucket` *plus a bucket policy allowing that role. For cross-account Knowledge Bases, the bucket policy must explicitly grant the KB role from the other account.*

---

12. Template Repositories for Prompt Management

Task 1.6.3 references S3 as a storage location for prompt templates.

Patterns:

- Store prompt templates as versioned JSON or YAML files in S3 with a clear prefix structure: `prompts/{use-case}/{version}/template.json`
- Use S3 versioning to track template changes over time.
- Use object tags to categorize templates: `model=claude-v3`, `task=summarization`, `status=active`.
- Use Lifecycle rules to archive deprecated templates.
- Use S3 event notifications to trigger template validation/deployment pipelines when new

templates are uploaded.

Bedrock Prompt Management stores managed prompts in the service, but S3 serves as the backing store for custom prompt management systems and for prompt templates used in custom orchestration code.

```
// TypeScript SDK v3 -- Retrieve a versioned prompt template from S3
import { S3Client, GetObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({ region: "us-east-1" });

const response = await s3.send(
  new GetObjectCommand({
    Bucket: "my-prompt-templates",
    Key: "prompts/summarization/v3/template.json",
    VersionId: "specific-version-id-for-reproducibility",
  })
);

const template = JSON.parse(await response.Body!.transformToString());
```

S3 Key Metrics for Monitoring

Metric	Source	What It Tells You
`BucketSizeBytes`	CloudWatch (daily)	Total storage size per storage class
`NumberOfObjects`	CloudWatch (daily)	Object count (track dataset growth)
`AllRequests`	S3 request metrics	Total API calls (monitor pipeline activity)
`4xxErrors` / `5xxErrors`	S3 request metrics	Access issues / service errors
`FirstByteLatency`	S3 request metrics	Time to first byte (performance)
`ReplicationLatency`	CRR metrics	Time for object to replicate to destination
`BytesPendingReplication`	CRR metrics	Replication backlog
S3 Storage Lens	Dashboard	Org-wide visibility into usage, cost, security posture

---

Amazon EBS

1. Block Storage for EC2/GPU Instances

Amazon Elastic Block Store provides block-level storage volumes for EC2 instances, including GPU instances used for ML training and inference (P4d, P5, G5, Inf2, Trn1).

Key characteristics:

- Attached to a single EC2 instance (except io2 with multi-attach).
- Persist independently from instance lifecycle (unlike instance store).
- Exist within a single Availability Zone.
- Snapshot to S3 for backup and cross-Region copy.

When to use EBS for AI:

- Root volumes for SageMaker notebook instances and EC2-based training.
- High-IOPS storage for model checkpoint writing during training.

- Local cache for model weights on inference instances.
- Scratch space for data preprocessing on GPU instances.

2. Volume Types for AI Workloads

Volume Type	Max IOPS	Max Throughput	Durability	AI Use Case
gp3	16,000	1,000 MiB/s	99.8-99.9%	Default for most workloads. Notebooks, general training.
gp2	16,000	250 MiB/s	99.8-99.9%	Legacy. Prefer gp3 (same price, better baseline).
io2 Block Express	256,000	4,000 MiB/s	99.999%	Large model checkpoint I/O, high-throughput training.
io1	64,000	1,000 MiB/s	99.999%	Provisioned IOPS for latency-sensitive inference.
st1	500	500 MiB/s	99.8-99.9%	Sequential reads of large training datasets.
sc1	250	250 MiB/s	99.8-99.9%	Cold storage for infrequently accessed data.

gp3 vs gp2: gp3 has independently configurable IOPS (up to 16,000) and throughput (up to 1,000 MiB/s) at no additional cost above the baseline 3,000 IOPS / 125 MiB/s. gp2 ties IOPS to volume size (3 IOPS per GiB, burst to 3,000). Always prefer gp3 for new workloads.

io2 Block Express: The highest-performance EBS option. Use for training jobs that write frequent, large checkpoints and need consistent sub-millisecond latency. Supports multi-attach for shared access from up to 16 Nitro instances in the same AZ.

3. Snapshots for Model Artifacts

EBS snapshots are incremental, point-in-time copies stored in S3 (managed by AWS).

AI use cases:

- Snapshot a volume containing a fine-tuned model before deploying to production.
- Copy snapshots cross-Region for DR.
- Create AMIs with pre-loaded model weights for faster instance launch.
- Use Amazon Data Lifecycle Manager to automate snapshot creation/retention/deletion.

Key behaviors:

- First snapshot captures the entire volume. Subsequent snapshots are incremental.
- Snapshots can be shared cross-account or made public.
- Snapshots can be encrypted (and copied to a different KMS key).
- Use `fast snapshot restore` to eliminate the first-access latency penalty on volumes created from snapshots.

> ***Exam tip:*** *If a question asks about the fastest boot time for an inference instance with a large model pre-loaded, the answer is an AMI backed by an EBS snapshot with fast snapshot restore enabled. For the highest IOPS for training checkpoints, use io2 Block Express.*

Amazon EFS

1. Shared File System for SageMaker

Amazon Elastic File System is a fully managed, elastic NFS file system that can be mounted by multiple EC2 instances, containers, and Lambda functions concurrently.

Key characteristics:

- POSIX-compliant (NFS v4.1).
- Elastic -- grows and shrinks automatically, no provisioning needed.
- Regional (Standard) or single-AZ (One Zone) deployment.
- 99.999999999% (11 nines) durability for Standard storage class.
- Supports encryption at rest (KMS) and in transit (TLS).

SageMaker integration:

- SageMaker training jobs can read input data from EFS using `FileSystemDataSource` with `FileSystemType: "EFS"` and `FileSystemAccessMode: "ro"` (read-only) or `"rw"` (read-write).
- SageMaker Studio can mount custom EFS file systems for shared notebooks and datasets across team members.
- EFS is ideal for datasets that are updated frequently and need to be available to multiple concurrent training jobs without copying.

SageMaker data input modes comparison:

Mode	Source	Startup Latency	Best For
File mode	S3	High (downloads first)	Small/medium datasets
Fast file mode	S3	Low (streams on demand)	Large datasets, sequential reads
Pipe mode	S3	Low (named pipes)	Legacy streaming
EFS / FSx	File system	None (already mounted)	Shared datasets, iterative training

2. Lambda + EFS Integration for Large Models

Lambda functions can mount EFS file systems for access to files larger than the 512 MB `/tmp` limit or the 250 MB deployment package limit.

Use cases for GenAI:

- Store large ML model files (ONNX, PyTorch) on EFS and load them in Lambda for inference.
- Share embedding caches across Lambda invocations.
- Store large prompt template libraries that exceed deployment package limits.

Requirements:

- Lambda must be in a VPC with subnets that can route NFS traffic (port 2049) to EFS mount targets.
- Lambda execution role needs `elasticfilesystem:ClientMount` and `elasticfilesystem:ClientWrite` permissions.
- Use EFS Access Points to control the POSIX user/group and root directory per function.
- Use at least two AZs for availability.

3. Throughput Modes for AI Workloads

Mode	Behavior	AI Use Case
Elastic (default)	Auto-scales up to 10 GiB/s read, 3 GiB/s write	Spiky training workloads, variable model loading
Provisioned	User-specified throughput regardless of storage size	Known sustained throughput needs (e.g., continuous data ingestion)
Bursting	Throughput scales with storage size, burst credits	Small file systems with periodic high-throughput needs

Performance modes:

- **General Purpose** (default): Lowest per-operation latency. Use for most AI workloads.
- **Max I/O**: Higher throughput for highly parallelized workloads. Higher latencies. Previous

generation -- consider Elastic throughput + General Purpose instead.

EFS storage classes:

- Standard -- frequently accessed files
- Infrequent Access (IA) -- lower cost, per-access charge
- EFS Lifecycle Management automatically transitions files to IA after a configurable period

(7, 14, 30, 60, 90, 180, 270, or 365 days).

> ***Exam tip:*** *EFS Elastic throughput is the best default for ML workloads because training jobs create bursty I/O patterns. Use Provisioned throughput only if you consistently need more throughput than what Elastic provides for your file system size. EFS is the answer when multiple SageMaker training jobs or Lambda functions need concurrent access to the same data.*

AWS DataSync

1. Data Migration for AI Datasets

AWS DataSync is an online data transfer service that simplifies and accelerates moving data between on-premises storage, edge locations, other clouds, and AWS storage services.

Key features:

- Transfers data up to 10x faster than open-source tools.
- Handles encryption, data integrity validation, network optimization automatically.
- Supports NFS, SMB, HDFS, and object storage protocols on-premises.
- Transfers to/from S3, EFS, FSx for Windows, FSx for Lustre, FSx for OpenZFS, FSx for ONTAP.
- Supports scheduled transfers (e.g., daily sync of new training data).
- Supports filtering by file name pattern, size, or modification time.
- Preserves file metadata and permissions.
- Pay-per-GB transferred.

Architecture:

- Deploy a DataSync agent on-premises (as a VM) near the source storage.
- Agent connects to the DataSync service over the internet or AWS Direct Connect.
- For cloud-to-cloud transfers (e.g., S3 in one Region to S3 in another), no agent is needed.

AI data migration patterns:

Pattern	How
Initial dataset migration	Deploy agent on-prem, schedule full sync to S3
Continuous data ingestion	Schedule recurring DataSync tasks (e.g., every 6 hours)
Knowledge Base population	Sync document repositories from on-prem file servers to S3
Cross-Region dataset copy	S3-to-S3 transfer across Regions (agentless)
Hybrid training	Sync training data to S3, then use SageMaker fast file mode

2. On-Premises to S3 Transfer

Setup steps:

1. Deploy the DataSync agent VM in your on-premises environment.
2. Activate the agent in the AWS console (connects via port 443).
3. Create a source location (NFS/SMB/HDFS path on-premises).
4. Create a destination location (S3 bucket + prefix).
5. Create a task linking source to destination with transfer options.
6. Run the task manually or on a schedule.

Transfer options:

- Verify data integrity (checksums).
- Preserve file metadata (timestamps, permissions, ownership).
- Filter files by include/exclude patterns.
- Configure bandwidth throttling.
- Choose handling of deleted files (preserve or delete at destination).

Cross-account transfers:

- The DataSync IAM role in the source account needs permissions on the destination S3 bucket.
- The destination bucket policy must grant the DataSync role from the source account.
- ACLs must be disabled on the destination bucket.

> ***Exam tip:*** *DataSync is the answer when a question describes migrating large datasets from on-premises to S3 for AI training. For one-time large transfers where network bandwidth is limited, AWS Snow Family (Snowball) is the answer. DataSync is for ongoing/repeated transfers over the network.*

AWS Transfer Family

1. SFTP/FTPS for Data Ingestion

AWS Transfer Family provides fully managed file transfer endpoints using standard protocols: SFTP, FTPS, FTP, and AS2. Files land directly in S3 or EFS.

Key features:

- No infrastructure to manage -- fully managed, HA, multi-AZ.
- Supports multiple identity providers: service-managed, AWS Managed Microsoft AD, custom

(Lambda-backed for integration with LDAP, Okta, etc.).

- Files stored natively in S3 (or EFS).
- Integrates with IAM, KMS, CloudTrail, CloudWatch for security and monitoring.

- Supports managed workflows for post-upload processing (decrypt, tag, copy, invoke Lambda).

Protocols:

Protocol	Port	Encryption	Notes
SFTP	22	SSH	Most common. Publicly accessible or VPC-hosted.
FTPS	990/21	TLS	FTP over TLS. NAT/firewall-friendly with passive mode.
FTP	21	None	Unencrypted. VPC-only (not internet-facing).
AS2	443	S/MIME	B2B EDI exchanges.

2. Integration with S3

AI data ingestion patterns:

- External partners upload training data, evaluation datasets, or documents via SFTP.
- Files land in an S3 bucket (scoped per user via IAM + home directory mappings).
- S3 event notifications trigger downstream processing (parsing, embedding, indexing).
- Managed workflows can automatically tag, validate, or transform uploaded files.

User access configuration:

- Each user gets a home directory mapped to an S3 prefix (e.g., `s3://bucket/partner-a/`).
- Logical directory mappings allow presenting a virtual directory structure while files are stored in different S3 paths.
- Session policies control S3 API actions per user (e.g., read-only, write-only, specific prefix).

Endpoint types:

- **Public** -- internet-accessible with an AWS-provided hostname or custom domain (Route 53).
- **VPC** -- accessible only within a VPC (or via VPN/Direct Connect). Supports fixed Elastic IPs for allowlisting.
- **VPC Internal** -- no internet access, only VPC-internal.

```
// TypeScript SDK v3 -- Create a Transfer Family SFTP server backed by S3
import {
  TransferClient,
  CreateServerCommand,
} from "@aws-sdk/client-transfer";

const client = new TransferClient({ region: "us-east-1" });

const response = await client.send(
  new CreateServerCommand({
    Protocols: ["SFTP"],
    IdentityProviderType: "SERVICE_MANAGED",
    EndpointType: "PUBLIC",
    Domain: "S3",
    Tags: [{ Key: "project", Value: "genai-data-ingestion" }],
  })
);
```

> ***Exam tip:*** *Transfer Family is the answer when a question involves external partners uploading data via SFTP/FTPS. DataSync is for bulk migration from on-premises storage. Transfer Family = protocol-based file transfer from external parties. DataSync = automated, high-performance bulk data movement.*

Cross-Cutting Exam Gotchas

Topic	Gotcha
S3 default encryption	SSE-S3 is always on. You do not need to "enable" encryption.
S3 metadata immutability	User-defined metadata (`x-amz-meta-*`) cannot be changed after upload.
Bedrock KB metadata files	Must be `.metadata.json` sidecar files, not S3 metadata or tags.
Versioning	Cannot be disabled once enabled, only suspended. Required for CRR.
CRR existing objects	CRR only applies to new objects. Use Batch Replication for existing.
Lifecycle timing	Minimum 30 days before Standard-to-IA transition. Glacier min durations apply.
Intelligent-Tiering fees	No retrieval fee, but there is a small monthly monitoring charge per object.
S3 Select	Deprecated for new customers. Prefer Athena for new workloads.
SQS FIFO	Not supported as S3 event notification destination.
EBS AZ scope	EBS volumes are AZ-scoped. Cannot be shared across AZs (use EFS for that).
EFS throughput	Elastic throughput is the default and best for spiky ML I/O patterns.
DataSync vs Snowball	DataSync = network transfer. Snowball = physical device for limited bandwidth.
DataSync vs Transfer Family	DataSync = automated migration. Transfer Family = protocol endpoints for external parties.
S3 Bucket Keys	Reduce KMS costs by up to 99%. Always enable when using SSE-KMS at scale.
EFS + Lambda	Lambda must be in a VPC to mount EFS. Requires port 2049 (NFS) access.
gp3 vs gp2	Always prefer gp3. Same cost, independently configurable IOPS and throughput.

Amazon Connect -- GenAI Study Guide (AIP-C01)

Amazon Connect is a **minor exam topic** listed under "Customer Engagement" in the in-scope services. It will not be the focus of many questions, but it appears in scenarios involving AI-powered contact centers, agent assist, and customer self-service. Know it well enough to pick it as the right answer when the scenario is about a cloud contact center with GenAI capabilities.

---

1. Service Overview

Amazon Connect is a **fully managed, cloud-based contact center** service. It supports voice, chat, email, and task channels from a single platform. Agents interact with customers through a web-based Contact Control Panel (CCP).

Core Components

Component	Purpose
Contact Flows	Visual workflow builder that defines the customer experience (IVR menus, routing logic, bot handoffs, prompts)
Queues	Hold contacts waiting for an agent; routing profiles determine which queues an agent can handle
Routing Profiles	Map agents to queues with priority and delay settings
Contact Control Panel (CCP)	Web-based UI where agents handle interactions
Customer Profiles	Unified customer data from multiple sources
Cases	Track and manage customer issues across multiple contacts

Key Characteristic

Connect is **pay-per-use** with no upfront commitment, minimum fees, or long-term contracts. You pay per minute of voice usage and per message for chat.

---

2. GenAI Features

Amazon Connect's AI capabilities are **built on Amazon Bedrock**. Connect fully manages the underlying AI including model selection, prompt definition, and capacity provisioning. It may use **cross-region inference** to process requests optimally.

Contact Lens (Analytics and Quality Management)

Contact Lens is the analytics engine built into Connect. It is powered by Bedrock and provides:

- **Conversational analytics** for voice, chat, and email -- sentiment analysis, issue detection, contact categorization
- **Real-time analytics** -- monitor live calls/chats for sentiment shifts, keyword triggers, compliance violations
- **Post-contact analytics** -- transcription, talk-time distribution, trend detection

- **Generative AI-powered post-contact summaries** -- auto-generated summaries of voice, chat, and email contacts so agents and supervisors do not need to read full transcripts
- **Semantic rule matching** -- categorize contacts using natural language statements (GenAI-powered) instead of rigid keyword lists
- **Sensitive data redaction** -- automatically redacts PII from transcripts, audio files, and emails
- **Automated agent performance evaluations** -- GenAI analyzes conversation transcripts to answer evaluation form questions, enabling managers to evaluate up to 100% of interactions

Connect AI Agents (formerly Amazon Q in Connect)

Connect AI agents are the **real-time GenAI assistant** for both self-service and agent assist:

For End Customers (Self-Service):

- AI agents engage directly with customers over voice and chat
- Answer questions, take actions (process returns, update accounts), resolve issues autonomously
- Seamlessly escalate to human agents when needed, preserving full conversation context

For Human Agents (Agent Assist):

- Detect customer intent in real-time during calls, chats, tasks, and emails
- Provide generative responses, suggested actions, and links to relevant knowledge articles
- Agents can query the AI agent directly using natural language
- Auto-generate case summaries and email responses

Customization:

- Custom AI prompts and AI guardrails
- Integration with step-by-step guides
- Multiple knowledge bases per AI agent (including Amazon Bedrock Knowledge Bases)
- Supports Model Context Protocol (MCP) for accessing organizational resources
- GDPR compliant and HIPAA eligible

Generative AI Email Support

- Auto-generates email conversation overviews, suggested actions, and draft responses
- Uses knowledge bases and custom prompts to tailor outputs

---

3. Integration with Bedrock and Lex

Amazon Bedrock

- Connect's AI features (Contact Lens, AI agents, summaries, evaluations) all use models via **Amazon Bedrock**
- Connect manages model selection -- you do not choose which FM to use for built-in features
- Data protection: prompts and responses are never stored, logged, or shared by Bedrock, and are never used to train models
- Supports multiple Amazon Bedrock Knowledge Bases per AI agent, including existing Bedrock KBs without data duplication

Amazon Lex

Amazon Lex provides the **traditional conversational AI** (intent-based chatbot) layer in Connect:

- Added to contact flows via the **"Get customer input" block**
- Handles structured interactions: DTMF input ("press 1 for..."), slot-filling ("what is your account number?"), intent detection
- Routes contacts to different flow branches based on detected intents
- Lex bots can be built directly in the Connect admin website

Lex + Connect AI Agents (Complementary, Not Competing)

This is an important distinction for the exam:

Capability	Amazon Lex	Connect AI Agents
Approach	Intent-based, rule-driven	Generative, context-aware
Best for	Predefined workflows, slot collection, DTMF menus	Open-ended questions, knowledge retrieval, dynamic actions
Conversation style	Structured (intents + slots)	Free-form natural language
Underlying tech	NLU/NLP models	LLMs via Bedrock

They are designed to work together: Lex handles well-defined intents and structured data collection, while Connect AI agents handle open-ended queries and generative responses. A single contact flow can use both.

---

4. Exam Relevance

When Connect Is the Answer

- Scenario describes a **cloud contact center** needing AI-powered agent assistance
- Question mentions **real-time agent coaching** or **post-contact summarization** for voice/chat
- Scenario involves **automated quality evaluation** of customer-agent conversations
- Question asks about **AI-powered self-service** over voice or chat in a contact center context
- Scenario requires **sentiment analysis on live customer calls**
- Question mentions **omnichannel** (voice + chat + email) customer engagement with AI

When Connect Is NOT the Answer

Scenario	Correct Service
Building a standalone chatbot (no contact center)	**Amazon Lex** (or Bedrock Agents)
General-purpose RAG application	**Amazon Bedrock Knowledge Bases**
Custom ML model for customer analytics	**Amazon SageMaker AI**
Email marketing campaigns	**Amazon SES** or **Amazon Pinpoint**
Push notifications or SMS campaigns	**Amazon Pinpoint**
Custom GenAI app with full model control	**Amazon Bedrock** directly

Key Differentiator

Connect is the answer when the question is specifically about a **contact center** use case. If the scenario does not mention agents handling calls/chats or a contact center, Connect is likely not the answer.

---

5. Exam Gotchas

1. **Contact Lens vs. Amazon Comprehend**: Contact Lens handles sentiment analysis *within Connect*. If the question is about sentiment analysis on arbitrary text outside a contact center, the answer is Amazon Comprehend. Inside a contact center scenario, it is Contact Lens.
2. **Connect AI Agents vs. Bedrock Agents**: Connect AI agents are purpose-built for the contact center (agent assist, self-service over phone/chat). Bedrock Agents are for general-purpose agentic workflows. If the scenario is a contact center, Connect AI agents is the answer.
3. **Lex alone vs. Lex in Connect**: Lex can be used independently for chatbots. When Lex appears in a Connect question, it is specifically the front-end IVR/chatbot component of the contact center flow. The exam may test whether you know Lex is added via the "Get customer input" flow block.
4. **Connect manages its own AI**: You do not pick a specific Bedrock model for Contact Lens or AI agents. Connect handles model selection, prompt engineering, and capacity internally. This is different from using Bedrock directly where you choose the model.
5. **Cross-region inference**: Connect may process AI inference requests in a different region than your Connect instance for optimal performance. All data stays encrypted on Amazon's network (never public internet). This may appear as a distractor about data residency.
6. **Post-contact summaries require Contact Lens enabled**: Generative AI summaries are a Contact Lens feature, not a standalone capability. The "Set recording and analytics behavior" flow block must be configured.

---

6. When to Use / When NOT to Use

Use Amazon Connect When

- You need a **cloud-based contact center** with built-in AI capabilities
- You want **GenAI-powered agent assist** (real-time suggestions, auto-summaries, knowledge retrieval) in a call/chat center
- You need **automated quality management** with AI-evaluated agent performance
- You want **AI self-service** that can handle customer issues autonomously over voice and chat, with human escalation
- You need **omnichannel routing** (voice, chat, email, tasks) with AI analytics across all channels

Do NOT Use Amazon Connect When

- You do not need a full contact center -- just a chatbot (use Lex or Bedrock Agents)
- You need general-purpose GenAI application development (use Bedrock directly)
- You need custom ML models for customer analytics (use SageMaker AI)
- You need marketing campaign management (use Pinpoint)
- You need only speech-to-text transcription without a contact center (use Amazon Transcribe)

---

7. Key Concepts Quick Reference

Concept	Definition
Contact Flow	Defines the end-to-end customer experience: IVR menus, prompts, Lex bot handoffs, routing, AI agent integration
Queue	Where contacts wait to be routed to an available agent
Routing Profile	Determines which queues an agent handles and at what priority
Contact Lens	Built-in analytics: transcription, sentiment, categorization, summaries, evaluations
Connect AI Agents	GenAI assistant for both self-service and agent assist, powered by Bedrock
Amazon Q in Connect	Previous name for Connect AI agents (may still appear in exam materials)
Get Customer Input block	Flow block that invokes Amazon Lex bots for structured input collection
Connect Assistant block	Flow block that activates Connect AI agents for GenAI capabilities
Post-Contact Summary	GenAI-generated summary of a completed interaction (voice, chat, or email)
Semantic Rule Matching	GenAI-powered contact categorization using natural language instead of keyword lists
Customer Profiles	Unified customer data that AI agents use for personalized responses
Cases	Track customer issues across multiple contacts; AI can auto-summarize case history

---

8. Architecture Patterns for Exam

Pattern 1: AI-Powered IVR with Escalation

```
Customer Call -&gt; Contact Flow -&gt; Lex Bot (intent detection, slot filling)
  -&gt; Resolved? -&gt; End
  -&gt; Not Resolved? -&gt; Queue -&gt; Human Agent (with Connect AI agent assist)
```

Pattern 2: GenAI Self-Service with Human Fallback

```
Customer Chat -&gt; Contact Flow -&gt; Connect AI Agent (autonomous resolution)
  -&gt; Resolved? -&gt; End
  -&gt; Complex Issue? -&gt; Escalate to Human Agent (full context preserved)
```

Pattern 3: Quality Management Loop

```
Contact Completed -&gt; Contact Lens (transcription, sentiment, categorization)
  -&gt; GenAI Post-Contact Summary
  -&gt; Automated Performance Evaluation (GenAI answers evaluation criteria)
  -&gt; Supervisor Review
```

Summary for Exam Day

- Amazon Connect = **cloud contact center** with built-in GenAI
- All AI features run on **Amazon Bedrock** (managed by Connect, you do not pick the model)
- **Contact Lens** = analytics, sentiment, transcription, summaries, evaluations

- **Connect AI Agents** = real-time agent assist + customer self-service
- **Amazon Lex** = structured chatbot/IVR within Connect flows (complementary to AI agents)
- Pick Connect when the scenario is about a **contact center**; pick other services when it is not
- Connect AI agents are different from Bedrock Agents: Connect AI agents are purpose-built for contact center use cases